

enumext

ENUMERATE EXERCISE SHEETS

V2.1 2026-01-15*

©2024–2026 by Pablo González L†

CTAN: <https://www.ctan.org/pkg/enumext>

 <https://github.com/pablgonz/enumext>

Abstract

This package provides enumerated list environments compatible with *tagging* PDF for creating “*simple exercise sheets*” along with “*multiple choice questions*”, storing the “*answers*” to these in memory using *multicol* package.

Contents

1	Introduction	1	5.7	Keys for multicol	11
1.1	Description and usage	2	5.8	Keys for minipage	12
1.2	The concept of left margin	3	5.8.1	The command <code>\miniright</code>	12
1.3	User interface	3	5.8.2	The key <code>mini-right</code>	12
1.3.1	Public counters	3	6	The storage system	12
1.3.2	Public dimension	3	6.1	Keys for storage system	12
1.3.3	Support for <i>multicol</i>	3	6.1.1	Keys for <code>label</code> and <code>ref</code>	13
1.3.4	Support for <i>minipage</i>	4	6.1.2	Keys for wrap and marks	13
1.3.5	The <code>\label</code> and <code>\ref</code> system	4	6.1.3	Keys for debug and checking	14
1.3.6	Support for <code>\footnote</code>	4	6.2	The command <code>\anskey</code>	14
2	The environments provided	5	6.2.1	Keys for <code>\anskey</code>	15
2.1	The environment <code>enumext</code>	5	6.3	The environment <code>anskey*</code>	15
2.2	The environment <code>enumext*</code>	5	6.3.1	Keys for <code>anskey*</code>	15
2.3	The command <code>\item*</code>	5	6.4	The environment <code>keyans</code>	16
2.3.1	Keys for <code>\item*</code>	6	6.4.1	The <code>\item*</code> in <code>keyans</code>	17
2.4	The command <code>\item</code> in <code>enumext*</code>	6	6.5	The environment <code>keyanspic</code>	17
3	The command <code>\setenumext</code>	6	6.5.1	Keys for <code>keyanspic</code>	18
4	The command <code>\setenumextmeta</code>	6	6.5.2	The command <code>\anspic</code>	18
5	The <code>keyval</code> system	7	6.6	Printing stored content	19
5.1	Keys for <code>label</code> and <code>ref</code>	7	6.6.1	The command <code>\getkeyans</code>	19
5.2	Keys for penalties	8	6.6.2	The command <code>\foreachkeyans</code>	19
5.3	Keys for spaces	8	6.6.3	The command <code>\printkeyans</code>	20
5.3.1	Vertical spaces	8	7	Full examples	21
5.3.2	Horizontal spaces	9	8	Tagged PDF examples	24
5.4	Keys for add code	10	9	The way of non-enumerated lists	24
5.5	Keys for start, series and resume	10	10	References	26
5.6	Keys for reset	11	11	Change history	27
5.6.1	The command <code>\resetenumext</code>	11	12	Index of Documentation	28
			13	Implementation	30
			14	Index of Implementation	151

Motivation and acknowledgments

Usually, it is enough to use the classic *enumerate* environment to generate “*simple exercise sheets*” or “*multiple choice questions*”, the basic idea behind *enumext* is to cover three points:

1. To have a simple interface to be able to write “*lists of exercises*” with “*answers*”.
2. To have a simple interface for writing “*multiple choice questions*”.
3. To have a simple interface for placing “*columns*” and “*drawings*” or “*tables*”.

This package would not be possible without Phelype Oleinik who has collaborated and adapted a large part of the code and all *TeX* team for their great work and to the different members of the *TeX-SX* community who have provided great answers and ideas. Here are notes on the main ones:

1. Answer given by Alan Munn in `\topsep`, `\itemsep`, `\partopsep`, `\parsep` - what do they each mean (and what about the bottom)?
2. Answer given by Enrico Gregorio in *Understanding minipages - aligning at top*
3. Answer given by Ulrich Diez in *Different mechanics of hyperlink vs. hyperref*
4. Answer given by Enrico Gregorio in *Minipage and multicol, vertical alignment*

*This file describes a documentation for v2.1, last revised 2026-01-15.

†E-mail: pablgonz@educarchile.cl.

License and Requirements

Permission is granted to copy, distribute and/or modify this software under the terms of the LaTeX Project Public License (lpp), version 1.3 or later (<https://www.latex-project.org/lppl.txt>). The software has the status “maintained”.
The enumext package loads and requires multicol[3] package, need to have a modern T_EX distribution such as T_EX Live or MiK_TTeX. It has been tested with the standard classes provided by L^AT_EX: book, report, article and letter on 10pt, 11pt and 12pt.

The minimum requirement is L^AT_EX release 2025-11-01.

1 Introduction

In the L^AT_EX world there are many useful packages and classes for creating “lists of exercises”, “worksheets” or “multiple choice questions”, classes like exam[1] and packages like xsim[2] do the job perfectly, but they don’t always fit the basic day to day needs.

In my work (and in the work of many teachers) it is common to use “simple exercise sheets” also known as “informal lists of exercises”, for example:

1. Factor $x^2 - 2x + 1$

2. Factor $3x + 3y + 3z$

3. True False

(a) $\alpha > \delta$

(b) L^AT_EXze is cool?

4. Related to Linux
- (a) You use Linux?

(b) Usually uses the package manager?

(c) Rate the following package and class

i. xsim-exam

ii. xsim

iii. exsheets

Sometimes we are also interested in showing the “answers” along with the questions:

1. Factor $x^2 - 2x + 1$

*

$(x - 1)^2$

2. Factor $3x + 3y + 3z$

*

$3(x + y + z)$

3. True False

(a) $\alpha > \delta$

*

False

(b) L^AT_EXze is cool?

*

Very True!

4. Related to Linux
- (a) You use Linux?

*

Yes

(b) Usually uses the package manager?

*

Yes, dnf

(c) Rate the following package and class

i. xsim-exam

*

doesn’t exist for now :(

ii. xsim

*

very good

iii. exsheets

*

obsolete

Or we are interested in referring to a specific question and its “answer”, for example:

The answer to 3.(b) is “Very True!” and the answer to 4.(c).ii is “very good”.

Or we are interested in printing all the “answers”:

1. $(x - 1)^2$

⌘

2. $3(x + y + z)$

⌘

3. (a) False

(b) Very True!

⌘

4. (a) Yes

⌘
- (b) Yes, dnf

⌘

(c) i. doesn’t exist for now :(

⌘

ii. very good

⌘

iii. obsolete

⌘

Another very common thing to use in my work is “multiple choice questions”, for example:

1. First type of questions

A) value

C) value

B) correct

D) value

2. Second type of questions

I. $2\alpha + 2\delta = 90^\circ$

II. $\alpha = \delta$

III. $\angle EDF = 45^\circ$

A) I only

D) I and III only

B) II only

E) I, II, and III

C) I and II only

★ 3. Third type of questions

(1) $2\alpha + 2\delta = 90^\circ$

(2) $\angle EDF = 45^\circ$

A) value

D) value

B) value

E) value

C) value
4. Question with image and label below:

A

A)

B

B)

A

C)

A

D)



E)

5. Question with image on right side:

A) value

B) value

C) value

D) correct

E) value

B

Where we are interested in the `\label` and a “short note” that we leave as an explanation, and then print them:

1. B) $x = 5$

2. D)

3. C) some note
- ⌘ 4. E) A duck

⌘ 5. D) “other note”

⌘

The `enumext` package was created and designed to meet these small requirements in the creation of “simple worksheets” and “multiple choice questions”.

- These “simple worksheets” or “multiple choice questions” appear to be easy to obtain using a combination of the `enumerate`, `minipage` and `multicols` environments, but like many things, what “looks simple” is not so simple.

1.1 Description and usage

The `enumext` package defines enumerated environments using the `list` environment provided by $\text{\TeX}$, but “does not redefine” any internal commands associated with it such as `\list`, `\endlist` or `\item` outside of the “scope” in which they are defined.

- This package is NOT intended to replace the `enumerate` environment nor replace the powerful `enumitem`[6], the approach is intended to work without hindering either of them.

This package can be used with `xelatex`, `lualatex`, `pdflatex` and the classical `latex`»`dvips`»`ps2pdf` and is present in $\text{\TeX}$ Live and $\text{\MiKTeX}$, use the package manager to install. For manual installation, download `enumext.zip` and unzip it, run `luatex enumext.ins` and move all files to appropriate locations, then run `mktexlsr`. To produce the documentation run `arara enumext.dtx`.

<code>enumext.sty</code>	»	<code>TDS:tex/latex/enumext/</code>
<code>enumext.pdf</code>	»	<code>TDS:doc/latex/enumext/</code>
<code>README.md</code>	»	<code>TDS:doc/latex/enumext/</code>
<code>enumext.dtx</code>	»	<code>TDS:source/latex/enumext/</code>
<code>enumext.ins</code>	»	<code>TDS:source/latex/enumext/</code>

The package is loaded in the usual way:

```
\usepackage{enumext}
```

1.2 The concept of left margin

There is a direct relationship between the parameters `\leftmargin`, `\itemindent`, `\labelwidth` and `\labelsep` plus an “extra space”, which makes it difficult to obtain the desired *horizontal spaces* in a `list` environment. Usually we don’t want the `list` to go beyond the left margin of the page, but since these four values are related, that causes a problem.

The `enumitem`[6] package adds the `\labelindent` parameter to solve some of these problems. A simplified representation of this in the figure 1.

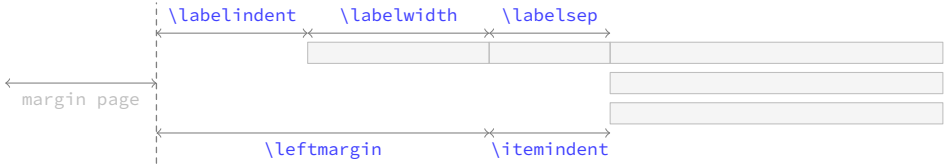


Figure 1: Representation of horizontal lengths in `enumitem`.

The `enumext` package does NOT provide a user interface to set the values for `\leftmargin` and `\itemindent`, instead it provides the keys `list-offset` and `list-indent` which internally set the values for `\leftmargin` and `\itemindent`. The concepts of `\leftmargin` and `\itemindent` are different in `enumext`. The figure 2 shows the visual representation of idea.

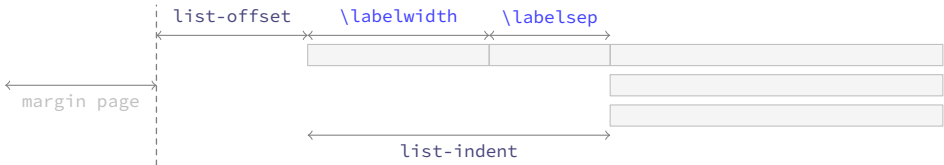


Figure 2: Representation of horizontal lengths concept in `enumext`.

In this way we reduce a *little* the amount of parameters we have to pass. With the default values of keys `list-offset`, `list-indent`, `labelwidth` and `labelsep` the lists will have the (usually) expected output for “simple worksheets”. The figure 3 shows the visual representation.

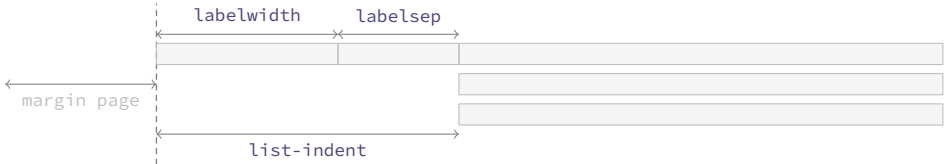


Figure 3: Default horizontal lengths `list-offset=0pt`, `list-indent=\labelwidth+\labelsep` in `enumext`.

1.3 User interface

The user interface consists of two main list environments `enumext` (vertical) and `enumext*` (horizontal), the environment `anskey*` and the command `\anskey` to “store content” and the environments `keyans`, `keyans*` and `keyanspic` for multiple choice. It also provides the commands `\getkeyans` to print individual stored content, `\printkeyans` to print all stored content, `\foreachkeyans` to print a range of stored content, `\miniright` for `minipage`, `\setenumext` to config [`\key = val`], `\setenumextmeta` to add a “meta-key” and `\resetenumext` to reset counters.

1.3.1 Public counters

The package `enumext` uses the `enumXi`, `enumXii`, `enumXiii`, `enumXiv` counters for the *four* nesting levels of the `enumext` environment, the `enumXv` counter for the `keyans` environment, the `enumXvi` counter for the `keyanspic` environment, the counter `enumXvii` for `enumext*` environment and the counter `enumXviii` for `keyans*` environment.

- If any package defines these *counters* or they are user-defined in the document, the package will return a “fatal error” and abort the load.

1.3.2 Public dimension

The package `enumext` only provides a *single public dimension* `\itemwidth` and is intended for user convenience only and is NOT for internal use as such. The dimension `\itemwidth` is a *rigid length* and contains the “width of the content” of each `\item` regardless of `labelwidth` and `labelsep`.

- If any package defines `\itemwidth` or the user defines `\itemwidth` in the document, the package will overwrite it without warning.

1.3.3 Support for multicol

The package provides direct support for using the `multicol`[3] package. This allows for a two-column output as shown in the figure 4.

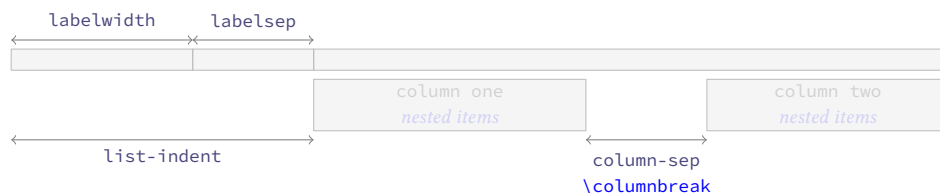


Figure 4: Representation of the two column output for a nested level in `enumext` environment.

The “non starred” version of the `multicols` environment is always used together with the `\raggedcolumns` command and is controlled by `columns` and `columns-sep` keys. It can be used in all nesting levels of the environment `enumext` and the environment `keyans` and can together with the `mini-env` key. If you need to force a start a new column `\columnbreak` must be used (see §5.7).

- The `\columnseprule` command is not available as a key and is set to “zero” for the inner levels and the `keyans` environment. If the value of this is set inside the document, it will affect “all environments” that use the `columns` key.

1.3.4 Support for minipage

The package provides direct support for `minipage` environment, this allows you to obtain an output like the one shown in figure 5.



Figure 5: Representation of the `mini-env` output for a nested level `enumext` environment.

The `minipage` environments on the “left side” and “right side” are always used with “aligned on top” [`t`]. It can be used in all nesting levels of the environment `enumext` and the environment `keyans` and is controlled by `mini-env` and `mini-sep` keys. In order to switch from the “left” side `minipage` environment to the “right” side one must use the command `\miniright` (see §5.8).

1.3.5 The \label and \ref system

This package provides a user interface like the `enumitem`[6] package to customize the references which is activated by the `ref` key (§5.1), the standard $\TeX$ `\label` and `\ref` commands work as usual. It also provides an “internal reference” system for the “stored content” by means of the key `save-ref` (§6.1.1) when the key `save-ans` is active (§6.1).

1.3.6 Support for \footnote

The `enumext*` and `keyans*` environments and the `mini-env` key use the `minipage` environment in their implementation but in a transparent way for the user, it is only used for typesetting. The implementation supports `\footnote` and is compatible with `hyperref` and works the same way as if used anywhere in the document.

Unfortunately, if `hyperref` is loaded and `tagged` PDF is not active, it will not produce the “links”, the internal implementation uses `\footnotetext[⟨number⟩]` and `\footnotemark[⟨number⟩]{⟨text⟩}` and “links” for this are not supported by the `hyperref` package.

The best way to solve this if `tagged` PDF is NOT active is to use Jean-François Burnol’s `footnotehyper`[9] package. It will support keeping the “links” if `hyperref` is loaded with the `hyperfootnotes=true` option (default). Load it as follows:

```
\IfDocumentMetadataF
{
  \usepackage{footnotehyper}
  \makesavenoteenv{enumext}
  \makesavenoteenv{enumext*}
}
```

At the moment the `footnotehyper` package v1.1f (2025/11/15) is not compatible with `tagged` PDF.

2 The environments provided

The package `enumext` provides two main list environments, the *vertical* environment `enumext` and the *horizontal* environment `enumext*`.

enumext	<code>\begin{enumext}[⟨keyval list⟩]</code>	<code>\begin{enumext*}[⟨keyval list⟩]</code>
enumext*	<code>\item ⟨item content⟩</code>	<code>\item ⟨item content⟩</code>
	<code>\item[⟨custom⟩] ⟨item content⟩</code>	<code>\item[⟨custom⟩] ⟨item content⟩</code>
	<code>\item*[⟨symbol⟩][⟨offset⟩] ⟨item content⟩</code>	<code>\item*[⟨symbol⟩][⟨offset⟩] ⟨item content⟩</code>
	<code>\end{enumext}</code>	<code>\end{enumext*}</code>

2.1 The environment enumext

The `enumext` is an environment that works in the same way as the standard `enumerate` environment provided by L^AT_EX, `\item` and `\item[⟨custom⟩]` commands work in the usual way. The environment can be nested with at most “four levels” and the options can be configured globally using `\setenumext` command and locally using `[⟨key = val⟩]` in the environment.

Example with `columns=2`

1. This text is in the first level.
- A. This text is in the fourth level.
- (a) This text is in the second level.
- X This text is in the first level.
- i. This text is in the third level.
- ★ 2. This text is in the first level.

2.2 The environment enumext*

The `enumext*` is a *horizontal list environment* similar to the `shortenumerate` or `tasks` environments provided by the `shortlst`[16] and `tasks`[17] packages, `\item` and `\item[⟨custom⟩]` work as usual. The options can be configured globally using `\setenumext` command and locally using `[⟨key = val⟩]` in the environment.

Some considerations to take into account for this environment:

- The environment cannot be nested within itself or in the environment `keyans*`, but it can be nested within `enumext` and vice versa.
- Each “item content” in the environment is placed within a `minipage` environment whose *width* is stored in the dimension `\itemwidth` does NOT include `labelwidth` and `labelsep`, only the *width of the content*.
- You cannot have floating environments like `figure` or `table` or `\marginpar` but `\footnote` with `hyperref` is supported. If you want to activate links “without” `tagged` PDF active you must load `footnotehyper` and `hyperref` packages (see §1.3.6).
- You cannot have any standard list environments like `itemize`, `enumerate`, `description`, `quote`, `quotation`, `verse`, `center`, `flushleft`, `flushright`, `verbatim`, `tabbing`, `trivlist`, `list` and all environments created with `\newtheorem`.

Example with `columns=2`

1. This text is in the first level.
2. This text is in the first level.
- X This text is in the first level.
- ★ 4. This text is in the first level.

2.3 The command \item*

`\item*` `\item*[⟨symbol⟩][⟨offset⟩]`

The `\item*`, `\item*[⟨symbol⟩]` and `\item*[⟨symbol⟩][⟨offset⟩]` works like the numbered `\item`, but places a `⟨symbol⟩` to the “left” of the `⟨label⟩` separated from it by the `⟨offset⟩` set by the the *second optional argument*. The *starred argument* “*” cannot be separated by spaces ‘ ’ from the command, i.e. `\item*` and the *first optional argument* does NOT support *verbatim content*. It can be configured with the keys `item-sym*` and `item-pos*` locally in the environment or globally using `\setenumext` command (§3).

• The behavior of `\item*` in the `enumext` and `enumext*` environments is NOT the same as in the `keyans` and `keyans*` environments.

2.3.1 Keys for `\item*`

`item-sym*` = { $\langle symbol \rangle$ } default: `\textborn`
Sets the *symbol* to be displayed to the “left” of the box containing the current $\langle label \rangle$ set by `labelwidth` key for `\item*` in `enumext` and `enumext*`. The *symbol* can be in *text* or *math* mode, for example `item-sym*={\star}`.

`item-pos*` = { $\langle rigid length \rangle$ } default: *by levels*
Sets the *offset* between the box containing the current $\langle label \rangle$ defined by `labelwidth` key and the $\langle symbol \rangle$ set by `item-sym*` key. The default values are set by `labelsep` key at each level. If positive values are passed, it will *offset to the left*; if negative values are passed it will *offset to the right*.

2.4 The command `\item` in `enumext*`

The `\item` command for the `enumext*` environment provides a “first optional argument” `\item( $\langle columns \rangle$ )` which “joins items” between columns. Let’s consider the following examples adapted directly from the `tasks` package:

```
\begin{enumext*}[widest=10,columns=4]
  \item The first
  \item* The second
  \item The third
  \item The fourth
  \item(3)* The fifth item is way too long for this and needs three columns
  \item The sixth
  \item The seventh
  \item(2)[X] The eighth item is way too long for this and needs two columns
    (\the\itemwidth)
  \item The ninth
  \item[Z] The tenth (\the\itemwidth)
\end{enumext*}
```

1. The first
- ★ 2. The second
3. The third
4. The fourth
- ★ 5. The fifth item is way too long for this and needs three columns
6. The sixth
7. The seventh
- X 8. The eighth item is way too long for this and needs two columns (196.17749pt)
9. The ninth
- Z 10. The tenth (89.28171pt)

3 The command `\setenumext`

<code>\setenumext</code>	<code>\setenumext{$\langle key = val \rangle$}</code>	<code>\setenumext[$\langle keyans* \rangle$]{$\langle key = val \rangle$}</code>
	<code>\setenumext[$\langle enumext, level \rangle$]{$\langle key = val \rangle$}</code>	<code>\setenumext[$\langle print, level \rangle$]{$\langle key = val \rangle$}</code>
	<code>\setenumext[$\langle enumext* \rangle$]{$\langle key = val \rangle$}</code>	<code>\setenumext[$\langle print, * \rangle$]{$\langle key = val \rangle$}</code>
	<code>\setenumext[$\langle keyans \rangle$]{$\langle key = val \rangle$}</code>	<code>\setenumext[$\langle print* \rangle$]{$\langle key = val \rangle$}</code>

The command `\setenumext` sets the $\langle keys \rangle$ on a global basis for environments `enumext`, `enumext*`, `keyans`, `keyans*` and the `\printkeyans` command. It can be used both in the preamble and in the body of the document as many times as desired.

The $\langle keys \rangle$ set in the *optional argument* of environments and commands have the *highest precedence*, overriding both options passed by `\setenumext`. If the *optional argument* is not passed, the *first level* of the environment `enumext` will be taken by default.

• For security reasons the keys `resume` “with value”, `resume*`, `reset`, `reset*`, `series` and `save-ans` they can NOT be set by this command and are ignored. The key `save-ans` that activate the “storage system” must be passed directly in the *optional argument* of the “first level” of the environment in which they are executed.

4 The command `\setenumextmeta`

<code>\setenumextmeta</code>	<code>\setenumextmeta[$\langle 1 \rangle$]{$\langle key name \rangle$}{$\langle key-one = val, key-two = val, ... \rangle$}</code>
	<code>\setenumextmeta[$\langle 2 \rangle$]{$\langle key name \rangle$}{$\langle key-one = val, key-two = val, ... \rangle$}</code>
	<code>\setenumextmeta[$\langle 3 \rangle$]{$\langle key name \rangle$}{$\langle key-one = val, key-two = val, ... \rangle$}</code>
	<code>\setenumextmeta[$\langle 4 \rangle$]{$\langle key name \rangle$}{$\langle key-one = val, key-two = val, ... \rangle$}</code>
	<code>\setenumextmeta[$\langle * \rangle$]{$\langle key name \rangle$}{$\langle key-one = val, key-two = val, ... \rangle$}</code>
	<code>\setenumextmeta*{$\langle key name \rangle$}{$\langle key-one = val, key-two = val, ... \rangle$}</code>

The command `\setenumextmeta` adds a new “meta-key” for the environments `enumext` and `enumext*`, the $\{ \langle key name \rangle \}$ must be different from those defined by the package. The *optional argument* of the form `[1]`, `[2]`, `[3]`, `[4]` adds a new “meta-key” for levels 1, 2, 3 and 4 of the `enumext` environment, the form `[*]` adds a new “meta-key” for the `enumext*` environment. If it is run *without* the *optional argument*, it will add a new “meta-key” for the “first level” of the `enumext` environment.

The *starred argument* ‘*’ will create the new “meta-key” for the environment `enumext*` and for “all levels” of the environment `enumext`. For example: `\setenumextmeta*{midsep}{topsep=3pt, partopsep=0pt}` will create a new key `midsep` available for all levels of the `enumext` environment and the `enumext*` environment and we can use it like any other key so `\begin{enumext}[midsep]` and `\begin{enumext*}[midsep]` will be valid.

5 The keyval system

The `<key = val>` system used by the `enumext` package is implemented using `l3keys` so it must be noted that `<keys>` marked as “value forbidden”, that is `<key>` is different from `<key=>`.

All `<keys>` described in this section are available for the `enumext`, `enumext*`, `keyans` and `keyans*` environments with the exception of the keys `series`, `resume`, `resume*`, `reset` and `reset*` which are only available for the `enumext` and `enumext*` environments; and the keys `mini-right`, `mini-right*` which are only available for the `enumext*` and `keyans*` environments.

All `<keys>` related to vertical or horizontal spacing accept a “skip” or “dim” expression if passed between braces, i.e. you do not need to use `\dimeval` or `\dimexpr` to perform calculations.

- It should be kept in mind that using any `<key>` that sets *rubber lengths* or *rigid lengths* for vertical or horizontal space on a level will influence the vertical and horizontal space for *inners levels* and `keyans`, `keyans*` and `keyanspic` environments.

5.1 Keys for `label` and `ref`

`mode-box` `<value forbidden>` default: *not used*

This is a “switch-key” that does not receive an argument and is “only” available for the “first level” of the `enumext` environment and the `enumext*` environment. When this is set the `label`, `font`, `wrap-label` and `wrap-label*` keys are executed within `\makebox` for the `enumext` and `keyans` environments.

- This key is intended for compatibility with tagged PDF and is forcibly “enabled” when `\DocumentMetadata` is present. If you want to get the same document output whether `\DocumentMetadata` is active or not, you must enable this key.
- In the `enumext*` and `keyans*` environments `\makelabel` is redefined using `\makebox` by default. If `enumext` or `keyans` is used in the `enumext*` environment the key must be activated manually.

`label = {(\alph* | \Alph* | \arabic* | \roman* | \Roman*)}` default: *by levels*

Sets the `<label>` that will be printed at the *current level* and default value for `labelwidth` key. The default value for the first level of the environments `enumext` and `enumext*` is `\arabic*`, for second level are `(\alph*)`, for third level are `\roman*` and for fourth level is `\Alph*`. For `keyans` and `keyans*` environments the default value is `\Alph*`.

- This key is intended to give the basic structure with which the `<label>` will be displayed, and the form in which it is used by standard “label and ref” and the “internal label and ref” system with the `save-ref` key. You cannot use commands with `<label>` as an argument, for example `\emph{(\alph*)}` will return an error. For full customization of how `<label>` is displayed use the `font`, `wrap-label` and/or `wrap-label*` keys.

`labelsep = {<rigid length>}` default: `0.3333em`

Sets the *horizontal space* between the box containing the current `<label>` defined by `label` key and the text of an item on the first line. Internally sets the value of `\labelsep` for the current level.

`labelwidth = {<rigid length>}` default: *by label*

Sets the *width* of the label box containing the current `<label>` set by the `label` key. Internally sets the value of `\labelwidth` for the current level. The default values are calculated by means of the *width* of a box by setting a *value* to the current counter set by `label` key using ‘0’ for `\arabic*`, ‘M’ for `\Alph*`, ‘m’ for `\alph*`, ‘VIII’ for `\Roman*` and ‘viii’ for `\roman*`.

`widest = {<integer | string>}` default: *empty*

Sets the `labelwidth` key passing the `<integer>` or converting the `<string>` of the form `\Alph`, `\alph`, `\Roman` or `\roman` to a *value* for the current counter defined by `label` key, then calculating the *width* by means of a box. For example `widest={XXIII}` or `widest={23}` are equivalent. This key is useful when the default values of the `labelwidth` key are smaller than those actually used.

`font = {<font commands>}` default: *empty*

Sets the *font style* for the current `<label>` defined by `label` key. You cannot use commands with an argument, for example `font={\small\textbf{#1}}` will return an error, but `font={\bfseries\small}` it will work. For full customization of how `<label>` is displayed use `wrap-label` and/or `wrap-label*` keys.

`align = {<left | right | center>}` default: *left*

Sets the *aligned* of `<label>` defined by `label` key on the current level in the label box of width set by `labelwidth`.

`wrap-label = {<code {#1} more code>}` default: *empty*

Wraps the *current* `<label>` defined by `label` key referenced by `{#1}` after executing the `align` and `font` keys. The `{<code>}` must be passed between braces and it does not modify the value set by the `labelwidth` key and is applied *only* on `\item` and `\item*`. When using it in the `\setenumext` command it is necessary to use the *double* ‘`##1`’. For example `wrap-label={\fbox{#1}}` or you can create a command:

```
\NewDocumentCommand \mywrap { s m }
{
  \IfBooleanTF{#1}
  {\textcolor{red}{\textbf{Q}}\textcolor{blue}{\textbf{.}}\textcolor{gray}{#2}}
  {\textcolor{blue}{\textbf{Q}}\textcolor{red}{\textbf{.}}\textcolor{gray}{#2}}
}
```

and then pass it through the key `wrap-label={\mywrap{#1}}` or `wrap-label={\mywrap*{#1}}`.

`wrap-label* = {⟨code {#1} more code⟩}` default: *empty*

The same as the `wrap-label` key but also applies on `\item[⟨custom⟩]`.

`ref = {⟨code {\alph*|\Alph*|\arabic*|\roman*|\Roman*} more code⟩}` default: *empty*

Modifies the way *cross references* are displayed. The `label` key sets the default form of the *cross references*, by using this key you can define a different format, for example: `ref=\emph{⟨\alph*⟩}` is valid.

Internally it renews the command associated with each counter when it is executed, i.e., in the environment `enumext` the command `\theenumXi` is modified when the key is executed at the first level, `\theenumXii` when it is executed at the second level and `\theenumXiii` together with `\theenumXiv` when it is executed at the third and fourth levels.

- This must be kept in mind, since the values set by the `label` and `ref` keys are not cumulative by levels, so if you have used the `ref` key in the first level and then want to associate the counter with `label` or `ref` in the second level you must use the direct commands, i.e. `\arabic{enumXi}` to indicate the count of the first level instead of using `\theenumXi`.

5.2 Keys for penalties

Page breaks in the provided environments are controlled by the following three parameters, which work together to ensure they look good, avoiding unsightly page breaks that could distort the output.

`beginpenalty = {⟨integer⟩}` default: *-51*

Set the *page breaking* penalty for breaking at the beginning of the `enumext`, `enumext*`, `keyans` and `keyans*` environments. Internally sets the value of `\@beginparpenalty`.

`midpenalty = {⟨integer⟩}` default: *-51*

Set the *page breaking* penalty for breaking between items of the `enumext`, `enumext*`, `keyans` and `keyans*` environments. Internally sets the value of `\@itempenalty`.

`endpenalty = {⟨integer⟩}` default: *-51*

Set the *page breaking* penalty for breaking at the end of the `enumext`, `enumext*`, `keyans` and `keyans*` environments. Internally sets the value of `\@endparpenalty`.

- The values passed to these *⟨keys⟩* affect the nested environments in which they were set and cannot be reset. $\text{\LaTeX}$ default is `-\@lowpenalty`, that is, *-51*. Because it is negative, it somewhat encourages a page break at each spot. Change it with, e.g., `\@beginparpenalty=9999`; a value of `10000` prohibits a page break. Please, refer to your $\text{\LaTeX}$ or $\text{\TeX}$ manual about how penalties control page breaks.

5.3 Keys for spaces

`show-length = {⟨true|false⟩}` default: *false*

Displays on the terminal the values for *all list parameters* at the current level. For *vertical spaces* show the values of `\topsep`, `\itemsep`, `\parsep` and `\partopsep`. For *horizontal spaces* show the values of `\labelwidth`, `\labelsep`, `\itemindent`, `\listparindent` and `\leftmargin`.

5.3.1 Vertical spaces

`topsep = {⟨rubber length|rigid length⟩}` default: *by levels*

Set the *vertical space* added to both the top and bottom of the list. Internally sets the value of `\topsep` for the current level. The default value for the first level of the environments `enumext` and `enumext*` are `8.0pt` plus `2.0pt` minus `4.0pt`, for second level are `4.0pt` plus `2.0pt` minus `1.0pt`, for third and fourth level are `2.0pt` plus `1.0pt` minus `1.0pt`. For `keyans` and `keyans*` environments the default value is `4.0pt` plus `2.0pt` minus `1.0pt`.

`parsep = {⟨rubber length|rigid length⟩}` default: *by levels*

Set the *vertical space* between paragraphs within an item. Internally sets the value of `\parsep` for the current level. The default value for the first level of the environments `enumext` and `enumext*` are `4.0pt` plus `2.0pt` minus `1.0pt`, for second level are `2.0pt` plus `1.0pt` minus `1.0pt`, for third and fourth level are `0pt`. For `keyans` and `keyans*` environments the default value is `2.0pt` plus `1.0pt` minus `1.0pt`.

- In the `enumext*` and `keyans*` environments this value is passed to `\parskip` within the `minipage` environment where “item content” is placed.

`partopsep = {⟨rubber length|rigid length⟩}` default: *by levels*

Set the *vertical space* added, beyond `topsep`, to the “top” and “bottom” of the entire environment if the environment instance is preceded by a “blank line” or `\par` command. Internally sets the value of `\partopsep` for the current level. The default values for first and second level in environment `enumext` are `2.0pt` plus `1.0pt` minus `1.0pt`, for third and fourth level are `1.0pt` minus `1.0pt`. For the `keyans` environment the

default value is 2.0pt plus 1.0pt minus 1.0pt , and for the `keyans*` and `enumext*` environments it is available but *without* effect.

- The value of this parameter also affects the *inner levels* and the environments `keyans`, `keyanspic` and `keyans*`. Caution should be taken with “blank lines” or `\par` command “before” each environment or nested level when formatting the source code of document. $\TeX$ will enter *vertical mode* and apply this value to the “top” and “bottom” the environment or nested level.

`itemsep` = { *rubber length* | *rigid length* } default: *by levels*

Set the *vertical space* between items, beyond the `parsep`. Internally sets the value of `\itemsep` for the current level. The default value for the first level of the environments `enumext` and `enumext*` are 4.0pt plus 2.0pt minus 1.0pt , for the rest of the levels are 2.0pt plus 1.0pt minus 1.0pt . For `keyans` and `keyans*` environments the default value is 4.0pt plus 2.0pt minus 1.0pt .

- In the `enumext*` and `keyans*` environments this value corresponds to the separation between rows.

`noitemsep` *value forbidden* default: *not used*

This is a “meta-key” that does not receive an argument. Set `itemsep` and `parsep` equal to 0pt the entire level of environment.

`nosep` *value forbidden* default: *not used*

This is a “meta-key” that does not receive an argument. Sets all keys for vertical spacing equal to 0pt the entire level of environment.

`base-fix` *value forbidden* default: *not used*

This is a “switch-key” that does not receive an argument available *only* for the “first level” of environment `enumext`. Fix the *baseline* when an environment `enumext` is nested in `enumext*` and there is no material between the `\item` and the start of the environment for example `\item \begin{enumext}` within the environment `enumext*`. Internally sets the keys `topsep`, `above` and `above*` at 0pt .

- This key is provided as a way to work around this minor issue, but you should be aware that if for some reason you have the `itemindent` key set in the `enumext*` environment it will be lost and you will need to adjust it using the `list-offset` key in the `enumext` environment.

Extra vertical spaces

- The following *keys* should be used with “caution”, they are intended to be used at the “top” and “bottom” of the environment when the `columns` or `mini-env` keys do not provide adequate *vertical spaces*. The values passed can be *rubber* or *rigid* lengths, the way they are applied is the way you differ, using the star ‘*’ *keys* applies `\vspace*` so that $\TeX$ does *not discard* this space at page break.

`above` = { *rubber length* | *rigid length* } default: *not used*

Set the *extra vertical space* added, beyond `topsep`, to the top of the entire level of environment. This key is intended to give a “fine adjustment” of the vertical space “above” the environment without hindering the value of the `topsep` key. The space is added with `\vspace` so is “discordable”.

`above*` = { *rubber length* | *rigid length* } default: *not used*

Set the *extra vertical space* added, beyond `topsep`, to the top of the entire level of environment. This key is intended to give a “fine adjustment” of the vertical space “above” the environment without hindering the value of the `topsep` key. The space is added with `\vspace*` so is “not discordable”.

`below` = { *rubber length* | *rigid length* } default: *not used*

Set the *extra vertical space* space added, beyond `topsep`, to the bottom of the entire level of environment. This key is intended to give a “fine adjustment” of the vertical space on the “below” the environment without hindering the value of the `topsep` key. The space is added with `\vspace` so is “discordable”.

`below*` = { *rubber length* | *rigid length* } default: *not used*

Set the *extra vertical space* space added, beyond `topsep`, to the bottom of the entire level of environment. This key is intended to give a “fine adjustment” of the vertical space on the “below” the environment without hindering the value of the `topsep` key. The space is added with `\vspace*` so is “not discordable”.

5.3.2 Horizontal spaces

`list-offset` = { *rigid length* } default: 0pt

Sets the *horizontal translation* of the entire environment level from the left edge of the box defined by the `labelwidth` key. Internally sets the values of `\leftmargin` and `\itemindent` for the current level.

`list-indent` = { *rigid length* } default: `labelwidth + labelsep`

Sets the *indentation* of the whole environment under the box defined by `labelwidth` and `labelsep` keys. Internally sets the value of `\leftmargin` and `\itemindent` for the current level. If `list-indent=0pt` is set in the environments `enumext` and `keyans` the *label* will be part of the text, separated by the value of the `labelsep` key and the *first word*, in simple terms it will look like a “common paragraph”.

- The `enumext*` and `keyans*` environments are implemented using `\makebox` and `minipage` which causes “list indent” to always be equal to the value passed to `labelwidth` plus `labelsep`. Passing a value to this key is equivalent to setting the value for the `list-offset` key.

`itemindent = {⟨rigid length⟩}` default: `0pt`

Sets the extra *horizontal indentation*, beyond `labelsep`, of the “*first line*” of each `\item` that is not followed by a “*blank line*” or the `\par` command. This value must be greater than or equal to `0pt` and is applied internally using `\hspace` without modifying the value of `\itemindent`.

- This key is intended for the `enumext*` and `keyans*` environments where, by their implementation, it is not possible to adjust `labelwidth` and `list-indent` without modifying the output. If you use `enumext` or `keyans` and want to get around the *blank line* limitation or the `\par` command followed by `\item` you can modify `labelwidth` and `list-indent` and get the same effect.

`rightmargin = {⟨rigid length⟩}` default: `0pt`

Set the *horizontal space* between the right margin of the environment and the right margin of the enclosing environment, the value it takes must be greater than or equal to `0pt`. Internally sets the value of `\rightmargin` for the current level.

`listparindent = {⟨rigid length⟩}` default: `0pt`

Sets the *horizontal space* indentation, beyond `list-indent`, for second and subsequent paragraphs within a list item. Internally sets the value of `\listparindent` for the current level.

- In the `enumext*` and `keyans*` environments this value is passed to `\parindent` within the `minipage` environment where “*item content*” is placed.

5.4 Keys for add code

The following *⟨keys⟩* should be used with “*caution*”, they are intended to inject `{⟨code⟩}` into different parts of the defined environments. We must keep in mind that the defined environments are based on the `list` base environment provided by $\text{\LaTeX}$ which is defined (simplified) as plain form `\list{⟨arg one⟩}{⟨arg two⟩}`. Using the `before*` key does not allow access to the `list` parameters defined by `[⟨key = val⟩]`.

`before = {⟨code⟩}` default: *not used*

Execute `{⟨code⟩}` “*before*” the environment starts. The `{⟨code⟩}` must be passed between braces, is executed “*after*” all calculations related to the *list parameters* in the environment and the *⟨keys⟩* sets by `[⟨key = val⟩]` have been performed, that is, in the *second argument* of the list: `\begin{list}{⟨arg one⟩}{⟨arg two⟩}{⟨code⟩}`.

`before* = {⟨code⟩}` default: *not used*

Execute `{⟨code⟩}` “*before*” the environment starts. The `{⟨code⟩}` must be passed between braces, is executed “*before*” performing all calculations related to the *list parameters* and the *⟨keys⟩* sets in `[⟨key = val⟩]` of the environment that is, “*before*” the arguments defining the list environment are executed: `{⟨code⟩}\begin{list}{⟨arg one⟩}{⟨arg two⟩}`.

`first = {⟨code⟩}` default: *not used*

Executes `{⟨code⟩}` when “*starting*” the environment. The `{⟨code⟩}` must be passed between braces, is executed right “*after*” all *list parameters* are done, after the second argument of list, just before the first occurrence of `\item`: `\begin{list}{⟨arg one⟩}{⟨arg two⟩}{⟨code⟩}\item`.

- Keep in mind that the `{⟨code⟩}` set in this *⟨key⟩* will affect the entire “*body*” of the environment and therefore the inner levels of the list and the `keyans`, `keyans*` and `keyanspic` environments. It is recommended to set this *⟨key⟩* per level. In the `enumext*` and `keyans*` environments this *⟨key⟩* is executed “*after*” the `listparindent`, `parsep` and `itemindent` *⟨keys⟩* within the `minipage` environment in which the “*item content*” is placed.

`after = {⟨code⟩}` default: *not used*

Execute `{⟨code⟩}` “*after*” finishing the environment. The `{⟨code⟩}` must be passed between braces.

5.5 Keys for start, series and resume

`start = {⟨integer | integer expression⟩}` default: `1`

Sets the *start value* of the numbering on the “*current level*”. The `{⟨integer expression⟩}` must be passed between braces, internally is evaluated and pass to the “*counter*” defined by `label` key on the current level, i.e. it is equivalent to enter `start={\dimeval{100*\value{chapter}}}` or `start={100*\value{chapter}}`.

`start* = {⟨integer | string⟩}` default: *not used*

Sets the *start value* of the numbering on the “*current level*”. Internally *⟨string⟩* is converted and passed as value to the “*counter*” defined by `label` key on the current level, i.e. it is equivalent to enter `start*=5`, `start*=E` or `start*=v`.

- The *start value* are set in the *second argument* of the `list` environment and “*before*” the execution of the keys `before` and `first`: `\begin{list}{⟨arg one⟩}{⟨arg two⟩}\setcounter{}`.

- The following *⟨keys⟩* are available only for the `enumext` and `enumext*` environments.

`series = {⟨series name⟩}` default: *not used*

Stores the *keys* of the *optional argument* of the “*current level*” of the environment in which it is executed in `{⟨series name⟩}` which is used as an *argument* in the `resume` key. The *⟨keys⟩* stored in `{⟨series name⟩}` are NOT cumulative and are overwritten if the same `{⟨series name⟩}` is used again at the “*same level*” at which the key was executed.

- For security reasons the `series` key will never save in `{⟨series name⟩}` the *⟨keys⟩* `series`, `resume`, `resume*`, `reset`, `reset*`, `save-ans`, `save-key`, `start*` and `start`.

`resume = {⟨series name⟩}` default: *not used*

Sets the *start value* and *options* for the “current level” continuing the numbering and *options* of the “same level” as the environment in which the `series={⟨series name⟩}` key was executed, the *start value* will continue numbering according to the last execution of `resume={⟨series name⟩}`. If passed “without value” this will only set *start value* continue the numbering of the “same level” from the last environment and level in which `series={⟨series name⟩}` or `resume={⟨series name⟩}` is NOT present and if the `save-ans` key is active (on the left) it will continue the numbering from the “last” environment in which it was executed. The *start value* can be overwritten using `start` or `start*` keys.

- The `resume` key passed “without value” must be exactly “without value”, i.e. `resume=` cannot be used and if executed before `resume*` it will affect the *start value*.

`resume*` `⟨value forbidden⟩` default: *not used*

Sets the *start value* and *options* for the “current level” continuing the numbering and options of the “same level” as the last environment and level in which the `series={⟨series name⟩}` or `resume={⟨series name⟩}` keys are NOT present and if the `save-ans` key is active (on the left) it will continue the numbering and options from the “last” environment in which it was executed. The *start value* can be overwritten using `start` or `start*` keys.

- When using the key `resume={⟨series name⟩}` or `resume*` you will have hierarchy in the `⟨keys⟩` that are stored in `{⟨series name⟩}` or in an internal version of `{⟨series name⟩}` in the case of `resume*`. If you want to *reset* the value of a `⟨key⟩` that is already stored in `{⟨series name⟩}` or in an internal version of `{⟨series name⟩}` this must be placed to the *right* of the key `resume={⟨series name⟩}` or `resume*`.
- When the `resume*` key is executed consecutively, it does not rewrite the `⟨keys⟩` stored in the internal version of `{⟨series name⟩}` and if the environment that precedes it does not have the *optional argument*, it will just continue with the numbering.

5.6 Keys for reset

`reset` `⟨value forbidden⟩` default: *not used*

Resets the *start value* of the “counters” in the `enumext` and `enumext*` environments along with the “internal counters” used by the `resume without value` and `resume*` keys at the “level” at which it is executed. The *start value* can be overwritten using the `start` or `start*` keys.

`reset*` `⟨value forbidden⟩` default: *not used*

Resets the *start value* of the “counters” in the `enumext` and `enumext*` environments along with the “internal counters” used by the `resume without value` and `resume*` keys at the “level” at which it is executed and in the “levels below” it in the case of the `enumext` environment. The *start value* can be overwritten using the `start` or `start*` keys.

- These keys are intended to be used in cases where the `\resetenumext` command does not work, e.g. after an unnumbered chapter. It should preferably be set *only* on the *first level*, although it is available for all levels.

5.6.1 The command `\resetenumext`

	<code>\resetenumext[⟨1⟩]{⟨some counter⟩}</code>	<code>\resetenumext[⟨4⟩]{⟨some counter⟩}</code>
	<code>\resetenumext[⟨2⟩]{⟨some counter⟩}</code>	<code>\resetenumext[⟨*⟩]{⟨some counter⟩}</code>
	<code>\resetenumext[⟨3⟩]{⟨some counter⟩}</code>	<code>\resetenumext*{⟨some counter⟩}</code>

The `\resetenumext` command “resets” the *start value* of the “counters” for the `enumext` and `enumext*` environments along with the “internal counters” used by the keys `resume without value` and `resume*` according to the value of `{⟨some counter⟩}`. For example `\resetenumext{chapter}` will “reset” the numbering of “all levels” of the `enumext` environment for each execution of a “numbered” chapter.

The *optional argument* of the form `[⟨1⟩]`, `[⟨2⟩]`, `[⟨3⟩]`, `[⟨4⟩]` “reset” the values for levels `1`, `2`, `3` and `4` of the `enumext` environment, the form `[⟨*⟩]` “reset” the values for the `enumext*` environment. If is run *without* the *optional argument*, it will “reset” the values for “all levels” of the `enumext` environment.

The *starred argument* ‘`*`’ will “reset” the values for “all levels” of the `enumext` and `enumext*` environments.

5.7 Keys for multicols

`columns = {⟨integer⟩}` default: `1`

Set the *number of columns* to be used by the `multicols` environment within the environments `enumext` and `keyans`. The value must be a positive integer less than or equal to `10`. In the `enumext*` and `keyans*` environments they correspond to the default number of columns (without joining) and internally adjust the value of `\itemwidth`.

`columns-sep = {⟨rigid length⟩}` default: *by level*

Set the *space between columns* used by the `multicols` environment within the environments `enumext` and `keyans`. Internally sets the value of `\columnsep`, by default its value is equal to the sum of the values set in the keys `labelwidth` and `labelsep` of the current level. In the `enumext*` and `keyans*` environments they correspond to the *space between columns* (without joining) and internally adjust the value of `\itemwidth`.

5.8 Keys for minipage

`mini-env = {⟨rigid length⟩}`

default: *not used*

Sets the *width* of the `minipage` environment on the “right side”. This value added to the value set by the `mini-sep` key to determines the *width* of the `minipage` environment on the “left side”, taking `\linewidth` as the maximum reference value.

`mini-sep = {⟨rigid length⟩}`

default: `0.3333em`

Sets the *space between* the `minipage` environment on the “left side” and the `minipage` environment on the “right side”. This separation is applied together with `\hfill`.

5.8.1 The command `\miniright`

```
\miniright \begin{enumext}[mini-env={⟨rigid length⟩}] ⟨item's before⟩ \item \miniright ⟨content⟩ \end{enumext}
\begin{enumext}[mini-env={⟨rigid length⟩}] ⟨item's before⟩ \item \miniright*⟨content⟩ \end{enumext}
```

The `\miniright` command close the `minipage` environment on the “left side” and opens the `minipage` environment on the “right side” by starting it with the `\centering` command. It must be placed “after” the last `\item` of the current environment and “before” starting the material to be placed on the “right side”.

The *starred argument* “*” inhibits the use of `\centering` command i.e. the usual $\TeX$ justification is maintained in the `minipage` on the “right side”.

5.8.2 The key `mini-right`

In the *horizontal list environments* `enumext*` and `keyans*` it is not possible to use the `\miniright` command and the `mini-right` key must be used instead.

`mini-right = {⟨content⟩}`

default: *not used*

Set the *content* for the drawing or tabular to be placed in the `minipage` environment on the “right side” by starting it with `\centering`. The `{⟨content⟩}` must be passed between braces.

`mini-right* = {⟨content⟩}`

default: *not used*

Same as above, but *without* starting with `\centering`.

6 The storage system

The entire mechanism for “*storing content*” it is activated according to `save-ans` key on the “*first level*” of `enumext` or `enumext*` environments and it is ignored if they are established when they are nested inside each other. Only when this `⟨key⟩` is “*active*” the `\anskey` command and the environments `anskey*`, `keyans`, `keyans*` and `keyanspic` are available.

<pre>\begin{enumext}[save-ans={⟨store name⟩}] \item Text \anskey{answer} \item Text \begin{keyans} ... \end{keyans} \end{enumext}</pre>	<pre>\begin{enumext}[save-ans={⟨store name⟩}] \item Text \anskey{answer} \item Text \begin{keyanspic} ... \end{keyanspic} \end{enumext}</pre>
---	---

By executing the key `save-ans={⟨store name⟩}` the entire “*structure*” of the environment (excluding the *first level*) including the *optional argument* passed to the inner levels or the environment nested in it, along with the `⟨content⟩` passed to `\anskey` or `anskey*`, the current `⟨labels⟩` for `\item*` and `\anspic*` in the environments `keyans`, `keyans*` and `keyanspic` will be “*stored*” in a *sequence* `{⟨store name⟩}` and at the same time will be “*stored*” (without the “*structure*” or *optional argument*) in a *prop list* `{⟨store name⟩}`.

For security reasons the *optional argument* of the inner levels or the nested environment are *filtered* by excluding all `⟨keys⟩` related to the “*storage system*” (§6.1) along with the keys `mini-env`, `mini-sep`, `mini-right`, `mini-right*`, `series`, `resume` and `resume*` when storing in *sequence* `{⟨store name⟩}` set by `save-ans` key.

6.1 Keys for storage system

The only `⟨keys⟩` available for all levels of the `enumext` environment and the `enumext*` environment are `no-store` and `save-key`, the rest of the `⟨keys⟩` described in this section must be passed directly in the *optional argument* of the “*first level*” of the environment in which the key `save-ans` is executed. The key `save-ans` should NOT be passed with the command `\setenumext`.

`save-ans = {⟨store name⟩}`

default: *not set*

Sets the *name* of the *sequence* and *prop list* in which the `{⟨contents⟩}` will be “*stored*” by `\anskey` and `anskey*` in `enumext` and `enumext*` environments and the current `⟨labels⟩` for `\item*` and `\anspic*` in the environments `keyans`, `keyans*` and `keyanspic`. If the *sequence* or *prop list* `{⟨store name⟩}` does not exist, it will be created globally and will not be *overwritten* if the key is used again.

`save-key = {⟨key list⟩}`

default: *not set*

This key *overrides* the default “*stored keys*” of the *optional argument* of the inner levels or nested environment that will be passed to the *sequence* `{⟨store name⟩}` set by `save-ans` key. The `⟨key list⟩` passed to this key ignores any `⟨keys⟩` in the “*stored structure*” and must be passed between braces. For example, if we execute at a second level:


```

\begin{enumext}[save-ans={\<store name>}]
  \item Text \anskey{answer}
  \item Text
    \begin{enumext}[nosep, columns=2, save-key={columns=3}]
      ...
    \end{enumext}
\end{enumext}

```

The “stored keys” by default in the sequence $\{\langle store name \rangle\}$ would be `nosep` and `columns=2`, but using the key `save-key={columns=3}` will overwrite and the “stored key” in the sequence $\{\langle store name \rangle\}$ are only `columns=3` ignoring all the others.

`save-sep = {\<text symbol>}` default: {,}

Sets the *text symbol* that will separate the current $\langle label \rangle$ to the *optional argument* passed to the `\item*` and `\anspic*` in the environments `keyans`, `keyans*` and `keyanspic` and storing them in the *sequence* and *prop list* $\{\langle store name \rangle\}$ set by `save-ans` key. The $\{\langle text symbol \rangle\}$ must always be passed between braces, whitespace ‘`␣`’ is preserved within the braces and only affects the “stored content” and not what is displayed when using the `show-ans` or `show-pos` keys.

`no-store <value forbidden>` default: not used

This is a “switch-key” that does not receive an argument and disables the “storing content” in the *sequence* and *prop list* $\{\langle store name \rangle\}$ set by `save-ans` key at the entire level or a nested environment in which it runs. This key is intended for use in internal levels or nested `enumext` or `enumext*` environments in which you want to use `enumext` or `enumext*` but “without” using the `\anskey` command or use `anskey*` environment and “without” interfering with the `check-ans` key.

6.1.1 Keys for label and ref

`save-ref = {\<true | false>}` default: false

Activates the “internal label and ref” mechanism for referencing “stored content” in *prop list* $\{\langle store name \rangle\}$ set by `save-ans` key. To reference the location of the “stored content” within the environment you must use `\ref{\<store name : position>}`, where $\langle position \rangle$ corresponds to the position occupied by the “stored content” in the *prop list* $\{\langle store name \rangle\}$ returned by the `show-pos` key. For example `\ref{test:4}` will return `3`. (b) which corresponds to the location of the “stored content” at position `4` in *prop list* `test` within the environment in which the key `save-ans=test` was set.

`mark-ref = {\<symbol>}` default: \textreferencemark

Sets the *symbol* that will be displayed by the `\printkeyans` command only if the `hyperref` package is detected and the `save-ref` key are active. This “symbol” is used as a “link” between the environment in which the `save-ans` key was used and the place where the command is executed.

6.1.2 Keys for wrap and marks

The `enumext` package provides a set of $\langle keys \rangle$ to set and manipulate “symbol marks” associated with “answers” and how they are displayed and stored in the *sequence* and *prop list*.

The $\langle keys \rangle$ available for the `\anskey` command and the `anskey*` environment can be passed “only” in the *optional argument* in the “first level” of the `enumext` or `enumext*` environment.

The $\langle keys \rangle$ available for the `keyans` and `keyans*` environments can be passed locally in the *optional argument*, at the “first level” of the `enumext` or `enumext*` environment or via the `\setenumext` command with one minor difference, when $\langle keys \rangle$ are passed through the “first level” of the `enumext` or `enumext*` environment they are set in “both” environments, but when they are passed using the `\setenumext` command they are set “individually” in each environment.

`show-ans = {\<true | false>}` default: false

Display the *symbol* set by the `mark-ans` key to the left of the *mandatory argument* $\langle content \rangle$ passed to the `\anskey` command and $\langle body \rangle$ for the `anskey*` environment using the `wrap-ans` key if set. For `\item*` and `\anspic*` the `keyans`, `keyans*` and `keyanspic` environments it will display the *symbol* set by the `mark-ans*` key to the left of the current $\langle label \rangle$ and *optional argument*. If the *optional argument* is present in `\item*` or `\anspic*` it will be shown using `wrap-opt` key.

Keys for \anskey and anskey*

`mark-ans = {\<symbol>}` default: \textasteriskcentered

Sets the *symbol* to be displayed in the left margin for `\anskey` command and `anskey*` environment when using the key `show-ans`. The “symbol” is placed in a box of width equal to the value of `labelwidth` at the current level, separated by the value of the key `mark-sep` and aligned by the value of the key `mark-pos`. This key is not affected by the keys `font` or `wrap-label` so if you want to apply *styling* you have to do it directly, for example: `mark-ans={\textcolor{red}{\textbf{\textasteriskcentered}}}`.

`mark-pos = {\<left | right | center>}` default: left

Sets the *aligned* of the “symbol” defined by `mark-ans` key for `\anskey` command and `anskey*` environment. The “symbol” is aligned in a box with the same dimensions of the label box defined by `labelwidth` key on the current level and separated by the value of the `mark-sep` key.

`mark-sep = {⟨rigid length⟩}` default: `labelsep`
 Sets the *horizontal space* between the box containing the “symbol” defined by `mark-ans` key and the *mandatory argument* ⟨*content*⟩ passed to the `\anskey` command and the ⟨*body*⟩ in `anskey*` environment.

`wrap-ans = {⟨code {#1} more code⟩}` default: `\fbox+\parbox{#1}`
 Wraps the *mandatory argument* ⟨*content*⟩ passed to the `\anskey` and the ⟨*body*⟩ in `anskey*` environment referenced by {#1} when using the `show-ans` or `show-pos` keys. The {⟨*code*⟩} must be passed between braces and only affects how the ⟨*content*⟩ or ⟨*body*⟩ is displayed and NOT the “stored content” in the *sequence* and *prop list* {⟨*store name*⟩} set by `save-ans` key. If this key is passed using `\setenumext` it is necessary to use double ‘{##1}’.

Keys for `keyans`, `keyans*` and `keyanspic`

`mark-ans* = {⟨symbol⟩}` default: `\textasteriskcentered`
 Sets the *symbol* to be displayed in the left margin for `\item*` and `\anspic*` for the `keyans`, `keyans*` and `keyanspic` environments when using the key `show-ans`. The “symbol” is placed in a box of width equal to the value of `labelwidth` of the environment in which it is executed, separated by the value of the key `mark-sep*` and aligned by the value of the key `mark-pos*`. This key is not affected by the keys `font` or `wrap-label` so if you want to apply *styling* you have to do it directly, for example:
`mark-ans*={\textcolor{red}{\textbf{\textasteriskcentered}}}`.

`mark-pos* = {⟨left | right | center⟩}` default: `left`
 Sets the *aligned* of the “symbol” defined by `mark-ans*` key for the `keyans`, `keyans*` and `keyanspic` environments. The “symbol” is aligned in a box with the same dimensions of the label box defined by `labelwidth` key of the environment in which it is executed and separated by the value of the `mark-sep*` key.

`mark-sep* = {⟨rigid length⟩}` default: `labelsep`
 Sets the *horizontal space* between the box containing the “symbol” defined by `mark-ans*` key and the current ⟨*label*⟩ for `\item*` and `\anspic*` in the `keyans`, `keyans*` and `keyanspic` environments.

`wrap-ans* = {⟨code {#1} more code⟩}` default: `not used`
 Wraps the *current* ⟨*label*⟩ when using the `show-ans` key for `\item*` and `\anspic*` referenced by {#1} in the `keyans`, `keyans*` and `keyanspic` environments after executing the `align` and `font` keys. The {⟨*code*⟩} must be passed between braces and *only* affects how the ⟨*label*⟩ is displayed and NOT the “stored label” in the *sequence* and *prop list* {⟨*store name*⟩} set by `save-ans` key. This key overwrites the key `wrap-label` and if is passed using `\setenumext` it is necessary to use double ‘{##1}’. For example, if you want the ⟨*label*⟩ to be displayed in red when using `show-ans` you just set `wrap-ans*={\textcolor{red}{#1}}`.

`wrap-opt = {⟨code {#1} more code⟩}` default: `[{#1}]`
 Wraps the *optional argument* passed to the `\item*` and `\anspic*` referenced by {#1} in the `keyans`, `keyans*` and `keyanspic` environments when using the `show-ans` or `show-pos` keys. The {⟨*code*⟩} must be passed between braces and only affects the current *optional argument* and NOT the “stored content” in the *sequence* and *prop list* {⟨*store name*⟩} set by `save-ans` key. If this key is passed using `\setenumext` it is necessary to use double ‘{##1}’.

6.1.3 Keys for debug and checking

`show-pos = {⟨true | false⟩}` default: `false`
 Displays the *position* occupied by the “stored content” by `\anskey`, `anskey*`, `\item*` and `\anspic*` in the *prop list* {⟨*store name*⟩} set by `save-ans` key. This position is used by the `\getkeyans` command and by the `\ref` command if the `save-ref` key is active.

`check-ans = {⟨true | false⟩}` default: `false`
 Enables the *checking answer* mechanism displaying an appropriate message on the terminal. This key works under the logic that each `\item` or `\item*` that does not open an inner level or nested environment contains “only one answer” or “only one execution” of the `\anskey` or `anskey*`. It is intended to be used in conjunction with the `no-store` key.

6.2 The command `\anskey`

`\anskey` `\anskey[⟨keys⟩]{⟨content⟩}`

The command `\anskey` takes a mandatory non empty argument {⟨*content*⟩} and “stores” it in the *sequence* and *prop list* {⟨*store name*⟩} set by `save-ans` key. By design the command cannot be nested or passed *verbatim material* in the argument and it is assumed that each *numbered* `\item` or `\item*` within the environment in which it is active it has a “single execution” of `\anskey` unless `\item` or `\item*` open a nested level or use the `no-store` key.

If `save-ref` key are active and the `hyperref`[8] package is detected, `\hyperlink` and `\hypertarget` will be used, otherwise the usual “label and ref” system provided by L^AT_EX will be used.

The `\anskey` command is available for all levels of the `enumext` environment and the `enumext*` environment, but is disabled for the `keyans`, `keyans*` and `keyanspic` environments.

6.2.1 Keys for `\anskey`

By default the *mandatory argument* $\langle content \rangle$ passed to `\anskey` when “storing” in the *sequence* $\{\langle store name \rangle\}$ has the form `\item  $\langle content \rangle$` , the following *keys* allow modifying the way in which it is “stored” in the *sequence*.

`break-col` $\langle value forbidden \rangle$ default: *not used*

Stores $\{\langle content \rangle\}$ in the *sequence* $\{\langle store name \rangle\}$ of the form `\columnbreak \item  $\langle content \rangle$` .

`item-join` = $\{\langle columns \rangle\}$ default: *not used*

Set the *number of columns* to be used for `\item( $\langle columns \rangle$ )` and stores $\{\langle content \rangle\}$ in the *sequence* $\{\langle store name \rangle\}$ of the form `\item( $\langle columns \rangle$ )  $\langle content \rangle$` .

`item-star` $\langle value forbidden \rangle$ default: *not used*

Stores $\{\langle content \rangle\}$ in the *sequence* $\{\langle store name \rangle\}$ of the form `\item*  $\langle content \rangle$` .

`item-sym*` = $\{\langle symbol \rangle\}$ default: *not set*

Sets the *symbol* for `\item*` when using the key `item-star` and stores $\{\langle content \rangle\}$ in the *sequence* $\{\langle store name \rangle\}$ of the form `\item*[ $\langle symbol \rangle$ ]  $\langle content \rangle$` . The *symbol* can be in text or math mode, for example `item-sym*={ $\$ \ast \$$ }` stores `\item*[ $\$ \ast \$$ ]  $\langle content \rangle$` .

`item-pos*` = $\{\langle rigid length \rangle\}$ default: *not set*

Sets the *offset* for `\item*` when using the keys `item-star` and `item-sym*` and stores $\{\langle content \rangle\}$ in the *sequence* $\{\langle store name \rangle\}$ of the form `\item*[ $\langle symbol \rangle$ ][ $\langle offset \rangle$ ]  $\langle content \rangle$` .

Example

```
\begin{enumext}[save-ans=test,show-ans=true]
  \item* Text containing our instructions or questions. \anskey{\first answer}
  \item Text containing our instructions or questions.
    \begin{enumext}
      \item Question.\anskey{\second answer}
    \end{enumext}
  \item Text containing our instructions or questions. \anskey{\third answer}
  \item Text containing our instructions or questions. \anskey{\fourth answer}
\end{enumext}
```

- | | |
|---|--|
| <p>* 1. Text containing our instructions or questions.</p> <p>* first answer</p> <p>2. Text containing our instructions or questions.</p> <p> (a) Question.</p> <p> * second answer</p> | <p>3. Text containing our instructions or questions.</p> <p>* third answer</p> <p>4. Text containing our instructions or questions.</p> <p>* fourth answer</p> |
|---|--|

6.3 The environment `anskey*`

`anskey*` `\begin{anskey*}[(key = val)]  $\langle body content \rangle$  \end{anskey*}`

The environment `anskey*` takes a mandatory $\{\langle body content \rangle\}$ and “stores it” in the *sequence* and *prop list* $\{\langle store name \rangle\}$ set by `save-ans` key. If `save-ref` key are active and the `hyperref`[8] package is detected `\hyperlink` and `\hypertarget` will be used, otherwise the usual “label and ref” system provided by $\LaTeX$ will be used.

By design the environment cannot be nested but full supports “verbatim material” in the $\langle body \rangle$ and it is assumed that “each numbered” `\item` or `\item*` within the environment in which it is active it has a “single execution” unless `\item` or `\item*` open a nested level or use the `no-store` key.

The `anskey*` environment is implemented using the new “collect code” c-type argument part of $\LaTeX$ release 2025-06-01[13]. `\begin{anskey*}` and `\end{anskey*}` must be in different lines and should not appear within verbatim environments or commands. All *keys* must be passed separated by commas and “without separation” of the start of the environment.

Comments ‘%’ or *any character* after `\begin{anskey*}` or $[(key = val)]$ on the same line are NOT supported, $\LaTeX$ will return an “error” message if this happens. In a similar way comments ‘%’ or *any character* after `\end{anskey*}` on the same line $\LaTeX$ will return a “warning” message.

6.3.1 Keys for `anskey*`

The `anskey*` environment uses the same *keys* as the `\anskey` command next to the *keys* `write-env`, `overwrite` and `force-eol`. The environment is available for all levels of the `enumext` environment and the `enumext*` environment, but it is disabled for the `keyans`, `keyans*` and `keyanspic` environments.

`write-env` = $\{\langle file.ext \rangle\}$ default: *not used*

Sets the *name* of the $\langle external file \rangle$ in which the $\langle contents \rangle$ of the environment will be written. The $\langle file.ext \rangle$ will be created in the working directory, relative or absolute paths are not supported. If $\langle file.ext \rangle$ does not exist, it will be created or overwritten if the `overwrite` key is used.

`overwrite = {\langle true | false \rangle}`

default: *false*

Sets whether the *\file.ext* generated by *write-env* from the *anskey** environment will be rewritten.

`force-eol = {\langle true | false \rangle}`

default: *false*

Sets if the *last end of line* for the *\stored content* is hidden or not. This key is necessary only if the last line is the closing of some environment defined by the *fancyvrb* package as *\end{Verbatim}* or another environment that does not support a comments ‘%’ after closing *\end{Verbatim}*%.

Example

```
\begin{enumext}[save-ans=test,show-pos=true,start=5]
  \item* Text containing our instructions or questions.
  \begin{anskey*}[item-star]
    \langle first answer \rangle
  \end{anskey*}

  \item Text containing our instructions or questions.
  \begin{enumext}
    \item Question.
    \begin{anskey*}
      \langle second answer \rangle
    \end{anskey*}
  \end{enumext}

  \item Text containing our instructions or questions.
  \begin{anskey*}
    \langle third answer \rangle
  \end{anskey*}

  \item Text containing our instructions or questions.
  \begin{anskey*}
    \langle fourth answer \rangle
  \end{anskey*}
\end{enumext}
```

★ 5. Text containing our instructions or questions.

7. Text containing our instructions or questions.

[5] First answer with verbatim

[7] third answer

6. Text containing our instructions or questions.

8. Text containing our instructions or questions.

(a) Question.

[8] fourth answer

[6] second answer

6.4 The environments *keyans* and *keyans**

keyans `\begin{keyans}[\langle key = val \rangle] \item \item[\langle custom \rangle] \item* \item*[\langle content \rangle] \end{keyans}`

*keyans** `\begin{keyans*}[\langle key = val \rangle] \item \item[\langle custom \rangle] \item* \item*[\langle content \rangle] \end{keyans*}`

The *keyans* and *keyans** environments are “*enumerated list*” environments designed for “*multiple choice*” questions activated by the *save-ans* key.

This environments can NOT be nested and must always be at the “*first level*” of the *enumext* environment, the commands *\item* and *\item[\langle custom \rangle]* work in the usual and the command *\item(\langle columns \rangle)* is available for the *keyans** environment.

- The behavior of *\item** in *keyans* and *keyans** environments is NOT the same as in the *enumext* or *enumext** environments.

```
\begin{enumext}[save-ans=test]
  \item \langle item content \rangle
  \begin{keyans}[\langle key = val \rangle]
    \item \langle item content \rangle
    \item [\langle custom \rangle] \langle item content \rangle
    \item* \langle item content \rangle
    \item*[\langle content \rangle] \langle item content \rangle
  \end{keyans}
\end{enumext}
```

```
\begin{enumext}[save-ans=test]
  \item \langle item content \rangle
  \begin{keyans*}[\langle key = val \rangle]
    \item \langle item content \rangle
    \item [\langle custom \rangle] \langle item content \rangle
    \item* \langle item content \rangle
    \item*[\langle content \rangle] \langle item content \rangle
  \end{keyans*}
\end{enumext}
```

The *\langle keys \rangle* set in the *optional argument* of the environment are the same (almost) as those of the *enumext* and *enumext** environments and have *higher precedence* than those set by *\setenumext[\langle keyans \rangle]{\langle key = val \rangle}* or *\setenumext[\langle keyans* \rangle]{\langle key = val \rangle}*. If the *optional argument* is not passed or the *\langle keys \rangle* are not set by *\setenumext*, the default values will be the same as the “*second level*” of the *enumext* environment with the difference in the *\langle label \rangle* which will be set to *label=\Alph**.

The keys *mark-ans**, *mark-pos**, *mark-sep**, *save-sep*, *wrap-opt*, *wrap-ans**, *show-ans* and *show-pos* are available for both environments.

6.4.1 The \item* in keyans and keyans*

`\item*` `\item*`
`\item*[\langle content \rangle]`

The `\item*` and `\item*[\langle content \rangle]` command “store” the current `\label` set by `label` key next to the *optional argument* `\langle content \rangle` in *sequence* and *prop list* `\{ \langle store name \rangle \}` set by `save-ans` key in the “first level” of the `enumext` or `enumext*` environments.

The *starred argument* ‘*’ cannot be separated by spaces ‘ ’ from the command, i.e. `\item*` and the *optional argument* does “NOT” support *verbatim content*. By design it is assumed that the `\item*` will only appear “once” within the environment.

Example

```
\begin{enumext}[save-ans=test,columns=2,show-ans=true]
  \item Text containing a question.

  \begin{keyans*}[nosep,columns=2]
    \item Choice
    \item* Correct choice
    \item Choice
    \item Choice
    \item Choice
  \end{keyans*}

  \item Text containing a question and image.

  \begin{keyans}[nosep,mini-env={0.4\linewidth}]
    \item Choice
    \item Choice
    \item Choice
    \item Choice
    \item*[\note] Correct choice
    \miniright
    \includegraphics[scale=0.25]{example-image-a}
    Some text
  \end{keyans}
\end{enumext}
```

1. Text containing a question.
A) Choice * B) Correct choice
C) Choice D) Choice
E) Choice

2. Text containing a question and image.
A) Choice
B) Choice
C) Choice
D) Choice
* E) [note] Correct choice


Some text

6.5 The environment keyanspic

`keyanspic` `\begin{keyanspic}[\langle key = val \rangle] \anspic*[\langle content \rangle]{\langle drawing or tabular \rangle} \end{keyanspic}`

The `keyanspic` environment is an “enumerated list” environment activated by the `save-ans` key that has the same configuration for “spacing” and `\label` as the `keyans` environment that uses the `\anspic` command instead of `\item`. It is intended for placing *drawings or tabular* with `\label` centered *above* or *below* in a *single line* or *upper and lower* layout style.

When the `keyanspic` environment is used *without keys* the `\labels` are centered *below* the *drawings or tabular* in a *single line* layout style.

A representation of the output can be seen in the figure 6.

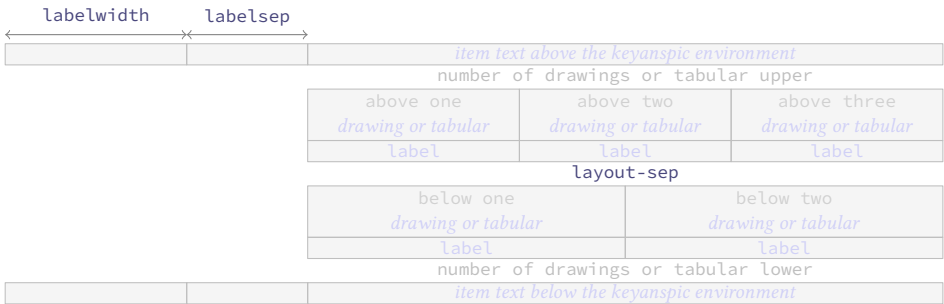


Figure 6: Representation of the `keyanspic` environment with `layout-sty={\langle 3, 2 \rangle}` in `enumext`.

This environment cannot be nested and must *always* be at the “first level” of the `enumext` environment, the `\item` command is disabled and `\keys` cannot be set using `\setenumext`.

6.5.1 Keys for keyanspic

`label-pos = {⟨above | below⟩}`

default: *below*

Set the *position* of ⟨*label*⟩ to be centered “above” or “below” *drawings* or *tabular* when the `\anspic` command is executed.

`label-sep = {⟨rubber length | rigid length⟩}`

default: *internal adjustment*

Set the *vertical spacing* between the ⟨*label*⟩ centered “above” or “below” and *drawings* or *tabular* when running the `\anspic` command.

`layout-sty = {⟨n° upper , n° lower⟩}`

default: *not set*

Set the *number of drawings* or *tabular* that will be distributed “upper” and “lower” within the environment when executing the `\anspic` command. The value must be passed in braces and if not set or the ⟨*n° lower*⟩ is omitted the *drawings* or *tabular* will be put on a *single line*.

`layout-sep = {⟨rubber length | rigid length⟩}`

default: *adjusted parsep from keyans*

Set the *vertical separation* between the number of *drawings* or *tabular* placed at the “upper” and “lower” within the environment when executing the `\anspic` command. Internally adjusts the `parsep` value taken from the `keyans` environment.

`layout-top = {⟨rubber length | rigid length⟩}`

default: *adjusted topsep from keyans*

Set the *vertical space* added to both the top and bottom of the environment. Internally adjust the value of `topsep` taken from `keyans` environment.

The keys `mark-ans*`, `mark-pos*`, `mark-sep*`, `save-sep`, `wrap-opt`, `wrap-ans*`, `show-ans` and `show-pos` are available for this environment.

6.5.2 The command `\anspic`

`\anspic` `\anspic{⟨drawing or tabular⟩}`
`\anspic*` `\anspic*[⟨content⟩]{⟨drawing or tabular⟩}`

The `\anspic` command take three arguments, the *starred argument* ‘*’ store the current ⟨*label*⟩ next to the *optional argument* ⟨*content*⟩ in *sequence* and *prop list* {⟨*store name*⟩} set by `save-ans` key.

The *starred argument* ‘*’ cannot be separated by spaces ‘`␣`’ from the command, i.e. `\anspic*` and the *optional argument* does “NOT” support *verbatim content*. By design it is assumed that the *starred argument* ‘*’ will only appear “once” within the environment.

Example

```
\begin{enumext}[save-ans=test,show-ans=true,nosep]
  \item Question with images and labels below.

  \begin{keyanspic}[layout-sty={3,2}]
    \anspic{\includegraphics[scale=0.15]{example-image-a}}
    \anspic{\includegraphics[scale=0.15]{example-image-b}}
    \anspic{\includegraphics[scale=0.15]{example-image-a}}
    \anspic{\includegraphics[scale=0.15]{example-image-a}}
    \anspic*[note]{\includegraphics[scale=0.15]{example-image-a}}
  \end{keyanspic}

  \item Question with images and labels above.

  \begin{keyanspic}[label-pos=above, layout-sty={3,2},layout-sep=0.25cm]
    \anspic{\includegraphics[scale=0.15]{example-image-a}}
    \anspic{\includegraphics[scale=0.15]{example-image-b}}
    \anspic{\includegraphics[scale=0.15]{example-image-a}}
    \anspic{\includegraphics[scale=0.15]{example-image-a}}
    \anspic*[note]{\includegraphics[scale=0.15]{example-image-a}}
  \end{keyanspic}

  \item Question with images and labels below on a single line.

  \begin{keyanspic}
    \anspic{\includegraphics[scale=0.15]{example-image-a}}
    \anspic{\includegraphics[scale=0.15]{example-image-b}}
    \anspic{\includegraphics[scale=0.15]{example-image-a}}
    \anspic{\includegraphics[scale=0.15]{example-image-a}}
    \anspic*[note]{\includegraphics[scale=0.15]{example-image-a}}
  \end{keyanspic}

\end{enumext}
```


1. Question with images and labels below.



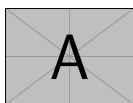
A)



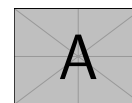
B)



C)

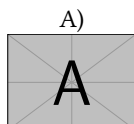


D)

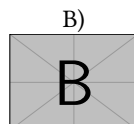


* E) [note]

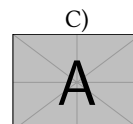
2. Question with images and labels above.



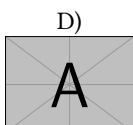
A)



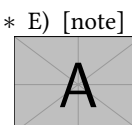
B)



C)



D)



* E) [note]

3. Question with images and labels below on a single line.



A)



B)



C)



D)



* E) [note]

Remember to pass the `alt={⟨description⟩}` key to the `\includegraphics` command when creating a *tagged* PDF.

6.6 Printing stored content

6.6.1 The command `\getkeyans`

`\getkeyans` `\getkeyans{⟨store name⟩ : ⟨position⟩}`

The command `\getkeyans` prints the “stored content” in *prop list* `{⟨store name⟩}` defined by `save-ans` key in the `⟨position⟩` returned by the `show-pos` key.

The “stored content” can only be accessed *after* it is stored, if `{⟨store name⟩}` does not exist the command will return an error.

The form taken by the argument `{⟨store name⟩ : ⟨position⟩}` is the same as that used to generate the “internal label and ref” system when `save-ref` key are active, so to refer to a “stored content”. For example `\getkeyans{test:4}` will return the “stored content” at position 4 of the environment in which the key `save-ans=test` was set.

6.6.2 The command `\foreachkeyans`

`\foreachkeyans` `\foreachkeyans[⟨key = val⟩]{⟨store name⟩}`

The command `\foreachkeyans` goes through and executes the command `\getkeyans` on the contents in *prop list* `{⟨store name⟩}`. If you pass without options run `\getkeyans` on all contents in *prop list* `{⟨store name⟩}`.

Options for command

`sep = {⟨code⟩}` default: `{;}`

Establishes the *separation* between “each” `{⟨content⟩}` stored in *prop list* `{⟨store name⟩}`. For example, you can use `sep={\\[10pt]}` for vertical separation of stored contents.

`step = {⟨integer⟩}` default: `1`

Sets the *step* (increment) applied to the value set by key `start` for “each” `{⟨content⟩}` stored in *prop list* `{⟨store name⟩}`. The value must be a *positive integer*.

`start = {⟨integer⟩}` default: `1`

Sets the *position* of the *prop list* `{⟨store name⟩}` from which execution will start. The value must be a *positive integer*.

`stop = {⟨integer⟩}` default: `0`

Sets the *position* of the *prop list* `{⟨store name⟩}` from which execution will finish. The value must be a *positive integer*.

`before = {⟨code⟩}` default: *empty*
 Sets the {⟨code⟩} that will be executed ⟨before⟩ each {⟨content⟩} stored in *prop list* {⟨store name⟩}. The {⟨code⟩} must be passed between braces.

`after = {⟨code⟩}` default: *empty*
 Sets the {⟨code⟩} that will be executed ⟨after⟩ each {⟨content⟩} stored in *prop list* {⟨store name⟩}. The {⟨code⟩} must be passed between braces.

`wrapper = {⟨code #1⟩ more code}` default: *empty*
 Wraps the {⟨content⟩} stored in *prop list* {⟨store name⟩} referenced by {#1}. The {⟨code⟩} must be passed between braces. For example `\foreachkeyans[wrapper={\makebox[1em][l]{#1}}]{⟨store name⟩}`.

6.6.3 The command `\printkeyans`

```
\printkeyans {⟨store name⟩}
\printkeyans [⟨keys⟩]{⟨store name⟩}
\printkeyans * [⟨keys⟩]{⟨store name⟩}
```

The command `\printkeyans` prints “all stored content” in *sequence* {⟨store name⟩} defined by `save-ans` key placing this inside the `enumext` or `enumext*` environment if the *starred argument* ‘*’ is used.

The “stored content” can only be accessed *after* it is stored in the *sequence*, if {⟨store name⟩} does not exist the command will return an error.

The *optional argument* allows managing the ⟨keys⟩ in the “first level” of the environment in which the “stored content” of the *sequence* {⟨store name⟩} will be printed, if the *starred argument* ‘*’ is used it will be `enumext*` otherwise `enumext`.

The default values for the “first level” are the same as the default values for the `enumext` and `enumext*` environments along with the keys `nosep`, `first=\small`, `font=\small` and `columns=2`. For the inner levels of the environment `enumext` saved in the *sequence* {⟨store name⟩} the default values are the same as those established for the second, third and fourth levels plus the keys `nosep`, `first=\small`, `font=\small`. If the environment `enumext*` is saved within the *sequence* {⟨store name⟩} it will have the same default values plus the keys `nosep`, `first=\small`, `font=\small`.

Since the command encapsulates by default the `enumext` environment or the `enumext*` environment, we must take some considerations:

- If we execute `\printkeyans*{⟨store name⟩}` and the *sequence* {⟨store name⟩} already contains any `enumext*` environment an error will be returned as we cannot nest.
- If we execute `\printkeyans*{⟨store name⟩}` and the *sequence* {⟨store name⟩} contains any `enumext` environments, they will start with the ⟨keys⟩ set for the first level unless they are set in the *optional argument* or `save-key` is used to modify it.
- If we execute `\printkeyans{⟨store name⟩}` and the *sequence* {⟨store name⟩} contains any environment `enumext*`, they will start with the ⟨keys⟩ set by default unless they are set in the *optional argument* or `save-key` is used to modify it.

The default values for the “first level” of `\printkeyans` commands and `\printkeyans*` are established using `\setenumext[⟨print , 1⟩]{⟨keys⟩}` and `\setenumext[⟨print*⟩]{⟨keys⟩}`.

If we need to set the ⟨keys⟩ for the environment `enumext` “saved” in the *sequence* {⟨store name⟩} we will use `\setenumext[⟨print , level⟩]{⟨keys⟩}` and if we need to set the ⟨keys⟩ for the environment `enumext*` “saved” in the *sequence* {⟨store name⟩} we will use `\setenumext[⟨print , *⟩]{⟨keys⟩}`.

Example

```
\begin{enumext}[save-ans=sample,columns=1,show-pos=true,nosep,save-ref=true]
  \item Factor $3x+3y+3z$. \anskey{$3(x+y+z)}$
  \item True False

  \begin{enumext}[nosep]
    \item \LaTeX2e\ is cool? \anskey{Very True!}
  \end{enumext}

  \item Related to Linux

  \begin{enumext}[nosep]
    \item You use Linux? \anskey{Yes}
    \item Rate the following package and class
      \begin{enumext}[nosep]
        \item \texttt{xsim} \anskey{very good}
        \item \texttt{exsheets} \anskey{obsolete}
      \end{enumext}
    \end{enumext}
\end{enumext}
```

```
The answer to \ref{sample:4} is \getkeyans{sample:4} and the answers to
all the worksheets are as follows:

\printkeyans{sample}
```

1. Factor $3x + 3y + 3z$.
- [1]
2. True False
- (a) ~~TeX~~ze is cool?
- [2]
3. Related to Linux
- (a) You use Linux?
- [3]
- (b) Rate the following package and class
- i. `xsim`
- [4]
- ii. `exsheets`
- [5]

The answer to 3.(b).i is very good and the answers to all the worksheets are as follows:

1. $3(x + y + z)$ ✖
2. (a) Very True! ✖
3. (a) Yes ✖
- (b) i. very good ✖
- ii. obsolete ✖

7 Full examples

Here I will leave as an example some adaptations questions taken from TeX-SX. The examples are attached to this documentation and can be extracted from your PDF viewer or from the command line by running:

```
$ pdfdetach -saveall enumext.pdf
```

and then you can use the excellent arara¹ tool to compile them.

Example 1

Adapted from the response given by Enrico Gregorio in Squares for answer choice options and perfect alignment to mathematical answers .

1. La velocità di $1,00 \times 10^2$ m/s espressa in km/h è:
- ☐ A 36 km/h.
- ☐ B 360 km/h.
- ☐ C 27,8 km/h.
- ☐ D $3,60 \times 10^8$ km/h.
3. La velocità di $1,00 \times 10^2$ m/s espressa in km/h è:
- ☐ A 36 km/h.
- ☐ B 360 km/h.
- ☐ C 27,8 km/h.
- ☐ D $3,60 \times 10^8$ km/h.
2. In fisica nucleare si usa l'angstrom (simbolo: $1 \text{ \AA} = 1 \times 10^{-10}$ m) e il fermi o femtometro ($1 \text{ fm} = 1 \times 10^{-15}$ m). Qual è la relazione tra queste due unità di misura?
- ☐ A $1 \text{ \AA} = 1 \times 10^5 \text{ fm}$.
- ☐ B $1 \text{ \AA} = 1 \times 10^{-5} \text{ fm}$.
- ☐ C $1 \text{ \AA} = 1 \times 10^{-15} \text{ fm}$.
- ☐ D $1 \text{ \AA} = 1 \times 10^3 \text{ fm}$.
4. In fisica nucleare si usa l'angstrom (simbolo: $1 \text{ \AA} = 1 \times 10^{-10}$ m) e il fermi o femtometro ($1 \text{ fm} = 1 \times 10^{-15}$ m). Qual è la relazione tra queste due unità di misura?
- ☐ A $1 \text{ \AA} = 1 \times 10^5 \text{ fm}$.
- ☐ B $1 \text{ \AA} = 1 \times 10^{-5} \text{ fm}$.
- ☐ C $1 \text{ \AA} = 1 \times 10^{-15} \text{ fm}$.
- ☐ D $1 \text{ \AA} = 1 \times 10^3 \text{ fm}$.

1. B
2. A
3. B
4. A

Example 2

Adapted from the response given by Florent Rougon in Multiple choice questions with proposed answers in random order — addition of automatic correction (cross mark) .

¹The cool TeX automation tool: <https://www.ctan.org/pkg/arara>

1. La velocità di $1,00 \times 10^2$ m/s espressa in km/h è:

A 36 km/h.

✓ B 360 km/h.

C 27,8 km/h.

D $3,60 \times 10^8$ km/h.
2. In fisica nucleare si usa l'angstrom (simbolo: $1 \text{ \AA} = 1 \times 10^{-10}$ m) e il fermi o femtometro ($1 \text{ fm} = 1 \times 10^{-15}$ m). Qual è la relazione tra queste due unità di misura?

✓ A $1 \text{ \AA} = 1 \times 10^5 \text{ fm}$.

B $1 \text{ \AA} = 1 \times 10^{-5} \text{ fm}$.

C $1 \text{ \AA} = 1 \times 10^{-15} \text{ fm}$.

D $1 \text{ \AA} = 1 \times 10^3 \text{ fm}$.
3. La velocità di $1,00 \times 10^2$ m/s espressa in km/h è:

A 36 km/h.

✓ B 360 km/h.

C 27,8 km/h.

D $3,60 \times 10^8$ km/h.
4. In fisica nucleare si usa l'angstrom (simbolo: $1 \text{ \AA} = 1 \times 10^{-10}$ m) e il fermi o femtometro ($1 \text{ fm} = 1 \times 10^{-15}$ m). Qual è la relazione tra queste due unità di misura?

✓ A $1 \text{ \AA} = 1 \times 10^5 \text{ fm}$.

B $1 \text{ \AA} = 1 \times 10^{-5} \text{ fm}$.

C $1 \text{ \AA} = 1 \times 10^{-15} \text{ fm}$.

D $1 \text{ \AA} = 1 \times 10^3 \text{ fm}$.
1. B

✖ 2. A

✖

3. B

✖ 4. A

✖
- Example 3
- A “simple multiple choice” test .
1. First type of questions

A value

C value

B correct

D value

2. Second type of questions

I. $2\alpha + 2\delta = 90^\circ$

II. $\alpha = \delta$

III. $\angle EDF = 45^\circ$

A I only

B II only

C I and II only

D I and III only

E I, II, and III

3. Third type of questions

(1) $2\alpha + 2\delta = 90^\circ$

(2) $\angle EDF = 45^\circ$

A value

B value

C value

D value

E value

4. Question with image and label below:

A

A

B

B

A

C

A

D



E

5. Question with image on right side:

A value

B value

C value

D correct

E value

B

Test keys

1. B, $x = 5$

✖ 4. E, A duck

✖

2. D

✖ 5. D, other note

✖

3. C, some note

✖

Example 4

A “simple worksheet” using ducks :) .

 Factor $x^2 - 2x + 1$

 Factor $3x + 3y + 3z$

The following questions need to be cuaqtified :)

 True False

(a) $\alpha > \delta$

(b) $\text{\LaTeX}$ is cool?

 Related to Linux

(a) You use Linux?

©2024–2026 by Pablo González L

22 / 167

- (b) Usually uses the package manager?
- (c) Rate the following package and class
 - i. xsim-exam
 - ii. xsim
 - iii. exsheets

The answer to 1 is $(x - 1)^2$ and the answer to 3.(a) is False.

- | | | | |
|-------------------|---|---------------------------------|---|
| 1. $(x - 1)^2$ | ⌘ | (b) Yes, dnf | ⌘ |
| 2. $3(x + y + z)$ | ⌘ | (c) i. doesn't exist for now :(| ⌘ |
| 3. (a) False | ⌘ | ii. very good | ⌘ |
| (b) Very True! | ⌘ | iii. obsolete | ⌘ |
| 4. (a) Yes | ⌘ | | |

Example 5

Adapted from the response given by Stephen in SAT like question format .

1 Which choice best describes what happens in the passage? A) One character argues with another character who intrudes on her home. B) One character receives a surprising request from another character. C) One character reminisces about choices she has made over the years. D) One character criticizes another character for pursuing an unexpected course of action.	3 Which choice best describes what happens in the passage? A) One character argues with another character who intrudes on her home. B) One character receives a surprising request from another character. C) One character reminisces about choices she has made over the years. D) One character criticizes another character for pursuing an unexpected course of action.
2 Which choice best describes what happens in the passage? A) One character argues with another character who intrudes on her home. B) One character receives a surprising request from another character. C) One character reminisces about choices she has made over the years. D) One character criticizes another character for pursuing an unexpected course of action.	4 Which choice best describes what happens in the passage? A) One character argues with another character who intrudes on her home. B) One character receives a surprising request from another character. C) One character reminisces about choices she has made over the years. D) One character criticizes another character for pursuing an unexpected course of action.

1. A) 2. C) 3. B) 4. D)

Example 6

Adapted from the response to Environment for enumerate environment .

- 8.5a, KSC 10. sample
- A sample
 - ✓ B answer
 - C sample
 - D sample
- 9.5a, KSC 11. sample
- A sample
 - B sample
 - C sample
 - ✓ D answer
12. sample
- A sample
 - B answer
 - C sample
 - D sample
13. sample
- A sample
 - B sample
 - C sample
 - D answer

10. B (8.5a, KSC)
11. D (9.5a, KSC)
12. B (10.5a, KSC)
13. D (11.5a, KSC)

8 Tagged PDF examples

This section is just to show the compatibility of `enumext` with *tagged* PDF using `lualatex`. The attached files here are just for testing and are intended as examples and, in a way, to simplify the time of Matthew Bertucci (@mbertucci) when he sees this excellent package and adds it to *The LaTeX Tagged PDF repository*. To compile the tests with `lualatex-dev` the packages `multicol`, `unicode-math`, `geometry`, `graphicx`, `luamml` and `hyperref` are required along with the line:

```
\DocumentMetadata{lang = en-US, pdfversion = 2.0, pdfstandard = ua-2, tagging=on,}
```

- All examples have been checked using `veraPDF` together with `ngpdf`.
 - The file `enumext-01.tex` contains the basic tests for the `enumext` and `enumext*` environments and the nesting between them plus the use of the `label`, `labelwidth`, `labelsep`, `ref`, `align` and `wrap-label` keys. Source file  and *tagged* PDF .
 - The file `enumext-02.tex` contains the tests for the `enumext` and `enumext*` environments and the support for `minipage` and `multicols` environments using the keys `columns`, `columns-sep`, `mini-env`, `mini-right` and `\miniright` command. Source file  and *tagged* PDF .
 - The file `enumext-03.tex` contains the tests for the `enumext` and `keyanspic` environments activated by the `save-ans` key together with the `save-sep` and `save-ref` keys and the `\printkeyans` command. Source file  and *tagged* PDF .
 - The file `enumext-04.tex` contains the tests for the `\anskey` command and the `anskey*` environment activated by the `save-ans` key along with the `\getkeyans` and `\printkeyans` commands. Source file  and *tagged* PDF .
 - The file `enumext-05.tex` contains the tests for the environments `keyans`, `keyans*` and `keyanspic` activated by the key `save-ans` together with the keys `no-store` and `show-ans` and the commands `\setenumext`, `\setenumextmeta`, `\printkeyans` and `\foreachkeyans`. Source file  and *tagged* PDF .
 - The file `enumext-06.tex` contains the tests for the environments `enumext` and `enumext*` for *fake itemize* and *description*. Source file  and *tagged* PDF .
 - The file `enumext-07.tex` contains the tests for starting the environments with `\setenumext{resume}`, the `\resetenumext` command and the `series`, `resume`, `resume*` and `reset` keys. Source file  and *tagged* PDF .

9 The way of non-enumerated lists

It is possible to use (or abuse) the `enumext` and `enumext*` environments to mimic *non-enumerated* list environments such as *itemize* and *description*, clearly the *(keys)* to “store answers”, the `keyans`, `keyans*` and `keyanspic` environments lose their sense and it is not the focus of `enumext` package, but, why not to do it?. Here I leave as an example other uses of the `enumext` environment that can be helpful for specific purposes. The *trick* to generate these “fake environments” is set `label={}` or `label={\some}` and play with the `list-indent`, `list-offset`, `font` and `wrap-label` keys.

Fake itemize environment

Here we set the `label` key using the default settings in `TEX` for the four levels `\textbullet`, `\textendash`, `\textasteriskcentered` and `\textperiodcentered` together with the `nosep` key to reduce the vertical spaces in the left side example and set the `label` key in *mathematical mode* for the right side as `\ast`, `\diamond`, `\circ` and `\star` for the four levels together with the `nosep` key

- First level item
 - Second level item
 - * Third level item
 - Fourth level item
 - First level item
- * First level item
 - ◇ Second level item
 - Third level item
 - ★ Fourth level item
 - * First level item

Fake description environment

Here we set `label={}` and `list-indent=2.5em`, `font=\bfseries`.

- SomeThing** A short one-line description.

This is an entry *without* a label.

Something A short *one-line* description text.

Something long A much *longer* description text may take more than one line or more than one paragraph.

 Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

If we add `list-indent=0pt` you get *widest style*:

SomeThing A short one-line description.
This is an entry *without* a label.
Something A short *one-line* description text.
Something long A much *longer* description text may take more than one line or more than one paragraph.
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

- The small space at the beginning of the “unlabeled entry” corresponds to `\labelsep` and can be removed using `\hspace{-\labelsep}` at the beginning of the line.
- When *tagged* PDF is active the default `description` style is NOT available due to the redefinition of `\makeLabel` for the `align` key which uses `\makebox` in this case, meaning that `\item[⟨content⟩]` will not extend beyond `\labelwidth` which causes overlaps,

Description indented by label

Here we set `label={}` and we will give a convenient value to `labelsep` and `labelwidth`, for example we can take as reference our *longest label* and pass it as value using:

```
\newlength{\descitemwd}  
\settowidth{\descitemwd}{\textbf{Something long}}  
  
and then use labelsep=4pt,labelwidth=\descitemwd,font=\bfseries.
```

SomeThing A short one-line description.
This is an entry *without* a label.
Something A short one-line description.
Something long A much longer description. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris.

The environment can be translated so that the *⟨labels⟩* are on the left margin calculating the value passed to the `list-offset` key, in this case it will be equal to the sum of the values set by the `labelwidth` and `labelsep` keys finally resulting as `list-offset={-\descitemwd - 4pt}`.

SomeThing A short one-line description.
This is an entry *without* a label.
Something A short one-line description.
Something long A much longer description. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris.

If we add `align=right` it will look like this:

SomeThing A short one-line description.
This is an entry *without* a label.
Something A short one-line description.
Something long A much longer description. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris.

- At this point we have used `list-offset={-\descitemwd - 4pt}` instead of `list-offset={-\labelwidth - \labelsep}`, this is because the parameters `\labelwidth` and `\labelsep` take the default values, as if we had not set `label`.

Description with multi-line labels

The `label` key does not accept *multiline material*, this is where the `wrap-label` and `wrap-label*` keys comes into play. Unlike the `enumitem` package, the `align` key only supports three options, so what we will do is create a command in the style `\parleft` of `enumitem` that allows us to place *multiline labels* using `\parbox`.

```
\NewDocumentCommand \labelbx { s +m }  
{%  
  \SuspendTagging{\parbox}%  
  \IfBooleanTF{#1}  
  {%  
    {\strut\smash{\parbox[t]{\labelwidth}{\raggedright{#2}}}}%  
    {\strut\smash{\parbox[t]{\labelwidth}{\raggedleft{#2}}}}%  
  }\ResumeTagging{\parbox}%  
}
```

Now we just need to set `wrap-label*={\labelbx{#1}}`.

SomeThing A short one-line description.
This is an entry *without* a label.
Something A short one-line description.
Something long A much longer description. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris.
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris.

SoMeThInG A much longer description. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum
LoNg ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris.

Final notes

The original implementation (if you can call it that) of the ideas that led to the creation of `enumext` were some macros using the `enumerate[5]` package for personal use created in early 2003, the code was quite questionable, but functional for these simple requirements.

With the great answers given by Christian Hupfer in [Create a fake label ref using list](#) and the answer given by David Carlisle in [Change the use of label ref by data save in an array \(list\)](#) I managed to create a more solid code than the original version, now using the `l3prop[11]` and `l3seq[11]` modules together with the `hyperref[8]` and `enumitem[6]` packages, which did the job, but with some limitations.

As time went by I took these limitations as a personal challenge which I called “*reinventing the wheel*”, since there were packages and classes that did more or less what I was looking for, but did not fit my simple requirements. This “*reinventing the wheel*” finally ended up becoming `enumext`.

Why list environments?

The answer is simple, first I love the beauty of its syntax and many of what I had already written used the `enumerate` environment or lists created using the `enumitem` package. In my mind I thought: how complicated could it be to write a package that looked like `enumitem`? It seemed simple enough, of course I didn’t have in mind the mess I was getting into working with `list` environments, `minipage` and adding support for the `multicol` and `hyperref` packages.

Of course, seeing the final result of the experiment “*reinventing the wheel*” I am quite satisfied.

Why not random questions and other utilities

The “*random*” type questions I love and hate them at the same time, although they simplify a lot the work when creating a multiple choice test, but you lose the beauty of typesetting a document with $\text{\LaTeX}$, that is to say the output does not always look as nice as it should, even if they are only alternatives these must follow a certain order when presented either numerical or presentation, that said handling that using *nested lists* is quite complicated so I do not classify to be implemented.

Why has it taken so long?

One of the setbacks, beyond my laziness, was including compatibility with *tagged* PDF. To be honest, it’s something I never considered at any point, but I firmly believe that being able to create *accessible documents* provides a great opportunity in the world of mathematics education. From my perspective as a *high school* teacher, beyond theorems and deep mathematics, the use of exercise lists is one of the most common things. Being able to open the way to work in parallel with those who have different abilities is really important and I regret not having looked into this in the past. I hope that `enumext` serves this purpose and inspires more users and authors to follow this path.

10 References

- [1] HIRSCHHORN, PHILIP. “Using the exam document class”. Available from CTAN, <https://www.ctan.org/pkg/exam>, 2023.
- [2] NIEDERBERGER, CLEMENS. “xsim – eXercise Sheets IMproved”. Available from CTAN, <https://www.ctan.org/pkg/xsim>, 2023.
- [3] MITTELBACH, FRANK. “An environment for multicolumn output”. Available from CTAN, <https://www.ctan.org/pkg/multicol>, 2025.
- [4] GONZÁLEZ, PABLO. “scontents - Stores $\text{\LaTeX}$ contents in memory or files”. Available from CTAN, <https://www.ctan.org/pkg/scontents>, 2025.
- [5] The $\text{\LaTeX}$ Project. “enumerate – Enumerate with redefinable labels”. Available from CTAN, <https://www.ctan.org/pkg/enumerate>, 2025.
- [6] BEZOS, JAVIER. “Customizing lists with the enumitem package”. Available from CTAN, <https://www.ctan.org/pkg/enumitem>, 2025.
- [7] BERRY, KARL. “ $\text{\LaTeX}_{2\epsilon}$: An Unofficial Reference Manual”. Available from CTAN, <https://ctan.org/pkg/latex2e-help-texinfo>, 2025.
- [8] The $\text{\LaTeX}$ Project. “Extensive support for hypertext in $\text{\LaTeX}$ ”. Available from CTAN, <https://www.ctan.org/pkg/hyperref>, 2025.
- [9] BURNOL, JEAN-FRANÇOIS. “The footnotehyper package”. Available from CTAN, <https://www.ctan.org/pkg/footnotehyper>, 2025.

- [10] The $\text{\LaTeX}$ Project. “The expl3 package”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2025.
- [11] The $\text{\LaTeX}$ Project. “The $\text{\LaTeX}$ 3 Interfaces”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2025.
- [12] The $\text{\LaTeX}$ Project. “The $\text{\LaTeX}$ 2 ϵ sources”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2025.
- [13] The $\text{\LaTeX}$ Project. “ $\text{\LaTeX}$ News, Issue 41, June 2025”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2025.
- [14] The $\text{\LaTeX}$ Project. “ $\text{\LaTeX}$ for authors current version”. Available from CTAN, <https://ctan.org/pkg/latex-base>, 2025.
- [15] GUNDLACH, PATRICK. “The lua-visual-debug package”. Available from CTAN, <https://www.ctan.org/pkg/lua-visual-debug>, 2023.
- [16] LEMVIG, MOGENS. “The shortlst package”. Available from CTAN, <https://www.ctan.org/pkg/shortlst>, 1998.
- [17] NIEDERBERGER, CLEMENS. “tasks – Horizontally columned lists”. Available from CTAN, <https://www.ctan.org/pkg/tasks>, 2022.
- [18] FISCHER, ULRIKE. “tagpdf – $\text{\LaTeX}$ kernel code for PDF tagging”. Available from CTAN, <https://www.ctan.org/pkg/tagpdf>, 2025.
- [19] The $\text{\LaTeX}$ Project. “latex-lab – $\text{\LaTeX}$ laboratory”. Available from CTAN, <https://www.ctan.org/pkg/latex-lab>, 2025.
- [20] MITTELBACH, FRANK. “ $\text{\LaTeX}$ ’s socket management”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2025.

11 Change history

- | | |
|--------------------------------|--|
| v2.1 (ctan), 2026-01-15 | – Fixed bad interaction between <code>\setenumext</code> and the <code>ref</code> key. |
| v2.0 (ctan), 2025-12-02 | – Update requirement to <code>multicol</code> release 2025-10-21. |
| | – Adjustments in the documentation. |
| v1.9 (ctan), 2025-11-01 | – Update requirement to $\text{\LaTeX}$ release 2025-11-01. |
| | – Move <code>\setcounter</code> to <code>\begin{list}{\langle arg one \rangle}{\langle arg two \rangle \setcounter}</code> . |
| | – Update code for <i>tagged</i> PDF. |
| v1.8 (ctan), 2025-10-04 | – Replacing <code>\scantokens</code> (primitive) with <code>\tl_retokenize:n</code> . |
| | – Cleanup warnings and details returned by <code>expltools</code> . |
| v1.7 (ctan), 2025-07-10 | – Fixed <code>\setenumext{enumext*}{resume}</code> . |
| | – Fixed bad interaction between <code>\setenumext</code> and the <code>resume</code> key. |
| | – The behavior of the key <code>resume*</code> has been updated and documented. |
| v1.6 (ctan), 2025-07-04 | – Syntax simplification for <code>\setenumextmeta</code> . |
| | – Environments can be started with the key <code>resume without value</code> . |
| | – Add <code>\resetenumext</code> , <code>reset</code> and <code>reset*</code> keys. |
| | – The <code>resume</code> , <code>resume*</code> and <code>series</code> keys can now be set per level. |
| | – Fixed bad interaction between <code>\printkeyans</code> and the <code>resume</code> , <code>resume*</code> keys. |
| v1.5 (ctan), 2025-06-11 | – Replacing <code>\regex_match:</code> (deprecated) with <code>\regex_if_match:</code> . |
| | – Add keys <code>beginpenalty</code> , <code>midpenalty</code> and <code>endpenalty</code> . |
| v1.4 (ctan), 2025-06-09 | – Improved implementation of the <code>start</code> key for <i>tagged</i> PDF. |
| | – Improved implementation of the <code>ref</code> key. |
| | – Fixed the behavior of the <code>save-sep</code> key. |
| | – Fixed the behavior of the <code>resume*</code> key. |
| v1.3 (ctan), 2025-06-01 | – Removed dependency on the <code>scontents</code> package. |
| | – The <code>anskey*</code> environment has been rewritten using the new <code>c</code> -type argument. |
| v1.2 (ctan), 2025-03-28 | – Replace signature (prevent expansion for optional argument). |
| | – Solve Inconsistent local/global assignment. |
| v1.1 (ctan), 2024-11-14 | – Fixed implementation for <code>font</code> and <code>base-fix</code> keys. |
| | – Added new keys for symbol marks. |
| | – Update and improvements in the internal code. |
| | – Adjustments in the documentation. |
| v1.0 (ctan), 2024-11-01 | – First public release. |

12 Index of Documentation

The italic numbers denote the pages where the corresponding entry is described.

C

Document class:

article 2

book 2

exam 2

letter 2

report 2

\columnbreak 4, 15

\columnsep 11

Commands provide by enumext:

\anskey 12–15

\anspic 12–14, 17, 18

\foreachkeyans 19

\getkeyans 14, 19

\item* 5–7, 12–14, 16, 17

\item 5–8, 10, 12, 14, 16, 17

\miniright 12

\printkeyans 6, 13, 20

\resetenumext 11

\setenumextmeta 6

\setenumext 5–7, 12, 14, 16, 20

Counters defined by enumext:

enumXiii 4

enumXii 4

enumXiv 4

enumXi 4

enumXviii 4

enumXvii 4

enumXvi 4

enumXv 4

D

Dimension defined by enumext:

\itemwidth 4, 5, 11

E

Environments provide by enumext:

anskey* 12–16, 24

enumext* 4–17, 20, 24

enumext 4–17, 20, 24

keyans* 4–16, 24

keyanspic 4, 7, 9, 10, 12–15, 17, 24

keyans 4, 6–18, 24

Environments:

Verbatim 16

center 5

description 5, 24, 25

enumerate 1, 3, 5, 26

figure 5

flushleft 5

flushright 5

itemize 5, 24

list 3, 5, 10, 26

minipage 3–5, 8–10, 12, 24, 26

multicols 3, 4, 11, 24

quotation 5

quote 5

shortenumerate 5

tabbing 5

table 5

tasks 5

F

trivlist 5

verbatim 5

verse 5

I

\footnote 5

\itemsep 9

\itemwidth 4, 5, 11

K

Keys for \anskey provide by enumext:

break-col 15

item-join 15

item-pos* 15

item-star 15

item-sym* 15

Keys for \foreachkeyans provide by enumext:

after 20

before 20

sep 19

start 19

step 19

stop 19

wrapper 20

Keys for anskey* provide by enumext:

break-col 15

force-eol 16

item-join 15

item-pos* 15

item-star 15

item-sym* 15

overwrite 16

write-env 15

Keys for environments provide by enumext:

above* 9

above 9

after 10

align 7, 14, 24, 25

base-fix 9

before* 10

before 10

beginpenalty 8

below* 9

below 9

check-ans 13, 14

columns-sep 4, 11, 24

columns 4, 9, 11, 24

endpenalty 8

first 10

font 7, 13, 14

item-pos* 5, 6

item-sym* 5, 6

itemindent 9, 10

itemsep 9

label-pos 18

label-sep 18

labelsep 3–7, 9–11, 24, 25

labelwidth 3–7, 9–11, 13, 14, 24, 25

label 7, 8, 10, 16, 17, 24, 25

layout-sep	18	wrap-label	7, 8, 13, 14, 24, 25
layout-sty	17, 18	wrap-opt	13, 14, 16, 18
layout-top	18	write-env	16
list-indent	3, 9, 10		
list-offset	3, 9, 25	L	
listparindent	10	\label	4
mark-ans*	13, 14, 16, 18	Labels provide by enumext :	
mark-ans	13, 14	\Alph*	7, 8, 16
mark-pos*	14, 16, 18	\Roman*	7, 8
mark-pos	13	\alph*	7, 8
mark-ref	13	\arabic*	7, 8
mark-sep*	14, 16, 18	\roman*	7, 8
mark-sep	13, 14	\labelsep	3, 7
midpenalty	8	\labelwidth	3, 7
mini-env	4, 9, 12, 24	\linewidth	12
mini-right*	7, 12	\listparindent	10
mini-right	7, 12, 24		
mini-sep	4, 12	P	
mode-box	7	Packages:	
no-store	12-15, 24	enumerate	26
noitemsep	9	enumext	1-5, 7, 13, 17, 24, 26
nosep	9, 24	enumitem	3, 4, 25, 26
overwrite	15	fancyvrb	16
parsep	8-10, 18	footnotehyper	5
partopsep	8	geometry	24
ref	4, 8, 24	graphicx	24
reset*	6, 7, 10, 11	hyperref	4, 5, 13-15, 24, 26
reset	6, 7, 10, 11, 24	l3keys	7
resume*	6, 7, 10-12, 24	l3prop	26
resume	6, 7, 10-12, 24	l3seq	26
rightmargin	10	luamml	24
save-ans	4, 6, 10-20, 24	multicol	1, 2, 4, 24, 26, 27
save-key	10, 12, 13, 20	scontents	27
save-ref	4, 7, 13-15, 19, 24	shortlst	5
save-sep	13, 16, 18, 24	tasks	5, 6
series	6, 7, 10-12, 24	unicode-math	24
show-ans	13, 14, 16, 18, 24	xsim	2
show-length	8	\parsep	8
show-pos	13, 14, 16, 18, 19	\partopsep	8
start*	10, 11		
start	10, 11	R	
topsep	8, 9, 18	\raggedcolumns	4
widest	7	\ref	4
wrap-ans*	14, 16, 18	\rightmargin	10
wrap-ans	13, 14		
wrap-label*	7, 8, 25	T	
		\topsep	8

13 Implementation

The most recent publicly released version of `enumext` is available at CTAN: <https://www.ctan.org/pkg/enumext>. While general feedback via email is welcomed, specific bugs or feature requests should be reported through the issue tracker: <https://github.com/pablgonz/enumext/issues>.

- The documentation presented here is far from professional, it contains a lot of obvious information that to the eye of a TeXpert are superfluous, but, after so many years developing this project is the only way to remember what does what.

13.1 General conventions

Variables containing `i`, `ii`, `iii` and `iv` are associated by level with the `enumext` environment, variables containing `v` are associated with the `keyans` environment, variables containing `vi` are associated with the `keyanspic` environment, variables containing `vii` are associated with the `enumext*` environment and variables containing `viii` are associated with the `keyans*` environment.

To simplify writing and documentation some variables and functions that are common to the different levels of the environments are described using a capital “X”.

The temporary function `\__enumext_tmp:n` is used in different parts of the package code for variable creation or execution of other functions that are grouped into this one.

All variables and functions defined in this package are private and are NOT intended to work or be used by another package or module.

13.2 Initial set up

Start the DocStrip guards.

```
1 (*package)
```

Identify the internal prefix (L^AT_EX3 DocStrip convention) for l3doc class.

```
2 (@@=enumext)
```

13.3 Declaration of the package

First we will make sure we have a minimum (super updated) version of L^AT_EX to work correctly.

```
3 \NeedsTeXFormat{LaTeX2e}[2025-11-01]
```

Now declare the `enumext` package.

```
4 \ProvidesExplPackage {enumext} {2026-01-15} {2.1} {Enumerate exercise sheets}
```

Finally check if the `multicol` package are loaded, if not we load it.

```
5 \hook_gput_code:nnn {begindocument} {enumext}
6 {
7   \IfPackageLoadedTF { multicol }
8   {
9     \msg_info:nnn { enumext } { package-load } { multicol }
10  }
11  {
12    \msg_info:nnn { enumext } { package-not-load } { multicol }
13    \RequirePackage{multicol}[2025-10-21]
14  }
15 }
```

13.4 Definition of variables

Variables that do not appear in this section are created by means of `\keys_define:nn` or some function described below.

Integer variables will control the nesting levels of the environments, `anskey*` environment and `\anskey` command.

```
\__enumext_level_int
\__enumext_level_h_int
\__enumext_anskey_level_int
\__enumext_keyans_level_int
\__enumext_keyans_level_h_int
\__enumext_keyans_pic_level_int

16 \int_new:N \__enumext_level_int
17 \int_new:N \__enumext_level_h_int
18 \int_new:N \__enumext_anskey_level_int
19 \int_new:N \__enumext_keyans_level_int
20 \int_new:N \__enumext_keyans_level_h_int
21 \int_new:N \__enumext_keyans_pic_level_int
```

(End of definition for `\__enumext_level_int` and others.)

```

\l__enumext_starred_bool
\g__enumext_starred_bool
  \l_enumext_starred_first_bool
\l__enumext_standar_bool
\g__enumext_standar_bool
  \l_enumext_standar_first_bool
\l__enumext_keyans_env_bool
\g__enumext_start_line_tl
\g__enumext_envir_name_tl
\l__enumext_envir_name_tl

```

Internal variables used by functions `\__enumext_is_not_nested:`, `\__enumext_is_on_first_level:` and `\__enumext_keyans_name_and_start:` (§13.5.1).

```

22 \bool_new:N \l__enumext_starred_bool
23 \bool_new:N \g__enumext_starred_bool
24 \bool_new:N \l__enumext_starred_first_bool
25 \bool_new:N \l__enumext_standar_bool
26 \bool_new:N \g__enumext_standar_bool
27 \bool_new:N \l__enumext_standar_first_bool
28 \bool_new:N \l__enumext_keyans_env_bool
29 \tl_new:N \g__enumext_start_line_tl
30 \tl_new:N \g__enumext_envir_name_tl
31 \tl_new:N \l__enumext_envir_name_tl

```

(End of definition for `\l__enumext_starred_bool` and others.)

```

\l__enumext_counter_i_tl
\l__enumext_counter_ii_tl
\l__enumext_counter_iii_tl
\l__enumext_counter_iv_tl
\l__enumext_counter_v_tl
\l__enumext_counter_vi_tl
\l__enumext_counter_vii_tl
\l__enumext_counter_viii_tl

```

Variables to store the “*name of the counters*” `enumXi`, `enumXii`, `enumXiii` and `enumXiv` for `enumext` environment, `enumXv` for `keyans` environment and `enumXvi` for the `keyanspic` environment. The counters `enumXvii` and `enumXviii` are used by `enumext*` and `keyans*` environments.

The initial values of these variables are set by the function `\__enumext_define_counter:Nn` (§13.11) and then modified by the function `\__enumext_label_style:Nnn` used by `label` key (§13.14).

```

32 \cs_set_protected:Npn \__enumext_tmp:n #1
33 {
34   \tl_new:c { l__enumext_counter_#1_tl }
35 }
36 \clist_map_inline:nn { i, ii, iii, iv, v, vi, vii, viii } { \__enumext_tmp:n {#1} }

```

(End of definition for `\l__enumext_counter_i_tl` and others.)

```

\l__enumext_ref_key_arg_tl
\l__enumext_ref_the_count_tl
  \l_enumext_renew_counter_X_tl
\l__enumext_the_counter_X_tl

```

Internal variables used by `ref` key (§13.14).

```

37 \tl_new:N \l__enumext_ref_key_arg_tl
38 \tl_new:N \l__enumext_ref_the_count_tl
39 \cs_set_protected:Npn \__enumext_tmp:n #1
40 {
41   \tl_new:c { l__enumext_renew_counter_#1_tl }
42   \tl_new:c { l__enumext_the_counter_#1_tl }
43   \tl_set:ce { l__enumext_the_counter_#1_tl } { \exp_not:c { theenumX#1 } }
44 }
45 \clist_map_inline:nn { i, ii, iii, iv, v, vi, vii, viii } { \__enumext_tmp:n {#1} }

```

(End of definition for `\l__enumext_ref_key_arg_tl` and others.)

```

\l__enumext_series_name_tl
  \l_enumext_resume_count_bool
  \l_enumext_resume_count_X_bool
  \l_enumext_resume_series_X_bool
\l__enumext_resume_star_key_X_bool
\g__enumext_resume_last_keys_X_tl

```

Internal variables used by `resume`, `resume*` and `series` keys (§13.26).

```

46 \tl_new:N \l__enumext_series_name_tl
47 \bool_new:N \l__enumext_resume_count_bool
48 \cs_set_protected:Npn \__enumext_tmp:n #1
49 {
50   \bool_new:c { l__enumext_resume_count_#1_bool }
51   \bool_new:c { l__enumext_resume_series_#1_bool }
52   \bool_new:c { l__enumext_resume_star_key_#1_bool }
53   \tl_new:c { g__enumext_resume_last_keys_#1_tl }
54 }
55 \clist_map_inline:nn { i, ii, iii, iv, vii } { \__enumext_tmp:n {#1} }

```

(End of definition for `\l__enumext_series_name_tl` and others.)

```

\l_enumext_current_widest_dim
\g__enumext_counter_styles_tl
\g__enumext_widest_label_tl
  \l_enumext_label_width_by_box

```

The variable `\l__enumext_current_widest_dim` stores the current label width, the variable `\g__enumext_counter_styles_tl` stores the default *label style* and the variable `\g__enumext_widest_label_tl` the label width. These variables are used by `widest` (§13.15) and `label` (§13.13) keys.

```

56 \dim_new:N \l__enumext_current_widest_dim
57 \tl_new:N \g__enumext_counter_styles_tl
58 \tl_new:N \g__enumext_widest_label_tl
59 \box_new:N \l__enumext_label_width_by_box

```

(End of definition for `\l__enumext_current_widest_dim` and others.)

```
\l__enumext_leftmargin_tmp_X_bool
\l__enumext_leftmargin_tmp_X_dim
\l__enumext_leftmargin_X_dim
\l__enumext_itemindent_X_dim
```

The boolean variable `\l__enumext_leftmargin_tmp_X_bool` and the dimensional variable `\l__enumext_leftmargin_tmp_X_dim` are used by the `list-indent` key (§13.19). The variables `\l__enumext_leftmargin_X_dim` and `\l__enumext_itemindent_X_dim` are used and set by the function `\__enumext_calc_hspace:NNNNNNNNNN` (§13.41.1).

```
60 \cs_set_protected:Npn \__enumext_tmp:n #1
61 {
62   \bool_new:c { \l__enumext_leftmargin_tmp_#1_bool }
63   \dim_new:c { \l__enumext_leftmargin_tmp_#1_dim }
64   \dim_new:c { \l__enumext_leftmargin_#1_dim }
65   \dim_new:c { \l__enumext_itemindent_#1_dim }
66 }
67 \clist_map_inline:nn { i, ii, iii, iv, v, vi, vii, viii } { \__enumext_tmp:n {#1} }
```

(End of definition for `\l__enumext_leftmargin_tmp_X_bool` and others.)

```
\l__enumext_multicols_above_X_skip
\l__enumext_multicols_below_X_skip
\g__enumext_multicols_right_X_skip
\l__enumext_align_label_pos_X_str
```

Internal variables used by `columns` key (§13.23) and `align` key (§13.13).

```
68 \cs_set_protected:Npn \__enumext_tmp:n #1
69 {
70   \skip_new:c { \l__enumext_multicols_above_#1_skip }
71   \skip_new:c { \l__enumext_multicols_below_#1_skip }
72   \skip_new:c { \g__enumext_multicols_right_#1_skip }
73   \str_new:c { \l__enumext_align_label_pos_#1_str }
74 }
75 \clist_map_inline:nn { i, ii, iii, iv, v } { \__enumext_tmp:n {#1} }
```

(End of definition for `\l__enumext_multicols_above_X_skip` and others.)

```
\g__enumext_minipage_stat_int
\l__enumext_minipage_temp_skip
\l__enumext_minipage_left_skip
\l__enumext_minipage_right_skip
\l__enumext_minipage_after_skip
\g__enumext_minipage_right_skip
\g__enumext_minipage_after_skip
\l__enumext_minipage_left_X_dim
\l__enumext_minipage_active_X_bool
```

Internal variables used by `\miniright` command (§13.24.4) and the keys `mini-right`, `mini-right*`, `mini-env` and `mini-sep` (§13.22, §13.24).

```
76 \int_new:N \g__enumext_minipage_stat_int
77 \skip_new:N \l__enumext_minipage_temp_skip
78 \skip_new:N \l__enumext_minipage_left_skip
79 \skip_new:N \l__enumext_minipage_right_skip
80 \skip_new:N \l__enumext_minipage_after_skip
81 \skip_new:N \g__enumext_minipage_right_skip
82 \skip_new:N \g__enumext_minipage_after_skip
83 \cs_set_protected:Npn \__enumext_tmp:n #1
84 {
85   \dim_new:c { \l__enumext_minipage_left_#1_dim }
86   \bool_new:c { \l__enumext_minipage_active_#1_bool }
87 }
88 \clist_map_inline:nn { i, ii, iii, iv, v, vii, viii } { \__enumext_tmp:n {#1} }
```

(End of definition for `\g__enumext_minipage_stat_int` and others.)

```
\l__enumext_wrap_label_X_bool
\l__enumext_wrap_label_opt_X_bool
\l__enumext_start_X_int
\l__enumext_fake_item_indent_X_tl
\l__enumext_label_fill_left_X_tl
\l__enumext_label_fill_right_X_tl
\l__enumext_vspace_a_star_X_bool
\l__enumext_vspace_b_star_X_bool
```

The bool vars `\l__enumext_wrap_label_X_bool` and `\l__enumext_wrap_label_opt_X_bool` are used by `wrap-label` and `wrap-label*` keys (§13.13), the integer `\l__enumext_start_X_int` are used by the `start` and `start*` keys (§13.15), the token list `\l__enumext_fake_item_indent_X_tl` is used by `itemindent` key (§13.19.1), the variables `\l__enumext_label_fill_left_X_tl` and `\l__enumext_label_fill_right_X_tl` are used by the `align` key (§13.13). The boolean vars `\l__enumext_vspace_a_star_X_bool`, `\l__enumext_vspace_b_star_X_bool` are used by `above`, `above*`, `below` and `below*` keys (§13.21).

```
89 \cs_set_protected:Npn \__enumext_tmp:n #1
90 {
91   \bool_new:c { \l__enumext_wrap_label_#1_bool }
92   \bool_new:c { \l__enumext_wrap_label_opt_#1_bool }
93   \int_new:c { \l__enumext_start_#1_int }
94   \tl_new:c { \l__enumext_fake_item_indent_#1_tl }
95   \tl_new:c { \l__enumext_label_fill_left_#1_tl }
96   \tl_new:c { \l__enumext_label_fill_right_#1_tl }
97   \bool_new:c { \l__enumext_vspace_a_star_#1_bool }
98   \bool_new:c { \l__enumext_vspace_b_star_#1_bool }
99 }
100 \clist_map_inline:nn { i, ii, iii, iv, v, vii, viii } { \__enumext_tmp:n {#1} }
```

(End of definition for `\l__enumext_wrap_label_X_bool` and others.)

```

\l__enumext_store_active_bool
\l__enumext_store_name_tl
\g__enumext_store_name_tl
\l__enumext_store_current_label_tl
\l__enumext_store_current_opt_arg_tl

```

The variable `\l__enumext_store_active_bool` setting by `save-ans` key (§13.29.1) activates all the mechanism related to `\anskey`, `anskey*`, `keyans`, `keyans*` and `keyanspic` environments.

The variable `\l__enumext_store_name_tl` saves the $\{\langle store\ name\rangle\}$ set by the `save-ans` key of the *sequence* and *prop list* in which we will store, the variable `\g__enumext_store_name_tl` it's just a global copy of $\{\langle store\ name\rangle\}$ used by different functions.

The variables `\l__enumext_store_current_label_tl` and `\l__enumext_store_current_opt_arg_tl` save the *current label* and *optional argument* of `\item*` (§13.40) and `\anspic*` (§13.45.2) for the `keyans`, `keyans*` and `keyanspic` environments.

```

101 \bool_new:N \l__enumext_store_active_bool
102 \tl_new:N \l__enumext_store_name_tl
103 \tl_new:N \g__enumext_store_name_tl
104 \tl_new:N \l__enumext_store_current_label_tl
105 \tl_new:N \l__enumext_store_current_opt_arg_tl

```

(End of definition for `\l__enumext_store_active_bool` and others.)

```

\l__enumext_store_anskey_arg_tl
\l__enumext_store_anskey_env_tl
\l__enumext_write_anskey_env_bool
\l__enumext_write_anskey_env_file_name_tl
\l__enumext_write_anskey_env_file_iow

```

The variable `\l__enumext_store_anskey_arg_tl` save the *argument* of `\anskey` (§13.33) and the variables `\l__enumext_store_anskey_env_tl` save the $\langle body\rangle$ of the environment `anskey*` (§13.34).

The variables `\l__enumext_write_anskey_env_bool`, `\l__enumext_write_anskey_env_file_name_tl` and `\l__enumext_write_anskey_env_file_iow` they are used by the `write-env` and `overwrite` keys in the `anskey*` environment implementation.

```

106 \tl_new:N \l__enumext_store_anskey_arg_tl
107 \tl_new:N \l__enumext_store_anskey_env_tl
108 \bool_new:N \l__enumext_write_anskey_env_bool
109 \tl_new:N \l__enumext_write_anskey_env_file_name_tl
110 \iow_new:N \l__enumext_write_anskey_env_file_iow

```

(End of definition for `\l__enumext_store_anskey_arg_tl` and others.)

```

\c__enumext_anskey_env_hidden_space_str

```

The `\c__enumext_anskey_env_hidden_space_str` is a constant *string* to used to hide the $\langle forced\ space\rangle$ added by T_EX when recording content in a macro. This *string* contains the *reserved phrase* ‘`%^^Aenumextheol%`’ which is added to the end of the argument stored in *sequence* and *prop list* when the key `force-eol` is false.

```

111 \str_const:Nc \c__enumext_anskey_env_hidden_space_str
112 { \c_percent_str \c_circumflex_str \c_circumflex_str A enumextheol \c_percent_str }

```

(End of definition for `\c__enumext_anskey_env_hidden_space_str`.)

```

\l__enumext_setkey_tmpa_tl
\l__enumext_setkey_tmpb_tl
\l__enumext_setkey_tmpa_int
\l__enumext_setkey_tmpa_seq
\l__enumext_setkey_tmpb_seq

```

Internal variables used by the command `\setenumext` (§13.51).

```

113 \tl_new:N \l__enumext_setkey_tmpa_tl
114 \tl_new:N \l__enumext_setkey_tmpb_tl
115 \int_new:N \l__enumext_setkey_tmpa_int
116 \seq_new:N \l__enumext_setkey_tmpa_seq
117 \seq_new:N \l__enumext_setkey_tmpb_seq

```

(End of definition for `\l__enumext_setkey_tmpa_tl` and others.)

```

\l__enumext_meta_path_str
\l__enumext_foreach_print_seq
\l__enumext_foreach_name_prop_tl
\l__enumext_foreach_default_keys_tl

```

Internal variables used by the `\setenumextmeta` command (§13.52) and `\foreachkeyans` command (§13.53).

```

118 \str_new:N \l__enumext_meta_path_str
119 \seq_new:N \l__enumext_foreach_print_seq
120 \tl_new:N \l__enumext_foreach_name_prop_tl
121 \tl_new:N \l__enumext_foreach_default_keys_tl

```

(End of definition for `\l__enumext_meta_path_str` and others.)

```

\l__enumext_print_keyans_starred_tl
\l__enumext_print_keyans_star_bool
\l__enumext_print_keyans_cmd_bool
\l__enumext_mark_position_str
\l__enumext_mark_position_v_str
\l__enumext_mark_position_viii_str
\l__enumext_mark_sep_tmpa_dim
\l__enumext_mark_sep_tmpb_dim
\l__enumext_show_pos_tmp_int
\g__enumext_item_symbol_aux_tl
\l__enumext_print_keyans_X_tl
\l__enumext_store_save_key_X_tl
\l__enumext_store_save_key_X_bool
\l__enumext_store_upper_level_X_bool

```

Internal variables used by command `\printkeyans` (§13.50), `show-pos`, `show-ans`, `mark-pos`, `mark-sep` keys (§13.30), `item-sym*` key (§13.38), `save-key` key (§13.30.3) and “*storing structure*”.

```

122 \tl_new:N \l__enumext_print_keyans_starred_tl
123 \bool_new:N \l__enumext_print_keyans_star_bool
124 \bool_new:N \l__enumext_print_keyans_cmd_bool
125 \str_new:N \l__enumext_mark_position_str
126 \str_new:N \l__enumext_mark_position_v_str
127 \str_new:N \l__enumext_mark_position_viii_str
128 \dim_new:N \l__enumext_mark_sep_tmpa_dim
129 \dim_new:N \l__enumext_mark_sep_tmpb_dim
130 \int_new:N \l__enumext_show_pos_tmp_int
131 \tl_new:N \g__enumext_item_symbol_aux_tl
132 \cs_set_protected:Npn \l__enumext_tmp:n #1
133 {
134   \tl_new:c { l__enumext_print_keyans_#1_tl }

```

```

135     \tl_new:c { \__enumext_store_save_key_#1_tl }
136     \bool_new:c { \__enumext_store_save_key_#1_bool }
137     \bool_new:c { \__enumext_store_upper_level_#1_bool }
138   }
139   \clist_map_inline:nn { i, ii, iii, iv, vii } { \__enumext_tmp:n {#1} }

```

(End of definition for `\__enumext_print_keyans_starred_tl` and others.)

Internal variables used by `keyanspic` environment and `\anspic` command (§13.45.1).

```

140 \seq_new:N \__enumext_anspic_args_seq
141 \dim_new:N \__enumext_anspic_mini_width_dim
142 \int_new:N \__enumext_anspic_above_int
143 \int_new:N \__enumext_anspic_below_int
144 \bool_new:N \__enumext_anspic_label_above_bool
145 \str_new:N \__enumext_anspic_mini_pos_str
146 \box_new:N \__enumext_anspic_label_box
147 \box_new:N \__enumext_anspic_body_box
148 \dim_new:N \__enumext_anspic_label_htdp_dim
149 \dim_new:N \__enumext_anspic_body_htdp_dim

```

(End of definition for `\__enumext_anspic_args_seq` and others.)

Internal variables used by “*internal check answer*” mechanism (§13.29.3) used by the `check-ans`, `no-store`, `wrap-ans*` keys and check for starred commands `\item*` in `keyans` and `keyans*` environments and `\anspic*` in `keyanspic` environment.

```

150 \bool_new:N \__enumext_check_answers_bool
151 \bool_new:N \g__enumext_check_ans_key_bool
152 \tl_new:N \__enumext_check_start_line_env_tl
153 \bool_new:N \__enumext_item_wrap_key_bool
154 \int_new:N \g__enumext_check_starred_cmd_int
155 \int_new:N \g__enumext_item_anskey_int
156 \int_new:N \g__enumext_item_number_int
157 \bool_new:N \__enumext_item_number_bool
158 \int_new:N \g__enumext_item_answer_diff_int

```

(End of definition for `\__enumext_check_answers_bool` and others.)

The boolean variable `\__enumext_hyperref_bool` will determine if the `hyperref` package is present or load in memory (§13.7). The boolean variable `\__enumext_footnotes_key_bool` determine if `hyperref` is load with key `hyperfootnotes=true`.

```

159 \bool_new:N \__enumext_hyperref_bool
160 \bool_new:N \__enumext_footnotes_key_bool

```

(End of definition for `\__enumext_hyperref_bool` and `\__enumext_footnotes_key_bool`.)

Internal variables used by `save-ref` key (§13.30). The variables `\__enumext_label_copy_X_tl` correspond to temporary copies of the *⟨labels⟩* defined by level on which operations will be performed.

The variables `\__enumext_newlabel_arg_one_tl` and `\__enumext_newlabel_arg_two_tl` will be used to form the arguments passed to the function `\__enumext_newlabel:nn` (§13.7) and the variable `\__enumext_write_aux_file_tl` will be in charge of executing the writing code in the `.aux` file.

```

161 \tl_new:N \__enumext_newlabel_arg_one_tl
162 \tl_new:N \__enumext_newlabel_arg_two_tl
163 \tl_new:N \__enumext_write_aux_file_tl
164 \cs_set_protected:Npn \__enumext_tmp:n #1
165 {
166   \tl_new:c { \__enumext_label_copy_#1_tl }
167 }
168 \clist_map_inline:nn { i, ii, iii, iv, v, vi, vii, viii } { \__enumext_tmp:n {#1} }

```

(End of definition for `\__enumext_newlabel_arg_one_tl` and others.)

Internal variables used for redefinition of `\footnote` (§13.8).

```

169 \int_new:N \g__enumext_footnote_standar_int
170 \int_new:N \g__enumext_footnote_starred_int
171 \seq_new:N \g__enumext_footnote_standar_arg_seq
172 \seq_new:N \g__enumext_footnote_starred_arg_seq
173 \seq_new:N \g__enumext_footnote_standar_int_seq
174 \seq_new:N \g__enumext_footnote_starred_int_seq

```

(End of definition for `\g__enumext_footnote_standar_int` and others.)

```

\l__enumext_item_starred_X_bool
\l__enumext_item_column_pos_X_int
\g__enumext_item_count_all_X_int
\l__enumext_joined_item_X_int
\l__enumext_joined_item_aux_X_int
\l__enumext_tmpa_X_int
\l__enumext_tmpa_X_dim
\l__enumext_item_text_X_box
\l__enumext_joined_width_X_dim
\l__enumext_item_width_X_dim
\g__enumext_item_symbol_aux_X_tl
\l__enumext_align_label_X_str
\g__enumext_minipage_active_X_bool
\l__enumext_miniright_code_X_box
\g__enumext_minipage_center_X_bool
\g__enumext_minipage_right_X_dim
\g__enumext_minipage_right_X_skip

```

Internal variables used by `enumext*` and `keyans*` environments.

```

175 \cs_set_protected:Npn \__enumext_tmp:n #1
176 {
177   \bool_new:c { \l__enumext_item_starred_#1_bool }
178   \int_new:c { \l__enumext_item_column_pos_#1_int }
179   \int_new:c { \g__enumext_item_count_all_#1_int }
180   \int_new:c { \l__enumext_joined_item_#1_int }
181   \int_new:c { \l__enumext_joined_item_aux_#1_int }
182   \int_new:c { \l__enumext_tmpa_#1_int }
183   \dim_new:c { \l__enumext_tmpa_#1_dim }
184   \box_new:c { \l__enumext_item_text_#1_box }
185   \dim_new:c { \l__enumext_joined_width_#1_dim }
186   \dim_new:c { \l__enumext_item_width_#1_dim }
187   \tl_new:c { \g__enumext_item_symbol_aux_#1_tl }
188   \str_new:c { \l__enumext_align_label_#1_str }
189   \bool_new:c { \g__enumext_minipage_active_#1_bool }
190   \box_new:c { \l__enumext_miniright_code_#1_box }
191   \bool_new:c { \g__enumext_minipage_center_#1_bool }
192   \dim_new:c { \g__enumext_minipage_right_#1_dim }
193   \skip_new:c { \g__enumext_minipage_right_#1_skip }
194 }
195 \clist_map_inline:nn { vii, viii } { \__enumext_tmp:n {#1} }

```

(End of definition for `\l__enumext_item_starred_X_bool` and others.)

```
\c__enumext_all_envs_clist
```

An internal `clist-var` variable to run with `\__enumext_tmp:n`.

```

196 \clist_const:Nn \c__enumext_all_envs_clist
197 {
198   {level-1}{i}, {level-2}{ii}, {level-3}{iii}, {level-4}{iv},
199   {keyans}{v}, {enumext*}{vii}, {keyans*}{viii}
200 }

```

(End of definition for `\c__enumext_all_envs_clist`.)

13.5 Some utility functions

```

\keys_precompile:neN
\seq_use:NV

```

Non-standard kernel variants used by the `\printkeyans` command (§13.50) and `\foreachkeyans` command (§13.53).

```

201 \cs_generate_variant:Nn \keys_precompile:nnN { neN }
202 \cs_generate_variant:Nn \seq_use:Nn { NV }

```

(End of definition for `\keys_precompile:neN` and `\seq_use:NV`.)

```
\__enumext_at_begin_document:n
```

A internal “hook” function used for copying plain `list` and `minipage` environments definition and `hyperref` detection.

```

203 \cs_new_protected:Npn \__enumext_at_begin_document:n #1
204 {
205   \hook_gput_code:nnn {begindocument} {enumext} { #1 }
206 }

```

(End of definition for `\__enumext_at_begin_document:n`.)

```

\__enumext_after_env:nn
\__enumext_before_env:nn

```

A internal “hook” functions for execute code `mini-right` and `mini-right*` keys outside the `enumext*` and `keyans*` environments and print check-ans outside the `enumext` and `enumext*` environments.

```

207 \cs_new_protected:Npn \__enumext_after_env:nn #1 #2
208 {
209   \hook_gput_code:nnn {env/#1/after} {enumext} {#2}
210 }
211 \cs_new_protected:Npn \__enumext_before_env:nn #1 #2
212 {
213   \hook_gput_code:nnn {env/#1/before} {enumext} {#2}
214 }

```

(End of definition for `\__enumext_after_env:nn` and `\__enumext_before_env:nn`.)

```
\__enumext_level:
```

Function for check current level in `enumext`.

```

215 \cs_new:Nn \__enumext_level:
216 {
217   \int_to_roman:n { \l__enumext_level_int }
218 }

```


(End of definition for `\__enumext_level:`.)

`\__enumext_if_is_int:nT` A conditional function to determine whether the variable we are passing is an integer used by `start` and `widest` keys. This function is taken directly from the answer given by Henri Menke in [How to test if an expl3 function argument is an integer expression?](#).

```

219 \prg_new_protected_conditional:Npnn \__enumext_if_is_int:n #1 { T, F, TF }
220 {
221   \regex_if_match:nnTF { ^[\+|-]?[\d]+$ } {#1} % $
222   { \prg_return_true: }
223   { \prg_return_false: }
224 }

```

(End of definition for `\__enumext_if_is_int:nT`, `\__enumext_if_is_int:nF`, and `\__enumext_if_is_int:nTF`.)

`\__enumext_show_length:nnn` Internal function used by `show-length` key to show “all lengths” calculated and used in `enumext`, `enumext*`, `keyans` and `keyans*` environments.

```

225 \cs_new:Npn \__enumext_show_length:nnn #1 #2 #3
226 {
227   *~#2
228   \prg_replicate:nn { 14 - \str_count:n {#2} } {~}
229   =~\use:c { #1_use:c } { \__enumext_#2_#3_#1 } \\
230 }

```

(End of definition for `\__enumext_show_length:nnn`.)

`\__enumext_unskip_unkern:` The function `\__enumext_unskip_unkern:` will remove the last `⟨skip⟩` or `⟨kern⟩` at execution time using the values `11` and `12` of `\lastnodetype` to apply `\unskip` or `\unkern` depending on the case.

```

231 \cs_new_protected:Nn \__enumext_unskip_unkern:
232 {
233   \int_case:nnT { \lastnodetype }
234   {
235     { 11 } { \unskip }
236     { 12 } { \unkern }
237   }
238 }

```

(End of definition for `\__enumext_unskip_unkern:`.)

13.5.1 Utilities for environments and levels

`\__enumext_is_not_nested:` The function `\__enumext_is_not_nested:` sets the variables `\g__enumext_standar_bool` and `\g__enumext_starred_bool` to “true” only if the environments `enumext` and `enumext*` are NOT nested in each other and saves the environment name in `\l__enumext_envir_name_tl`.

```

239 \cs_new_protected:Nn \__enumext_is_not_nested:
240 {
241   \str_case:en { \@currenvir }
242   {
243     {enumext}
244     {
245       \tl_set:Nn \l__enumext_envir_name_tl { enumext }
246       \bool_lazy_and:nnT
247       { \bool_not_p:n { \g__enumext_standar_bool } }
248       { \int_compare_p:nNn { \l__enumext_level_h_int } = { 0 } }
249       {
250         \bool_gset_true:N \g__enumext_standar_bool
251       }
252     }
253     {enumext*}
254     {
255       \tl_set:Nn \l__enumext_envir_name_tl { enumext* }
256       \bool_lazy_and:nnT
257       { \bool_not_p:n { \g__enumext_starred_bool } }
258       { \int_compare_p:nNn { \l__enumext_level_int } = { 0 } }
259       {
260         \bool_gset_true:N \g__enumext_starred_bool
261       }
262     }
263   }
264 }

```

The function `\__enumext_is_on_first_level:` will sets the variables `\l__enumext_standar_first_bool` (§13.29.1), `\l__enumext_starred_first_bool` (§13.29.1) to “true” only if the environment is not nested and we are in the “first level” of it . We will also save the *start line number* of each environment in the variable `\g__enumext_start_line_tl` and the *name* of each environment in the variable `\g__enumext_envir_name_tl` to use in messages related to the `check-ans` key and `.log` file.

```

265 \cs_new_protected:Nn \__enumext_is_on_first_level:
266 {
267   \bool_lazy_all:nT
268   {
269     { \bool_if_p:N \g__enumext_standar_bool }
270     { \int_compare_p:nNn { \l__enumext_level_int } = { 1 } }
271     { \int_compare_p:nNn { \l__enumext_level_h_int } = { 0 } }
272   }
273   {
274     \bool_set_true:N \l__enumext_standar_first_bool
275     \tl_gset:Nn \g__enumext_envir_name_tl { enumext }
276     \tl_gset:Ne \g__enumext_start_line_tl
277       {
278         on~line~\exp_not:V \inputlineno
279       }
280   }
281   \bool_lazy_all:nT
282   {
283     { \bool_if_p:N \g__enumext_starred_bool }
284     { \int_compare_p:nNn { \l__enumext_level_h_int } = { 1 } }
285     { \int_compare_p:nNn { \l__enumext_level_int } = { 0 } }
286   }
287   {
288     \bool_set_true:N \l__enumext_starred_first_bool
289     \tl_gset:Nn \g__enumext_envir_name_tl { enumext* }
290     \tl_gset:Ne \g__enumext_start_line_tl
291       {
292         on~line~\exp_not:V \inputlineno
293       }
294   }
295 }

```

(End of definition for `\__enumext_is_not_nested:` and `\__enumext_is_on_first_level:`.)

`\__enumext_keyans_name_and_start:`

The function `\__enumext_keyans_name_and_start:` will save the start line number and name of the environments `keyans`, `keyans*` and `keyanspic` in the variables `\l__enumext_check_start_line_env_tl` and `\l__enumext_envir_name_tl` to use in the `\__enumext_check_starred_cmd:n` function.

```

296 \cs_new_protected:Nn \__enumext_keyans_name_and_start:
297 {
298   \str_case:en { \@currentenvir }
299   {
300     {keyans}
301     {
302       \tl_set:Nn \l__enumext_envir_name_tl { keyans }
303       \tl_set:Ne \l__enumext_check_start_line_env_tl
304         {
305           in~'keyans'~start~on~line~\exp_not:V \inputlineno
306         }
307     }
308     {keyans*}
309     {
310       \tl_set:Nn \l__enumext_envir_name_tl { keyans* }
311       \tl_set:Ne \l__enumext_check_start_line_env_tl
312         {
313           in~'keyans*'~start~on~line~\exp_not:V \inputlineno
314         }
315     }
316     {keyanspic}
317     {
318       \tl_set:Nn \l__enumext_envir_name_tl { keyanspic }
319       \tl_set:Ne \l__enumext_check_start_line_env_tl
320         {
321           in~'keyanspic'~start~on~line~\exp_not:V \inputlineno
322         }
323     }
324   }

```

```

324     }
325 }

```

(End of definition for `\__enumext_keyans_name_and_start:`)

13.5.2 Utilities for log and terminal

The function `\__enumext_reset_global_vars:` will be passed to the function `\__enumext_execute_after_env:` and will return the global variables to their default values after being used.

```

326 \cs_new_protected:Nn \__enumext_reset_global_vars:
327 {
328     \__enumext_reset_global_int:
329     \__enumext_reset_global_bool:
330     \__enumext_reset_global_tl:
331 }
332 \cs_new_protected:Nn \__enumext_reset_global_int:
333 {
334     \int_gzero:N \g__enumext_item_number_int
335     \int_gzero:N \g__enumext_item_anskey_int
336     \int_gzero:N \g__enumext_item_answer_diff_int
337 }
338 \cs_new_protected:Nn \__enumext_reset_global_bool:
339 {
340     \bool_gset_false:N \g__enumext_check_ans_key_bool
341     \bool_gset_false:N \g__enumext_standar_bool
342     \bool_gset_false:N \g__enumext_starred_bool
343 }
344 \cs_new_protected:Nn \__enumext_reset_global_tl:
345 {
346     \tl_gclear:N \g__enumext_store_name_tl
347     \tl_gclear:N \g__enumext_start_line_tl
348     \tl_gclear:N \g__enumext_envir_name_tl
349 }

```

(End of definition for `\__enumext_reset_global_vars:` and others.)

The function `\__enumext_log_global_vars:` will be passed to the function `\__enumext_execute_after_env:` and write to the `.log` file the number of elements saved in the *prop list* and *sequence* created by the `save-ans` key along with the value of the integer variable created for the `resume` key.

```

350 \cs_new_protected:Nn \__enumext_log_global_vars:
351 {
352     \msg_log:nneeee { enumext } { prop-seq-int-hook }
353     { \g__enumext_store_name_tl }
354     { \prop_count:c { g__enumext_ \g__enumext_store_name_tl _prop } }
355     { \seq_count:c { g__enumext_ \g__enumext_store_name_tl _seq } }
356     { \int_use:c { g__enumext_resume_ \g__enumext_store_name_tl _int } }
357 }

```

The function `\__enumext_log_answer_vars:` will be passed to the function `\__enumext_execute_after_env:` and write to the `.log` file the number of items and answers along with the difference between them.

```

358 \cs_new_protected:Nn \__enumext_log_answer_vars:
359 {
360     \msg_log:nneeee { enumext } { item-answer-hook }
361     { \int_use:N \g__enumext_item_number_int }
362     { \int_use:N \g__enumext_item_anskey_int }
363     { \int_eval:n { \g__enumext_item_number_int - \g__enumext_item_anskey_int } }
364 }

```

(End of definition for `\__enumext_log_global_vars:` and `\__enumext_log_answer_vars:`)

13.6 Copying list and minipage environments

The `list` environment provided by $\TeX$ has the following plain form:

```

\list{⟨arg one⟩}{⟨arg two⟩}
  \item[⟨opt⟩]
\endlist

```

And `minipage` environment provided by $\TeX$ has the following (simplified) plain form:

```

\minipage[⟨pos⟩][⟨height⟩][⟨inner-pos⟩]{⟨width⟩}
  ⟨internal implement⟩
\endminipage

```

As a precaution we copy them using `\__enumext_at_begin_document:n` in case any package redefines the `\list` environment or a related command.

For compatibility with *tagged* PDF we should use `\NewCommandCopy` and not `\cs_new_eq:NN` for `\item`. When *tagged* PDF is active `\item` is redefined using `\ltxcmd` (see `latex-lab-block`[19]).

```
\__enumext_start_list:nn
  \__enumext_stop_list:
  \__enumext_item_std:w
  \__enumext_minipage:w
  \__enumext_endminipage:
```

The functions `\__enumext_start_list:nn` and `\__enumext_stop_list:` correspond to copies of `\list` and `\endlist` from plain definition of `\list` environment, the function `\__enumext_item_std:w` is a copy of the `\item` command.

```
365 \__enumext_at_begin_document:n
366 {
367   \cs_new_eq:NN \__enumext_start_list:nn \list
368   \cs_new_eq:NN \__enumext_stop_list: \endlist
369   \NewCommandCopy \__enumext_item_std:w \item
370 }
```

The functions `\__enumext_minipage:w` and `\__enumext_endminipage:` correspond to copies of `\minipage` and `\endminipage` from plain definition of `\minipage` environment.

```
371 \__enumext_at_begin_document:n
372 {
373   \cs_new_eq:NN \__enumext_minipage:w \minipage
374   \cs_new_eq:NN \__enumext_endminipage: \endminipage
375 }
```

(End of definition for `\__enumext_start_list:nn` and others.)

13.7 Compatibility with hyperref and footnotehyper

First we define the necessary rules using “hooks” to determine if the `hyperref` package is loaded.

```
\__enumext_after_hyperref:
  \__enumext_hypertarget:nn
  \__enumext_phantomsection:
```

```
376 \hook_gput_code:nnn { begindocument } { enumext } { \__enumext_after_hyperref: }
377 \hook_gset_rule:nnnn { begindocument } { enumext } { after } { hyperref }
```

The function `\__enumext_after_hyperref:` sets the state of the boolean variable `\l__enumext_hyperref_bool` to “true” if the package is loaded. At this point we will use the public macro `\IfHyperBoolean` to determine if the `hyperfootnotes=true` key is present, if so, we set the state of the boolean variable `\__enumext_footnotes_key_bool` to “true”.

```
378 \cs_new_protected:Nn \__enumext_after_hyperref:
379 {
380   \IfPackageLoadedT { hyperref }
381   {
382     \msg_info:nnn { enumext } { package-load } { hyperref }
383     \bool_set_true:N \l__enumext_hyperref_bool
384     \IfHyperBoolean{hyperfootnotes}
385     {
386       \bool_set_true:N \l__enumext_footnotes_key_bool
387     }
388   }
389 }
```

If the state of the variable `\l__enumext_footnotes_key_bool` is true we will check if the package `footnotehyper` is loaded, in case it is not present, we will set the value of `\l__enumext_footnotes_key_bool` to false and we will redefine `\footnote`.

```
390 \bool_if:NT \l__enumext_footnotes_key_bool
391 {
392   \IfPackageLoadedTF { footnotehyper }
393   {
394     \msg_info:nnn { enumext } { package-load } { footnotehyper }
395   }
396   {
397     \bool_set_false:N \l__enumext_footnotes_key_bool
398   }
399 }
```

The functions `\__enumext_hypertarget:nn` and `\__enumext_phantomsection:` correspond to the internal copies of `\hypertarget` and `\phantomsection`. If the boolean variable `\l__enumext_hyperref_bool` is false the functions `\__enumext_hypertarget:nn` and `\__enumext_phantomsection:` will be disabled.

```
400 \bool_if:NTF \l__enumext_hyperref_bool
401 {
402   \cs_new_eq:NN \__enumext_hypertarget:nn \hypertarget
403   \cs_new_eq:NN \__enumext_phantomsection: \phantomsection
404 }
405 {
406   \cs_new_eq:NN \__enumext_hypertarget:nn \use_none:nn
```

```

407         \cs_new_eq:NN \__enumext_phantomsection: \prg_do_nothing:
408     }
409 }

```

(End of definition for `\__enumext_after_hyperref:`, `\__enumext_hypertarget:nn`, and `\__enumext_phantomsection:`.)

`\__enumext_newlabel:nn` The function `\__enumext_newlabel:nn` write the information to the `.aux` file when using the `save-ref` key. The arguments taken by the function are:

#1: `\l__enumext_newlabel_arg_one_tl`

#2: `\l__enumext_newlabel_arg_two_tl`

🔗 The trick here is to manage the number of arguments passed to `\newlabel{#1}{#2}` according to the presence of the `hyperref` package.

```

410 \cs_new_protected:Npn \__enumext_newlabel:nn #1 #2
411 {
412     \protected@write \@auxout { }
413     {
414         \token_to_str:N \newlabel {#1}
415         {
416             {#2}
417             \bool_if:NT \__enumext_hyperref_bool
418             { { \thepage } {#2} {#1} }
419             { }
420         }
421     }
422     \__enumext_hypertarget:nn {#1} { }
423     \__enumext_phantomsection:
424 }

```

(End of definition for `\__enumext_newlabel:nn`.)

13.8 Internal redefining `\footnote` command

To keep the correct numbering of `\footnote` and to make it work correctly in the `enumext*` and `keyans*` environments and `mini-env` key it is necessary to redefine the `\footnote` command. This implementation is adapted from the answer given by Clea F. Rees (@cfr) in [footnotes in boxes compatible with hyperref](#).

`\__enumext_footnotetext:nn` `\__enumext_renew_footnote:` `\__enumext_print_footnote:` `\__enumext_renew_footnote_mini:` `\__enumext_print_footnote_mini:` Redefinition of the `\footnote` command using `\footnotetext` and `\footnotemark` for the `mini-env` key in the `enumext` and `keyans` environments.

```

425 \cs_new_protected:Nn \__enumext_footnotetext:nn
426 {
427     \footnotetext[#1]{#2}
428 }
429 \cs_new_protected:Nn \__enumext_renew_footnote:
430 {
431     \RenewDocumentCommand \footnote { o +m }
432     {
433         \tl_if_novalue:nTF {##1}
434         {
435             \stepcounter{footnote}
436             \int_gset_eq:Nc \g__enumext_footnote_standar_int { c@footnote }
437         }
438         {
439             \int_gset:Nn \g__enumext_footnote_standar_int { ##1 }
440         }
441         \footnotemark [ \g__enumext_footnote_standar_int ]
442         \seq_gput_right:Nn \g__enumext_footnote_standar_arg_seq { ##2 }
443         \seq_gput_right:NV
444             \g__enumext_footnote_standar_int_seq \g__enumext_footnote_standar_int
445     }
446 }
447 \cs_new_protected:Nn \__enumext_print_footnote:
448 {
449     \seq_if_empty:NF \g__enumext_footnote_standar_int_seq
450     {
451         \seq_map_pairwise_function:NNN
452             \g__enumext_footnote_standar_int_seq
453             \g__enumext_footnote_standar_arg_seq
454             \__enumext_footnotetext:nn
455     }
456     \seq_gclear:N \g__enumext_footnote_standar_arg_seq
457     \seq_gclear:N \g__enumext_footnote_standar_int_seq
458 }

```

The `enumext*` and `keyans*` environments are implemented using `minipage` so we must also redefine `\footnote` to keep these numbering as if it were part of the document.

```

459 \cs_new_protected:Nn \__enumext_renew_footnote_mini:
460 {
461   \RenewDocumentCommand \footnote { o +m }
462   {
463     \tl_if_novalue:nTF {##1}
464     {
465       \stepcounter{footnote}
466       \int_gset_eq:Nc \g__enumext_footnote_starred_int { c@footnote }
467     }
468     {
469       \int_gset:Nn \g__enumext_footnote_starred_int { ##1 }
470     }
471     \footnotemark [ \g__enumext_footnote_starred_int ]
472     \seq_gput_right:Nn \g__enumext_footnote_starred_arg_seq { ##2 }
473     \seq_gput_right:NV
474       \g__enumext_footnote_starred_int_seq \g__enumext_footnote_starred_int
475   }
476 }
477 \cs_new_protected:Nn \__enumext_print_footnote_mini:
478 {
479   \seq_if_empty:NF \g__enumext_footnote_starred_int_seq
480   {
481     \seq_map_pairwise_function:NNN
482       \g__enumext_footnote_starred_int_seq
483       \g__enumext_footnote_starred_arg_seq
484       \__enumext_footnotetext:nn
485   }
486   \seq_gclear:N \g__enumext_footnote_starred_arg_seq
487   \seq_gclear:N \g__enumext_footnote_starred_int_seq
488 }

```

(End of definition for `\__enumext_footnotetext:nn` and others.)

```

\__enumext_renew_footnote_standar:
\__enumext_print_footnote_standar:
\__enumext_renew_footnote_starred:
\__enumext_print_footnote_starred:

```

We encapsulate the redefinition of `\footnote` to pass it to internal `__enumext_mini_page` environment used by the `mini-env` key in the `enumext` and `keyans` environments. We will run the redefinition when *tagged* PDF is active or when the `footnotehyper` package is not loaded.

```

489 \cs_new_protected:Nn \__enumext_renew_footnote_standar:
490 {
491   \bool_if:NT \g__enumext_standar_bool
492   {
493     \IfDocumentMetadataTF
494     {
495       \__enumext_renew_footnote:
496     }
497     {
498       \bool_if:NF \l__enumext_footnotes_key_bool
499       {
500         \__enumext_renew_footnote:
501       }
502     }
503   }
504 }
505 \cs_new_protected:Nn \__enumext_print_footnote_standar:
506 {
507   \bool_if:NT \g__enumext_standar_bool
508   {
509     \IfDocumentMetadataTF
510     {
511       \__enumext_print_footnote:
512     }
513     {
514       \bool_if:NF \l__enumext_footnotes_key_bool
515       {
516         \__enumext_print_footnote:
517       }
518     }
519   }
520 }

```


We encapsulate the redefinition of `\footnote` to pass it to the `enumext*` and `keyans*` environments. We will run the redefinition when *tagged* PDF is active or when the `footnotehyper` package is not loaded.

```

521 \cs_new_protected:Nn \__enumext_renew_footnote_starred:
522 {
523   \IfDocumentMetadataTF
524   {
525     \__enumext_renew_footnote_mini:
526   }
527   {
528     \bool_if:NF \l__enumext_footnotes_key_bool
529     {
530       \__enumext_renew_footnote_mini:
531     }
532   }
533 }
534 \cs_new_protected:Nn \__enumext_print_footnote_starred:
535 {
536   \IfDocumentMetadataTF
537   {
538     \__enumext_print_footnote_mini:
539   }
540   {
541     \bool_if:NF \l__enumext_footnotes_key_bool
542     {
543       \__enumext_print_footnote_mini:
544     }
545   }
546 }

```

In `enumext*` and `keyans*` environments we need to use “hooks” to print `\footnote` with support for *tagged* PDF.

```

547 \__enumext_after_env:nn { enumext* }
548 {
549   \__enumext_print_footnote_starred:
550 }
551 \__enumext_after_env:nn { keyans* }
552 {
553   \__enumext_print_footnote_starred:
554 }

```

(End of definition for `\__enumext_renew_footnote_standar:` and others.)

13.9 The internal minipage environment

```

\__enumext_internal_mini_page:
__enumext_mini_env*

```

The function `\__enumext_internal_mini_page:` creates a internal `\__enumext_mini_page` environment (*custom version* of `minipage`) setting the `\if@minipage` switch to “false” to allow spaces at the “above” of the environment, plus we will add `\skip_vertical:N \c_zero_skip` to maintain alignment on “top” in the first part and `\skip_vertical:N \c_zero_skip` in the second part to allow spaces “below”. This environment will be used internally by the `mini-env` key, it is NOT documented in the user interface and is for internal use only. Within this environment we redefine `\footnote` to make them look the same as if they were elsewhere in the document. This implementation is adapted from the answer given by Max Chernoff (@MaxChernoff) in [Customize minipage to support spaces below it](#).

- This function is passed to the function `\__enumext_safe_exec:` in the `enumext` environment definition (§13.42) and `\__enumext_safe_exec_vii:` in the `enumext*` environment definition (§13.47).

```

555 \cs_new_protected:Nn \__enumext_internal_mini_page:
556 {
557   \int_compare:nNnT { \l__enumext_level_int } = { 0 }
558   {
559     \DeclareDocumentEnvironment{__enumext_mini_page}{ m }
560     {
561       \__enumext_renew_footnote_standar:
562       \__enumext_minipage:w [ t ] { ##1 }
563       \legacy_if_gset_false:n { @minipage }
564       \skip_vertical:N \c_zero_skip
565     }
566     {
567       \skip_vertical:N \c_zero_skip
568       \__enumext_endminipage:
569       \__enumext_print_footnote_standar:
570     }
571   }

```

```
572 }
```

(End of definition for `\__enumext_internal_mini_page`: and `\__enumext_mini_env*`.)

13.10 Definition of public dimension

The package `enumext` only provides a single public dimension `\itemwidth` and is intended for user convenience only and is not for internal use as such. This dimension is set in all environments and is only used by the `wrap-ans` key at its default value.

```
573 \dim_zero_new:N \itemwidth
```

13.11 Definition of counters

To create the necessary “counters” we must first make sure that they are not already defined by the user or a package such as `enumitem`, otherwise a error will be returned and the package loading will be aborted. The arguments taken by the function are:

- #1**: A token list `\__enumext_counter_X_tl` for “store” the counter’s name.
#2: The counter’s name.

```
enumXi
enumXii
enumXiii
enumXiv
enumXv
enumXvi
enumXvii
enumXviii
574 \cs_new_protected:Npn \__enumext_define_counter:Nn #1 #2
575 {
576   \cs_if_exist:cTF { c@ #2 }
577   { \msg_fatal:nnn { enumext } { counters }{ #2 } }
578   {
579     \tl_set:Nn #1 { #2 }
580     \newcounter { #2 }
581   }
582 }
```

The counters created here are `enumXi`, `enumXii`, `enumXiii` and `enumXiv` for `enumext` environment, `enumXv` for `keyans` environment, `enumXvi` for `keyanspic` environment, `enumXvii` for `enumext*` and `enumXviii` for the `keyans*` environments.

```
583 \__enumext_define_counter:Nn \__enumext_counter_i_tl { enumXi }
584 \__enumext_define_counter:Nn \__enumext_counter_ii_tl { enumXii }
585 \__enumext_define_counter:Nn \__enumext_counter_iii_tl { enumXiii }
586 \__enumext_define_counter:Nn \__enumext_counter_iv_tl { enumXiv }
587 \__enumext_define_counter:Nn \__enumext_counter_v_tl { enumXv }
588 \__enumext_define_counter:Nn \__enumext_counter_vi_tl { enumXvi }
589 \__enumext_define_counter:Nn \__enumext_counter_vii_tl { enumXvii }
590 \__enumext_define_counter:Nn \__enumext_counter_viii_tl { enumXviii }
```

(End of definition for `\__enumext_define_counter:Nn` and others.)

In version 1.6 the command `\resetenumext` (§13.27) was added which internally uses `\counterwithin*` so for its correct operation, we will create “real counters” instead of the “integer variables” for the keys `resume` and `resume*`.

```
\c@__enumext_resume_i_int
\c@__enumext_resume_ii_int
\c@__enumext_resume_iii_int
\c@__enumext_resume_iv_int
\c@__enumext_resume_vii_int
591 \cs_set_protected:Npn \__enumext_tmp:n #1
592 {
593   \cs_if_exist:cTF { c@ __enumext_resume_#1_int }
594   { \msg_fatal:nne { enumext } { counters }{ __enumext_resume_#1_int } }
595   {
596     \newcounter { __enumext_resume_#1_int }
597   }
598 }
599 \clist_map_inline:nn {i,ii,iii,iv,vii} { \__enumext_tmp:n {#1} }
```

(End of definition for `\c@__enumext_resume_i_int` and others.)

13.12 Definition of labels

This part of the code is inspired by the `enumitem` package. The idea is to be able to access the counters using `\arabic*`, `\Alph*`, `\alph*`, `\Roman*` and `\roman*` to use them in the `label` key.

- Direct support for this is provided since $\text{\LaTeX}$ release 2025-06-01[13], but we will keep the original implementation so as not to hinder the internal “label and ref” system.

These `\counters` will be used as default `\labels` if the `label` key is not used for the different levels of the `enumext`, `enumext*`, `keyans` and `keyans*` environments, so it is necessary to get a default value for `labelwidth` from these `\labels` at the same time.

```
600 \cs_new_protected:Npn \__enumext_register_default_label_wd:Nn #1 #2
601 {
602   \tl_const:cn { c__enumext_widest_ \cs_to_str:N #1 _tl } {#2}
603   \tl_gput_right:Nn \g__enumext_counter_styles_tl {#1}
```

```

604 }
605 \__enumext_register_default_label_wd:Nn \arabic { 0 }
606 \__enumext_register_default_label_wd:Nn \Alph { M }
607 \__enumext_register_default_label_wd:Nn \alph { m }
608 \__enumext_register_default_label_wd:Nn \Roman { VIII }
609 \__enumext_register_default_label_wd:Nn \roman { viii }

```

(End of definition for __enumext_register_default_label_wd:Nn.)

```

\__enumext_label_width_by_box:Nn
\__enumext_label_width_by_box:cv

```

The function __enumext_label_width_by_box:Nn set the default \labelwidth using a box width if no \labelwidth key is passed.

```

610 \cs_new_protected:Npn \__enumext_label_width_by_box:Nn #1 #2
611 {
612   \hbox_set:Nn \l__enumext_label_width_by_box {#2}
613   \dim_set:Nn #1 { \box_wd:N \l__enumext_label_width_by_box }
614 }
615 \cs_generate_variant:Nn \__enumext_label_width_by_box:Nn { cv }

```

(End of definition for __enumext_label_width_by_box:Nn.)

```

\__enumext_label_style:Nnn
\__enumext_label_style:cvn

```

The function __enumext_label_style:Nnn is used by the \label key to creates the variables containing the *<label style>* and will allow to use \arabic*, \Alph*, \alph*, \Roman* and \roman* as arguments. It loops through the defined counter styles in \g__enumext_counter_styles_tl (\arabic, \alph, \Alph, \roman and \Roman) for example, looking for \roman* and replacing that by \roman{<counter>}, and doing the same for the \g__enumext_widest_label_tl to keep both in sync.

```

616 \cs_new_protected:Npn \__enumext_label_style:Nnn #1 #2 #3
617 {
618   \tl_clear_new:N #1
619   \tl_put_right:Ne #1 { \tl_trim_spaces:n {#3} }
620   \tl_gset_eq:NN \g__enumext_widest_label_tl #1
621   \tl_map_inline:Nn \g__enumext_counter_styles_tl
622   {
623     \tl_replace_all:Nne #1 { ##1* } { \exp_not:N ##1 {#2} }
624     \tl_greplace_all:Nne \g__enumext_widest_label_tl { ##1* }
625     { \tl_use:c { c__enumext_widest_ \cs_to_str:N ##1 _tl } }
626   }
627   \__enumext_label_width_by_box:Nn \l__enumext_current_widest_dim
628   { \tl_use:N \g__enumext_widest_label_tl }
629   \tl_set_eq:cN { the #2 } #1
630 }
631 \cs_generate_variant:Nn \__enumext_label_style:Nnn { cvn }

```

(End of definition for __enumext_label_style:Nnn.)

13.13 Setting keys associated with label

When *tagged* PDF is active \makeLabel is redefined using \makebox to work correctly (§13.37). From the user side it is convenient to have a key that allows using this redefinition with \makebox without having \IfDocumentMetadataTF active.

mode-box

We define the key `mode-box` only for the “first level” of `enumext` and `enumext*` environments.

```

632 \cs_set_protected:Npn \__enumext_tmp:n #1
633 {
634   \keys_define:nn { enumext / #1 }
635   {
636     mode-box .bool_set:N = \l__enumext_mode_box_bool,
637     mode-box .initial:n = false,
638     mode-box .value_forbidden:n = true,
639   }
640 }
641 \clist_map_inline:nn { level-1, enumext* } { \__enumext_tmp:n {#1} }

```

(End of definition for mode-box.)

```

font
labelsep
labelwidth
wrap-label
wrap-label*

```

Definition of keys `font`, `labelsep`, `labelwidth`, `wrap-label` and `wrap-label*` keys for `enumext` and `keyans` environments.

```

642 \cs_set_protected:Npn \__enumext_tmp:nn #1 #2
643 {
644   \keys_define:nn { enumext / #1 }
645   {
646     font .tl_set:c = { \__enumext_label_font_style_#2_tl },

```

```

647     font .value_required:n = true,
648     labelsep .dim_set:c = { l__enumext_labelsep_#2_dim },
649     labelsep .initial:n = {0.3333em},
650     labelsep .value_required:n = true,
651     labelwidth .dim_set:c = { l__enumext_labelwidth_#2_dim },
652     labelwidth .value_required:n = true,
653     wrap-label .cs_set_protected:cp = { __enumext_wrapper_label_#2:n } ##1,
654     wrap-label .initial:n = {##1},
655     wrap-label .value_required:n = true,
656     wrap-label* .code:n = {
657         \bool_set_true:c { l__enumext_wrap_label_opt_#2_bool }
658         \keys_set:nn { enumext / #1 } { wrap-label = {##1} }
659     },
660     wrap-label* .value_required:n = true,
661 }
662 }
663 \clist_map_inline:Nn \c__enumext_all_envs_clist { \__enumext_tmp:nn #1 }

```

(End of definition for font and others.)

align The align key is implemented differently for “starred” and “non starred” environments. For compatibility with tagged PDF we must set \l__enumext_align_label_pos_X_str.

```

664 \cs_set_protected:Npn \__enumext_tmp:nn #1 #2
665 {
666     \keys_define:nn { enumext / #1 }
667     {
668         align .choice:,
669         align / left .code:n =
670             {
671                 \tl_clear:c { l__enumext_label_fill_left_#2_tl }
672                 \tl_set:cn { l__enumext_label_fill_right_#2_tl } { \hfill }
673                 \str_set:cn { l__enumext_align_label_pos_#2_str } { l }
674             },
675         align / right .code:n =
676             {
677                 \tl_set:cn { l__enumext_label_fill_left_#2_tl } { \hfill }
678                 \tl_clear:c { l__enumext_label_fill_right_#2_tl }
679                 \str_set:cn { l__enumext_align_label_pos_#2_str } { r }
680             },
681         align / center .code:n =
682             {
683                 \tl_set:cn { l__enumext_label_fill_left_#2_tl } { \hfill }
684                 \tl_set:cn { l__enumext_label_fill_right_#2_tl } { \hfill }
685                 \str_set:cn { l__enumext_align_label_pos_#2_str } { c }
686             },
687         align / unknown .code:n =
688             \msg_error:nneee { enumext } { unknown-choice }
689             { align } { left,~right,~ center } { \exp_not:n {##1} },
690         align .initial:n = left,
691         align .value_required:n = true,
692     }
693 }
694 \clist_map_inline:nn
695 {
696     {level-1}{i}, {level-2}{ii}, {level-3}{iii}, {level-4}{iv}, {keyans}{v}
697 }
698 { \__enumext_tmp:nn #1 }
699 \cs_set_protected:Npn \__enumext_tmp:nn #1 #2
700 {
701     \keys_define:nn { enumext / #1 }
702     {
703         align .choice:,
704         align / left .code:n = \str_set:cn { l__enumext_align_label_#2_str } { l },
705         align / right .code:n = \str_set:cn { l__enumext_align_label_#2_str } { r },
706         align / center .code:n = \str_set:cn { l__enumext_align_label_#2_str } { c },
707         align / unknown .code:n =
708             \msg_error:nneee { enumext } { unknown-choice }
709             { align } { left,~right,~ center } { \exp_not:n {##1} },
710         align .initial:n = left,
711         align .value_required:n = true,

```

```

712     }
713   }
714   \clist_map_inline:nn { {enumext*}{vii}, {keyans*}{viii} } { \__enumext_tmp:nn #1 }

```

(End of definition for `align`.)

13.14 Setting label and ref keys

The implementation of the keys `label` and `ref` are part of the core of the package `enumext`, here the default values for `\label`, the value of the variables `\l__enumext_label_X_tl`, the default values for `\labelwidth` and the “*label and ref*” system.

13.14.1 Define and set label and ref keys for enumext environment

Here we set the default `\labels` of the *four levels* of `enumext` environment, along with the default values for `labelwidth`, `widest` and `ref` keys.

```

label
ref
\__enumext_label_i_tl 715 \cs_set_protected:Npn \__enumext_tmp:nnn #1 #2 #3
\__enumext_label_ii_tl 716 {
\__enumext_label_iii_tl 717   \keys_define:nn { enumext / #1 }
\__enumext_label_iv_tl 718   {
719     label .code:n = {
720       \__enumext_label_style:cvn { \__enumext_label_#2_tl }
721       { \__enumext_counter_#2_tl } {##1}
722       \dim_set_eq:cN { \__enumext_labelwidth_#2_dim }
723       \__enumext_current_widest_dim
724     },
725     label .initial:n = #3,
726     label .value_required:n = true,
727     ref .code:n = \__enumext_standar_ref:nn {#2} {##1},
728     ref .value_required:n = true,
729   }
730 }
731 \__enumext_tmp:nnn { level-1 } { i } { \arabic*. }
732 \__enumext_tmp:nnn { level-2 } { ii } { (\alph*. ) }
733 \__enumext_tmp:nnn { level-3 } { iii } { \roman*. }
734 \__enumext_tmp:nnn { level-4 } { iv } { \Alph*. }

```

(End of definition for `label` and others.)

`\__enumext_standar_ref:nn` The `\__enumext_standar_ref:nn` function will first pass the key *argument* `ref` to the variable `\l__enumext_ref_key_arg_tl` and analyze its state, if it is not *empty* it will set a copy of the *current counter style* save in `\l__enumext_the_counter_X_tl` to `\l__enumext_ref_the_count_tl` and then set the variable `\l__enumext_renew_counter_X_tl` which will modify `\theenumX`.

```

735 \cs_new_protected:Npn \__enumext_standar_ref:nn #1 #2
736 {
737   \tl_set:Nn \l__enumext_ref_key_arg_tl {#2}
738   \tl_if_empty:NTF \l__enumext_ref_key_arg_tl
739   {
740     \msg_error:nnn { enumext } { key-ref-empty } { enumext }
741   }
742   {
743     \tl_set_eq:Nc \l__enumext_ref_the_count_tl { \__enumext_the_counter_#1_tl }
744     \tl_set:ce { \__enumext_renew_counter_#1_tl }
745     {
746       \exp_not:N \renewcommand { \exp_not:V \l__enumext_ref_the_count_tl }
747       { \exp_not:V \l__enumext_ref_key_arg_tl }
748     }
749   }
750 }

```

Finally the function `\__enumext_standar_ref:` will execute the modification for the reference system in the second argument of the environment definition `enumext`.

```

751 \cs_new_protected:Npn \__enumext_standar_ref:
752 {
753   \tl_if_empty:cF { \__enumext_renew_counter_ \__enumext_level: _tl }
754   {
755     \tl_use:c { \__enumext_renew_counter_ \__enumext_level: _tl }
756   }
757 }

```

(End of definition for `\__enumext_standar_ref:nn` and `\__enumext_standar_ref:`.)

13.14.2 Define and set label and ref keys for enumext* and keyans* environments

label Here we set the default $\langle labels \rangle$ for `enumext*` and `keyans*` environments, along with the default value for
ref `labelwidth` key and `ref` key.

```

\l__enumext_label_vii_tl 758 \cs_set_protected:Npn \l__enumext_tmp:nnn #1 #2 #3
\l__enumext_label_viii_tl 759 {
760   \keys_define:nn { enumext / #1 }
761   {
762     label .code:n = {
763       \__enumext_label_style:cvn { \l__enumext_label_#2_tl }
764       { \l__enumext_counter_#2_tl } {##1}
765       \dim_set_eq:cN { \l__enumext_labelwidth_#2_dim }
766       \l__enumext_current_widest_dim
767     },
768     label .initial:n = #3,
769     label .value_required:n = true,
770     ref .code:n = \l__enumext_starred_ref:n {##1},
771     ref .value_required:n = true,
772   }
773 }
774 \l__enumext_tmp:nnn { enumext* } { vii } { \arabic*.}
775 \l__enumext_tmp:nnn { keyans* } { viii } { \Alph*.}

```

(End of definition for label and others.)

__enumext_starred_ref:n The implementation of `\__enumext_starred_ref:n` is the same as that used for the environment `enumext`.
__enumext_starred_ref:

```

776 \cs_new_protected:Npn \__enumext_starred_ref:n #1
777 {
778   \tl_set:Nn \l__enumext_ref_key_arg_tl {#1}
779   \int_compare:nNnT { \l__enumext_level_h_int } = { 1 }
780   {
781     \tl_if_empty:NTF \l__enumext_ref_key_arg_tl
782     {
783       \msg_error:nnn { enumext } { key-ref-empty } { enumext* }
784     }
785     {
786       \tl_set_eq:NN \l__enumext_ref_the_count_tl \l__enumext_the_counter_vii_tl
787       \tl_set:Ne \l__enumext_renew_counter_vii_tl
788       {
789         \exp_not:N \renewcommand { \exp_not:V \l__enumext_ref_the_count_tl } { \exp_not:V
790       }
791     }
792   }
793   \int_compare:nNnT { \l__enumext_keyans_level_h_int } = { 1 }
794   {
795     \tl_if_empty:NTF \l__enumext_ref_key_arg_tl
796     {
797       \msg_error:nnn { enumext } { key-ref-empty } { keyans* }
798     }
799     {
800       \tl_set_eq:NN \l__enumext_ref_the_count_tl \l__enumext_the_counter_viii_tl
801       \tl_set:Ne \l__enumext_renew_counter_viii_tl
802       {
803         \exp_not:N \renewcommand { \exp_not:V \l__enumext_ref_the_count_tl } { \exp_not:V
804       }
805     }
806   }
807 }

```

Finally the function `\__enumext_starred_ref:` will execute the modification for the reference system in the second argument of the `enumext*` and `keyans*` environment definition.

```

808 \cs_new_protected:Nn \__enumext_starred_ref:
809 {
810   \int_compare:nNnT { \l__enumext_level_h_int } = { 1 }
811   {
812     \tl_if_empty:NF \l__enumext_renew_counter_vii_tl
813     {
814       \tl_use:N \l__enumext_renew_counter_vii_tl
815     }
816   }
817   \int_compare:nNnT { \l__enumext_keyans_level_h_int } = { 1 }
818   {

```



```

819         \tl_if_empty:NF \l__enumext_renew_counter_viii_tl
820         {
821             \tl_use:N \l__enumext_renew_counter_viii_tl
822         }
823     }
824 }

```

(End of definition for `\__enumext_starred_ref:n` and `\__enumext_starred_ref:.`)

13.14.3 Define and set `label` and `ref` keys for `keyans` and `keyanspic` environments

Here we set the default `<label>` for `keyans` and `keyanspic` environment, along with the default value for `labelwidth` if it has not been established and `ref` key. The `keyanspic` environment use the same `<label>` as the `keyans` environment.

```

\__enumext_label_v_tl
\__enumext_label_vi_tl
825 \keys_define:nn { enumext / keyans }
826 {
827     label .code:n = {
828         \__enumext_label_style:cnv { \__enumext_label_v_tl }
829         { \__enumext_counter_v_tl } {#1}
830         \__enumext_label_style:cnv { \__enumext_label_vi_tl }
831         { \__enumext_counter_vi_tl } {#1}
832         \dim_set_eq:NN
833         \__enumext_labelwidth_v_dim \__enumext_current_widest_dim
834     },
835     label .initial:n = \Alph*,
836     label .value_required:n = true,
837     ref .code:n = \__enumext_keyans_ref:n {#1},
838     ref .value_required:n = true,
839 }

```

(End of definition for `label` and others.)

The implementation of `\__enumext_keyans_ref:n` is the same as that used for the environment `enumext`.

```

\__enumext_keyans_ref:n
\__enumext_keyans_ref:
840 \cs_new_protected:Npn \__enumext_keyans_ref:n #1
841 {
842     \tl_set:Nn \l__enumext_ref_key_arg_tl {#1}
843     \tl_if_empty:NTF \l__enumext_ref_key_arg_tl
844     {
845         \msg_error:nnn { enumext } { key-ref-empty } { keyans }
846     }
847     {
848         \tl_set_eq:NN \l__enumext_ref_the_count_tl \l__enumext_the_counter_v_tl
849         \tl_put_right:Ne \l__enumext_renew_counter_v_tl
850         {
851             \exp_not:N \renewcommand { \exp_not:V \l__enumext_ref_the_count_tl } { \exp_not:V \l__
852         }
853     }
854 }

```

Finally the function `\__enumext_keyans_ref:` will execute the modification for the reference system in the second argument of the `keyans*` environment definition.

```

855 \cs_new_protected:Nn \__enumext_keyans_ref:
856 {
857     \tl_if_empty:NF \l__enumext_renew_counter_v_tl
858     {
859         \tl_use:N \l__enumext_renew_counter_v_tl
860     }
861 }

```

(End of definition for `\__enumext_keyans_ref:n` and `\__enumext_keyans_ref:.`)

13.15 Setting `start`, `start*` and `widest` keys

The function `\__enumext_start_from:NNn` used by `start` and `start*` keys take three arguments:

```

\__enumext_start_from:ccn #1: \l__enumext_label_X_tl
\__enumext_start_from:cce #2: \l__enumext_start_X_int
\__enumext_start_from:cce #3: <integer or string>

```

The first argument of this function are the “*counter style*” set by `label` key, the second argument is returned by the function, the third argument can be an `<integer>` or `<string>` of the form `\Alph`, `\alph`, `\Roman` or `\roman`. This effectively allows `start*=A` or `start=1` to be used.

- In version 1.6 it is allowed to pass the `resume` key *without value* by means of the command `\setenumext`, for the correct operation of this we must set the boolean variable `\l__enumext_resume_count_bool` set by the `resume` key *without value* to “false” (§13.26). This is necessary to be able to “reset” the *start value* by means of the `start` or `start*` keys.

```

862 \cs_new_protected:Npn \__enumext_start_from:NNn #1 #2 #3
863 {
864   \bool_set_false:N \l__enumext_resume_count_bool
865   \__enumext_if_is_int:nTF { #3 }
866   {
867     \int_set:Nn #2 {#3}
868   }
869   {
870     \regex_if_match:nVT { \c{Alph} | \c{alph} } #1
871     { \int_set:Nn #2 { \int_from_alph:n {#3} } }
872     \regex_if_match:nVT { \c{Roman} | \c{roman} } #1
873     { \int_set:Nn #2 { \int_from_roman:n {#3} } }
874   }
875 }
876 \cs_generate_variant:Nn \__enumext_start_from:NNn { ccn, cce }

```

(End of definition for `\__enumext_start_from:NNn`.)

```

\__enumext_widest_from:nNNn
\__enumext_widest_from:nccn

```

The function `\__enumext_widest_from:nNNn` used by the `widest` key take four arguments:

- #1: The counter associated with the environment level
- #2: `\l__enumext_label_X_tl`
- #3: `\l__enumext_labelwidth_X_dim`
- #4: *integer or string*

The second and third arguments of this function are the values set by `label` and `labelwidth` keys, the four argument can be an *integer* or *string* of the form `\Alph`, `\alph`, `\Roman` or `\roman`. The value of the four argument is set temporarily for the identified counter in this point (level), then the value is expanded into a “box” and the “width” of the “box” is returned.

```

877 \cs_new_protected:Npn \__enumext_widest_from:nNNn #1 #2 #3 #4
878 {
879   \__enumext_if_is_int:nTF {#4}
880   {
881     \setcounter{enumX#1} { #4 }
882   }
883   {
884     \regex_if_match:nVT { \c{Alph} | \c{alph} } #2
885     { \setcounter{enumX#1} { \int_from_alph:n {#4} } }
886     \regex_if_match:nVT { \c{Alph} | \c{alph} } #2
887     { \setcounter{enumX#1} { \int_from_roman:n {#4} } }
888   }
889   \__enumext_label_width_by_box:cv
890   { \l__enumext_labelwidth_#1_dim } { \l__enumext_label_#1_tl }
891 }
892 \cs_generate_variant:Nn \__enumext_widest_from:nNNn { nccn }

```

(End of definition for `\__enumext_widest_from:nNNn`.)

Now define and set `start*`, `start` and `widest` keys for `enumext`, `enumext*`, `keyans` and `keyans*` environments.

`start*`

`widest`

```

893 \cs_set_protected:Npn \__enumext_tmp:nn #1 #2
894 {
895   \keys_define:nn { enumext / #1 }
896   {
897     start* .code:n = {
898       \__enumext_start_from:ccn
899       { \l__enumext_label_#2_tl }
900       { \l__enumext_start_#2_int } {##1}
901     },
902     start* .value_required:n = true,
903     start .code:n = {
904       \__enumext_start_from:cce
905       { \l__enumext_label_#2_tl }
906       { \l__enumext_start_#2_int } { \int_eval:n {##1} }
907     },
908     start .initial:n = 1,
909     start .value_required:n = true,
910     widest .code:n = {

```

```

911             \__enumext_widest_from:nccn {#2}
912             { \__enumext_label_#2_tl }
913             { \__enumext_labelwidth_#2_dim } {##1}
914         },
915         widest .value_required:n = true,
916     }
917 }
918 \clist_map_inline:Nn \c__enumext_all_envs_clist { \__enumext_tmp:nn #1 }

```

(End of definition for `start`, `start*`, and `widest`.)

13.16 Setting keys for penaltys

The three parameters `\@beginparpenalty`, `\@itempenalty` and `\@endparpenalty` work together to ensure that list environments look good, avoiding unsightly page breaks that can break the flow of the `list`, so it's a good idea to have a `<keys>` to access these.

```

919 \cs_set_protected:Npn \__enumext_tmp:nn #1 #2
920 {
921     \keys_define:nn { enumext / #1 }
922     {
923         beginpenalty .int_set:c = { \__enumext_beginparpenalty_#2_int },
924         beginpenalty .initial:n = -51,
925         beginpenalty .value_required:n = true,
926         midpenalty .int_set:c = { \__enumext_itempenalty_#2_int },
927         midpenalty .initial:n = -51,
928         midpenalty .value_required:n = true,
929         endpenalty .int_set:c = { \__enumext_endparpenalty_#2_int },
930         endpenalty .initial:n = -51,
931         endpenalty .value_required:n = true,
932     }
933 }
934 \clist_map_inline:Nn \c__enumext_all_envs_clist { \__enumext_tmp:nn #1 }

```

(End of definition for `beginpenalty`, `midpenalty`, and `endpenalty`.)

13.17 Setting keys for vertical spaces

Define and set `topsep`, `partopsep`, `parsep`, `itemsep`, `noitemsep` and `nosep` keys for `enumext`, `enumext*`, `keyans` and `keyans*` environments.

```

935 \cs_set_protected:Npn \__enumext_tmp:nnnnnn #1 #2 #3 #4 #5 #6
936 {
937     \keys_define:nn { enumext / #1 }
938     {
939         topsep .skip_set:c = { \__enumext_topsep_#2_skip },
940         topsep .initial:n = {#3},
941         topsep .value_required:n = true,
942         partopsep .skip_set:c = { \__enumext_partopsep_#2_skip },
943         partopsep .initial:n = {#4},
944         partopsep .value_required:n = true,
945         parsep .skip_set:c = { \__enumext_parsep_#2_skip },
946         parsep .initial:n = {#5},
947         parsep .value_required:n = true,
948         itemsep .skip_set:c = { \__enumext_itemsep_#2_skip },
949         itemsep .initial:n = {#6},
950         itemsep .value_required:n = true,
951         noitemsep .meta:n = { itemsep = 0pt, parsep = 0pt },
952         noitemsep .value_forbidden:n = true,
953         nosepp .meta:n = {
954             itemsep = 0pt, parsep = 0pt,
955             topsep = 0pt, partopsep = 0pt,
956         },
957         nosepp .value_forbidden:n = true,
958     }
959 }

```

Now we set the values based on standard `article` class in `10pt`.

```

960 \__enumext_tmp:nnnnnn { level-1 } { i } { 8.0pt plus 2.0pt minus 4.0pt }
961 { 2.0pt plus 1.0pt minus 1.0pt } { 4.0pt plus 2.0pt minus 1.0pt }
962 { 4.0pt plus 2.0pt minus 1.0pt }
963 \__enumext_tmp:nnnnnn { level-2 } { ii } { 4.0pt plus 2.0pt minus 1.0pt }
964 { 2.0pt plus 1.0pt minus 1.0pt } { 2.0pt plus 1.0pt minus 1.0pt }
965 { 2.0pt plus 1.0pt minus 1.0pt }

```

```

966 \__enumext_tmp:nnnnnn { level-3 } { iii } { 2.0pt plus 1.0pt minus 1.0pt }
967   { 1.0pt minus 1.0pt } { 0pt } { 2.0pt plus 1.0pt minus 1.0pt }
968 \__enumext_tmp:nnnnnn { level-4 } { iv } { 2.0pt plus 1.0pt minus 1.0pt }
969   { 1.0pt minus 1.0pt } { 0pt } { 2.0pt plus 1.0pt minus 1.0pt }
970 \__enumext_tmp:nnnnnn { keyans } { v } { 4.0pt plus 2.0pt minus 1.0pt }
971   { 2.0pt plus 1.0pt minus 1.0pt } { 2.0pt plus 1.0pt minus 1.0pt }
972   { 2.0pt plus 1.0pt minus 1.0pt }
973 \__enumext_tmp:nnnnnn { enumext* } { vii } { 8.0pt plus 2.0pt minus 4.0pt }
974   { 2.0pt plus 1.0pt minus 1.0pt } { 4.0pt plus 2.0pt minus 1.0pt }
975   { 4.0pt plus 2.0pt minus 1.0pt }
976 \__enumext_tmp:nnnnnn { keyans* } { viii } { 4.0pt plus 2.0pt minus 1.0pt }
977   { 2.0pt plus 1.0pt minus 1.0pt } { 2.0pt plus 1.0pt minus 1.0pt }
978   { 2.0pt plus 1.0pt minus 1.0pt }

```

(End of definition for *topsep* and others.)

13.18 Setting base-fix key

When nesting starting right after `\item` (without material between them) there is a problem with the alignment of the *baseline* between the two environments. One way to get around this problem is to place `\mode_leave_vertical`: apply `\vspace{-\baselineskip}` and set `\topsep=0pt` for the “first level” of the nested `enumext` environment.

`base-fix` We define the key `base-fix` only for the “first level” of `enumext` environment.

```

\__enumext_nested_base_line_fix:
979 \keys_define:nn { enumext / level-1 }
980   {
981     base-fix .bool_set:N = \__enumext_base_line_fix_bool,
982     base-fix .initial:n = false,
983     base-fix .value_forbidden:n = true,
984   }

```

The function `\__enumext_nested_base_line_fix`: passed to the `\__enumext_parse_keys:n` function in the definition of the `enumext` environment (§13.42) will be responsible for applying the *baseline correction* and adjusting the `\keys` for the `enumext` environment and the `\printkeyans` with *starred argument* ‘*’ (§13.50).

We will first implement the function code from the user side of the `base-fix` key, that is, only the user knows when it is necessary to apply it within the document in which case the variable `\__enumext_print_keyans_star_bool` set by the `\printkeyans` command is false and the variable `\__enumext_base_line_fix_bool` is true.

We set the values of the keys `topsep`, `above` and `above*` for the “first level” of `enumext` environment equal to `0pt` and finally set the variable `\__enumext_base_line_fix_bool` to false.

```

985 \cs_new_protected:Nn \__enumext_nested_base_line_fix:
986   {
987     \bool_lazy_all:nT
988       {
989         { \bool_if_p:N \__enumext_starred_first_bool }
990         { \bool_if_p:N \__enumext_base_line_fix_bool }
991         { \bool_not_p:n { \__enumext_print_keyans_star_bool } }
992       }
993     {
994       \mode_leave_vertical:
995       \vspace { -\dim_eval:n { \baselineskip + \parsep } }
996       \keys_set:nn { enumext / level-1 }
997         {
998           topsep = 0pt, above = 0pt, above* = 0pt,
999         }
1000     }

```

When we are running the `\printkeyans` command with the *starred argument* ‘*’ the variable `\__enumext_print_keyans_star_bool` is true and we can run a simplified version of `\vspace` using `\skip_vertical:n`.

```

1001 \bool_lazy_and:nnT
1002   { \bool_if_p:N \__enumext_starred_first_bool }
1003   { \bool_if_p:N \__enumext_print_keyans_star_bool }
1004   {
1005     \mode_leave_vertical:
1006     \skip_vertical:n { -\baselineskip }
1007     \skip_vertical:N \c_zero_skip
1008     \keys_set:nn { enumext / level-1 }
1009       {
1010         topsep = 0pt, above = 0pt, above* = 0pt,

```

```

1011     }
1012   }
1013   \bool_set_false:N \__enumext_base_line_fix_bool
1014 }

```

(End of definition for `base-fix` and `\__enumext_nested_base_line_fix:`.)

13.19 Setting keys for horizontal spaces

Define and set `itemindent`, `rightmargin`, `listparindent`, `list-offset` and `list-indent` keys for `enumext`, `enumext*`, `keyans` and `keyans*` environments.

```

1015 \cs_set_protected:Npn \__enumext_tmp:nn #1 #2
1016 {
1017   \keys_define:nn { enumext / #1 }
1018   {
1019     itemindent .dim_set:c = { l__enumext_fake_item_indent_#2_dim },
1020     itemindent .value_required:n = true,
1021     rightmargin .dim_set:c = { l__enumext_rightmargin_#2_dim },
1022     rightmargin .value_required:n = true,
1023     listparindent .dim_set:c = { l__enumext_listparindent_#2_dim },
1024     listparindent .value_required:n = true,
1025     list-offset .dim_set:c = { l__enumext_listoffset_#2_dim },
1026     list-offset .value_required:n = true,
1027     list-indent .code:n =
1028       \bool_set_true:c { l__enumext_leftmargin_tmp_#2_bool }
1029       \dim_set:cn { l__enumext_leftmargin_tmp_#2_dim } {##1},
1030     list-indent .value_required:n = true,
1031   }
1032 }
1033 \clist_map_inline:nn
1034 {
1035   {level-1}{i}, {level-2}{ii}, {level-3}{iii}, {level-4}{iv}, {keyans}{v}
1036 }
1037 { \__enumext_tmp:nn #1 }

```

(End of definition for `itemindent` and others.)

For `enumext*` and `keyans*` environments the situation is a bit different, the `list-indent` key behaves like the `list-offset` key.

```

1038 \cs_set_protected:Npn \__enumext_tmp:nn #1 #2
1039 {
1040   \keys_define:nn { enumext / #1 }
1041   {
1042     itemindent .dim_set:c = { l__enumext_fake_item_indent_#2_dim },
1043     itemindent .value_required:n = true,
1044     rightmargin .dim_set:c = { l__enumext_rightmargin_#2_dim },
1045     rightmargin .value_required:n = true,
1046     listparindent .dim_set:c = { l__enumext_listparindent_#2_dim },
1047     listparindent .value_required:n = true,
1048     list-offset .dim_set:c = { l__enumext_listoffset_#2_dim },
1049     list-offset .value_required:n = true,
1050     list-indent .meta:n = { list-offset = ##1 },
1051     list-indent .value_required:n = true,
1052   }
1053 }
1054 \clist_map_inline:nn
1055 {
1056   {enumext*}{vii}, {keyans*}{viii}
1057 }
1058 { \__enumext_tmp:nn #1 }

```

13.19.1 Functions for setting the fake `itemindent`

The `itemindent` key does not set the value of `\itemindent`, it only sets the value of the *horizontal space* applied using `\skip_horizontal:N`. We will store this value in the variable and only apply it when it is greater than `0pt`. Here I will need to place `\mode_leave_vertical:` and the plain TeX macro `\ignorespaces` to avoid unwanted extra space when using the `itemindent` key.

```

1059 \cs_set_protected:Nn \__enumext_fake_item_indent:
1060 {
1061   \dim_compare:nNt
1062     { \dim_use:c { l__enumext_fake_item_indent_ \__enumext_level: _dim } }
1063     >
1064     { \c_zero_dim }

```

```

1065     {
1066       \tl_set:ce { l__enumext_fake_item_indent_ \__enumext_level: _tl }
1067       {
1068         \exp_not:N \mode_leave_vertical:
1069         \exp_not:n { \skip_horizontal:n }
1070         { \dim_use:c { l__enumext_fake_item_indent_ \__enumext_level: _dim } }
1071         \exp_not:N \ignorespaces
1072       }
1073     }
1074   }
1075   \cs_set_protected:Nn \__enumext_keyans_fake_item_indent:
1076   {
1077     \dim_compare:nNnT
1078     { \l__enumext_fake_item_indent_v_dim } > { \c_zero_dim }
1079     {
1080       \tl_set:Ne \l__enumext_fake_item_indent_v_tl
1081       {
1082         \exp_not:N \mode_leave_vertical:
1083         \exp_not:N \skip_horizontal:N \l__enumext_fake_item_indent_v_dim
1084         \exp_not:N \ignorespaces
1085       }
1086     }
1087   }
1088   \cs_set_protected:Nn \__enumext_fake_item_indent_vii:
1089   {
1090     \dim_compare:nNnT
1091     { \l__enumext_fake_item_indent_vii_dim } > { \c_zero_dim }
1092     {
1093       \tl_set:Ne \l__enumext_fake_item_indent_vii_tl
1094       {
1095         \exp_not:N \skip_horizontal:N \l__enumext_fake_item_indent_vii_dim
1096         \exp_not:N \ignorespaces
1097       }
1098     }
1099   }
1100   \cs_set_protected:Nn \__enumext_fake_item_indent_viii:
1101   {
1102     \dim_compare:nNnT
1103     { \l__enumext_fake_item_indent_viii_dim } > { \c_zero_dim }
1104     {
1105       \tl_set:Ne \l__enumext_fake_item_indent_viii_tl
1106       {
1107         \exp_not:N \skip_horizontal:N \l__enumext_fake_item_indent_viii_dim
1108         \exp_not:N \ignorespaces
1109       }
1110     }
1111   }

```

(End of definition for `\__enumext_fake_item_indent:` and others.)

13.20 Setting show-length key

show-length

Define and set `show-length` key for `enumext`, `enumext*`, `keyans` and `keyans*` environments. The function sets the boolean variable `\l__enumext_show_length_X_bool` used in the definition of all environments to “true” and calls the function `\__enumext_show_length:nnn` which prints all the values of the “vertical” and “horizontal” parameters calculated and used.

```

1112 \cs_set_protected:Npn \__enumext_tmp:nn #1 #2
1113 {
1114   \keys_define:nn { enumext / #1 }
1115   {
1116     show-length .bool_set:c = { l__enumext_show_length_#2_bool },
1117     show-length .initial:n = false,
1118   }
1119 }
1120 \clist_map_inline:Nn \c__enumext_all_envs_clist { \__enumext_tmp:nn #1 }

```

(End of definition for `show-length`.)

13.21 Setting before, after and first keys

before
before*
after
first

Define and set `before`, `before*`, `after` and `first` keys for `enumext`, `enumext*`, `keyans` and `keyans*` environments.


```

1121 \cs_set_protected:Npn \__enumext_tmp:nn #1 #2
1122 {
1123   \keys_define:nn { enumext / #1 }
1124   {
1125     before .tl_set:c = { l__enumext_before_no_starred_key_#2_tl },
1126     before .value_required:n = true,
1127     before* .tl_set:c = { l__enumext_before_starred_key_#2_tl },
1128     before* .value_required:n = true,
1129     after .tl_set:c = { l__enumext_after_stop_list_#2_tl },
1130     after .value_required:n = true,
1131     first .tl_set:c = { l__enumext_after_list_args_#2_tl },
1132     first .value_required:n = true,
1133   }
1134 }
1135 \clist_map_inline:Nn \c__enumext_all_envs_clist { \__enumext_tmp:nn #1 }

```

(End of definition for *before* and others.)

13.21.1 Functions for before, after and first keys in enumext

The function `\__enumext_before_args_exec:` executes the `{\code}` set by the `before*` key “before” the `enumext` environment is started. The `{\code}` is executed “without” knowing any definition of the `{\arg two}` of the list: `{\code}\list{\arg one}{\arg two}`.

```

1136 \cs_new_protected:Nn \__enumext_before_args_exec:
1137 {
1138   \tl_use:c { l__enumext_before_starred_key_ \__enumext_level: _tl }
1139 }

```

The function `\__enumext_before_keys_exec:` executes the `{\code}` set by the `before` key “before” the `enumext` environment is started in *second argument* of the list. The `{\code}` is executed “knowing” all definition and values provides by `\keys: \list{\arg one}{\arg two}{\code}`

```

1140 \cs_new_protected:Nn \__enumext_before_keys_exec:
1141 {
1142   \tl_use:c { l__enumext_before_no_starred_key_ \__enumext_level: _tl }
1143 }

```

The function `\__enumext_after_stop_list:` executes the `{\code}` set by the `after` key “after” the `enumext` environment has finished: `\endlist{\code}`.

```

1144 \cs_new_protected:Nn \__enumext_after_stop_list:
1145 {
1146   \tl_use:c { l__enumext_after_stop_list_ \__enumext_level: _tl }
1147 }

```

The function `\__enumext_after_args_exec:` executes the `{\code}` set by the `first` key after the end of the second argument of the list defining the `enumext` environment, just before the first occurrence of `\item: \list{\arg one}{\arg two}{\code}\item.`

```

1148 \cs_new_protected:Nn \__enumext_after_args_exec:
1149 {
1150   \tl_use:c { l__enumext_after_list_args_ \__enumext_level: _tl }
1151 }

```

(End of definition for `\__enumext_before_args_exec:` and others.)

13.21.2 Functions for before, after and first keys in keyans

Same implementation as the one used in the `enumext` environment.

```

\__enumext_before_args_exec_v:
\__enumext_before_keys_exec_v:
\__enumext_after_stop_list_v:
\__enumext_after_args_exec_v:
1152 \cs_new_protected:Nn \__enumext_before_args_exec_v:
1153 {
1154   \tl_use:N \l__enumext_before_starred_key_v_tl
1155 }
1156 \cs_new_protected:Nn \__enumext_before_keys_exec_v:
1157 {
1158   \tl_use:N \l__enumext_before_no_starred_key_v_tl
1159 }
1160 \cs_new_protected:Nn \__enumext_after_stop_list_v:
1161 {
1162   \tl_use:N \l__enumext_after_stop_list_v_tl
1163 }
1164 \cs_new_protected:Nn \__enumext_after_args_exec_v:
1165 {
1166   \tl_use:N \l__enumext_after_list_args_v_tl
1167 }

```

(End of definition for `\__enumext_before_args_exec_v:` and others.)

13.21.3 Functions for before, after and first keys in enumext* and keyans*

Same implementation as the one used in the `enumext` environment.

```

\__enumext_before_args_exec_vii:
\__enumext_before_keys_exec_vii:
\__enumext_after_stop_list_vii:
\__enumext_after_args_exec_vii:
1168 \cs_new_protected:Nn \__enumext_before_args_exec_vii:
1169 {
1170   \tl_use:N \l__enumext_before_starred_key_vii_tl
1171 }
1172 \cs_new_protected:Nn \__enumext_before_args_exec_viii:
1173 {
1174   \tl_use:N \l__enumext_before_starred_key_viii_tl
1175 }
1176 \cs_new_protected:Nn \__enumext_before_keys_exec_vii:
1177 {
1178   \tl_use:N \l__enumext_before_no_starred_key_vii_tl
1179 }
1180 \cs_new_protected:Nn \__enumext_before_keys_exec_viii:
1181 {
1182   \tl_use:N \l__enumext_before_no_starred_key_viii_tl
1183 }
1184 \cs_new_protected:Nn \__enumext_after_stop_list_vii:
1185 {
1186   \tl_use:N \l__enumext_after_stop_list_vii_tl
1187 }
1188 \cs_new_protected:Nn \__enumext_after_stop_list_viii:
1189 {
1190   \tl_use:N \l__enumext_after_stop_list_viii_tl
1191 }
1192 \cs_new_protected:Nn \__enumext_after_args_exec_vii:
1193 {
1194   \tl_use:N \l__enumext_after_list_args_vii_tl
1195 }
1196 \cs_new_protected:Nn \__enumext_after_args_exec_viii:
1197 {
1198   \tl_use:N \l__enumext_after_list_args_viii_tl
1199 }

```

(End of definition for `\__enumext_before_args_exec_vii:` and others.)

13.22 Setting keys for multicol and minipage

The default value of the `columns-sep` key is handled by the state of the boolean variable `\l__enumext_columns_sep_X_bool` which is handled in the internal definition of the `enumext` and `keyans` environments. Define and set `mini-env`, `mini-sep`, `columns-sep` and `columns` keys for `enumext`, `enumext*`, `keyans` and `keyans*` environments.

```

1200 \cs_set_protected:Npn \__enumext_tmp:nn #1 #2
1201 {
1202   \keys_define:nn { enumext / #1 }
1203   {
1204     mini-env .dim_set:c = { l__enumext_minipage_right_#2_dim },
1205     mini-env .value_required:n = true,
1206     mini-sep .dim_set:c = { l__enumext_minipage_hsep_#2_dim },
1207     mini-sep .initial:n = 0.3333em,
1208     mini-sep .value_required:n = true,
1209     columns-sep .dim_set:c = { l__enumext_columns_sep_#2_dim },
1210     columns-sep .value_required:n = true,
1211     columns .int_set:c = { l__enumext_columns_#2_int },
1212     columns .initial:n = 1,
1213     columns .value_required:n = true,
1214   }
1215 }
1216 \clist_map_inline:Nn \c__enumext_all_envs_clist { \__enumext_tmp:nn #1 }

```

For `enumext*` and `keyans*` environments the situation is a bit different, the command `\miniright` is not available, so we will add the keys `mini-right` and `mini-right*` to implement support for `minipage` environment.

```

1217 \cs_set_protected:Npn \__enumext_tmp:nn #1 #2
1218 {
1219   \keys_define:nn { enumext / #1 }
1220   {
1221     mini-right .tl_gset:c = { g__enumext_miniright_code_#2_tl },
1222     mini-right .value_required:n = true,
1223     mini-right* .code:n = {

```

```

1224         \bool_gset_true:c { g__enumext_minipage_center_#2_bool }
1225         \keys_set:nn { enumext / #1 } { mini-right = {##1} }
1226     },
1227     mini-right* .value_required:n = true,
1228 }
1229 }
1230 \clist_map_inline:nn { {enumext*}{vii}, {keyans*}{viii} } { \__enumext_tmp:nn #1 }

```

(End of definition for `mini-env` and others.)

13.23 Adjustment of vertical spaces for multicols

When nesting a “list environment” inside the `multicols` environment, the values of the “vertical spaces” are lost, basically the `multicols` environment takes control over them. Graphically, this can be seen in figure 7.



Figure 7: Representation of the vertical space in `multicols` for a nested level.

To keep the desired spaces *above* and *below* in the “list environment” (`\topsep` + `[\partopsep]`) it is necessary to “adjust” the spaces added by the `multicols` environment. The most appropriate option in this case is to use a “context sensitive” vertical space with `\addvspace`.

❗ I should make it clear that the implementation here is a “bit questionable”. At first glance doing `\multicolsep=\topsep` seemed right, but the results were not always as expected. An almost *imperceptible* detail is that in some cases the `\itemsep` values of are “stretched”, possibly due to the use of `\raggedcolumns` and this affects the lower space when closing the environment, which is “smaller” than expected. My attempts to find the correct values using `\showoutput` and `\showboxdepth` absolutely failed.

13.23.1 Adjustment of vertical spaces for multicols in enumext

`\__enumext_multi_set_vskip:` The function `\__enumext_multi_set_vskip:` will take care of determining the “adjusted spaces” that we will apply “above” and “below” the `multicols` environment in `enumext`.

We will set the default values taking into account that $\TeX$ is in *horizontal mode*, then we will make the settings for the *vertical mode* in which `\partopsep` comes into play.

Set the values of `\l__enumext_multicols_above_X_skip` and `\l__enumext_multicols_below_X_skip` equal to the value of `\topsep` in the *current level*.

```

1231 \cs_new_protected:Nn \__enumext_multi_set_vskip:
1232 {
1233     \skip_set:cn { l__enumext_multicols_above_ \__enumext_level: _skip }
1234     {
1235         \skip_use:c { l__enumext_topsep_ \__enumext_level: _skip }
1236     }
1237     \skip_set:cn { l__enumext_multicols_below_ \__enumext_level: _skip }
1238     {
1239         \skip_use:c { l__enumext_topsep_ \__enumext_level: _skip }
1240     }
1241     \__enumext_add_pre_parsep:
1242 }

```

(End of definition for `\__enumext_multi_set_vskip:`)

`\__enumext_add_pre_parsep:` The function `\__enumext_add_pre_parsep:` “adjusted” the value of `\l__enumext_multicols_above_X_skip` detecting the value of `\parsep` from the previous level. This is necessary since `\parsep` from the previous level affects the *vertical spaces*.

```

1243 \cs_new_protected:Nn \__enumext_add_pre_parsep:
1244 {
1245     \int_case:nn { \l__enumext_level_int }
1246     {
1247         { 2 }{
1248             \skip_if_eq:nnF { \l__enumext_parsep_i_skip } { \c_zero_skip }
1249             {
1250                 \skip_add:Nn \l__enumext_multicols_above_ii_skip
1251                 {
1252                     \l__enumext_parsep_i_skip
1253                 }
1254             }
1255         }
1256         { 3 }{

```

```

1257         \skip_if_eq:nnF { \l__enumext_parsep_ii_skip } { \c_zero_skip }
1258         {
1259             \skip_add:Nn \l__enumext_multicols_above_iii_skip
1260             {
1261                 \l__enumext_parsep_ii_skip
1262             }
1263         }
1264     }
1265 { 4 }{
1266     \skip_if_eq:nnF { \l__enumext_parsep_iii_skip } { \c_zero_skip }
1267     {
1268         \skip_add:Nn \l__enumext_multicols_above_iv_skip
1269         {
1270             \l__enumext_parsep_iii_skip
1271         }
1272     }
1273 }
1274 }
1275 }

```

(End of definition for `\__enumext_add_pre_parsep:`)

`\__enumext_multi_addvspace:` The function `\__enumext_multi_addvspace:` will apply the spaces set using `\addvspace` “above” the `multicols` environment in `enumext`, taking into account whether $\text{\TeX}$ is in $\langle horizontal\ mode \rangle$ or $\langle vertical\ mode \rangle$.

```

1276 \cs_new_protected:Nn \__enumext_multi_addvspace:
1277 {
1278     \__enumext_multi_set_vskip:
1279     \mode_if_vertical:T
1280     {
1281         \skip_add:cn { \l__enumext_multicols_above_ \__enumext_level: _skip }
1282         {
1283             \skip_use:c { \l__enumext_partopsep_ \__enumext_level: _skip }
1284         }
1285         \skip_add:cn { \l__enumext_multicols_below_ \__enumext_level: _skip }
1286         {
1287             \skip_use:c { \l__enumext_partopsep_ \__enumext_level: _skip }
1288         }
1289     }
1290     \par\nopagebreak
1291     \addvspace{ \skip_use:c { \l__enumext_multicols_above_ \__enumext_level: _skip } }
1292 }

```

(End of definition for `\__enumext_multi_addvspace:`)

13.23.2 Adjustment of vertical spaces for multicols in keyans

`\__enumext_keyans_multi_set_vskip:` The function `\__enumext_keyans_multi_set_vskip:` will take care of determining the “adjusted spaces” that we will apply “above” and “below” the `multicols` environment in `keyans`. The implementation of this function is the same as the one used in `enumext`.

`\__enumext_keyans_multi_addvspace:`

```

1293 \cs_new_protected:Nn \__enumext_keyans_multi_set_vskip:
1294 {
1295     \skip_set:Nn \l__enumext_multicols_above_v_skip
1296     {
1297         \l__enumext_topsep_v_skip
1298     }
1299     \skip_set:Nn \l__enumext_multicols_below_v_skip
1300     {
1301         \l__enumext_topsep_v_skip
1302     }
1303 }
1304 \cs_new_protected:Nn \__enumext_keyans_multi_addvspace:
1305 {
1306     \__enumext_keyans_multi_set_vskip:
1307     \mode_if_vertical:T
1308     {
1309         \skip_add:Nn \l__enumext_multicols_above_v_skip
1310         {
1311             \skip_use:N \l__enumext_partopsep_v_skip
1312         }
1313         \skip_add:Nn \l__enumext_multicols_below_v_skip
1314         {

```

```

1315         \skip_use:N \l__enumext_partopsep_v_skip
1316     }
1317 }
1318 \par\nopagebreak
1319 \addvspace{ \l__enumext_multicols_above_v_skip }
1320 }

```

(End of definition for `\__enumext_keyans_multi_set_vskip:` and `\__enumext_keyans_multi_addvspace:`)

13.24 Adjustment of vertical spaces for minipage

When nesting a “list environment” within the `minipage` environment, the values of the “vertical spaces” are lost. Graphically it can be seen like in the figure 8.



Figure 8: Representation of the `minipage` spacing adjustment for a nested level.

Since we want to keep the “left” and “right” environments “aligned on top”, preserving the `\baselineskip` and keep the desired “spaces” (`\topsep` + `[\partopsep]`) it is necessary to “adjust” the “vertical spaces” for `minipage` environments.

Here there are several complications that we must circumvent, the `minipage` environment eliminates the “top” spaces, the `multicols` environment can be nested in the `minipage` environment, the “top” and “bottom” spaces are affected when `topsep=0pt` and to this is added the `\partopsep` parameter that comes into action according to whether $\TeX$ is in *horizontal mode* or *vertical mode*. Depending on these cases, small adjustments must be made using `\vspace` and `\addvspace` to obtain the “desired vertical spacing”.

- Again I must make clear that the implementation here is a “bit questionable”, but hunting the spaces (glue) produced by the `minipage` environment is quite complicated, even more if `multicols` is nested. The setting of the values was more “trial and error” (approx to `\strutbox`), using the help of the `lua-visual-debug`[15] package, again my attempts to find the correct values using `\showoutput` and `\showboxdepth` absolutely failed.

13.24.1 Adjustment of vertical spaces for minipage in enumext

```

\__enumext_minipage_set_skip:
\__enumext_minipage_add_space:

```

The function `\__enumext_minipage_set_skip:` will take care of determining the “adjust” spaces that we will apply “above” and “below” the `\__enumext_mini_page` environment in `enumext`.

First we will set the value of `\l__enumext_minipage_right_skip` equal to `\topsep`, then we will see if $\TeX$ is in *vertical mode* and we will add `\partopsep`, followed by that we set the value of `\l__enumext_minipage_after_skip`.

```

1321 \cs_new_protected:Nn \__enumext_minipage_set_skip:
1322 {
1323     \skip_set:Nn \l__enumext_minipage_right_skip
1324     {
1325         \skip_use:c { \l__enumext_topsep_ \__enumext_level: _skip }
1326     }
1327     \mode_if_vertical:T
1328     {
1329         \skip_add:Nn \l__enumext_minipage_right_skip
1330         {
1331             \skip_use:c { \l__enumext_partopsep_ \__enumext_level: _skip }
1332         }
1333     }
1334     \skip_set_eq:NN \l__enumext_minipage_after_skip \l__enumext_minipage_right_skip

```

We will adjust the values `\l__enumext_multicols_above_X_skip` and `\l__enumext_multicols_below_X_skip` and call the function `\__enumext_pre_itemsep_skip:`.

```

1335     \skip_set_eq:cN
1336     { \l__enumext_multicols_above_ \__enumext_level: _skip } \l__enumext_minipage_right_skip
1337     \skip_set_eq:cN
1338     { \l__enumext_multicols_below_ \__enumext_level: _skip } \l__enumext_minipage_right_skip
1339     \__enumext_pre_itemsep_skip:

```

If the environment `multicols` is active, we set `\topskip=0pt` and then we make `\multicolsep` have the same value as `\l__enumext_multicols_above_X_skip`.

```

1340     \int_compare:nNnT
1341     { \int_use:c { \l__enumext_columns_ \__enumext_level: _int } } > { 1 }
1342     {
1343         \skip_zero:N \topskip
1344         \skip_set_eq:Nc \multicolsep { \l__enumext_multicols_above_ \__enumext_level: _skip }

```

```

1345     }
1346 }

```

The function `\__enumext_minipage_add_space:` will apply the spaces on the “*left side*” using `\addvspace` “*above*” the `\__enumext_minipage` environment, taking into account whether $\TeX$ is in $\langle horizontal\ mode \rangle$ or $\langle vertical\ mode \rangle$. Here we use the plain $\TeX$ macro `\nointerlineskip` to prevent baseline “*glue*” being added between the next pair of boxes in a *vertical list*. For the latter we will make some adjustments since the `\partopsep` parameter comes into play and this affects the *vertical spacing*.

```

1347 \cs_new_protected:Nn \__enumext_minipage_add_space:
1348 {
1349   \__enumext_minipage_set_skip:
1350   \__enumext_unskip_unkern:
1351   \mode_if_vertical:TF
1352   {
1353     \nopagebreak\nointerlineskip
1354   }
1355   {
1356     \par\nopagebreak\nointerlineskip
1357     \skip_zero:c { \l__enumext_partopsep_ \__enumext_level: \skip }
1358   }
1359   \int_compare:nNnTF
1360   { \int_use:c { \l__enumext_columns_ \__enumext_level: \int } } > { 1 }
1361   {
1362     \addvspace{ 0.445\box_ht:N \strutbox }
1363   }
1364   {
1365     \addvspace{ 0.250\box_ht:N \strutbox }
1366   }
1367 }

```

(End of definition for `\__enumext_minipage_set_skip:` and `\__enumext_minipage_add_space:`.)

`\__enumext_pre_itemsep_skip:`

The function `\__enumext_pre_itemsep_skip:` will adjust the spaces below the environment `minipage` and the environment `multicols` if it is nested in it, taking into account the value of `\itemsep` from the previous level.

```

1368 \cs_new_protected:Nn \__enumext_pre_itemsep_skip:
1369 {
1370   \int_case:nn { \l__enumext_level_int }
1371   {
1372     { 2 }{
1373       \skip_if_eq:nnTF
1374       { \l__enumext_itemsep_i_skip } { \l__enumext_minipage_after_skip }
1375       {
1376         \skip_set:Nn \l__enumext_minipage_after_skip { 0.150\box_ht:N \strutbox }
1377         \skip_set:Nn \l__enumext_multicols_below_ii_skip { 0.350\box_ht:N \strutbox }
1378       }
1379       {
1380         \dim_compare:nNnT
1381         { \l__enumext_itemsep_i_skip } < { \l__enumext_minipage_after_skip }
1382         {
1383           \skip_sub:Nn
1384           \l__enumext_minipage_after_skip { \l__enumext_itemsep_i_skip }
1385           \skip_sub:Nn
1386           \l__enumext_multicols_below_ii_skip { \l__enumext_itemsep_i_skip }
1387           \skip_add:Nn
1388           \l__enumext_minipage_after_skip { 0.150\box_ht:N \strutbox }
1389           \skip_add:Nn
1390           \l__enumext_multicols_below_ii_skip { 0.350\box_ht:N \strutbox }
1391         }
1392         \dim_compare:nNnT
1393         { \l__enumext_itemsep_i_skip } > { \l__enumext_minipage_after_skip }
1394         {
1395           \skip_set:Nn \l__enumext_minipage_temp_skip
1396           {
1397             \l__enumext_itemsep_i_skip - \l__enumext_minipage_after_skip
1398           }
1399           \skip_sub:Nn
1400           \l__enumext_minipage_after_skip { \l__enumext_itemsep_i_skip }
1401           \skip_sub:Nn
1402           \l__enumext_multicols_below_ii_skip { \l__enumext_itemsep_i_skip }
1403           \skip_add:Nn

```



```

1404         \l__enumext_minipage_after_skip
1405         { 0.150\box_ht:N \strutbox + \l__enumext_minipage_temp_skip }
1406     \skip_add:Nn
1407         \l__enumext_multicols_below_ii_skip
1408         { 0.350\box_ht:N \strutbox + \l__enumext_minipage_temp_skip }
1409     }
1410 }
1411 }
1412 { 3 }{
1413     \skip_if_eq:nnTF
1414     { \l__enumext_itemsep_ii_skip } { \c_zero_skip }
1415     {
1416         \skip_set:Nn \l__enumext_minipage_after_skip { 0.150\box_ht:N \strutbox }
1417         \skip_set:Nn \l__enumext_multicols_below_iii_skip { 0.350\box_ht:N \strutbox }
1418     }
1419     {
1420         \dim_compare:nNnT
1421         { \l__enumext_itemsep_ii_skip } < { \l__enumext_minipage_after_skip }
1422         {
1423             \skip_sub:Nn
1424             \l__enumext_minipage_after_skip { \l__enumext_itemsep_ii_skip }
1425             \skip_sub:Nn
1426             \l__enumext_multicols_below_iii_skip { \l__enumext_itemsep_ii_skip }
1427             \skip_add:Nn
1428             \l__enumext_minipage_after_skip { 0.150\box_ht:N \strutbox }
1429             \skip_add:Nn
1430             \l__enumext_multicols_below_iii_skip { 0.350\box_ht:N \strutbox }
1431         }
1432         \dim_compare:nNnT
1433         { \l__enumext_itemsep_ii_skip } > { \l__enumext_minipage_after_skip }
1434         {
1435             \skip_set:Nn \l__enumext_minipage_temp_skip
1436             {
1437                 \l__enumext_itemsep_ii_skip - \l__enumext_minipage_after_skip
1438             }
1439             \skip_sub:Nn
1440             \l__enumext_minipage_after_skip { \l__enumext_itemsep_ii_skip }
1441             \skip_sub:Nn
1442             \l__enumext_multicols_below_iii_skip { \l__enumext_itemsep_ii_skip }
1443             \skip_add:Nn
1444             \l__enumext_minipage_after_skip
1445             { 0.150\box_ht:N \strutbox + \l__enumext_minipage_temp_skip }
1446             \skip_add:Nn
1447             \l__enumext_multicols_below_iii_skip
1448             { 0.350\box_ht:N \strutbox + \l__enumext_minipage_temp_skip }
1449         }
1450     }
1451 }
1452 { 4 }{
1453     \skip_if_eq:nnTF { \l__enumext_itemsep_iii_skip } { \c_zero_skip }
1454     {
1455         \skip_set:Nn \l__enumext_minipage_after_skip { 0.150\box_ht:N \strutbox }
1456         \skip_set:Nn \l__enumext_multicols_below_iv_skip { 0.350\box_ht:N \strutbox }
1457     }
1458     {
1459         \dim_compare:nNnT
1460         { \l__enumext_itemsep_iii_skip } < { \l__enumext_minipage_after_skip }
1461         {
1462             \skip_sub:Nn
1463             \l__enumext_minipage_after_skip { \l__enumext_itemsep_iii_skip }
1464             \skip_sub:Nn
1465             \l__enumext_multicols_below_iv_skip { \l__enumext_itemsep_iii_skip }
1466             \skip_add:Nn
1467             \l__enumext_minipage_after_skip { 0.150\box_ht:N \strutbox }
1468             \skip_add:Nn
1469             \l__enumext_multicols_below_iv_skip { 0.350\box_ht:N \strutbox }
1470         }
1471         \dim_compare:nNnT
1472         { \l__enumext_itemsep_iii_skip } > { \l__enumext_minipage_after_skip }
1473         {
1474             \skip_set:Nn \l__enumext_minipage_temp_skip

```

```

1475         {
1476             \l__enumext_itemsep_iii_skip - \l__enumext_minipage_after_skip
1477         }
1478         \skip_sub:Nn
1479             \l__enumext_minipage_after_skip { \l__enumext_itemsep_iii_skip }
1480         \skip_sub:Nn
1481             \l__enumext_multicols_below_iv_skip { \l__enumext_itemsep_iii_skip }
1482         \skip_add:Nn
1483             \l__enumext_minipage_after_skip
1484             { 0.150\box_ht:N \strutbox + \l__enumext_minipage_temp_skip }
1485         \skip_add:Nn
1486             \l__enumext_multicols_below_iv_skip
1487             { 0.350\box_ht:N \strutbox + \l__enumext_minipage_temp_skip }
1488         }
1489     }
1490 }
1491 }
1492 }

```

(End of definition for \l__enumext_pre_itemsep_skip:.)

13.24.2 Adjustment of vertical spaces for minipage in keyans

```

\__enumext_keyans_minipage_set_skip:
\__enumext_keyans_minipage_add_space:
\__enumext_keyans_pre_itemsep_skip:

```

The function __enumext_keyans_mini_set_vskip: will take care of determining the “adjusted” spaces that we will apply “above” and “below” the `__enumext_mini_page` environment in `keyans`. The implementation of this function is the same as the one used in `enumext`.

```

1493 \cs_new_protected:Nn \__enumext_keyans_minipage_set_skip:
1494 {
1495     \skip_zero:N \l__enumext_minipage_after_skip
1496     \skip_zero:N \l__enumext_minipage_left_skip
1497     \skip_zero:N \l__enumext_minipage_right_skip
1498     \skip_set:Nn \l__enumext_minipage_right_skip
1499     {
1500         \l__enumext_topsep_v_skip
1501     }
1502     \mode_if_vertical:T
1503     {
1504         \skip_add:Nn \l__enumext_minipage_right_skip
1505         {
1506             \l__enumext_partopsep_v_skip
1507         }
1508     }
1509     \skip_set_eq:NN \l__enumext_minipage_after_skip \l__enumext_minipage_right_skip
1510     \skip_set_eq:NN \l__enumext_multicols_above_v_skip \l__enumext_minipage_right_skip
1511     \skip_set_eq:NN \l__enumext_multicols_below_v_skip \l__enumext_minipage_right_skip
1512     \__enumext_keyans_pre_itemsep_skip:
1513     \int_compare:nNnT { \l__enumext_columns_v_int } > { 1 }
1514     {
1515         \skip_zero:N \topskip
1516         \skip_set_eq:NN \multicolsep \l__enumext_minipage_right_skip
1517     }
1518 }
1519 \cs_new_protected:Nn \__enumext_keyans_minipage_add_space:
1520 {
1521     \__enumext_keyans_minipage_set_skip:
1522     \__enumext_unskip_unkern:
1523     \mode_if_vertical:TF
1524     {
1525         \nopagebreak\nointerlineskip
1526     }
1527     {
1528         \par\nopagebreak\nointerlineskip
1529         \skip_zero:N \l__enumext_partopsep_v_skip
1530     }
1531     \int_compare:nNnTF { \l__enumext_columns_v_int } > { 1 }
1532     {
1533         \addvspace{ 0.445\box_ht:N \strutbox }
1534     }
1535     {
1536         \addvspace{ 0.250\box_ht:N \strutbox }
1537     }
1538 }

```

```

1539 \cs_new_protected:Nn \__enumext_keyans_pre_itemsep_skip:
1540 {
1541   \skip_if_eq:nnTF
1542   { \l__enumext_itemsep_i_skip } { \l__enumext_minipage_after_skip }
1543   {
1544     \skip_set:Nn \l__enumext_minipage_after_skip { 0.150\box_ht:N \strutbox }
1545     \skip_set:Nn \l__enumext_multicols_below_v_skip { 0.350\box_ht:N \strutbox }
1546   }
1547   {
1548     \dim_compare:nNnT
1549     { \l__enumext_itemsep_i_skip } < { \l__enumext_minipage_after_skip }
1550     {
1551       \skip_sub:Nn \l__enumext_minipage_after_skip { \l__enumext_itemsep_i_skip }
1552       \skip_sub:Nn \l__enumext_multicols_below_v_skip { \l__enumext_itemsep_i_skip }
1553       \skip_add:Nn \l__enumext_minipage_after_skip { 0.150\box_ht:N \strutbox }
1554       \skip_add:Nn \l__enumext_multicols_below_v_skip { 0.350\box_ht:N \strutbox }
1555     }
1556     \dim_compare:nNnT
1557     { \l__enumext_itemsep_i_skip } > { \l__enumext_minipage_after_skip }
1558     {
1559       \skip_set:Nn \l__enumext_minipage_temp_skip
1560       {
1561         \l__enumext_itemsep_i_skip - \l__enumext_minipage_after_skip
1562       }
1563       \skip_sub:Nn \l__enumext_minipage_after_skip { \l__enumext_itemsep_i_skip }
1564       \skip_sub:Nn \l__enumext_multicols_below_v_skip { \l__enumext_itemsep_i_skip }
1565       \skip_add:Nn \l__enumext_minipage_after_skip
1566       { 0.150\box_ht:N \strutbox + \l__enumext_minipage_temp_skip }
1567       \skip_add:Nn \l__enumext_multicols_below_v_skip
1568       { 0.350\box_ht:N \strutbox + \l__enumext_minipage_temp_skip }
1569     }
1570   }
1571 }

```

(End of definition for `\__enumext_keyans_minipage_set_skip:`, `\__enumext_keyans_minipage_add_space:`, and `\__enumext_keyans_pre_itemsep_skip:`.)

13.24.3 Adjustment of vertical spaces for minipage in enumext* and keyans*

`\__enumext_mini_set_vskip_vii:`
`\__enumext_mini_set_vskip_viii:`

The functions `\__enumext_mini_set_vskip_vii:` and `\__enumext_mini_set_vskip_viii:` will take care of determining the “adjusted” spaces that we will apply “above” and “below” the `\__enumext_mini_page` environment in `enumext*` and `keyans*`.

```

1572 \cs_new_protected:Nn \__enumext_mini_set_vskip_vii:
1573 {
1574   \skip_zero_new:N \l__enumext_minipage_left_skip
1575   \skip_gzero_new:N \g__enumext_minipage_right_skip
1576   \skip_gzero_new:N \g__enumext_minipage_after_skip
1577   \skip_if_eq:nnTF { \l__enumext_topsep_vii_skip } { \c_zero_skip }
1578   {
1579     \skip_set:Nn \l__enumext_minipage_left_skip { 0.5\box_dp:N \strutbox }
1580     \skip_gset:Nn \g__enumext_minipage_right_skip { 0.325\box_dp:N \strutbox }
1581   }
1582   {
1583     \skip_set:Nn \l__enumext_minipage_left_skip { 0.5875\box_dp:N \strutbox }
1584     \skip_gset:Nn \g__enumext_minipage_right_skip
1585     {
1586       \l__enumext_topsep_vii_skip
1587     }
1588     \skip_gset:Nn \g__enumext_minipage_after_skip
1589     {
1590       0.325\box_dp:N \strutbox + \l__enumext_topsep_vii_skip
1591     }
1592   }
1593 }
1594 \cs_new_protected:Nn \__enumext_mini_set_vskip_viii:
1595 {
1596   \skip_zero_new:N \l__enumext_minipage_after_skip
1597   \skip_zero_new:N \l__enumext_minipage_left_skip
1598   \skip_zero_new:N \l__enumext_minipage_right_skip
1599   \skip_if_eq:nnTF { \l__enumext_topsep_viii_skip } { \c_zero_skip }
1600   {
1601     \skip_set:Nn \l__enumext_minipage_left_skip

```

```

1602         {
1603             0.5\box_dp:N \strutbox
1604         }
1605         \skip_set:Nn \l__enumext_minipage_right_skip
1606         {
1607             \l__enumext_partopsep_viii_skip
1608         }
1609         \skip_set:Nn \l__enumext_minipage_after_skip
1610         {
1611             1.6\box_dp:N \strutbox
1612         }
1613     }
1614     {
1615         \skip_set:Nn \l__enumext_minipage_left_skip
1616         {
1617             0.5875\box_dp:N \strutbox
1618         }
1619         \skip_set:Nn \l__enumext_minipage_right_skip
1620         {
1621             \l__enumext_topsep_viii_skip
1622         }
1623         \skip_set:Nn \l__enumext_minipage_after_skip
1624         {
1625             0.325\box_dp:N \strutbox + \l__enumext_topsep_viii_skip
1626         }
1627     }
1628 }

```

(End of definition for `\__enumext_mini_set_vskip_vii:` and `\__enumext_mini_set_vskip_viii:`)

`\__enumext_mini_addvspace_vii:`
`\__enumext_mini_addvspace_viii:`

The functions `\__enumext_mini_addvspace_vii:` and `\__enumext_mini_addvspace_viii:` will apply the vertical space “only above” the `\__enumext_mini_page` environment on the *left side* when the `mini-right` key is active in the `enumext*` and `keyans*` environments.

Here we will NOT take into account whether $\text{\TeX}$ is in $\langle\text{horizontal mode}\rangle$ or $\langle\text{vertical mode}\rangle$, since `\partopsep` is equal to `0pt` in both environments.

```

1629 \cs_new_protected:Nn \__enumext_mini_addvspace_vii:
1630 {
1631     \__enumext_mini_set_vskip_vii:
1632     \par\nopagebreak
1633     \addvspace { \l__enumext_minipage_left_skip }
1634 }
1635 \cs_new_protected:Nn \__enumext_mini_addvspace_viii:
1636 {
1637     \__enumext_mini_set_vskip_viii:
1638     \par\nopagebreak
1639     \addvspace { \l__enumext_minipage_left_skip }
1640 }

```

(End of definition for `\__enumext_mini_addvspace_vii:` and `\__enumext_mini_addvspace_viii:`)

13.24.4 The command `\miniright`

The command `\miniright` will close the `\__enumext_mini_page` environment on the “left side”, open the `\__enumext_mini_page` environment on the “right side” adding the *adjusted vertical space*. By default we will add `\centering` when starting the “right side” environment. The *starred argument* ‘`*`’ inhibits the use of `\centering` command i.e. the usual $\text{\TeX}$ justification is maintained in the `\__enumext_mini_page` on the “right side”.

`\miniright`

First we will perform some checks to prevent the command from being executed outside the `enumext` environment or somewhere inappropriate then we will call the internal functions to execute it in the `enumext` and `keyans` environments.

```

1641 \NewDocumentCommand \miniright { s }
1642 {
1643     \int_compare:nNt { \l__enumext_keyans_pic_level_int } = { 1 }
1644     {
1645         \msg_error:nnn { enumext } { wrong-miniright-place }
1646     }
1647     % outside
1648     \bool_lazy_and:nnT
1649     { \int_compare_p:nNn { \l__enumext_level_int } = { 0 } }
1650     { \int_compare_p:nNn { \l__enumext_level_h_int } = { 0 } }

```

```

1651     {
1652         \msg_error:nnn { enumext } { wrong-miniright-place }
1653     }
1654 % starred env
1655 \bool_lazy_and:nnT
1656 { \bool_if_p:N \g__enumext_starred_bool }
1657 { \bool_not_p:n { \l__enumext_standar_bool } }
1658 {
1659     \msg_error:nnn { enumext } { wrong-miniright-starred }
1660 }
1661 % exec
1662 \int_compare:nNnTF { \l__enumext_keyans_level_int } = { 1 }
1663 {
1664     \__enumext_keyans_mini_right_cmd:n {#1}
1665 }
1666 { \__enumext_mini_right_cmd:n {#1} }
1667 }

```

(End of definition for `\miniright`. This function is documented on page 12.)

`\__enumext_mini_right_cmd:n`

The function `\__enumext_mini_right_cmd:n` takes as argument the *starred* ‘`*`’ of the `\miniright` command in the `enumext` environment. We check if the `mini-env` key is active via the variable `\l__enumext_minipage_right_X_dim`, if so we close the `multicols` environment with the `\__enumext_mini_page` environment on the “left side”, then we open the `\__enumext_mini_page` environment on the “right side”, apply our adjusted “vertical spaces”, followed by adding the `\centering` command when the *starred argument* ‘`*`’ is not present and set zero `\g__enumext_minipage_stat_int`, otherwise we return an error.

```

1668 \cs_new_protected:Npn \__enumext_mini_right_cmd:n #1
1669 {
1670     \dim_compare:nNnTF
1671     { \dim_use:c { \l__enumext_minipage_right_ \__enumext_level: _dim } } > { \c_zero_dim }
1672     {
1673         \__enumext_multicols_stop:
1674         \int_compare:nNnT
1675         { \int_use:c { \l__enumext_columns_ \__enumext_level: _int } } = { 1 }
1676         {
1677             \par\addvspace{ \l__enumext_minipage_after_skip }
1678         }
1679         \end__enumext_mini_page
1680         \hfill
1681         \__enumext_mini_page{ \dim_use:c { \l__enumext_minipage_right_ \__enumext_level: _dim } }
1682         \par\nointerlineskip
1683         \addvspace { \l__enumext_minipage_right_skip }
1684         \bool_if:nF {#1}
1685         {
1686             \centering
1687         }
1688         \int_gzero:N \g__enumext_minipage_stat_int
1689     }
1690     { \msg_error:nnn { enumext } { wrong-miniright-use } }
1691 % paranoia
1692 \RenewDocumentCommand \miniright { s }
1693 {
1694     \msg_error:nn { enumext } { many-miniright-used }
1695 }
1696 }

```

(End of definition for `\__enumext_mini_right_cmd:n`.)

`\__enumext_keyans_mini_right_cmd:n`

The function `\__enumext_keyans_mini_right_cmd:n` takes as argument the *starred* ‘`*`’ of the `\miniright` command in the `keyans` environment. The implementation of this function is the same as that of the `\__enumext_mini_right_cmd:n` function of the `enumext` environment.

```

1697 \cs_new_protected:Npn \__enumext_keyans_mini_right_cmd:n #1
1698 {
1699     \dim_compare:nNnTF { \l__enumext_minipage_right_v_dim } > { \c_zero_dim }
1700     {
1701         \__enumext_keyans_multicols_stop:
1702         \int_compare:nNnT { \l__enumext_columns_v_int } = { 1 }
1703         {
1704             \par\addvspace{ \l__enumext_minipage_after_skip }
1705         }
1706         \end__enumext_mini_page

```

```

1707     \hfill
1708     \__enumext_mini_page{ \__enumext_minipage_right_v_dim }
1709     \par\nointerlineskip
1710     \addvspace { \__enumext_minipage_right_skip }
1711     \bool_if:nF {#1}
1712     {
1713         \centering
1714     }
1715     \int_gzero:N \g__enumext_minipage_stat_int
1716 }
1717 { \msg_error:nnn { enumext } { wrong-miniright-use } }
1718 % paranoia
1719 \RenewDocumentCommand \miniright { s }
1720 {
1721     \msg_error:nn { enumext } { many-miniright-used }
1722 }
1723 }

```

(End of definition for __enumext_keyans_mini_right_cmd:n.)

13.25 Setting above and below keys

While having controlled the *vertical spaces* within the `enumext` and `keyans` environments when using the `columns` or `mini-env` keys, sometimes the “vertical spaces above” or “vertical spaces below” the environments are not as expected and it is necessary to be able to apply a “fine correction” to these. As I have not been able to correct these *glitches*, the best option is to leave a couple of *keys* dedicated to this purpose, in this case it is best to use `\vspace` or `\vspace*` when convenient.

Define `above`, `above*`, `below` and `below*` keys for `enumext` and `keyans` environments.

```

1724 \cs_set_protected:Npn \__enumext_tmp:nn #1 #2
1725 {
1726     \keys_define:nn { enumext / #1 }
1727     {
1728         above .skip_set:c = { \__enumext_vspace_above_#2_skip },
1729         above .value_required:n = true,
1730         above* .code:n      = \bool_set_true:c { \__enumext_vspace_a_star_#2_bool }
1731                             \keys_set:nn { enumext / #1 } { above = {##1} },
1732         above* .value_required:n = true,
1733         below .skip_set:c = { \__enumext_vspace_below_#2_skip },
1734         below .value_required:n = true,
1735         below* .code:n      = \bool_set_true:c { \__enumext_vspace_b_star_#2_bool }
1736                             \keys_set:nn { enumext / #1 } { below = {##1} },
1737         below* .value_required:n = true,
1738     }
1739 }
1740 \clist_map_inline:Nn \c__enumext_all_envs_clist { \__enumext_tmp:nn #1 }

```

(End of definition for `above` and others.)

13.25.1 Functions for above and below keys in enumext

`\__enumext_vspace_above:` The function `\__enumext_vspace_above:` apply the *vertical space above* the `enumext` environment set by the `above*` and `above` keys.

```

1741 \cs_new_protected:Nn \__enumext_vspace_above:
1742 {
1743     \skip_if_eq:nnF
1744     { \skip_use:c { \__enumext_vspace_above_ \__enumext_level: _skip } } { \c_zero_skip }
1745     {
1746         \bool_if:cTF { \__enumext_vspace_a_star_ \__enumext_level: _bool }
1747         {
1748             \vspace*{ \skip_use:c { \__enumext_vspace_above_ \__enumext_level: _skip } }
1749         }
1750         {
1751             \vspace { \skip_use:c { \__enumext_vspace_above_ \__enumext_level: _skip } }
1752         }
1753     }
1754 }

```

(End of definition for `\__enumext_vspace_above:`.)

`\__enumext_vspace_below:` The function `\__enumext_vspace_below:` apply the *vertical space below* the `enumext` environment set by the `below*` and `below` keys.

```

1755 \cs_new_protected:Nn \__enumext_vspace_below:
1756 {
1757   \skip_if_eq:nnF
1758   { \skip_use:c { \l__enumext_vspace_below_ \__enumext_level: _skip } } { \c_zero_skip }
1759   {
1760     \bool_if:cTF { \l__enumext_vspace_b_star_ \__enumext_level: _bool }
1761     {
1762       \vspace*{ \skip_use:c { \l__enumext_vspace_below_ \__enumext_level: _skip } }
1763     }
1764     {
1765       \vspace { \skip_use:c { \l__enumext_vspace_below_ \__enumext_level: _skip } }
1766     }
1767   }
1768 }

```

(End of definition for `\__enumext_vspace_below:`.)

13.25.2 Functions for above and below keys in keyans

`\__enumext_vspace_above_v:` The function `\__enumext_vspace_above_v:` apply the *vertical space above* the `keyans` environment set by the `above` and `above*` keys.

```

1769 \cs_new_protected:Nn \__enumext_vspace_above_v:
1770 {
1771   \skip_if_eq:nnF { \l__enumext_vspace_above_v_skip } { \c_zero_skip }
1772   {
1773     \bool_if:NTF \l__enumext_vspace_a_star_v_bool
1774     {
1775       \vspace*{ \l__enumext_vspace_above_v_skip }
1776     }
1777     { \vspace { \l__enumext_vspace_above_v_skip } }
1778   }
1779 }

```

(End of definition for `\__enumext_vspace_above_v:`.)

`\__enumext_vspace_below_v:` The function `\__enumext_vspace_below_v:` apply the *vertical space below* the `keyans` environment set by the `below*` and `below` keys.

```

1780 \cs_new_protected:Nn \__enumext_vspace_below_v:
1781 {
1782   \skip_if_eq:nnF { \l__enumext_vspace_below_v_skip } { \c_zero_skip }
1783   {
1784     \bool_if:NTF \l__enumext_vspace_b_star_v_bool
1785     {
1786       \vspace*{ \l__enumext_vspace_below_v_skip }
1787     }
1788     { \vspace { \l__enumext_vspace_below_v_skip } }
1789   }
1790 }

```

(End of definition for `\__enumext_vspace_below_v:`.)

13.25.3 Functions for above and below keys in enumext* keyans*

`\__enumext_vspace_above_vii:` The functions `\__enumext_vspace_above_vii:` and `\__enumext_vspace_above_viii:` apply the *vertical space above* the `enumext*` and `keyans*` environments set by the `above` and `above*` keys.

`\__enumext_vspace_above_viii:`

```

1791 \cs_new_protected:Nn \__enumext_vspace_above_vii:
1792 {
1793   \skip_if_eq:nnF { \l__enumext_vspace_above_vii_skip } { \c_zero_skip }
1794   {
1795     \bool_if:NTF \l__enumext_vspace_a_star_vii_bool
1796     {
1797       \vspace*{ \l__enumext_vspace_above_vii_skip }
1798     }
1799     { \vspace { \l__enumext_vspace_above_vii_skip } }
1800   }
1801 }
1802 \cs_new_protected:Nn \__enumext_vspace_above_viii:
1803 {
1804   \skip_if_eq:nnF { \l__enumext_vspace_above_viii_skip } { \c_zero_skip }
1805   {

```

```

1806         \bool_if:NTF \l__enumext_vspace_a_star_viii_bool
1807         {
1808             \vspace*{ \l__enumext_vspace_above_viii_skip }
1809         }
1810         { \vspace { \l__enumext_vspace_above_viii_skip } }
1811     }
1812 }

```

(End of definition for `\__enumext_vspace_above_vii:` and `\__enumext_vspace_above_viii:`)

The functions `\__enumext_vspace_below_vii:` and `\__enumext_vspace_below_viii:` apply the *vertical space below* the `enumext*` and `keyans*` environments set by the `below*` and `below` keys.

```

1813 \cs_new_protected:Nn \__enumext_vspace_below_vii:
1814 {
1815     \skip_if_eq:nnF { \l__enumext_vspace_below_vii_skip } { \c_zero_skip }
1816     {
1817         \bool_if:NTF \l__enumext_vspace_b_star_vii_bool
1818         {
1819             \vspace*{ \l__enumext_vspace_below_vii_skip }
1820         }
1821         { \vspace { \l__enumext_vspace_below_vii_skip } }
1822     }
1823 }
1824 \cs_new_protected:Nn \__enumext_vspace_below_viii:
1825 {
1826     \skip_if_eq:nnF { \l__enumext_vspace_below_viii_skip } { \c_zero_skip }
1827     {
1828         \bool_if:NTF \l__enumext_vspace_b_star_viii_bool
1829         {
1830             \vspace*{ \l__enumext_vspace_below_viii_skip }
1831         }
1832         { \vspace { \l__enumext_vspace_below_viii_skip } }
1833     }
1834 }

```

(End of definition for `\__enumext_vspace_below_vii:` and `\__enumext_vspace_below_viii:`)

13.26 Setting series, resume and resume* keys

The `series` key is responsible for the whole process of the `resume` and `resume*` keys. The idea behind this is to be able to absorb the *⟨keys⟩* passed to the *optional argument* of the environments `enumext` and `enumext*`, but, discarding some specific *⟨keys⟩*.

We define the keys `series`, `resume` and `resume*` for the “all levels” of `enumext` and `enumext*`. Here we do not need to make sure that `\printkeyans` is not running otherwise the *start value* of the environments would be increased when using `resume` or `resume*` keys.

In version 1.6 it is allowed to pass the key `resume without value` by means of the command `\setenumext`, for the correct operation of this we must set the boolean variable `\l__enumext_resume_count_bool` set by the key `resume without value` to “true” to be later processed by the function `\__enumext_parse_series:n` in the definition of the environments `enumext` and `enumext*`.

```

1835 \cs_set_protected:Npn \__enumext_tmp:n #1 #2
1836 {
1837     \keys_define:nn { enumext / #1 }
1838     {
1839         series .str_set:N = \l__enumext_series_name_str,
1840         series .value_required:n = true,
1841         resume .code:n = {
1842             \bool_if:NF \l__enumext_print_keyans_cmd_bool
1843             {
1844                 \tl_set:Nn \l__enumext_series_name_tl {##1}
1845                 \tl_if_empty:NTF \l__enumext_series_name_tl
1846                 {
1847                     \bool_set_true:c { l__enumext_resume_count_#2_bool }
1848                     \bool_set_eq:Nc
1849                     \l__enumext_resume_count_bool
1850                     { l__enumext_resume_count_#2_bool }
1851                 }
1852                 {
1853                     \__enumext_resume:n {##1}
1854                 }
1855             }
1856         }
1857     }

```

```

1856         },
1857         resume* .code:n = {
1858             \bool_if:NF \l__enumext_print_keyans_cmd_bool
1859             {
1860                 \bool_set_true:c { \l__enumext_resume_count_#2_bool }
1861                 \bool_set_eq:Nc
1862                 \l__enumext_resume_count_bool { \l__enumext_resume_count_#2_bool }
1863                 \bool_set_true:c { \l__enumext_resume_star_key_#2_bool }
1864                 \__enumext_resume_star:
1865             }
1866         },
1867         resume* .value_forbidden:n = true,
1868     }
1869 }
1870 \clist_map_inline:nn
1871 {
1872     {level-1}{i}, {level-2}{ii}, {level-3}{iii}, {level-4}{iv},{enumext*}{vii},
1873 }
1874 { \__enumext_tmp:nn #1 }

```

(End of definition for *series*, *resume*, and *resume**.)

13.26.1 Internal function to save counter and integer values

```

\__enumext_standar_save_counter:
\__enumext_standar_save_counter_aux:
\__enumext_starred_save_counter:
\__enumext_starred_save_counter_aux:

```

The `\__enumext_standar_save_counter:` and `\__enumext_starred_save_counter:` functions will save the last counter value to `\g__enumext_series_⟨series name⟩_int` if the `series={⟨series name⟩}` key has been passed, to `\c@__enumext_resume_X_int` if it has passed the key `resume` *without value* and the key `series` is not active, in `\g__enumext_series_⟨series name⟩_X_int` if the key `resume={⟨series name⟩}` has been passed and in `\g__enumext_series_⟨store name⟩_X_int` if the key has been passed `save-ans={⟨store name⟩}`.

🔗 The variables `\l__enumext_series_name_str` and `\l__enumext_series_name_tl` contain the same `{⟨series name⟩}` but are executed at different moments, the integer variable with `\l__enumext_series_name_str` sets the value when execute `series={⟨series name⟩}` and the integer variable with `\l__enumext_series_name_tl` sets the subsequent values when use `resume={⟨series name⟩}`. This function is passed to the `enumext` environment definition (§13.42) and the `enumext*` environment definition (§13.47).

```

1875 \cs_new_protected:Nn \__enumext_standar_save_counter:
1876 {
1877     \bool_if:NTF \g__enumext_standar_bool
1878     {
1879         \__enumext_standar_save_counter_aux:
1880         \int_compare:nNnT { \l__enumext_level_int } = { 1 }
1881         {
1882             \int_if_exist:cT { g__enumext_resume_ \l__enumext_store_name_tl _int }
1883             {
1884                 \int_gset_eq:cN
1885                 { g__enumext_resume_ \l__enumext_store_name_tl _int } \value{enumXi}
1886             }
1887         }
1888     }
1889     {
1890         \__enumext_standar_save_counter_aux:
1891     }
1892 }
1893 \cs_new_protected:Nn \__enumext_standar_save_counter_aux:
1894 {
1895     \str_if_empty:NF \l__enumext_series_name_str
1896     {
1897         \int_gset_eq:cc
1898         { g__enumext_series_ \l__enumext_series_name_str _ \l__enumext_level: _int }
1899         { c@enumX \__enumext_level: }
1900     }
1901     \tl_if_empty:NTF \l__enumext_series_name_tl
1902     {
1903         \str_if_empty:NT \l__enumext_series_name_str
1904         {
1905             \tl_if_empty:NT \l__enumext_store_name_tl
1906             {
1907                 \int_gset_eq:cc
1908                 { c@__enumext_resume_ \l__enumext_level: _int } { c@enumX \l__enumext_level: }
1909             }
1910         }

```

```

1911     }
1912     {
1913         \int_if_exist:cT
1914         { g__enumext_series_ \l__enumext_series_name_tl _ \__enumext_level: _int }
1915         {
1916             \int_gset_eq:cc
1917             { g__enumext_series_ \l__enumext_series_name_tl _ \__enumext_level: _int }
1918             { c@enumX \__enumext_level: }
1919         }
1920     }
1921 }
1922 \cs_new_protected:Nn \__enumext_starred_save_counter:
1923 {
1924     \bool_if:NTF \g__enumext_starred_bool
1925     {
1926         \__enumext_starred_save_counter_aux:
1927         \int_if_exist:cT { g__enumext_resume_ \l__enumext_store_name_tl _int }
1928         {
1929             \int_gset_eq:cN
1930             { g__enumext_resume_ \l__enumext_store_name_tl _int } \value{enumXvii}
1931         }
1932     }
1933     {
1934         \__enumext_starred_save_counter_aux:
1935     }
1936 }
1937 \cs_new_protected:Nn \__enumext_starred_save_counter_aux:
1938 {
1939     \str_if_empty:NF \l__enumext_series_name_str
1940     {
1941         \int_gset_eq:cN
1942         { g__enumext_series_ \l__enumext_series_name_str _vii_int } \value{enumXvii}
1943     }
1944     \tl_if_empty:NTF \l__enumext_series_name_tl
1945     {
1946         \str_if_empty:NT \l__enumext_series_name_str
1947         {
1948             \tl_if_empty:NT \l__enumext_store_name_tl
1949             {
1950                 \int_gset_eq:cc { c@ __enumext_resume_vii_int } { c@enumXvii }
1951             }
1952         }
1953     }
1954     {
1955         \int_if_exist:cT { g__enumext_series_ \l__enumext_series_name_tl _vii_int }
1956         {
1957             \int_gset_eq:cN
1958             { g__enumext_series_ \l__enumext_series_name_tl _vii_int } \value{enumXvii}
1959         }
1960     }
1961 }

```

(End of definition for __enumext_standar_save_counter: and others.)

13.26.2 Internal function for resume counters

__enumext_resume_counter:

The __enumext_resume_counter: function is executed by the `resume*` key and `resume` key *without value*, only the “counters” for the “levels” of the environments in which it is executed will be set. If the `save-ans` key is active it will set the “counter” according to the value of the integer variable created by that key.

```

1962 \cs_new_protected:Nn \__enumext_resume_counter:
1963 {
1964     \cs_set:Npn \__enumext_tmp:n ##1
1965     {
1966         \bool_if:cT { l__enumext_resume_count_ \int_to_roman:n {##1} _bool }
1967         {
1968             \exp_args:Ne \int_set:cn { l__enumext_start_ \int_to_roman:n {##1} _int }
1969             {
1970                 \int_use:c { c@ __enumext_resume_ \int_to_roman:n {##1} _int } + 1
1971             }
1972         }
1973     }
1974     \int_compare:nNnT { \l__enumext_level_int } > { 0 }

```

```

1975     {
1976         \bool_lazy_and:nnTF
1977         { \bool_if_p:N \l__enumext_standar_first_bool }
1978         { \bool_if_p:N \l__enumext_store_active_bool }
1979         {
1980             \int_set:Nn \l__enumext_start_i_int
1981             {
1982                 \int_use:c { g__enumext_resume_ \l__enumext_store_name_tl _int } + 1
1983             }
1984         }
1985         {
1986             \int_step_function:nN { \l__enumext_level_int } \__enumext_tmp:n
1987         }
1988     }
1989     \int_compare:nNnT { \l__enumext_level_h_int } = { 1 }
1990     {
1991         \bool_lazy_and:nnTF
1992         { \bool_if_p:N \l__enumext_starred_first_bool }
1993         { \bool_if_p:N \l__enumext_store_active_bool }
1994         {
1995             \int_set:Nn \l__enumext_start_vii_int
1996             {
1997                 \int_use:c { g__enumext_resume_ \l__enumext_store_name_tl _int } + 1
1998             }
1999         }
2000         {
2001             \bool_if:NT \l__enumext_resume_count_vii_bool
2002             {
2003                 \int_set:Nn \l__enumext_start_vii_int
2004                 {
2005                     \int_use:c { c@ __enumext_resume_vii_int } + 1
2006                 }
2007             }
2008         }
2009     }
2010 }

```

(End of definition for __enumext_resume_counter:.)

13.26.3 Internal functions for series key

```

\__enumext_filter_series:n
  \__enumext_filter_series_key:n
  \__enumext_filter_series_pair:nn

```

The function `\__enumext_filter_series:n` will be in charge of filtering the *⟨keys⟩* we want to store where *#1* represents the *optional argument* passed to the environment. This implementation is adapted directly from the code provided by Jonathan P. Spratte (@Skillmon) in [chat-Tex-SX](#)

```

2011 \cs_new:Npn \__enumext_filter_series:n #1
2012 {
2013     \use:e
2014     {
2015         \keyval_parse:NNn
2016         \__enumext_filter_series_key:n
2017         \__enumext_filter_series_pair:nn {#1}
2018     }
2019 }

```

The function `\__enumext_filter_series_key:n` will be responsible for filtering the *⟨keys⟩* that are passed “without value” by excluding the `resume`, `resume*`, `reset`, `reset*` and `base-fix` keys.

```

2020 \cs_new:Npn \__enumext_filter_series_key:n #1
2021 {
2022     \str_case:nnF {#1}
2023     {
2024         { resume } {} { resume* } {} { reset } {} { reset* } {} { base-fix } {}
2025     }
2026     { , { \exp_not:n {#1} } }
2027 }

```

The function `\__enumext_filter_series_pair:nn` will be responsible for filtering the *⟨keys⟩* that are passed “with value” by excluding the `series`, `resume`, `start`, `start*`, `save-ans` and `save-key` keys.

```

2028 \cs_new:Npn \__enumext_filter_series_pair:nn #1#2
2029 {
2030     \str_case:nnF {#1}
2031     {
2032         { series } {} { resume } {} { start } {}
2033         { start* } {} { save-ans } {} { save-key } {}

```

```

2034     }
2035     { , { \exp_not:n {#1} } = { \exp_not:n {#2} } }
2036 }

```

(End of definition for `\__enumext_filter_series:n`, `\__enumext_filter_series_key:n`, and `\__enumext_filter_series_pair:nn`.)

`\__enumext_save_last_keys:n` The function `\__enumext_save_last_keys:n` will be in charge of saving the filtering (*keys*) when the keys `series={⟨series name⟩}` or `resume={⟨series name⟩}` or `resume*` are NOT active and will save them in the variable `\g__enumext_resume_last_keys_X_tl` for the `enumext` environment and in the variable `\g__enumext_resume_last_keys_vii_tl` for the `enumext*` environment.

- The boolean variable `\l__enumext_resume_series_X_bool` is set to “true” by the key `resume={⟨series name⟩}`, the boolean variable `\l__enumext_resume_star_key_X_bool` is set to “true” by the key `resume*`, in this case we need to make sure both variables are set to “false” so that they don’t override the default filtered (*keys*).

```

2037 \cs_new_protected:Npn \__enumext_save_last_keys:n #1
2038 {
2039   \int_compare:nNtT { \l__enumext_level_int } > { 0 }
2040   {
2041     \bool_if:cF { \l__enumext_resume_series_ \__enumext_level: _bool }
2042     {
2043       \bool_if:cF { \l__enumext_resume_star_key_ \__enumext_level: _bool }
2044       {
2045         \tl_gclear:c { g__enumext_resume_last_keys_ \__enumext_level: _tl }
2046         \tl_gset:ce
2047           { g__enumext_resume_last_keys_ \__enumext_level: _tl }
2048           { \__enumext_filter_series:n {#1} }
2049       }
2050     }
2051   }
2052   \int_compare:nNtT { \l__enumext_level_h_int } = { 1 }
2053   {
2054     \bool_if:NF \l__enumext_resume_series_vii_bool
2055     {
2056       \tl_gclear:N \g__enumext_resume_last_keys_vii_tl
2057       \tl_gset:Ne \g__enumext_resume_last_keys_vii_tl { \__enumext_filter_series:n {#1} }
2058     }
2059   }
2060 }

```

The `\__enumext_resume_last_counter:` function will be in charge of setting the “counters” when the keys `series={⟨series name⟩}` or `resume={⟨series name⟩}` are NOT active and the `resume` key is being used *without value* either in the *optional argument* of the environments or through the `\setenumext` command.

- The boolean variable `\l__enumext_resume_count_bool` is set to “true” by the keys `resume without value` and `resume*`, and set to “false” by the keys `start` and `start*` (§13.15).

```

2061 \cs_new_protected:Nn \__enumext_resume_last_counter:
2062 {
2063   \bool_lazy_and:nnT
2064     { \bool_if_p:N \l__enumext_resume_count_bool }
2065     { \tl_if_empty_p:N \l__enumext_series_name_tl }
2066   {
2067     \__enumext_resume_counter:
2068   }
2069 }

```

(End of definition for `\__enumext_save_last_keys:n` and `\__enumext_resume_last_counter:`.)

`\__enumext_parse_series:n` The `\__enumext_parse_series:n` function handled by the `series` key will be responsible for *storing* the *filtered* (*keys*) from the *optional arguments* of the `enumext` and `enumext*` environments for the `resume` and `resume*` keys. If the `series` key is NOT active it will call the `\__enumext_save_last_keys:n` function to *store* the *filtered* (*keys*) that will be used by the `resume*` key and then the `\__enumext_resume_last_counter:` function used by the `resume` key *without value* if it is active, otherwise *store* the *filtered* (*keys*) in the global variable `\g__enumext_series_⟨series name⟩_X_tl` along with the creation of the integer variable `\g__enumext_series_⟨series name⟩_X_int` used by the `resume` key *with value*.

- This function is passed to the function `\__enumext_parse_keys:n` in the `enumext` environment definition (§13.42) and to the function `\__enumext_parse_keys_vii:n` in the `enumext*` environment definition (§13.47).

```

2070 \cs_new_protected:Npn \__enumext_parse_series:n #1
2071 {
2072   \str_if_empty:NTF \l__enumext_series_name_str
2073   {

```

```

2074     \int_compare:nNnT { \l__enumext_level_int } > { 0 }
2075     {
2076         \__enumext_save_last_keys:n {#1}
2077         \__enumext_resume_last_counter:
2078     }
2079     \int_compare:nNnT { \l__enumext_level_h_int } = { 1 }
2080     {
2081         \__enumext_save_last_keys:n {#1}
2082         \__enumext_resume_last_counter:
2083     }
2084 }
2085 {
2086     \int_compare:nNnT { \l__enumext_level_int } > { 0 }
2087     {
2088         \tl_gclear_new:c
2089         { g__enumext_series_ \l__enumext_series_name_str _ \__enumext_level: _tl }
2090         \tl_gset:ce
2091         { g__enumext_series_ \l__enumext_series_name_str _ \__enumext_level: _tl }
2092         { \__enumext_filter_series:n {#1} }
2093         \int_if_exist:cF
2094         { g__enumext_series_ \l__enumext_series_name_str _ \__enumext_level: _int }
2095         {
2096             \int_new:c
2097             { g__enumext_series_ \l__enumext_series_name_str _ \__enumext_level: _int }
2098         }
2099     }
2100     \int_compare:nNnT { \l__enumext_level_h_int } = { 1 }
2101     {
2102         \tl_gclear_new:c { g__enumext_series_ \l__enumext_series_name_str _vii_tl }
2103         \tl_gset:ce
2104         { g__enumext_series_ \l__enumext_series_name_str _vii_tl }
2105         { \__enumext_filter_series:n {#1} }
2106         \int_if_exist:cF { g__enumext_series_ \l__enumext_series_name_str _vii_int }
2107         {
2108             \int_new:c { g__enumext_series_ \l__enumext_series_name_str _vii_int }
2109         }
2110     }
2111 }
2112 }

```

(End of definition for __enumext_parse_series:n.)

13.26.4 Internal functions for resume key with value

__enumext_resume:n

The function __enumext_resume:n will handle the *argument* $\langle series\ name \rangle$ passed to the `resume` key in `enumext` and `enumext*` environments. First we will check if the global variable `\g__enumext_series_-\langle series\ name \rangle_X_tl` exists, if so we will call the function `\__enumext_resume_series:n` and pass the $\langle keys \rangle$ stored in `\g__enumext_series_-\langle series\ name \rangle_X_tl` to the environments, otherwise we will return an error.

```

2113 \cs_new_protected:Npn \__enumext_resume:n #1
2114 {
2115     \int_compare:nNnT { \l__enumext_level_int } > { 0 }
2116     {
2117         \tl_if_exist:cTF { g__enumext_series_ \tl_to_str:n {#1} _ \__enumext_level: _tl }
2118         {
2119             \__enumext_resume_series:n {#1}
2120             \exp_args:Ne \keys_set:nv { enumext / level-\int_use:N \l__enumext_level_int }
2121             { g__enumext_series_ \tl_to_str:n {#1} _ \__enumext_level: _tl }
2122         }
2123         {
2124             \msg_error:nnn { enumext } { unknown-series-standar } {#1}
2125         }
2126     }
2127     \int_compare:nNnT { \l__enumext_level_h_int } = { 1 }
2128     {
2129         \tl_if_exist:cTF { g__enumext_series_ \tl_to_str:n {#1} _vii_tl }
2130         {
2131             \__enumext_resume_series:n {#1}
2132             \keys_set:nv { enumext / enumext* }
2133             { g__enumext_series_ \tl_to_str:n {#1} _vii_tl }
2134         }
2135         {
2136             \msg_error:nnn { enumext } { unknown-series-starred } {#1}

```



```

2137     }
2138 }
2139 }

```

(End of definition for `\__enumext_resume:n`.)

```

\__enumext_resume_series:n
\__enumext_resume_integer_series:

```

The function `\__enumext_resume_series:n` will set the variable `\l__enumext_resume_series_X_bool` to “true” and pass the `{⟨argument⟩}` to the variable `\l__enumext_series_name_tl` then call the function `\__enumext_resume_integer_series:`.

```

2140 \cs_new_protected:Npn \__enumext_resume_series:n #1
2141 {
2142   \int_compare:nNnT { \l__enumext_level_int } > { 0 }
2143   {
2144     \bool_set_true:c { \l__enumext_resume_series_ \__enumext_level: _bool }
2145     \tl_clear:N \l__enumext_series_name_tl
2146     \tl_set:Nn \l__enumext_series_name_tl {#1}
2147     \__enumext_resume_integer_series:
2148   }
2149   \int_compare:nNnT { \l__enumext_level_h_int } = { 1 }
2150   {
2151     \bool_set_true:N \l__enumext_resume_series_vii_bool
2152     \tl_clear:N \l__enumext_series_name_tl
2153     \tl_set:Nn \l__enumext_series_name_tl {#1}
2154     \__enumext_resume_integer_series:
2155   }
2156 }

```

The function `\__enumext_resume_integer_series:` will be executed when the `resume={⟨series name⟩}` key is active, setting the *start value* for the “counter” of the “current level” of the environments in which it is run according to the value of the “integer variables” created by the `series` key. If the `save-ans` key is active it will set the *start value* for the “counter” according to the value of the integer variable created by that key.

```

2157 \cs_new_protected:Nn \__enumext_resume_integer_series:
2158 {
2159   \cs_set:Npn \__enumext_tmp:n ##1
2160   {
2161     \int_if_exist:cT { g__enumext_series_ \l__enumext_series_name_tl _ \int_to_roman:n {##1} }
2162     {
2163       \exp_args:Ne \int_set:cn { \l__enumext_start_ \int_to_roman:n {##1} _int }
2164       {
2165         \int_use:c { g__enumext_series_ \l__enumext_series_name_tl _ \int_to_roman:n {##1} }
2166       }
2167     }
2168   }
2169   \int_compare:nNnT { \l__enumext_level_int } > { 0 }
2170   {
2171     \bool_lazy_and:nnTF
2172     { \bool_if_p:N \l__enumext_standar_first_bool }
2173     { \bool_if_p:N \l__enumext_store_active_bool }
2174     {
2175       \int_set:Nn \l__enumext_start_i_int
2176       {
2177         \int_use:c { g__enumext_resume_ \l__enumext_store_name_tl _int } + 1
2178       }
2179     }
2180     {
2181       \int_step_function:nN { \l__enumext_level_int } \l__enumext_tmp:n
2182     }
2183   }
2184   \int_compare:nNnT { \l__enumext_level_h_int } = { 1 }
2185   {
2186     \bool_lazy_and:nnTF
2187     { \bool_if_p:N \l__enumext_starred_first_bool }
2188     { \bool_if_p:N \l__enumext_store_active_bool }
2189     {
2190       \int_set:Nn \l__enumext_start_vii_int
2191       {
2192         \int_use:c { g__enumext_resume_ \l__enumext_store_name_tl _int } + 1
2193       }
2194     }
2195     {
2196       \int_set:Nn \l__enumext_start_vii_int

```

```

2197         {
2198             \int_use:c { g__enumext_series_ \l__enumext_series_name_tl _vii_int } + 1
2199         }
2200     }
2201 }
2202 }

```

(End of definition for __enumext_resume_series:n and __enumext_resume_integer_series:.)

13.26.5 Internal function for resume* key

__enumext_resume_star:

The function __enumext_resume_star: will handle the `resume*` key in the `enumext` and `enumext*` environments. This function will execute the filtered `<keys>` in the last one and will continue with the numbering and `<keys>` according to the last execution of the environment `enumext` or `enumext*` in which the keys `resume={<series name>}` or `series={<series name>}` were NOT active.

```

2203 \cs_new_protected:Nn \__enumext_resume_star:
2204 {
2205     \cs_set:Npn \__enumext_tmp:n ##1
2206     {
2207         \tl_if_empty:cTF { g__enumext_resume_last_keys_ \int_to_roman:n {##1} _tl }
2208         {
2209             \__enumext_resume_counter:
2210         }
2211         {
2212             \__enumext_resume_counter:
2213             \exp_args:Ne \keys_set:nv
2214             { enumext / level-\int_use:N \l__enumext_level_int }
2215             { g__enumext_resume_last_keys_ \int_to_roman:n {##1} _tl }
2216         }
2217     }
2218     \int_compare:nNt { \l__enumext_level_int } > { 0 }
2219     {
2220         \int_step_function:nN { \l__enumext_level_int } \__enumext_tmp:n
2221     }
2222     \int_compare:nNt { \l__enumext_level_h_int } = { 1 }
2223     {
2224         \tl_if_empty:NTF \g__enumext_resume_last_keys_vii_tl
2225         {
2226             \__enumext_resume_counter:
2227         }
2228         {
2229             \__enumext_resume_counter:
2230             \keys_set:nV { enumext / enumext* } \g__enumext_resume_last_keys_vii_tl
2231         }
2232     }
2233 }

```

(End of definition for __enumext_resume_star:.)

13.27 The \resetenumext command

Sometimes it is necessary to be able to reset the “counters” of the environments according to some value, for example `\chapter`. Since we use “internal counters” for the `resume` and `resume*` keys which set the *start value*, but are not accessible by the user, it is to provide a public command for this. This implementation is an adaptation of the answers given by Clea F. Rees (@cfr) and Jonathan P. Spratte (@Skillmon) in [Correct implementation of optional argument \(comma-separated\) in expl3](#).

\resetenumext

The `\resetenumext` command “resets” the *start value* of the “counters” for the `enumext` and `enumext*` environments along with the “internal counters” used by the keys `resume without value` and `resume*` according to the value of `{<some counter>}`.

```

2234 \NewDocumentCommand \resetenumext { s o m }
2235 {
2236     \bool_if:nTF {#1}
2237     {
2238         \__enumext_reset_count_resume_all:n {#3}
2239     }
2240     {
2241         \tl_if_novalue:nTF {#2}
2242         {
2243             \__enumext_reset_count_resume_levels:n {#3}
2244         }
2245         {

```

```

2246         \str_if_eq:nnTF {#2} { * }
2247         { \__enumext_starred_reset:n {#3} }
2248         {
2249             \bool_lazy_and:nnTF
2250             { \int_compare_p:nNn {#2} > 0 }
2251             { \int_compare_p:nNn {#2} < 5 }
2252             { \__enumext_standard_reset:nn {#2} {#3} }
2253             {
2254                 \msg_error:nne { enumext } { out-of-range } { \int_eval:n {#2} }
2255             }
2256         }
2257     }
2258 }
2259 }
2260 \cs_new_protected:Npn \__enumext_standard_reset:nn #1 %#2
2261 {
2262     \__enumext_reset_count_resume:en { \int_to_roman:n {#1} } {%#2}
2263 }
2264 \cs_new_protected:Npn \__enumext_starred_reset:n #1
2265 {
2266     \__enumext_reset_count_resume:nn { vii } {#1}
2267 }
2268 \cs_new_protected:Npn \__enumext_reset_count_resume:nn #1 #2
2269 {
2270     \counterwithin*{enumX#1}{#2}
2271     \counterwithin*{\__enumext_resume_#1_int}{#2}
2272 }
2273 \cs_generate_variant:Nn \__enumext_reset_count_resume:nn { e }
2274 \cs_new_protected:Npn \__enumext_reset_count_resume_all:n #1
2275 {
2276     \clist_map_inline:nn { i,ii,iii,iv,vii }
2277     {
2278         \__enumext_reset_count_resume:nn { ##1 } { #1 }
2279     }
2280 }
2281 \cs_new_protected:Npn \__enumext_reset_count_resume_levels:n #1
2282 {
2283     \clist_map_inline:nn { i,ii,iii,iv }
2284     {
2285         \__enumext_reset_count_resume:nn { ##1 } { #1 }
2286     }
2287 }

```

(End of definition for `\resetenumext` and others. This function is documented on page 11.)

13.28 The reset and reset* keys

The `\resetenumext` command does not work, for example, after an unnumbered chapter, so it is preferable to provide a pair of *keys* that adjust the internal variables if necessary.

```

reset We define the keys reset and reset* for the “all levels” of enumext and enumext*.
reset*
2288 \cs_set_protected:Npn \__enumext_tmp:n #1
2289 {
2290     \keys_define:nn { enumext / #1 }
2291     {
2292         reset .code:n = \__enumext_standard_reset_key:,
2293         reset .value_forbidden:n = true,
2294         reset* .code:n = \__enumext_standard_reset_key_star:,
2295         reset* .value_forbidden:n = true,
2296     }
2297 }
2298 \clist_map_inline:nn {level-1, level-2, level-3, level-4} { \__enumext_tmp:n {#1} }
2299 \keys_define:nn { enumext / enumext* }
2300 {
2301     reset .code:n = \__enumext_starred_reset_key:,
2302     reset .value_forbidden:n = true,
2303     reset* .code:n = \__enumext_starred_reset_key:,
2304     reset* .value_forbidden:n = true,
2305 }

```

(End of definition for `reset` and `reset*`.)

13.28.1 Internal functions for reset and reset* keys

The function `\__enumext_standard_reset_key:` will be handled by the `reset` key and will “reset” the counter `\c@__enumext_resume_X_int` to “zero” according to the *level* at which it is executed within the `enumext` environment.

```

2306 \cs_new_protected:Nn \__enumext_standard_reset_key:
2307 {
2308   \int_compare:nNtT { \__enumext_level_int } > { 0 }
2309   {
2310     \int_if_exist:cT { c@__enumext_resume_ \int_to_roman:n { \__enumext_level_int } _int }
2311     {
2312       \int_gzero:c { c@__enumext_resume_ \int_to_roman:n { \__enumext_level_int } _int }
2313     }
2314   }
2315 }

```

The function `\__enumext_standard_reset_key_star:` will be handled by the `reset*` key and will “reset” the counters `\c@__enumext_resume_X_int` to “zero” from the *level* at which it is executed within the `enumext` environment to the *lower levels*.

```

2316 \cs_new_protected:Nn \__enumext_standard_reset_key_star:
2317 {
2318   \cs_set:Npn \__enumext_tmp:n ##1
2319   {
2320     \int_if_exist:cT { c@__enumext_resume_ \int_to_roman:n { ##1 } _int }
2321     {
2322       \int_gzero:c { c@__enumext_resume_ \int_to_roman:n { ##1 } _int }
2323     }
2324   }
2325   \int_compare:nNtT { \__enumext_level_int } > { 0 }
2326   {
2327     \int_step_function:nnN { \__enumext_level_int } { 4 } \__enumext_tmp:n
2328   }
2329 }

```

The function `\__enumext_starred_reset_key:` will be handled by `reset` keys and `reset*` will “reset” the counter `\c@__enumext_resume_vii_int` to “zero” when executed in the `enumext*` environment.

```

2330 \cs_new_protected:Nn \__enumext_starred_reset_key:
2331 {
2332   \int_gzero:c { c@__enumext_resume_vii_int }
2333 }

```

(End of definition for `\__enumext_standard_reset_key:`, `\__enumext_standard_reset_key_star:`, and `\__enumext_starred_reset_key:`.)

13.29 Setting save-ans, check-ans and no-store keys

The key `save-ans` is directly associated with the keys `check-ans`, `no-store`, `resume` and `resume*`, this will activate the entire “storage system” in the `enumext` package.

13.29.1 Setting save-ans key

`save-ans` We define the keys `save-ans` only for the “first level” of `enumext` and `enumext*`.

```

2334 \cs_set_protected:Npn \__enumext_tmp:n #1
2335 {
2336   \keys_define:nn { enumext / #1 }
2337   {
2338     save-ans .code:n = \__enumext_storing_set:n { ##1 },
2339     save-ans .value_required:n = true,
2340   }
2341 }
2342 \clist_map_inline:nn { level-1, enumext* } { \__enumext_tmp:n { ##1 } }

```

(End of definition for `save-ans`.)

13.29.2 Internal functions for save-ans key

The functions `\__enumext_start_save_ans_msg:` and `\__enumext_stop_save_ans_msg:` will display in the terminal and .log file the environment in which the `save-ans` key was executed along with the line at the beginning and end of it. The function `\__enumext_start_save_ans_msg:` will be passed to `\__enumext_storing_set:n` and the function `\__enumext_stop_save_ans_msg:` will be passed to the function `\__enumext_execute_after_env:`.

```

2343 \cs_new_protected:Nn \__enumext_start_save_ans_msg:
2344 {
2345   \msg_term:nnVV { enumext } { save-ans-log }

```

```

2346     \g__enumext_envir_name_tl \l__enumext_store_name_tl
2347   }
2348 \cs_new_protected:Nn \__enumext_stop_save_ans_msg:
2349 {
2350   \msg_term:nnVV { enumext } { save-ans-log-hook }
2351   \g__enumext_envir_name_tl \g__enumext_store_name_tl
2352 }

```

(End of definition for __enumext_start_save_ans_msg: and __enumext_stop_save_ans_msg:.)

__enumext_storing_set:n
 __enumext_storing_exec:

The function __enumext_storing_set:n first pass the value of the `save-ans` key to the variable \l__enumext_store_name_tl which will contain the {<store name>} of the *sequence* and *prop list* we will use. If \l__enumext_store_name_tl is *empty* we return an error message, otherwise will return the appropriate message __enumext_start_save_ans_msg: and proceed to execute the function __enumext_storing_exec: for `enumext` and `enumext*` environments.

```

2353 \cs_new_protected:Npn \__enumext_storing_set:n #1
2354 {
2355   \tl_set:Nx \l__enumext_store_name_tl {#1}
2356   \tl_if_empty:NTF \l__enumext_store_name_tl
2357   {
2358     \bool_lazy_or:nnT
2359     { \l__enumext_standar_first_bool } { \l__enumext_starred_first_bool }
2360     {
2361       \msg_error:nnV { enumext } { save-ans-empty } \g__enumext_envir_name_tl
2362     }
2363   }
2364   {
2365     \bool_lazy_or:nnT
2366     { \l__enumext_standar_first_bool } { \l__enumext_starred_first_bool }
2367     {
2368       \__enumext_start_save_ans_msg:
2369       \__enumext_storing_exec:
2370     }
2371   }
2372 }

```

The function __enumext_storing_exec: will set to true the variable \l__enumext_store_active_bool which activates the use of the `\anskey` command and the `anskey*`, `keyans`, `keyans*` and `keyanspic` environments and will set to “true” the variable \l__enumext_check_answers_bool used for internal checking answers mechanism set by the `check-ans` and `no-store` keys, copy {<store name>} into the variable \g__enumext_store_name_tl.

```

2373 \cs_new_protected:Nn \__enumext_storing_exec:
2374 {
2375   \bool_set_true:N \l__enumext_store_active_bool
2376   \bool_set_true:N \l__enumext_check_answers_bool
2377   \tl_gset:NV \g__enumext_store_name_tl \l__enumext_store_name_tl

```

The *prop list* \g__enumext_series_<store name>_prop and the *sequence* \g__enumext_series_<store name>_seq will be created globally to “store content” in case they do not exist together with the integer variable \g__enumext_series_<store name>_int used by the keys `resume` and `resume*`.

```

2378   \prop_if_exist:cF { g__enumext_ \l__enumext_store_name_tl _prop }
2379   {
2380     \msg_log:nnV { enumext } { store-prop } \l__enumext_store_name_tl
2381     \prop_new:c { g__enumext_ \l__enumext_store_name_tl _prop }
2382   }
2383   \seq_if_exist:cF { g__enumext_ \l__enumext_store_name_tl _seq }
2384   {
2385     \msg_log:nnV { enumext } { store-seq } \l__enumext_store_name_tl
2386     \seq_new:c { g__enumext_ \l__enumext_store_name_tl _seq }
2387   }
2388   \int_if_exist:cF { g__enumext_resume_ \l__enumext_store_name_tl _int }
2389   {
2390     \msg_log:nnV { enumext } { store-int } \l__enumext_store_name_tl
2391     \int_new:c { g__enumext_resume_ \l__enumext_store_name_tl _int }
2392   }
2393 }

```

(End of definition for __enumext_storing_set:n and __enumext_storing_exec:.)

13.29.3 The check answer mechanism

The internal mechanism for “checking answers” follows this logic:

If the line begins with `\item` or `\item*` and does NOT *open a nested environment*, each `\item` or `\item*` must contain a *single* execution of the `\anskey` command, i.e. the counter of the executions of the `\anskey` command must be equal to the counter associated with the sum of executions of `\item` and `\item*`.

If the line begins with `\item` or `\item*` and *opens a nested environment* each `\item` or `\item*` in the nested environment must have a *single* execution of the `\anskey` command and the counter associated to the sum of `\item` and `\item*` executions must decrementing by “one” to maintain equality.

In order for the mechanism for the check-answer to work (not counting `keyans`, `keyans*` and `keyanspic`) we need:

1. We must keep track of the total number of `\item` and `\item*` (enumerated) that appear within the environment including the nested levels.
2. We must keep track of the total number of `\item` and `\item*` (enumerated) that appear per level of nesting.
3. Keeping track of the number of times the environment nests.

The integer variable associated to the sum of each `\item` and `\item*` in the environment `\g__enumext_item_number_int` must match the integer variable `\g__enumext_item_anskey_int` associated to the execution of the command `\anskey`. We analyze the cases:

- a) If the list only has one level the number of `\item` + `\item*` = `\anskey`
- b) If the list has *nested levels*, for each level of nesting we need to decrementing by one (for the `\item` or `\item*` that opens the nest) so that the account remains the same.

With `keyans`, `keyans*` and `keyanspic` it is enough to increase in one the integer of `\anskey`. The integers created must be global if they are not lost in the interior levels of nesting and to execute the test we will use a “hook” function after closing the *first level* of the environment.

13.29.4 Setting check-ans and no-store keys

Now we define the keys `check-ans` and `no-store` for all levels of `enumext` and `enumext*` environments.

```

check-ans 2394 \cs_set_protected:Npn \__enumext_tmp:n #1
no-store   2395 {
            2396   \keys_define:nn { enumext / #1 }
            2397   {
            2398     check-ans .bool_set:N = \__enumext_check_ans_key_bool,
            2399     check-ans .initial:n = false,
            2400     check-ans .value_required:n = true,
            2401     no-store .code:n = {
            2402               \bool_set_false:N \__enumext_check_answers_bool
            2403               \bool_set_false:N \__enumext_check_ans_key_bool
            2404             },
            2405     no-store .value_forbidden:n = true,
            2406   }
            2407 }
            2408 \clist_map_inline:nn
            2409 {
            2410   level-1, level-2, level-3, level-4, enumext*
            2411 }
            2412 { \__enumext_tmp:n {#1} }
```

(End of definition for `check-ans` and `no-store`.)

13.29.5 Set-up check answer mechanism

The function `\__enumext_check_ans_active:` will first check the state of the variable `\l__enumext_store_name_tl`, that is, the `save-ans` key is active, if so it will check the state of the variable `\l__enumext_check_answers_bool` handled by the key `no-store` and will execute the function `\__enumext_check_ans_level:` only if “true”, i.e. the key `no-store` is not active.

```

2413 \cs_new_protected:Nn \__enumext_check_ans_active:
2414 {
2415   \tl_if_empty:NF \l__enumext_store_name_tl
2416   {
2417     \bool_if:NT \l__enumext_check_answers_bool
2418     {
2419       \__enumext_check_ans_level:
2420     }
2421   }
2422 }
```

The function `\__enumext_check_ans_level:` will decrement by “one” the value of the variable `\g__enumext_item_number_int` which keeps track of the executions of `\item` and `\item*` for each level of nesting of the environment `enumext`, taking into account whether it is nested within `enumext*` or the opposite and set `\l__enumext_item_number_bool` to “false”.

```

2423 \cs_new_protected:Nn \__enumext_check_ans_level:
2424 {
2425   \int_case:nn { \l__enumext_level_int }
2426   {
2427     { 1 }{
2428       \bool_lazy_all:nT
2429       {
2430         { \bool_if_p:N \g__enumext_starred_bool }
2431         { \int_compare_p:nNn { \l__enumext_level_h_int } = { 1 } }
2432       }
2433       {
2434         \int_gdecr:N \g__enumext_item_number_int
2435         \bool_set_false:N \l__enumext_item_number_bool
2436       }
2437     }
2438     { 2 }{
2439       \int_gdecr:N \g__enumext_item_number_int
2440       \bool_set_false:N \l__enumext_item_number_bool
2441     }
2442     { 3 }{
2443       \int_gdecr:N \g__enumext_item_number_int
2444       \bool_set_false:N \l__enumext_item_number_bool
2445     }
2446     { 4 }{
2447       \int_gdecr:N \g__enumext_item_number_int
2448       \bool_set_false:N \l__enumext_item_number_bool
2449     }
2450   }

```

We should only execute this if `enumext*` is nested in the “first level” of `enumext`, for the rest of the cases the value of `\g__enumext_item_number_int` is already decreased.

```

2451   \int_case:nn { \l__enumext_level_h_int }
2452   {
2453     { 1 }{
2454       \bool_lazy_all:nT
2455       {
2456         { \bool_if_p:N \g__enumext_standar_bool }
2457         { \int_compare_p:nNn { \l__enumext_level_int } = { 1 } }
2458       }
2459       {
2460         \int_gdecr:N \g__enumext_item_number_int
2461         \bool_set_false:N \l__enumext_item_number_bool
2462       }
2463     }
2464   }
2465 }

```

(End of definition for `\__enumext_check_ans_active:` and `\__enumext_check_ans_level:`)

`\__enumext_check_ans_key_hook:`

The function `\__enumext_check_ans_key_hook:` will *export* the status of the local variable `\l__enumext_check_ans_key_bool` to the global variable `\g__enumext_check_ans_key_bool` only if the key `check-ans` is active.

```

2466 \cs_new_protected:Nn \__enumext_check_ans_key_hook:
2467 {
2468   \bool_lazy_and:nnT
2469   { \bool_if_p:N \l__enumext_check_ans_key_bool }
2470   { \bool_if_p:N \g__enumext_standar_bool }
2471   {
2472     \bool_gset_true:N \g__enumext_check_ans_key_bool
2473   }
2474   \bool_lazy_and:nnT
2475   { \bool_if_p:N \l__enumext_check_ans_key_bool }
2476   { \bool_if_p:N \g__enumext_starred_bool }
2477   {
2478     \bool_gset_true:N \g__enumext_check_ans_key_bool
2479   }
2480 }

```


(End of definition for `\__enumext_check_ans_key_hook:`.)

`\__enumext_item_answer_diff:` The function `\__enumext_item_answer_diff:` will set the value of the variable `\g__enumext_item_answer_diff_int` which is used by the functions `\__enumext_check_ans_show:` for the key `save-ans` and by the function `\__enumext_check_ans_log:` by the internal “*check answer*” mechanism. This function will be passed to the function `\__enumext_execute_after_env:`.

```
2481 \cs_new_protected:Nn \__enumext_item_answer_diff:
2482 {
2483   \int_gset:Nn \g__enumext_item_answer_diff_int
2484   {
2485     \int_sign:n { \g__enumext_item_number_int - \g__enumext_item_anskey_int }
2486   }
2487 }
```

(End of definition for `\__enumext_item_answer_diff:`.)

`\__enumext_check_ans_show:` The function `\__enumext_check_ans_show:` will be executed within the function `\__enumext_execute_after_env:` when the key `check-ans` is active, that is, when `\g__enumext_check_ans_key_bool` is “*true*” and will return the appropriate message according to the value of `\g__enumext_item_answer_diff_int` set by the function `\__enumext_item_answer_diff:`.

```
2488 \cs_new_protected:Nn \__enumext_check_ans_show:
2489 {
2490   \int_case:nn { \g__enumext_item_answer_diff_int }
2491   {
2492     { -1 } { \__enumext_check_ans_msg_less: }
2493     { 0 } { \__enumext_check_ans_msg_same_ok: }
2494     { 1 } { \__enumext_check_ans_msg_greater: }
2495   }
2496 }
2497 \cs_new_protected:Nn \__enumext_check_ans_msg_less:
2498 {
2499   \msg_warning:nnee { enumext } { item-less-answer } { \g__enumext_store_name_tl }
2500   { \g__enumext_envir_name_tl } { \g__enumext_start_line_tl }
2501 }
2502 \cs_new_protected:Nn \__enumext_check_ans_msg_same_ok:
2503 {
2504   \msg_term:nnee { enumext } { items-same-answer } { \g__enumext_store_name_tl }
2505   { \g__enumext_envir_name_tl } { \g__enumext_start_line_tl }
2506 }
2507 \cs_new_protected:Nn \__enumext_check_ans_msg_greater:
2508 {
2509   \msg_warning:nnee { enumext } { item-greater-answer } { \g__enumext_store_name_tl }
2510   { \g__enumext_envir_name_tl } { \g__enumext_start_line_tl }
2511 }
```

(End of definition for `\__enumext_check_ans_show:` and others.)

`\__enumext_check_ans_log:` The function `\__enumext_check_ans_log:` will be executed within the function `\__enumext_execute_after_env:` when the key `check-ans` is not active, that is, when `\g__enumext_check_ans_key_bool` is “*false*” and write in the log the appropriate message according to the value of `\g__enumext_item_answer_diff_int` set by the function `\__enumext_item_answer_diff:`.

```
2512 \cs_new_protected:Nn \__enumext_check_ans_log:
2513 {
2514   \int_case:nn { \g__enumext_item_answer_diff_int }
2515   {
2516     { -1 } { \__enumext_check_ans_log_msg_less: }
2517     { 0 } { \__enumext_check_ans_log_msg_same_ok: }
2518     { 1 } { \__enumext_check_ans_log_msg_greater: }
2519   }
2520 }
2521 \cs_new_protected:Nn \__enumext_check_ans_log_msg_less:
2522 {
2523   \msg_log:nnee { enumext } { item-less-answer } { \g__enumext_store_name_tl }
2524   { \g__enumext_envir_name_tl } { \g__enumext_start_line_tl }
2525 }
2526 \cs_new_protected:Nn \__enumext_check_ans_log_msg_same_ok:
2527 {
2528   \msg_log:nnee { enumext } { items-same-answer } { \g__enumext_store_name_tl }
2529   { \g__enumext_envir_name_tl } { \g__enumext_start_line_tl }
2530 }
```

```

2531 \cs_new_protected:Nn \__enumext_check_ans_log_msg_greater:
2532 {
2533   \msg_log:nneee { enumext } { item-greater-answer } { \g__enumext_store_name_tl }
2534   { \g__enumext_envir_name_tl } { \g__enumext_start_line_tl }
2535 }

```

(End of definition for __enumext_check_ans_log: and others.)

13.29.6 Check for \item* and \anspic* commands

__enumext_check_starred_cmd:n

The function __enumext_check_starred_cmd:n performs an *extra check* for the `keyans`, `keyans*` and `keyanspic` environments. Unlike the *check* executed by `check-ans` key this one is not controlled by any key, it is intended to prevent the forgetting of `\item*` or `\anspic*` in these environments.

```

2536 \cs_new_protected:Npn \__enumext_check_starred_cmd:n #1
2537 {
2538   \int_compare:nNnT
2539     { \g__enumext_check_starred_cmd_int } = { 0 }
2540     {
2541       \msg_warning:nnnV
2542         { enumext } { missing-starred }{ #1 } \l__enumext_check_start_line_env_tl
2543     }
2544   \int_compare:nNnT
2545     { \g__enumext_check_starred_cmd_int } > { 1 }
2546     {
2547       \msg_warning:nnnV
2548         { enumext } { many-starred }{ #1 } \l__enumext_check_start_line_env_tl
2549     }
2550   \int_gzero:N \g__enumext_check_starred_cmd_int
2551   \tl_clear:N \l__enumext_check_start_line_env_tl
2552 }

```

(End of definition for __enumext_check_starred_cmd:n.)

13.30 Keys and functions associated with storage

13.30.1 Keys for marks, wrap and show

The `enumext` package provides a set of *keys* for manipulating “symbol marks” associated with “answers” and how they are displayed and stored in the *sequence* and *prop list* as well as an internal “label and ref” system.

mark-ans*
mark-pos*
mark-sep*
wrap-ans*
wrap-opt
save-sep
show-ans
show-pos

For the `keyans` and `keyans*` environments we will only add the keys `mark-ans*`, `mark-pos*`, `mark-sep*`, `wrap-ans*`, `wrap-opt`, `save-sep`, `show-ans` and `show-pos`.

```

2553 \cs_set_protected:Npn \__enumext_tmp:nn #1 #2
2554 {
2555   \keys_define:nn { enumext / #1 }
2556   {
2557     mark-ans* .tl_set:c = { \l__enumext_mark_answer_sym_#2_tl },
2558     mark-ans* .initial:n = \textasteriskcentered,
2559     mark-ans* .value_required:n = true,
2560     mark-pos* .choice:,
2561     mark-pos* / left .code:n = \str_set:cn { \l__enumext_mark_position_#2_str } { l },
2562     mark-pos* / right .code:n = \str_set:cn { \l__enumext_mark_position_#2_str } { r },
2563     mark-pos* / center .code:n = \str_set:cn { \l__enumext_mark_position_#2_str } { c },
2564     mark-pos* / unknown .code:n =
2565       \msg_error:nneee { enumext } { unknown-choice }
2566       { mark-pos } { left,~right,~center } { \exp_not:n {##1} },
2567     mark-pos* .initial:n = right,
2568     mark-pos* .value_required:n = true,
2569     mark-sep* .dim_set:c = { \l__enumext_mark_sym_sep_#2_dim },
2570     mark-sep* .value_required:n = true,
2571     wrap-ans* .cs_set_protected:cp = { __enumext_keyans_wrapper_item_#2:n } ##1,
2572     wrap-ans* .value_required:n = true,
2573     wrap-opt .cs_set_protected:cp = { __enumext_keyans_wrapper_opt_#2:n } ##1,
2574     wrap-opt .initial:n = [{##1}],
2575     wrap-opt .value_required:n = true,
2576     save-sep .tl_set:c = { \l__enumext_store_keyans_item_opt_sep_#2_tl },
2577     save-sep .initial:n = {,~},
2578     save-sep .value_required:n = true,
2579     show-ans .bool_set:N = \l__enumext_show_answer_bool,
2580     show-ans .initial:n = false,
2581     show-ans .value_required:n = true,
2582     show-pos .bool_set:N = \l__enumext_show_position_bool,
2583     show-pos .initial:n = false,

```

```

2584         show-pos .value_required:n = true,
2585     }
2586 }
2587 \clist_map_inline:nn { {keyans}{v}, {keyans*}{viii} } { \__enumext_tmp:n #1 }

```

(End of definition for mark-ans* and others.)

We add the $\langle keys \rangle$ mark-ref and save-ref related to the “storage system” and internal mechanism of “label and ref” along with the $\langle keys \rangle$ show-ans, show-pos and the $\langle keys \rangle$ mark-ans, mark-pos, mark-sep and wrap-ans for the command $\backslash anskey$, the environment $anskey^*$ and the the $\langle keys \rangle$ for environments $keyans$ and $keyans^*$ only at the *first level* of $enumext$ and $enumext^*$.

```

2588 \cs_set_protected:Npn \__enumext_tmp:n #1
2589 {
2590     \keys_define:nn { enumext / #1 }
2591     {
2592         mark-ref .tl_set:N = \__enumext_mark_ref_sym_tl,
2593         mark-ref .initial:n = \textreferencemark,
2594         mark-ref .value_required:n = true,
2595         save-ref .bool_set:N = \__enumext_store_ref_key_bool,
2596         save-ref .initial:n = false,
2597         save-ref .value_required:n = true,
2598         show-ans .bool_set:N = \__enumext_show_answer_bool,
2599         show-ans .initial:n = false,
2600         show-ans .value_required:n = true,
2601         show-pos .bool_set:N = \__enumext_show_position_bool,
2602         show-pos .initial:n = false,
2603         show-pos .value_required:n = true,
2604         mark-ans .tl_set:N = \__enumext_mark_answer_sym_tl,
2605         mark-ans .initial:n = \textasteriskcentered,
2606         mark-ans .value_required:n = true,
2607         mark-sep .dim_set:N = \__enumext_mark_sym_sep_dim,
2608         mark-sep .value_required:n = true,
2609         mark-pos .choice:,
2610         mark-pos / left .code:n = \str_set:Nn \__enumext_mark_position_str { l },
2611         mark-pos / right .code:n = \str_set:Nn \__enumext_mark_position_str { r },
2612         mark-pos / center .code:n = \str_set:Nn \__enumext_mark_position_str { c },
2613         mark-pos / unknown .code:n =
2614             \msg_error:nnee { enumext } { unknown-choice }
2615             { mark-pos } { left,~right,~center } { \exp_not:n {##1} },
2616         mark-pos .initial:n = right,
2617         mark-pos .value_required:n = true,
2618
2619         wrap-ans .cs_set_protected:Np = \__enumext_anskey_wrapper:n ##1,
2620         wrap-ans .initial:n =
2621             {
2622                 \fbox{\parbox[t]{\dimeval{\itemwidth -2\fboxsep -2\fboxrule}}{##1}}
2623             },
2624         wrap-ans .value_required:n = true,
2625         mark-ans* .code:n = {
2626             \keys_set:nn { enumext / keyans } { mark-ans* = {##1} }
2627             \keys_set:nn { enumext / keyans* } { mark-ans* = {##1} }
2628         },
2629         mark-ans* .value_required:n = true,
2630         mark-pos* .code:n = {
2631             \keys_set:nn { enumext / keyans } { mark-pos* = {##1} }
2632             \keys_set:nn { enumext / keyans* } { mark-pos* = {##1} }
2633         },
2634         mark-pos* .value_required:n = true,
2635         mark-sep* .code:n = {
2636             \keys_set:nn { enumext / keyans } { mark-sep* = {##1} }
2637             \keys_set:nn { enumext / keyans* } { mark-sep* = {##1} }
2638         },
2639         mark-sep* .value_required:n = true,
2640         wrap-ans* .code:n = {
2641             \keys_set:nn { enumext / keyans } { wrap-ans* = {##1} }
2642             \keys_set:nn { enumext / keyans* } { wrap-ans* = {##1} }
2643         },
2644         wrap-ans* .value_required:n = true,
2645         wrap-opt .code:n = {
2646             \keys_set:nn { enumext / keyans } { wrap-opt = {##1} }
2647             \keys_set:nn { enumext / keyans* } { wrap-opt = {##1} }

```

```

2648         },
2649         wrap-opt .value_required:n = true,
2650         save-sep .code:n = {
2651             \keys_set:nn { enumext / keyans } { save-sep = {##1} }
2652             \keys_set:nn { enumext / keyans* } { save-sep = {##1} }
2653         },
2654         save-sep .value_required:n = true,
2655     }
2656 }
2657 \clist_map_inline:nn { level-1, enumext* } { \__enumext_tmp:n {#1} }

```

(End of definition for *mark-ref* and others.)

13.30.2 Storing structure of the environments

The idea behind “*storing structure*” in the *sequence* is to have a copy of the *structure of the environment* in which the key `save-ans` is being executed so we must capture the *optional argument* passed to the levels of the environment in which it is executed and “*storing*” this in the *sequence*.

```

\__enumext_store_active_keys:n
\__enumext_store_active_keys_vii:n

```

The functions `\__enumext_store_active_keys:n` and `\__enumext_store_active_keys_vii:n` will be responsible for the “*storing keys*” filtered from the *optional argument* of the environment in which the key `save-ans` is executed and the levels within this for the `enumext` and `enumext*` environments. We will execute this function only if the variable `\__enumext_store_save_key_X_bool` is false, that is, the key *store-key* is not active, establishing the variable `\__enumext_store_save_key_X_tl` with the filtered *keys*.

```

2658 \cs_new_protected:Npn \__enumext_store_active_keys:n #1
2659 {
2660     \bool_if:cF { \__enumext_store_save_key_ \__enumext_level: _bool }
2661     {
2662         \tl_clear:c { \__enumext_store_save_key_ \__enumext_level: _tl }
2663         \tl_set:ce
2664             { \__enumext_store_save_key_ \__enumext_level: _tl }
2665             { \__enumext_filter_save_key:n {#1} }
2666     }
2667 }
2668 \cs_new_protected:Npn \__enumext_store_active_keys_vii:n #1
2669 {
2670     \bool_if:NF \__enumext_store_save_key_vii_bool
2671     {
2672         \tl_clear:N \__enumext_store_save_key_vii_tl
2673         \tl_set:Ne \__enumext_store_save_key_vii_tl { \__enumext_filter_save_key:n {#1} }
2674     }
2675 }

```

(End of definition for `\__enumext_store_active_keys:n` and `\__enumext_store_active_keys_vii:n`.)

13.30.3 Setting save-key key

Since this “*storing structure*” in the *sequence* established by the `save-ans` key when executing `\anskey` or `anskey*`, we will not be able to modify it. The best thing here is to have a key that allows you to modify the *optional argument* of the “*storing structure*” in the *sequence*.

`save-key` The values set by this key passed in the *optional argument* of the `enumext` and `enumext*` environments will override the values of the `\__enumext_store_save_key_X_tl` variable set by the functions `\__enumext_store_active_keys:n` and `\__enumext_store_active_keys_vii:n`. Now define the key *save-key* for all levels of `enumext` and `enumext*` environments.

```

2676 \cs_set_protected:Npn \__enumext_tmp:n #1
2677 {
2678     \keys_define:nn { enumext / enumext* }
2679     {
2680         save-key .code:n = \__enumext_parse_save_key_vii:n {##1},
2681         save-key .value_required:n = true,
2682     }
2683     \keys_define:nn { enumext / #1 }
2684     {
2685         save-key .code:n = \__enumext_parse_save_key:n {##1},
2686         save-key .value_required:n = true,
2687     }
2688 }
2689 \clist_map_inline:nn { level-1, level-2, level-3, level-4 } { \__enumext_tmp:n {#1} }

```

(End of definition for *save-key*.)

```

__enumext_parse_save_key:n
  __enumext_parse_save_key_vii:n

```

The functions `__enumext_parse_save_key:n` and `__enumext_parse_save_key_vii:n` will be responsible for “*storing keys*” in the variable `l__enumext_store_save_key_X_tl` for `enumext` and `enumext*`.

```

2690 \cs_new_protected:Npn __enumext_parse_save_key:n #1
2691 {
2692   \bool_set_true:c { l__enumext_store_save_key_ __enumext_level: _bool }
2693   \tl_clear:c { l__enumext_save_key_ __enumext_level: _tl }
2694   \tl_set:ce
2695     { l__enumext_store_save_key_ __enumext_level: _tl }
2696     { __enumext_filter_save_key:n {#1} }
2697 }
2698 \cs_new_protected:Npn __enumext_parse_save_key_vii:n #1
2699 {
2700   \bool_set_true:N l__enumext_store_save_key_vii_bool
2701   \tl_clear:N l__enumext_store_save_key_vii_tl
2702   \tl_set:Ne l__enumext_store_save_key_vii_tl { __enumext_filter_save_key:n {#1} }
2703 }

```

(End of definition for `__enumext_parse_save_key:n` and `__enumext_parse_save_key_vii:n`.)

13.30.4 Internal functions to store optional arguments

```

__enumext_filter_save_key:n
  __enumext_filter_save_key_key:n
  __enumext_filter_save_key_pair:nn

```

The function `__enumext_filter_save_key:n` will be in charge of “*filtering keys*” we want to *stored in sequence* where `{#1}` represents the *optional argument* passed to the environment.

```

2704 \cs_new:Npn __enumext_filter_save_key:n #1
2705 {
2706   \use:e
2707     {
2708       \keyval_parse:NNn
2709         __enumext_filter_save_key_key:n
2710         __enumext_filter_save_key_pair:nn {#1}
2711     }
2712 }

```

The function `__enumext_filter_save_key_key:n` will be responsible for “*filtering keys*” that are passed “*without value*” by excluding the `resume`, `resume*`, `reset`, `reset*`, `no-store` and `base-fix` keys.

```

2713 \cs_new:Npn __enumext_filter_save_key_key:n #1
2714 {
2715   \str_case:nnF {#1}
2716     {
2717       { resume } {} { resume* } {} { reset } {} { reset* } {} { no-store } {} { base-fix } {}
2718     }
2719     { , { \exp_not:n {#1} } }
2720 }

```

The function `__enumext_filter_save_key_pair:nn` will be responsible for “*filtering keys*” that are passed “*with value*” by excluding the `series`, `resume`, `save-ans`, `save-ref`, `save-key`, `check-ans`, `show-ans`, `save-pos`, `mark-ans`, `mark-pos`, `mark-sep`, `wrap-ans`, `mark-ans*`, `mark-pos*`, `mark-sep*`, `wrap-ans*`, `wrap-opt`, `save-sep`, `mark-ref`, `mini-env`, `mini-sep`, `mini-right` and `mini-right*` keys.

```

2721 \cs_new:Npn __enumext_filter_save_key_pair:nn #1#2
2722 {
2723   \str_case:nnF {#1}
2724     {
2725       { series } {} { resume } {} { save-ans } {} { save-ref } {}
2726       { save-key } {} { check-ans } {} { show-ans } {} { show-pos } {}
2727       { mark-ans } {} { mark-pos } {} { mark-sep } {} { wrap-ans } {}
2728       { mark-ans* } {} { mark-pos* } {} { mark-sep* } {} { wrap-ans* } {}
2729       { wrap-opt } {} { save-sep } {} { mark-ref } {} { mini-env } {}
2730       { mini-sep } {} { mini-right } {} { mini-right* } {}
2731     }
2732     { , { \exp_not:n {#1} } = { \exp_not:n {#2} } }
2733 }

```

(End of definition for `__enumext_filter_save_key:n`, `__enumext_filter_save_key_key:n`, and `__enumext_filter_save_key_pair:nn`.)

13.30.5 Function for storing content in prop list

```

__enumext_store_addto_prop:n
__enumext_store_addto_prop:V

```

The function `__enumext_store_addto_prop:n` stores the `{\langle content \rangle}` in *prop list* defined by `save-ans` key. The “*stored content*” is retrieved by means of the `\getkeyans` command.

The form in which the `{\langle content \rangle}` is “*stored*” in the *prop list* is `{\langle position \rangle}{\langle content \rangle}`. This function is used by `\anskey` in `enumext` and `enumext*` environments, `\item*` in `keyans` and `keyans*` environments and `\anspic*` in `keyanspic` environment.

```

2734 \cs_new_protected:Npn __enumext_store_addto_prop:n #1

```

```

2735 {
2736   \prop_gput_if_not_in:cen { g__enumext_ \l__enumext_store_name_tl _prop }
2737   {
2738     \int_eval:n { \prop_count:c { g__enumext_ \l__enumext_store_name_tl _prop } + 1 }
2739   }
2740   { #1 }
2741 }
2742 \cs_generate_variant:Nn \__enumext_store_addto_prop:n { V }

```

(End of definition for `\__enumext_store_addto_prop:n`.)

13.30.6 Function for storing content in sequence

The function `\__enumext_store_addto_seq:n` stores the $\{\langle content \rangle\}$ in *sequence* defined by `save-ans` key. This function is used by `\anskey` in `enumext`, `\item*` in `keyans` and `\anspic` in `keyanspic`.

The form in which the $\{\langle content \rangle\}$ is stored in *sequence* is in a internal `enumext` or `enumext*` environments with the “*same structure*” in which the command was executed.

The “*stored content*” is retrieved by means of the `\printkeyans` command.

```

2743 \cs_new_protected:Npn \__enumext_store_addto_seq:n #1
2744 {
2745   \seq_gput_right:cn { g__enumext_ \l__enumext_store_name_tl _seq } { #1 }
2746 }
2747 \cs_generate_variant:Nn \__enumext_store_addto_seq:n { v, V }

```

(End of definition for `\__enumext_store_addto_seq:n`.)

13.30.7 Functions for storing structure in the sequence

The “*storing structure*” is handled by the functions `\__enumext_store_level_open:` and `\__enumext_store_level_close:` which are executed per level within the `enumext` environment.

```

2748 \cs_new_protected:Nn \__enumext_store_level_open:
2749 {
2750   \bool_if:NT \l__enumext_check_answers_bool
2751   {
2752     \tl_if_empty:cTF { l__enumext_store_save_key_ \__enumext_level: _tl }
2753     {
2754       \__enumext_store_addto_seq:n
2755       {
2756         \item \begin{enumext}
2757       }
2758     }
2759     {
2760       \tl_put_left:cn { l__enumext_store_save_key_ \__enumext_level: _tl }
2761       {
2762         \item \begin{enumext} [
2763       }
2764       \tl_put_right:cn { l__enumext_store_save_key_ \__enumext_level: _tl }
2765       {
2766         ]
2767       }
2768       \__enumext_store_addto_seq:v { l__enumext_store_save_key_ \__enumext_level: _tl }
2769     }
2770   }
2771 }
2772 \cs_new_protected:Nn \__enumext_store_level_close:
2773 {
2774   \bool_if:NT \l__enumext_check_answers_bool
2775   {
2776     \__enumext_store_addto_seq:n { \end{enumext} }
2777   }
2778 }

```

(End of definition for `\__enumext_store_level_open:` and `\__enumext_store_level_close:`.)

The “*storing structure*” is handled by the functions `\__enumext_store_level_open_vii:` and `\__enumext_store_level_close_vii:` which are executed in the `enumext*` environment.

```

2779 \cs_new_protected:Nn \__enumext_store_level_open_vii:
2780 {
2781   \bool_if:NT \l__enumext_check_answers_bool
2782   {
2783     \tl_if_empty:NTF \l__enumext_store_save_key_vii_tl
2784     {

```

```

2785         \__enumext_store_addto_seq:n
2786         {
2787             \item \begin{enumext*}
2788         }
2789     }
2790     {
2791         \tl_put_left:Nn \l__enumext_store_save_key_vii_tl
2792         {
2793             \item \begin{enumext*}[
2794             ]
2795             \tl_put_right:Nn \l__enumext_store_save_key_vii_tl
2796             {
2797                 ]
2798             }
2799             \__enumext_store_addto_seq:V \l__enumext_store_save_key_vii_tl
2800         }
2801     }
2802 }
2803 \cs_new_protected:Nn \__enumext_store_level_close_vii:
2804 {
2805     \bool_if:NT \l__enumext_check_answers_bool
2806     {
2807         \__enumext_store_addto_seq:n { \end{enumext*} }
2808     }
2809 }

```

(End of definition for `\__enumext_store_level_open_vii:` and `\__enumext_store_level_close_vii:`.)

13.30.8 Function for show marks and position

```

\__enumext_print_keyans_box:NN
\__enumext_print_keyans_box:cc

```

The function `\__enumext_print_keyans_box:NN` print a box in the left margin with `\l__enumext_mark_answer_sym_tl` used by the `wrap-ans`, `show-ans` and `show-pos` keys. The function takes two arguments:

#1: `\l__enumext_labelwidth_X_dim`
#2: `\l__enumext_labelsep_X_dim`

```

2810 \cs_new_protected:Nn \__enumext_print_keyans_box:NN
2811 {
2812     \mode_leave_vertical:
2813     \skip_horizontal:n { -\dim_use:N #2 }
2814     \hbox_overlap_left:n
2815     {
2816         \makebox[ \dim_use:N #1 ][ \l__enumext_mark_position_str ]
2817         {
2818             \tl_use:N \l__enumext_mark_answer_sym_tl
2819         }
2820     }
2821     \skip_horizontal:n { \dim_use:N #2 }
2822 }
2823 \cs_generate_variant:Nn \__enumext_print_keyans_box:NN { cc }

```

(End of definition for `\__enumext_print_keyans_box:NN`.)

13.31 The internal label and ref

The function `\__enumext_store_internal_ref:` handles the “*internal label and ref*” system used by the `save-ref` and `mark-ref` keys for `\anskey` will allow to execute `\ref{⟨store name : position⟩}` and will return `1.(a).i.A`.

```

\__enumext_store_internal_ref:

```

First we will remove the dots “.” from the current `⟨labels⟩`, we do not want to get double dots in our references, then we will place this in the variable `\l__enumext_newlabel_arg_two_tl`.

```

2824 \cs_new_protected:Nn \__enumext_store_internal_ref:
2825 {
2826     \cs_set_protected:Npn \__enumext_tmp:n ##1
2827     {
2828         \tl_set_eq:cc { \l__enumext_label_copy_##1_tl } { \l__enumext_label_##1_tl }
2829         \tl_reverse:c { \l__enumext_label_copy_##1_tl }
2830         \tl_remove_once:cn { \l__enumext_label_copy_##1_tl } { . }
2831         \tl_reverse:c { \l__enumext_label_copy_##1_tl }
2832     }
2833     \clist_map_inline:nn { i, ii, iii, iv, vii } { \__enumext_tmp:n {##1} }
2834     \cs_set:Npn \__enumext_tmp:n ##1
2835     { . \tl_use:c { \l__enumext_label_copy_ \int_to_roman:n {##1} _tl } }

```


Here we need to analyse the cases where the environment is started with `enumext*` and if `\anskey` or `anskey*` is running alone in it or if it is running in a nested `enumext` environment within the starting environment.

```

2836 \bool_lazy_all:nT
2837 {
2838   { \bool_if_p:N \g__enumext_starred_bool }
2839   { \int_compare_p:nNn { \l__enumext_level_int } = { 0 } }
2840 }
2841 {
2842   \tl_put_right:Ne \l__enumext_newlabel_arg_two_tl
2843   { \tl_use:N \l__enumext_label_copy_vii_tl }
2844 }
2845 \bool_lazy_all:nT
2846 {
2847   { \bool_not_p:n { \g__enumext_standar_bool } }
2848   { \bool_if_p:N \l__enumext_standar_bool }
2849   { \int_compare_p:nNn { \l__enumext_level_int } > { 0 } }
2850 }
2851 {
2852   \tl_put_right:Ne \l__enumext_newlabel_arg_two_tl
2853   {
2854     \tl_use:N \l__enumext_label_copy_vii_tl
2855     \int_step_function:nnN { 1 } { \l__enumext_level_int } \__enumext_tmp:n
2856   }
2857 }

```

If started with `enumext` and if `\anskey` or `anskey*` is running alone in it or if it is running in a nested `enumext*` environment within the starting environment.

```

2858 \bool_lazy_all:nT
2859 {
2860   { \bool_if_p:N \g__enumext_standar_bool }
2861   { \int_compare_p:nNn { \l__enumext_level_int } > { 0 } }
2862   { \int_compare_p:nNn { \l__enumext_level_h_int } = { 0 } }
2863 }
2864 {
2865   \tl_put_right:Ne \l__enumext_newlabel_arg_two_tl
2866   {
2867     \tl_use:N \l__enumext_label_copy_i_tl
2868     \int_step_function:nnN { 2 } { \l__enumext_level_int } \__enumext_tmp:n
2869   }
2870 }
2871 \cs_set:Npn \__enumext_tmp:n ##1
2872 { \tl_use:c { l__enumext_label_copy_ \int_to_roman:n {##1} _tl } . }
2873 \bool_lazy_all:nT
2874 {
2875   { \bool_if_p:N \g__enumext_standar_bool }
2876   { \bool_if_p:N \l__enumext_starred_bool }
2877   { \int_compare_p:nNn { \l__enumext_level_int } > { 0 } }
2878 }
2879 {
2880   \tl_put_right:Ne \l__enumext_newlabel_arg_two_tl
2881   {
2882     \int_step_function:nnN { 1 } { \l__enumext_level_int } \__enumext_tmp:n
2883     \tl_use:N \l__enumext_label_copy_vii_tl
2884   }
2885 }

```

Now we set the variable `\l__enumext_newlabel_arg_one_tl` which will contain $\langle \textit{store name} : \textit{position} \rangle$.

```

2886 \tl_put_right:Ne \l__enumext_newlabel_arg_one_tl
2887 {
2888   \l__enumext_store_name_tl \c_colon_str
2889   \int_eval:n { \prop_count:c { g__enumext_ \l__enumext_store_name_tl _prop } }
2890 }

```

Now execute the function `\__enumext_newlabel:nn` and save the result in the variable `\l__enumext_write_aux_file_tl` and finally we write in the `.aux` file.

```

2891 \tl_put_right:Ne \l__enumext_write_aux_file_tl
2892 {
2893   \__enumext_newlabel:nn
2894   { \exp_not:V \l__enumext_newlabel_arg_one_tl }
2895   { \l__enumext_newlabel_arg_two_tl }
2896 }
2897 \l__enumext_write_aux_file_tl

```

```
2898 }
```

(End of definition for `\__enumext_store_internal_ref:`)

13.32 Common functions for `\anskey` and `anskey*` environment

```
\__enumext_store_anskey_arg:n
```

The internal function `\__enumext_store_anskey_arg:n` first we pass the $\langle \textit{argument} \rangle$ to the *prop list*, then checks the state of the variable `\l__enumext_store_ref_key_bool` handled by the *save-ref* key and will call the function `\__enumext_store_internal_ref:` for the “*internal label and ref*” system. Followed by this if the *show-ans* or *show-pos* keys are active we will show the “*wrapped*” $\langle \textit{argument} \rangle$.

```
2899 \cs_new_protected:Npn \__enumext_store_anskey_arg:n #1
2900 {
2901   \int_gincr:N \g__enumext_item_anskey_int
2902   \__enumext_store_addto_prop:n {#1}
2903   \bool_if:NT \l__enumext_store_ref_key_bool
2904   {
2905     \__enumext_store_internal_ref:
2906   }
2907   \__enumext_anskey_show_wrap_left:n { #1 }
```

Now we start processing the $[\langle \textit{key} = \textit{val} \rangle]$ passed to the command to build our `\item` in the variable `\l__enumext_store_anskey_arg_tl` which we will “*store*” in the *sequence*. First we clear the variable `\l__enumext_store_anskey_arg_tl` and process the $\langle \textit{keys} \rangle$, if the *break-col* key is present and the command is running under *enumext* (not in *enumext**) we will add `\columnbreak` and then `\item`.

```
2908   \tl_clear:N \l__enumext_store_anskey_arg_tl
2909   \bool_lazy_and:nnT
2910   { \bool_if_p:N \l__enumext_store_columns_break_bool }
2911   { \bool_not_p:n { \l__enumext_starred_bool } }
2912   {
2913     \tl_put_left:Nn \l__enumext_store_anskey_arg_tl { \columnbreak }
2914   }
2915   \tl_put_right:Nn \l__enumext_store_anskey_arg_tl { \item }
```

If the *item-join* key is present and the command is running under *enumext** we will add $\langle \textit{number} \rangle$ to `\l__enumext_store_anskey_arg_tl`.

```
2916   \bool_lazy_and:nnT
2917   { \bool_not_p:n { \l__enumext_starred_bool } }
2918   { \int_compare_p:nNn { \l__enumext_store_item_join_int } > { 1 } }
2919   {
2920     \tl_put_right:Ne \l__enumext_store_anskey_arg_tl
2921     {
2922       ( \exp_not:V \l__enumext_store_item_join_int )
2923     }
2924   }
```

And now we will review the keys *item-star*, *item-sym** and *item-pos** and pass them to `\l__enumext_store_anskey_arg_tl` along with the $\langle \textit{argument} \rangle$ for `\anskey` or $\langle \textit{body} \rangle$ for `anskey*`.

```
2925   \bool_if:NTF \l__enumext_store_item_star_bool
2926   {
2927     \tl_put_right:Nn \l__enumext_store_anskey_arg_tl { * }
2928     \tl_if_empty:NF \l__enumext_store_item_symbol_tl
2929     {
2930       \tl_put_right:Ne \l__enumext_store_anskey_arg_tl
2931       {
2932         [ \exp_not:V \l__enumext_store_item_symbol_tl ]
2933       }
2934     }
2935     \dim_compare:nT
2936     {
2937       \l__enumext_store_item_symbol_sep_dim != \c_zero_dim
2938     }
2939     {
2940       \tl_put_right:Ne \l__enumext_store_anskey_arg_tl
2941       {
2942         [ \exp_not:V \l__enumext_store_item_symbol_sep_dim ]
2943       }
2944     }
2945     \tl_put_right:Nn \l__enumext_store_anskey_arg_tl {#1}
2946   }
2947   {
2948     \tl_put_right:Nn \l__enumext_store_anskey_arg_tl {#1}
2949   }
```

Finally we check if the `save-ref` key are active along with the `hyperref` package load, if both conditions are met, it will create the `\hyperlink` with “*symbol*” set by `mark-ref` key and then store in *sequence*.

```

2950 \bool_lazy_and:nnT
2951 { \bool_if_p:N \l__enumext_store_ref_key_bool }
2952 { \bool_if_p:N \l__enumext_hyperref_bool }
2953 {
2954   \tl_put_right:Ne \l__enumext_store_anskey_arg_tl
2955   {
2956     \hfill \exp_not:N \hyperlink { \exp_not:V \l__enumext_newlabel_arg_one_tl }
2957     { \exp_not:V \l__enumext_mark_ref_sym_tl }
2958   }
2959 }
2960 \__enumext_store_addto_seq:V \l__enumext_store_anskey_arg_tl
2961 }

```

(End of definition for `\__enumext_store_anskey_arg:n`.)

`\__enumext_anskey_show_wrap_arg:n`

The function `\__enumext_anskey_show_wrap_arg:n` “wraps” the $\langle argument \rangle$ passed to `\anskey` and the $\langle body \rangle$ for `anskey*` when using the `wrap-ans` and `wrap-sep` keys.

```

2962 \cs_new_protected:Npn \__enumext_anskey_show_wrap_arg:n #1
2963 {
2964   \par
2965   \bool_if:NTF \l__enumext_starred_bool
2966   {
2967     \dim_compare:nNnT { \l__enumext_mark_sym_sep_dim } = { \c_zero_dim }
2968     {
2969       \dim_set:Nn \l__enumext_mark_sym_sep_dim { \l__enumext_labelsep_vii_dim }
2970     }
2971     \__enumext_print_keyans_box:NN
2972     \l__enumext_labelwidth_vii_dim \l__enumext_mark_sym_sep_dim
2973   }
2974   {
2975     \dim_compare:nNnT { \l__enumext_mark_sym_sep_dim } = { \c_zero_dim }
2976     {
2977       \dim_set:Nn \l__enumext_mark_sym_sep_dim
2978       {
2979         \dim_use:c { \l__enumext_labelsep_ \__enumext_level: _dim }
2980       }
2981     }
2982     \__enumext_print_keyans_box:cc
2983     { \l__enumext_labelwidth_ \__enumext_level: _dim } { \l__enumext_mark_sym_sep_dim }
2984   }
2985   \__enumext_anskey_wrapper:n { #1 }
2986 }

```

(End of definition for `\__enumext_anskey_show_wrap_arg:n`.)

`\__enumext_anskey_show_wrap_left:n`

The function `\__enumext_anskey_show_wrap_left:n` will show the “*mark*” defined by the `mark-ans` key or the “*position*” of the $\langle content \rangle$ stored in the *prop list* when using the `show-pos` key on the left margin next to the “wraps” $\langle argument \rangle$ passed to `\anskey` and the $\langle body \rangle$ in `anskey*` on the right side when using the `show-ans` key.

```

2987 \cs_new_protected:Npn \__enumext_anskey_show_wrap_left:n #1
2988 {
2989   \bool_if:NT \l__enumext_show_answer_bool
2990   {
2991     \__enumext_anskey_show_wrap_arg:n { #1 }
2992   }
2993   \bool_if:NT \l__enumext_show_position_bool
2994   {
2995     \tl_set:Ne \l__enumext_mark_answer_sym_tl
2996     {
2997       \group_begin:
2998       \exp_not:N \normalfont
2999       \exp_not:N \footnotesize [ \int_eval:n
3000       {
3001         \prop_count:c { g__enumext_ \l__enumext_store_name_tl _prop }
3002       }
3003       ]
3004       \group_end:
3005     }

```

```

3006         \__enumext_anskey_show_wrap_arg:n { #1 }
3007     }
3008 }

```

(End of definition for __enumext_anskey_show_wrap_left:n.)

13.33 The command \anskey

Since we will be “*storing content*” in a `list` environment within *sequences* and can (more or less) manage the options passed to each level, it is necessary that we have a little more control over `\item` when storing.

The `\anskey` command will cover this point and give it similar behaviour to that of `\item` in the `enumext` and `enumext*` environments executed as follows `\anskey[⟨key = val⟩]{⟨content⟩}`.

First we'll add the keys `break-col`, `item-join`, `item-star`, `item-sym*` and `item-pos*`.

```

break-col \keys_define:nn { enumext / anskey }
item-join
item-star
item-sym*
item-pos*
unknown
\__enumext_anskey_unknown:n
\__enumext_anskey_unknown:nn
3009 \keys_define:nn { enumext / anskey }
3010 {
3011     break-col .bool_set:N = \l__enumext_store_columns_break_bool,
3012     break-col .default:n = true,
3013     break-col .value_forbidden:n = true,
3014     item-join .int_set:N = \l__enumext_store_item_join_int,
3015     item-join .value_required:n = true,
3016     item-star .bool_set:N = \l__enumext_store_item_star_bool,
3017     item-star .default:n = true,
3018     item-star .value_forbidden:n = true,
3019     item-sym* .tl_set:N = \l__enumext_store_item_symbol_tl,
3020     item-sym* .value_required:n = true,
3021     item-pos* .dim_set:N = \l__enumext_store_item_symbol_sep_dim,
3022     item-pos* .value_required:n = true,
3023     unknown .code:n = { \__enumext_anskey_unknown:n {#1} },
3024 }

```

The `⟨keys⟩` are stored in `\l_keys_key_str` and the value (if any) is passed as an argument to the function `\__enumext_anskey_unknown:n`.

```

3025 \cs_new_protected:Npn \__enumext_anskey_unknown:n #1
3026 {
3027     \exp_args:NV \__enumext_anskey_unknown:nn \l_keys_key_str {#1}
3028 }
3029 \cs_new_protected:Npn \__enumext_anskey_unknown:nn #1 #2
3030 {
3031     \tl_if_blank:nTF {#2}
3032     {
3033         \msg_error:nnn { enumext } { anskey-cmd-key-unknown } {#1}
3034     }
3035     {
3036         \msg_error:nnnn { enumext } { anskey-cmd-key-value-unknown } {#1} {#2}
3037     }
3038 }

```

(End of definition for `break-col` and others.)

- 🌱 The `\anskey` command will only be present when using the `save-ans` key in `enumext` and `enumext*` environments, otherwise it will return an error.

`\anskey` We will first call the function `\__enumext_anskey_safe_outer:` to be sure where we execute the command, then we will check the state of the variable `\l__enumext_check_answers_bool` set by the key `no-store`, if is true we will increment `\g__enumext_item_anskey_int` for the internal “*check answer*” system and execute the function `\__enumext_anskey_safe_inner:n` to ensure that the command is not nested and that the argument is not empty, finally search the `[⟨key = val⟩]` and call the function `\__enumext_store_anskey_arg:n`.

```

3039 \NewDocumentCommand \anskey { o +m }
3040 {
3041     \__enumext_anskey_safe_outer:
3042     \group_begin:
3043     \bool_if:NT \l__enumext_check_answers_bool
3044     {
3045         \tl_if_novalue:nF {#1}
3046         {
3047             \keys_set:nn { enumext / anskey } {#1}
3048         }
3049         \tl_if_blank:nTF {#2}
3050         {
3051             \msg_error:nn { enumext } { anskey-empty-arg }

```

```

3052         }
3053     {
3054         \__enumext_anskey_safe_inner:
3055         \__enumext_store_anskey_arg:n {#2}
3056     }
3057 }
3058 \group_end:
3059 }

```

(End of definition for `\anskey`. This function is documented on page 14.)

13.33.1 Internal functions for the command

```

\__enumext_anskey_safe_outer:
\__enumext_anskey_safe_inner:

```

The `\__enumext_store_anskey_safe_outer:` function will return the appropriate messages when the command is executed outside the environment in which the `save-ans` key was activated.

```

3060 \cs_new_protected:Nn \__enumext_anskey_safe_outer:
3061 {
3062     \bool_if:NF \l__enumext_store_active_bool
3063     {
3064         \msg_error:nnnn { enumext } { anskey-wrong-place }{ anskey }{ enumext }
3065     }
3066     \int_compare:nNnT { \l__enumext_keyans_level_int } = { 1 }
3067     {
3068         \msg_error:nnnn { enumext } { command-wrong-place }{ anskey }{ keyans }
3069     }
3070     \int_compare:nNnT { \l__enumext_keyans_level_h_int } = { 1 }
3071     {
3072         \msg_error:nnnn { enumext } { command-wrong-place }{ anskey }{ keyans* }
3073     }
3074     \int_compare:nNnT { \l__enumext_keyans_pic_level_int } = { 1 }
3075     {
3076         \msg_error:nnnn { enumext } { command-wrong-place }{ anskey }{ keyanspic }
3077     }
3078 }

```

The `\__enumext_anskey_safe_inner:` function will first check if the command is nested, if preceded by a not numbered `\item` or if it is in *math mode* returning the appropriate messages.

```

3079 \cs_new_protected:Nn \__enumext_anskey_safe_inner:
3080 {
3081     \int_incr:N \l__enumext_anskey_level_int
3082     \int_compare:nNnT { \l__enumext_anskey_level_int } > { 1 }
3083     {
3084         \msg_error:nn { enumext } { anskey-nested }
3085     }
3086     \bool_if:NF \l__enumext_item_number_bool
3087     {
3088         \msg_error:nn { enumext } { anskey-unnumber-item }
3089     }
3090     \mode_if_math:T
3091     {
3092         \msg_error:nne { enumext } { anskey-math-mode } { \c_backslash_str anskey }
3093     }
3094 }

```

(End of definition for `\__enumext_anskey_safe_outer:` and `\__enumext_anskey_safe_inner:`.)

13.34 The environment `anskey*`

The original implementation of the `anskey*` environment used non-public functions from the `scontents`[4] package, which was not the best approach. Fortunately L^AT_EX release 2025-06-01 implemented the new `c`-type argument in the `\lcmd`[13], with which we can record the *(body)* of the environment in *verbatim mode* and `\tl_retokenize:n` (wrapper around the `\scantokens`) provide by L^AT_EX3 release 2025-07-08 do the work as the original implementation.

```

break-col
item-join
item-star
item-sym*
item-pos*
force-eol
write-env
overwrite
unknown

```

First we add the same keys from the `\anskey` command along with the `force-eol`, `write-env` and `overwrite` keys that were in the original implementation that used the `scontents` support package for these.

```

3095 \keys_define:nn { enumext / anskey* }
3096 {
3097     break-col .bool_set:N = \l__enumext_store_columns_break_bool,
3098     break-col .default:n = true,
3099     break-col .value_forbidden:n = true,
3100     item-join .int_set:N = \l__enumext_store_item_join_int,

```

```

3101     item-join .value_required:n = true,
3102     item-star .bool_set:N = \l__enumext_store_item_star_bool,
3103     item-star .default:n = true,
3104     item-star .value_forbidden:n = true,
3105     item-sym* .tl_set:N = \l__enumext_store_item_symbol_tl,
3106     item-sym* .value_required:n = true,
3107     item-pos* .dim_set:N = \l__enumext_store_item_symbol_sep_dim,
3108     item-pos* .value_required:n = true,
3109     force-eol .bool_set:N = \l__enumext_anskey_env_force_eol_bool,
3110     force-eol .initial:n = false,
3111     force-eol .default:n = true,
3112     write-env .code:n = {
3113         \bool_set_true:N \l__enumext_write_anskey_env_bool
3114         \tl_set:Nn \l__enumext_write_anskey_env_file_name_tl {#1}
3115     },
3116     write-env .value_required:n = true,
3117     overwrite .bool_set:N = \l__enumext_anskey_env_overwrite_bool,
3118     overwrite .initial:n = false,
3119     overwrite .default:n = true,
3120     unknown .code:n = { \__enumext_anskey_env_unknown:n {#1} },
3121 }

```

(End of definition for `break-col` and others.)

__enumext_anskey_env_unknown:n
 __enumext_anskey_env_unknown:nn

The *⟨keys⟩* are stored in `\l_keys_key_str` and the value (if any) is passed as an argument to the function `\__enumext_anskey_env_unknown:n`.

```

3122 \cs_new_protected:Npn \__enumext_anskey_env_unknown:n #1
3123 {
3124     \exp_args:NV \__enumext_anskey_env_unknown:nn \l_keys_key_str {#1}
3125 }
3126 \cs_new_protected:Npn \__enumext_anskey_env_unknown:nn #1#2
3127 {
3128     \tl_if_blank:nTF {#2}
3129     {
3130         \msg_error:nnn { enumext } { anskey-env-key-unknown } {#1}
3131     }
3132     {
3133         \msg_error:nnnn { enumext } { anskey-env-key-value-unknown } {#1} {#2}
3134     }
3135 }

```

(End of definition for `\__enumext_anskey_env_unknown:n` and `\__enumext_anskey_env_unknown:nn`.)

__enumext_anskey_env_file_if_writable:n
 __enumext_anskey_env_file_if_writable:nT
 __enumext_anskey_env_file_if_writable:nF
 __enumext_anskey_env_file_if_writable:nTF

The conditional function `\__enumext_anskey_env_file_if_writable:n` used by the `write-env` and `overwrite` keys in the `anskey*` environment to determine whether the output file is written or overwritten.

```

3136 \prg_new_protected_conditional:Npnn \__enumext_anskey_env_file_if_writable:n #1 { T, F, TF }
3137 {
3138     \bool_if:NTF \l__enumext_write_anskey_env_bool
3139     {
3140         \file_if_exist:nTF {#1}
3141         {
3142             \bool_if:NTF \l__enumext_anskey_env_overwrite_bool
3143             {
3144                 \msg_warning:nne { enumext } { overwrite-file } {#1}
3145                 \prg_return_true:
3146             }
3147             {
3148                 \msg_warning:nne { enumext } { not-writing } {#1}
3149                 \prg_return_false:
3150             }
3151         }
3152         {
3153             \msg_warning:nne { enumext } { writing-file } {#1}
3154             \prg_return_true:
3155         }
3156     }
3157     { \prg_return_false: }
3158 }

```

The `\__enumext_anskey_env_file_write:nn` function is used by the `write-env` key in the `anskey*` environment to write the output file with the *⟨body⟩* of the environment.

```

3159 \cs_new_protected:Npn \__enumext_anskey_env_file_write:nn #1#2
3160 {
3161   \__enumext_anskey_env_file_if_writable:nT {#1}
3162   {
3163     \iow_open:Nn \__enumext_write_anskey_env_file_iow {#1}
3164     \iow_now:Nn \__enumext_write_anskey_env_file_iow {#2}
3165     \iow_close:N \__enumext_write_anskey_env_file_iow
3166   }
3167 }
3168 \cs_generate_variant:Nn \__enumext_anskey_env_file_write:nn { VV }

```

(End of definition for `\__enumext_anskey_env_file_if_writable:n` and others.)

anskey* First, we'll call the function `\__enumext_anskey_env_safe_outer:` to make sure where we're running the environment, then, we'll check the state of the variable `\l__enumext_check_answers_bool` set by the key `no-store`. If it's true, we'll look for `[⟨key = val⟩]` and verify that the *argument* c-type `⟨body⟩` is not empty. Finally, we'll run the internal check function `\__enumext_anskey_env_safe_inner:n` and call the function `\__enumext_store_anskey_arg:n`.

```

3169 \NewDocumentEnvironment{anskey*}{ o c }
3170 {
3171   \__enumext_anskey_env_safe_outer:
3172   \bool_if:NT \l__enumext_check_answers_bool
3173   {
3174     \tl_if_novalue:nF {#1}
3175     {
3176       \keys_set:nn { enumext / anskey* } {#1}
3177     }
3178     \tl_if_blank:nTF {#2}
3179     {
3180       \msg_error:nn { enumext } { anskey-empty-arg }
3181     }
3182     {
3183       \__enumext_anskey_env_safe_inner:
3184       \__enumext_store_anskey_env:n {#2}
3185     }
3186   }
3187 } { }

```

(End of definition for `anskey*`. This function is documented on page 15.)

13.34.1 Internal functions for the environment

`\__enumext_anskey_env_safe_outer:` The function `\__enumext_store_anskey_safe_outer:` will return the appropriate messages when `anskey*` is executed outside the environment in which the `save-ans` key was activated or within the `keyans`, `keyans*` or `keyanspic` environments.

```

3188 \cs_new_protected:Nn \__enumext_anskey_env_safe_outer:
3189 {
3190   \bool_if:NF \l__enumext_store_active_bool
3191   {
3192     \msg_error:nnn { enumext } { anskey-env-error } { anskey* }
3193   }
3194   \int_compare:nNnT { \l__enumext_keyans_level_int } = { 1 }
3195   {
3196     \msg_error:nnn { enumext } { anskey-env-wrong } { keyans }
3197   }
3198   \int_compare:nNnT { \l__enumext_keyans_level_h_int } = { 1 }
3199   {
3200     \msg_error:nnn { enumext } { anskey-env-wrong } { keyans* }
3201   }
3202   \int_compare:nNnT { \l__enumext_keyans_pic_level_int } = { 1 }
3203   {
3204     \msg_error:nnn { enumext } { anskey-env-wrong } { keyanspic }
3205   }
3206 }

```

The function `\__enumext_anskey_env_safe_inner:` will first check if preceded by a not numbered `\item` or if it is in *math mode* returning the appropriate messages.

```

3207 \cs_new_protected:Nn \__enumext_anskey_env_safe_inner:
3208 {
3209   \bool_if:NF \l__enumext_item_number_bool
3210   {
3211     \msg_error:nn { enumext } { anskey-unnumber-item }

```



```

3212     }
3213     \mode_if_math:T
3214     {
3215         \msg_error:nnn { enumext } { anskey-math-mode } { anskey* }
3216     }
3217 }

```

The `\__enumext_store_anskey_env:n` function will first pass the `c`-type argument *(body)* to the variable `\l__enumext_store_anskey_env_tl` and replace the macro `\obeyedline` with `^^J` and then execute the `write-env` and `overwrite` keys, check the state of the variable `\l__enumext_anskey_env_force_eol_bool` managed by the `force-eol` key and we will add `\c__enumext_anskey_env_hidden_space_str` if necessary. Finally we will use `\exp_args:N` on the `\__enumext_store_anskey_arg:n` to expand the `\tl_retokenize:n` function which rescans the `\l__enumext_store_anskey_env_tl` variable before processing it.

```

3218 \cs_new_protected:Npn \__enumext_store_anskey_env:n #1
3219 {
3220     \tl_set:Nn \l__enumext_store_anskey_env_tl {#1}
3221     \RenewDocumentCommand \obeyedline { } { \iow_char:N ^^J }
3222     \tl_replace_all:Nee \l__enumext_store_anskey_env_tl { \obeyedline } { \iow_char:N ^^J }
3223     \__enumext_anskey_env_file_write:VV
3224     \l__enumext_write_anskey_env_file_name_tl \l__enumext_store_anskey_env_tl
3225     \bool_if:NF \l__enumext_anskey_env_force_eol_bool
3226     {
3227         \tl_put_right:Nc \l__enumext_store_anskey_env_tl
3228         {
3229             \c__enumext_anskey_env_hidden_space_str
3230         }
3231     }
3232     \exp_args:Nc
3233     \__enumext_store_anskey_arg:n
3234     {
3235         \tl_retokenize:n { \l__enumext_store_anskey_env_tl }
3236     }
3237 }

```

Since `\obeyedline` can be redefined by the user, for example to `\mbox{}\par`, it is necessary to redefine it to `^^J` in order to use `\tl_replace_all:Nee` otherwise it returns an error.

(End of definition for `\__enumext_anskey_env_safe_outer:`, `\__enumext_anskey_env_safe_inner:`, and `\__enumext_store_anskey_env:n`)

13.35 Executing check-ans system and write .log

`\__enumext_execute_after_env:`

The `\__enumext_execute_after_env:` function will first return the appropriate message for the end of the environment in which the `save-ans` key is being executed, then call the `\__enumext_item_answer_diff:` function and then will write the values of the global variables used to the `.log` file. If the key `check-ans` is active it will execute the function `\__enumext_check_ans_show:` and show the result in the terminal, otherwise it will execute the function `\__enumext_check_ans_log:` and write the results in the `.log` file and finally we execute the function `\__enumext_reset_global_vars:` returning the used variables to their original state.

```

3238 \cs_new_protected:Nn \__enumext_execute_after_env:
3239 {
3240     \int_compare:nNtT { \l__enumext_level_int } = { 0 }
3241     {
3242         \tl_if_empty:NF \g__enumext_store_name_tl
3243         {
3244             \__enumext_stop_save_ans_msg:
3245             \__enumext_item_answer_diff:
3246             \__enumext_log_global_vars:
3247             \__enumext_log_answer_vars:
3248             \bool_if:NTF \g__enumext_check_ans_key_bool
3249             {
3250                 \__enumext_check_ans_show:
3251             }
3252             { \__enumext_check_ans_log: }
3253         }
3254         \__enumext_reset_global_vars:
3255     }
3256 }

```

This function is passed to the function `\__enumext_after_env:nn` for the environments `enumext` (§13.42) and `enumext*` (§13.47) and it is executed only when the environments are not nested or at some level of these..

(End of definition for `\__enumext_execute_after_env:`.)

13.36 Common functions for `keyans`, `keyans*` and `keyanspic`

13.36.1 Storing content in prop list

`\__enumext_keyans_addto_prop:n`

The function `\__enumext_keyans_addto_prop:n` will pass the the current $\langle label \rangle$ for `\item*` in `keyans` environment and the current $\langle label \rangle$ for `\anspic*` in `keyanspic` environment followed by the $\langle contents \rangle$ of the *optional argument* of both commands to the `\l__enumext_store_current_label_tl` variable, which will be stored to the *prop list* defined by the `save-ans` key using the function `\__enumext_store_addto_prop:V`.

```

3257 \cs_new_protected:Npn \__enumext_keyans_addto_prop:n #1
3258 {
3259   \tl_clear:N \l__enumext_store_current_label_tl
3260   \int_compare:nNnTF { \l__enumext_keyans_pic_level_int } = { 1 }
3261   {
3262     \tl_put_right:Ne \l__enumext_store_current_label_tl { \l__enumext_label_vi_tl }
3263   }
3264   {
3265     \tl_put_right:Ne \l__enumext_store_current_label_tl { \l__enumext_label_v_tl }
3266   }

```

If the *optional argument* is present and the `save-sep` key is not empty, we save it.

```

3267   \tl_if_novalue:nF { #1 }
3268   {
3269     \tl_if_empty:NF \l__enumext_store_keyans_item_opt_sep_v_tl
3270     {
3271       \tl_put_right:NV \l__enumext_store_current_label_tl \l__enumext_store_keyans_item_opt_sep_v_tl
3272     }
3273     \tl_put_right:Nn \l__enumext_store_current_label_tl { #1 }
3274   }
3275   \__enumext_store_addto_prop:V \l__enumext_store_current_label_tl
3276 }

```

(End of definition for `\__enumext_keyans_addto_prop:n`.)

13.36.2 The `save-ref` key for `keyans`, `keyans*` and `keyanspic`

The “*internal label and ref*” system for the `keyans`, `keyans*` and `keyanspic` environments has *slight differences* with the one implemented for `\anskey` basically because in this environments the interest is in the current $\langle label \rangle$ for `\item*` and `\anspic*` with the $\langle contents \rangle$ of the *optional argument*. The mechanism defined here will allow to execute `\ref{\langle store name : position \rangle}` and will return `1.(A)`.

`\__enumext_keyans_store_ref:`
`\__enumext_keyans_store_ref_aux_i:`
`\__enumext_keyans_store_ref_aux_ii:`

The function `\__enumext_keyans_store_ref:` handles the “*internal label and ref*” system used by the `save-ref` key for `\item*` and `\anspic*` commands. First we will create copies of the current $\langle labels \rangle$ and remove the dots “.” from them, we do not want to get double dots in references.

```

3277 \cs_new_protected:Nn \__enumext_keyans_store_ref:
3278 {
3279   \bool_if:NT \l__enumext_store_ref_key_bool
3280   {
3281     \cs_set_protected:Npn \__enumext_tmp:n ##1
3282     {
3283       \tl_set_eq:cc { \l__enumext_label_copy_##1_tl } { \l__enumext_label_##1_tl }
3284       \tl_reverse:c { \l__enumext_label_copy_##1_tl }
3285       \tl_remove_once:cn { \l__enumext_label_copy_##1_tl } { . }
3286       \tl_reverse:c { \l__enumext_label_copy_##1_tl }
3287     }
3288     \clist_map_inline:nn { i, v, vi, vii, viii } { \__enumext_tmp:n {##1} }
3289     \__enumext_keyans_store_ref_aux_i:
3290   }
3291 }

```

The auxiliary function `\__enumext_keyans_store_ref_aux_i:` set the variable `\l__enumext_newlabel_arg_one_tl` which will contain $\{\langle store name : position \rangle\}$ analyzing whether the environment in which they are executed is `enumext*` or `enumext`.

```

3292 \cs_new_protected:Nn \__enumext_keyans_store_ref_aux_i:
3293 {
3294   \bool_if:NT \g__enumext_starred_bool
3295   {
3296     \tl_set_eq:NN \l__enumext_label_copy_i_tl \l__enumext_label_copy_vii_tl
3297   }
3298   \int_compare:nNnT { \l__enumext_keyans_pic_level_int } = { 1 }
3299   {

```

```

3300     \tl_put_right:Ne \l__enumext_newlabel_arg_two_tl
3301     { \l__enumext_label_copy_i_tl . \l__enumext_label_copy_vi_tl }
3302   }
3303   \int_compare:nNt { \l__enumext_keyans_level_int } = { 1 }
3304   {
3305     \tl_put_right:Ne \l__enumext_newlabel_arg_two_tl
3306     { \l__enumext_label_copy_i_tl . \l__enumext_label_copy_v_tl }
3307   }
3308   \int_compare:nNt { \l__enumext_keyans_level_h_int } = { 1 }
3309   {
3310     \tl_put_right:Ne \l__enumext_newlabel_arg_two_tl
3311     { \l__enumext_label_copy_i_tl . \l__enumext_label_copy_viii_tl }
3312   }
3313   \tl_put_right:Ne \l__enumext_newlabel_arg_one_tl
3314   {
3315     \l__enumext_store_name_tl \c_colon_str
3316     \int_eval:n { \prop_count:c { g__enumext_ \l__enumext_store_name_tl _prop } }
3317   }
3318   \__enumext_keyans_store_ref_aux_ii:
3319 }

```

Now auxiliary function `\__enumext_keyans_store_ref_aux_ii:` save the result in the variable `\l__enumext_write_aux_file_tl` and finally we write in the `.aux` file.

```

3320 \cs_new_protected:Nn \__enumext_keyans_store_ref_aux_ii:
3321 {
3322   \tl_put_right:Ne \l__enumext_write_aux_file_tl
3323   {
3324     \__enumext_newlabel:nn
3325     { \exp_not:V \l__enumext_newlabel_arg_one_tl }
3326     { \l__enumext_newlabel_arg_two_tl }
3327   }
3328   \l__enumext_write_aux_file_tl
3329 }

```

(End of definition for `\__enumext_keyans_store_ref:`, `\__enumext_keyans_store_ref_aux_i:`, and `\__enumext_keyans_store_ref_aux_ii:`.)

13.36.3 Storing content in sequence

`\__enumext_keyans_addto_seq:n`
`\__enumext_keyans_addto_seq_link:`

The function `\__enumext_keyans_addto_seq:n` will pass the contents of the current *⟨label⟩* `\l__enumext_label_v_tl` for the `keyans` environment and the `\l__enumext_label_vi_tl` for the `keyanspic` environment when using `\item*` and `\anspic*`, followed by the *⟨contents⟩* of the *optional argument* of both commands to the `\l__enumext_store_current_label_tl` variable to the sequence defined by the `save-ans` key.

```

3330 \cs_new_protected:Npn \__enumext_keyans_addto_seq:n #1
3331 {
3332   \tl_clear:N \l__enumext_store_current_label_tl
3333   \int_compare:nNt { \l__enumext_keyans_pic_level_int } = { 1 }
3334   {
3335     \tl_put_right:Ne \l__enumext_store_current_label_tl { \item \l__enumext_label_vi_tl }
3336   }
3337   {
3338     \tl_put_right:Ne \l__enumext_store_current_label_tl { \item \l__enumext_label_v_tl }
3339   }
3340   \tl_if_no_value:nF { #1 }
3341   {
3342     \tl_if_empty:NF \l__enumext_store_keyans_item_opt_sep_v_tl
3343     {
3344       \tl_put_right:NV \l__enumext_store_current_label_tl \l__enumext_store_keyans_item_opt_
3345     }
3346     \tl_put_right:Nn \l__enumext_store_current_label_tl { #1 }
3347   }
3348   \__enumext_keyans_addto_seq_link:
3349 }

```

Checks if the `save-ref` key is active along with the `hyperref` package load, if both conditions are met, it will create the `\hyperlink` and then store using the `\__enumext_store_addto_seq:V` function. Finally, copy the contents of the variable `\l__enumext_store_current_label_tl` into the global variable `\g__enumext_check_ans_item_tl` to be used by the function `\__enumext_check_starred_cmd:n` and increment the value of the integer variable `\g__enumext_item_anskey_int` handled by the `check-ans` key.

```

3350 \cs_new_protected:Nn \__enumext_keyans_addto_seq_link:

```

```

3351 {
3352   \bool_lazy_and:nnT
3353   { \bool_if_p:N \l__enumext_store_ref_key_bool }
3354   { \bool_if_p:N \l__enumext_hyperref_bool }
3355   {
3356     \tl_put_right:Ne \l__enumext_store_current_label_tl
3357     {
3358       \hfill \exp_not:N \hyperlink
3359       {
3360         \exp_not:V \l__enumext_newlabel_arg_one_tl
3361       }
3362       { \exp_not:V \l__enumext_mark_ref_sym_tl }
3363     }
3364   }
3365   \__enumext_store_addto_seq:V \l__enumext_store_current_label_tl
3366   \bool_if:NT \l__enumext_check_answers_bool
3367   {
3368     \int_gincr:N \g__enumext_item_anskey_int
3369   }
3370 }

```

(End of definition for `\__enumext_keyans_addto_seq:n` and `\__enumext_keyans_addto_seq_link:.`)

13.36.4 The show-ans and show-pos keys for keyans and keyanspic

The function `\__enumext_keyans_save_item_opt:n` will save the optional argument of `\item*` and `\anspic*` in the variable `\l__enumext_store_current_opt_arg_tl`.

```

\__enumext_keyans_save_item_opt:n
\__enumext_keyans_show_item_opt:
\__enumext_keyans_show_item_opt_viii:

```

```

3371 \cs_new_protected:Npn \__enumext_keyans_save_item_opt:n #1
3372 {
3373   \tl_if_novalue:nF { #1 }
3374   {
3375     \tl_set:Nn \l__enumext_store_current_opt_arg_tl { #1 }
3376   }
3377 }

```

The function `\__enumext_keyans_show_item_opt:` will print the optional arguments of `\item*` and `\anspic*` when the show-ans or show-pos keys are set next to the key `wrap-opt` in `keyans` and `keyanspic` environments.

```

3378 \cs_new_protected:Nn \__enumext_keyans_show_item_opt:
3379 {
3380   \tl_if_empty:NF \l__enumext_store_current_opt_arg_tl
3381   {
3382     \bool_lazy_or:nnT
3383     { \bool_if_p:N \l__enumext_show_answer_bool }
3384     { \bool_if_p:N \l__enumext_show_position_bool }
3385     {
3386       \__enumext_keyans_wrapper_opt_v:n
3387       { \l__enumext_store_current_opt_arg_tl } \c_space_tl
3388     }
3389   }
3390 }

```

The function `\__enumext_keyans_show_item_opt_viii:` will print the optional argument of `\item*` when the `show-ans` or `show-pos` keys are set next to the key `wrap-opt` in `keyans*` environment.

```

3391 \cs_new_protected:Nn \__enumext_keyans_show_item_opt_viii:
3392 {
3393   \tl_if_empty:NF \l__enumext_store_current_opt_arg_tl
3394   {
3395     \bool_lazy_or:nnT
3396     { \bool_if_p:N \l__enumext_show_answer_bool }
3397     { \bool_if_p:N \l__enumext_show_position_bool }
3398     {
3399       \__enumext_keyans_wrapper_opt_viii:n
3400       { \l__enumext_store_current_opt_arg_tl } \c_space_tl
3401     }
3402   }
3403 }

```

(End of definition for `\__enumext_keyans_save_item_opt:n`, `\__enumext_keyans_show_item_opt:`, and `\__enumext_keyans_show_item_opt_viii:.`)

```

    \__enumext_keyans_pos_mark_set:
\__enumext_keyans_show_ans:
\__enumext_keyans_show_pos:

```

The function `\__enumext_keyans_pos_mark_set:` adjusts the horizontal spaces for the `mark-sep*` key taking into account the value of the `align` key and the width of $\langle label \rangle$.

```

3404 \cs_new_protected:Nn \__enumext_keyans_pos_mark_set:
3405 {
3406   \__enumext_label_width_by_box:Nn
3407   \l__enumext_mark_sep_tmpa_dim { \l__enumext_label_v_tl }
3408   \str_case:Vn \l__enumext_align_label_pos_v_str
3409   {
3410     { l }
3411     {
3412       \dim_set:Nn \l__enumext_mark_sep_tmpb_dim { \c_zero_dim }
3413     }
3414     { r }
3415     {
3416       \dim_set:Nn \l__enumext_mark_sep_tmpb_dim
3417       { \l__enumext_labelwidth_v_dim - \l__enumext_mark_sep_tmpa_dim }
3418     }
3419     { c }
3420     {
3421       \dim_set:Nn \l__enumext_mark_sep_tmpb_dim
3422       { 0.5\l__enumext_labelwidth_v_dim - 0.5\l__enumext_mark_sep_tmpa_dim }
3423     }
3424   }

```

Here we set the default values for the key `mark-ans*`, `mark-sep*` and `mark-pos*`.

```

3425   \dim_compare:nNtT { \l__enumext_mark_sym_sep_v_dim } = { \c_zero_dim }
3426   {
3427     \dim_set:Nn \l__enumext_mark_sym_sep_v_dim { \l__enumext_labelsep_v_dim }
3428   }
3429   \tl_set_eq:NN \l__enumext_mark_answer_sym_tl \l__enumext_mark_answer_sym_v_tl
3430   \dim_add:Nn \l__enumext_mark_sym_sep_v_dim { \l__enumext_mark_sep_tmpb_dim }
3431   \str_set_eq:NN \l__enumext_mark_position_str \l__enumext_mark_position_v_str
3432 }

```

The function `\__enumext_keyans_show_ans:` will print the $\langle symbol \rangle$ set by the `mark-ans*` key when the `show-ans` key is active.

```

3433 \cs_new_protected:Nn \__enumext_keyans_show_ans:
3434 {
3435   \bool_lazy_all:nT
3436   {
3437     { \bool_if_p:N \l__enumext_show_answer_bool }
3438     { \bool_if_p:N \l__enumext_item_wrap_key_bool }
3439   }
3440   {
3441     \__enumext_keyans_pos_mark_set:
3442     \__enumext_print_keyans_box:NN
3443     \l__enumext_labelwidth_v_dim \l__enumext_mark_sym_sep_v_dim
3444   }
3445 }

```

The function `\__enumext_keyans_show_pos:` will print the $\langle position \rangle$ of the stored content in *prop list*. Need add `1` to `\g__enumext_⟨store name⟩_prop` for `keyans` environment.

```

3446 \cs_new_protected:Nn \__enumext_keyans_show_pos:
3447 {
3448   \int_compare:nNtTF { \l__enumext_keyans_level_int } = { 1 }
3449   {
3450     \int_incr:N \l__enumext_show_pos_tmp_int
3451   }
3452   {
3453     \int_zero:N \l__enumext_show_pos_tmp_int
3454   }
3455   \bool_lazy_all:nT
3456   {
3457     { \bool_if_p:N \l__enumext_show_position_bool }
3458     { \bool_if_p:N \l__enumext_item_wrap_key_bool }
3459   }
3460   {
3461     \tl_set:Nc \l__enumext_mark_answer_sym_v_tl
3462     {
3463       \group_begin:
3464       \exp_not:N \normalfont
3465       \exp_not:N \footnotesize [ \int_eval:n

```

```

3466         {
3467             \prop_count:c { g__enumext_ \l__enumext_store_name_tl _prop }
3468             + \l__enumext_show_pos_tmp_int
3469         }
3470     ]
3471     \group_end:
3472 }
3473 \__enumext_keyans_pos_mark_set:
3474 \__enumext_print_keyans_box:NN
3475     \l__enumext_labelwidth_v_dim \l__enumext_mark_sym_sep_v_dim
3476 }
3477 }

```

(End of definition for `\__enumext_keyans_pos_mark_set:`, `\__enumext_keyans_show_ans:`, and `\__enumext_keyans_show_pos:`.)

13.37 Redefining `\item` and `\makelabel` in `enumext`

Redefining the `\item` command is not as simple as I thought. This command works in conjunction with the `\makelabel` command so I have to redefine both of them, in addition to this, we will have to use a couple of *global* variables to pass the values from one command to the other.

When *labeling* PDF is active `\makelabel` is redefined as `\hss #1` and the only way to get the `align` key to work correctly is to redefine `\makelabel` using `\makebox`. The best way to implement this is to use the conditional command `\IfDocumentMetadataTF` to force this redefinition and the dedicated `mode-box` key to manually activate it by the user.

The `\item` and `\item[⟨custom⟩]` commands work in the usual way on `enumext` and we will add `\item*`, `\item*[⟨symbol⟩]` and `\item*[⟨symbol⟩][⟨offset⟩]`.

`\__enumext_default_item:n`

First we will see if the *optional argument* is present, if it is NOT present we will check the state of the variable `\l__enumext_check_answers_bool` set by the key `no-store`, set the boolean variable `\l__enumext_wrap_label_X_bool` to “true” for the key `wrap-label` and execute `\__enumext_item_std:w` and the key `itemindent`, otherwise we will check the state of the boolean variable `\l__enumext_wrap_label_opt_X_bool` set by the key `wrap-label*` and execute `\__enumext_item_std:w` with the *optional argument* and the key `itemindent`.

```

3478 \cs_new_protected:Npn \__enumext_default_item:n #1
3479 {
3480     \tl_if_novalue:nTF {#1}
3481     {
3482         \bool_if:NT \l__enumext_check_answers_bool
3483         {
3484             \int_gincr:N \g__enumext_item_number_int
3485             \bool_set_true:N \l__enumext_item_number_bool
3486         }
3487         \bool_set_true:c { l__enumext_wrap_label_ \__enumext_level: _bool }
3488         \__enumext_item_std:w \tl_use:c { l__enumext_fake_item_indent_ \__enumext_level: _tl }
3489     }
3490     {
3491         \bool_set_eq:cc
3492         { l__enumext_wrap_label_ \__enumext_level: _bool }
3493         { l__enumext_wrap_label_opt_ \__enumext_level: _bool }
3494         \__enumext_item_std:w [ #1 ] \tl_use:c { l__enumext_fake_item_indent_ \__enumext_level: _tl }
3495     }
3496 }

```

(End of definition for `\__enumext_default_item:n`.)

`\__enumext_item_starred_exec:nn`

`\__enumext_item_starred_exec:`

The `\item*`, `\item*[⟨symbol⟩]` and `\item*[⟨symbol⟩][⟨offset⟩]` works like the *numbered \item*, but placing a `⟨symbol⟩` to the “left” of the “*label*” separated from it by the value the second *optional argument* `⟨offset⟩`.

`#1: \l__enumext_item_symbol_X_tl`

`#2: \l__enumext_item_symbol_sep_X_dim`

First we will make a copy of `\l__enumext_item_symbol_X_tl` which is set by the key `item-sym*` or passed as “*first optional argument*” in the global variable `\g__enumext_item_symbol_aux_tl`, followed by setting the variable `\l__enumext_item_symbol_sep_X_dim` set by the key `item-pos*` or by the “*second optional argument*”, then we will see the state of the variable `\l__enumext_check_answers_bool` set by the key `no-store`, set the boolean variable `\l__enumext_wrap_label_X_bool` to “true” for the key `wrap-label` and execute `\__enumext_item_std:w` and the key `itemindent`.

```

3497 \cs_new_protected:Npn \__enumext_item_starred_exec:nn #1 #2
3498 {
3499     \tl_if_novalue:nTF {#1}

```

```

3500     {
3501       \tl_gset_eq:Nc
3502       \g__enumext_item_symbol_aux_tl { \l__enumext_item_symbol_ \__enumext_level: _tl }
3503     }
3504     {
3505       \tl_gset:Nn \g__enumext_item_symbol_aux_tl {#1}
3506     }
3507   \tl_if_novalue:nTF {#2}
3508   {
3509     \dim_set_eq:cc
3510     { \l__enumext_item_symbol_sep_ \__enumext_level: _dim }
3511     { \l__enumext_labelsep_ \__enumext_level: _dim }
3512   }
3513   {
3514     \dim_set:cn { \l__enumext_item_symbol_sep_ \__enumext_level: _dim } {#2}
3515   }
3516   \bool_if:NT \l__enumext_check_answers_bool
3517   {
3518     \int_gincr:N \g__enumext_item_number_int
3519     \bool_set_true:N \l__enumext_item_number_bool
3520   }
3521   \bool_set_true:c { \l__enumext_wrap_label_ \__enumext_level: _bool }
3522   \__enumext_item_std:w \tl_use:c { \l__enumext_fake_item_indent_ \__enumext_level: _tl }
3523 }

```

The function `\__enumext_item_starred_exec:` will be responsible for executing `\item*` for the `enumext` environment.

```

3524 \cs_new_protected:Nn \__enumext_item_starred_exec:
3525 {
3526   \tl_if_empty:cF { \l__enumext_item_symbol_ \__enumext_level: _tl }
3527   {
3528     \mode_leave_vertical:
3529     \skip_horizontal:n { -\dim_use:c { \l__enumext_item_symbol_sep_ \__enumext_level: _dim } }
3530     \hbox_overlap_left:n { \g__enumext_item_symbol_aux_tl }
3531     \skip_horizontal:n { \dim_use:c { \l__enumext_item_symbol_sep_ \__enumext_level: _dim } }
3532   }
3533 }

```

(End of definition for `\__enumext_item_starred_exec:nn` and `\__enumext_item_starred_exec:.`)

`\__enumext_redefine_item:` The function `\__enumext_redefine_item:` will redefine the `\item` command in the `enumext` environment adding `\item*`. This function are passed to `\__enumext_list_arg_two_X:` used in the definition of the `enumext` environment (§13.42).

```

3534 \cs_new_protected:Nn \__enumext_redefine_item:
3535 {
3536   \RenewDocumentCommand \item { s o o }
3537   {
3538     \bool_if:nTF {##1}
3539     {
3540       \__enumext_item_starred_exec:nn {##2} {##3}
3541     }
3542     { \__enumext_default_item:n {##2} }
3543   }
3544 }

```

(End of definition for `\__enumext_redefine_item:.`)

`\__enumext_make_label:` The function `\__enumext_make_label:` redefine `\make_label` for the keys `mode-box`, `align`, `font`, `wrap-label`, `wrap-label*` and `\item*` for `enumext` environment. This function are passed to `\__enumext_list_arg_two_X:` used in the definition of the `enumext` environment (§13.42).

```

3545 \cs_new_protected:Nn \__enumext_make_label:
3546 {
3547   \IfDocumentMetadataTF
3548   {
3549     \__enumext_make_label_box:
3550   }
3551   {
3552     \bool_if:NTF \l__enumext_mode_box_bool
3553     {
3554       \__enumext_make_label_box:
3555     }

```



```

3556         {
3557             \__enumext_make_label_std:
3558         }
3559     }
3560 }

```

Standard definition when `\DocumentMetadata` is not active.

```

3561 \cs_new_protected:Nn \__enumext_make_label_std:
3562 {
3563     \RenewDocumentCommand \makeLabel { m }
3564     {
3565         \tl_use:c { l__enumext_label_fill_left_ \__enumext_level: _tl }
3566         \__enumext_item_starred_exec:
3567         \tl_use:c { l__enumext_label_font_style_ \__enumext_level: _tl }
3568         \bool_if:cTF { l__enumext_wrap_label_ \__enumext_level: _bool }
3569         {
3570             \use:c { __enumext_wrapper_label_ \__enumext_level: :n } { ##1 }
3571         }
3572         { ##1 }
3573         \tl_use:c { l__enumext_label_fill_right_ \__enumext_level: _tl }
3574         \tl_gclear:N \g__enumext_item_symbol_aux_tl
3575     }
3576 }

```

Definition using `\makebox` when `\DocumentMetadata` is active or `mode-box` is active.

- Here it is necessary to use `\strut\smash` to maintain text *alignment* in case the user wants to use `\labelbx` for example. In my experiments with *mimicking* the `description` environment it was the only way out and it seems to have no adverse effects and may serve in the future as a basis for a more generic `list` environment package than `enumext`.

```

3577 \cs_new_protected:Nn \__enumext_make_label_box:
3578 {
3579     \RenewDocumentCommand \makeLabel { m }
3580     {
3581         \strut\smash
3582         {
3583             \makebox
3584             [ \dim_use:c { l__enumext_labelwidth_ \__enumext_level: _dim } ]
3585             [ \str_use:c { l__enumext_align_label_pos_ \__enumext_level: _str } ]
3586             {
3587                 \__enumext_item_starred_exec:
3588                 \tl_use:c { l__enumext_label_font_style_ \__enumext_level: _tl }
3589                 \bool_if:cTF { l__enumext_wrap_label_ \__enumext_level: _bool }
3590                 {
3591                     \use:c { __enumext_wrapper_label_ \__enumext_level: :n } { ##1 }
3592                 }
3593                 { ##1 }
3594                 \tl_gclear:N \g__enumext_item_symbol_aux_tl
3595             }
3596         } % close smash
3597     }
3598 }

```

(End of definition for `\__enumext_make_label:`, `\__enumext_make_label_std:`, and `\__enumext_make_label_box:`.)

13.38 Setting `item-sym*` and `item-pos*` keys

In order to have a cleaner implementation of `\item*` for the `enumext` and `enumext*` environments it is best to define a couple of keys that allow us to control and set by default the `<symbol>` and its `<offset>`.

```

item-sym* Define and set item-sym* and item-pos* keys for enumext and enumext*.
item-pos*
3599 \cs_set_protected:Npn \__enumext_tmp:nn #1 #2
3600 {
3601     \keys_define:nn { enumext / #1 }
3602     {
3603         item-sym* .tl_set:c = { l__enumext_item_symbol_#2_tl },
3604         item-sym* .value_required:n = true,
3605         item-sym* .initial:n = {\textborn},
3606         item-pos* .dim_set:c = { l__enumext_item_symbol_sep_#2_dim },
3607         item-pos* .value_required:n = true,
3608     }
3609 }
3610 \clist_map_inline:nn
3611 {

```

```

3612     {level-1}{i}, {level-2}{ii}, {level-3}{iii}, {level-4}{iv}, {enumext*}{vii}
3613   }
3614   { \__enumext_tmp:nn #1 }

```

(End of definition for *item-sym** and *item-pos**.)

13.39 Handling unknown keys

At this point in the code I already know that I will NOT add more *⟨keys⟩* for and since I have already been quite *paranoid and restrictive* with the definitions of environments and commands, the only thing left to do is do it with the *⟨keys⟩* (you have to be consistent in life).

- Well, the paragraph above is not so real, after all I had to add more *⟨keys⟩* than I had planned, not everything turns out the way one thinks in life.

13.39.1 Handling unknown keys for keyans, keyans* and keyanspic

Define and set *unknown* key for *keyans*, *keyans** and *keyanspic* environments. Here it is necessary to set *\l__enumext_envir_name_tl* in case an *unknown* key is passed using *\setenumext*.

```

unknown
\__enumext_keyans_unknown_keys:n
\__enumext_keyans_unknown_keys:nn
3615 \cs_set_protected:Npn \__enumext_tmp:n #1
3616 {
3617   \keys_define:nn { enumext / #1 }
3618   {
3619     unknown .code:n = {
3620       \tl_set:Nn \l__enumext_envir_name_tl {#1}
3621       \__enumext_keyans_unknown_keys:n {##1}
3622     },
3623   }
3624 }
3625 \clist_map_inline:nn { keyans, keyans*, keyanspic } { \__enumext_tmp:n {#1} }

```

Internal functions for handling *unknown* key.

```

3626 \cs_new_protected:Npn \__enumext_keyans_unknown_keys:n #1
3627 {
3628   \exp_args:NV \__enumext_keyans_unknown_keys:nn \l_keys_key_str {#1}
3629 }
3630 \cs_new_protected:Npn \__enumext_keyans_unknown_keys:nn #1#2
3631 {
3632   \tl_if_blank:nTF {#2}
3633   {
3634     \msg_error:nne { enumext } { keyans-unknown-key } {#1}
3635   }
3636   {
3637     \msg_error:nnee { enumext } { keyans-unknown-key-value } {#1} {#2}
3638   }
3639 }

```

(End of definition for *unknown*, *__enumext_keyans_unknown_keys:n*, and *__enumext_keyans_unknown_keys:nn*.)

13.39.2 Handling unknown keys for enumext*

Define and set *unknown* key for *enumext** environment.

```

unknown
\__enumext_starred_unknown_keys:n
\__enumext_starred_unknown_keys:nn
3640 \keys_define:nn { enumext / enumext* }
3641 {
3642   unknown .code:n = {
3643     \tl_set:Nn \l__enumext_envir_name_tl { enumext* }
3644     \__enumext_starred_unknown_keys:n {#1}
3645   },
3646 }

```

Internal functions for handling *unknown* key.

```

3647 \cs_new_protected:Npn \__enumext_starred_unknown_keys:n #1
3648 {
3649   \exp_args:NV \__enumext_starred_unknown_keys:nn \l_keys_key_str {#1}
3650 }
3651 \cs_new_protected:Npn \__enumext_starred_unknown_keys:nn #1#2
3652 {
3653   \tl_if_blank:nTF {#2}
3654   {
3655     \msg_error:nne { enumext } { starred-unknown-key } {#1}
3656   }
3657   {
3658     \msg_error:nnee { enumext } { starred-unknown-key-value } {#1} {#2}
3659   }
3660 }

```

(End of definition for *unknown*, *__enumext_starred_unknown_keys:n*, and *__enumext_starred_unknown_keys:nn*.)

13.39.3 Handling unknown keys for enumext

unknown Defines and set the key `unknown` for `enumext` environment.

```

3661 \cs_set_protected:Npn \__enumext_tmp:n #1
3662 {
3663   \keys_define:nn { enumext / level-#1 }
3664   {
3665     unknown .code:n = {
3666       \int_set:Nn \__enumext_level_int { #1 }
3667       \tl_set:Nn \__enumext_envir_name_tl { enumext }
3668       \__enumext_standar_unknown_keys:n {##1}
3669     },
3670   }
3671 }
3672 \clist_map_inline:nn {1, 2, 3, 4} { \__enumext_tmp:n {#1} }

```

Internal functions for handling `unknown` key.

```

3673 \cs_new_protected:Npn \__enumext_standar_unknown_keys:n #1
3674 {
3675   \exp_args:NV \__enumext_standar_unknown_keys:nn \l_keys_key_str {#1}
3676 }
3677 \cs_new_protected:Npn \__enumext_standar_unknown_keys:nn #1#2
3678 {
3679   \tl_if_blank:nTF {#2}
3680   {
3681     \msg_error:nne { enumext } { standar-unknown-key } {#1}
3682   }
3683   {
3684     \msg_error:nnee { enumext } { standar-unknown-key-value } {#1} {#2}
3685   }
3686 }

```

(End of definition for `unknown`, `\__enumext_standar_unknown_keys:n`, and `\__enumext_standar_unknown_keys:nn`.)

13.40 Redefining `\item` and `\makeLabel` in keyans

The `\item` and `\item[⟨custom⟩]` commands work in the usual way in `keyans`, but the `\item*` and `\item*[⟨content⟩]` commands *store* the current `⟨label⟩` next to the `⟨content⟩` if it is present in the *sequence* and *prop list* defined by `save-ans` key.

`\__enumext_keyans_default_item:n` The function `\__enumext_keyans_default_item:n` executes the original behavior of the `\item` along with the keys `wrap-label`, `wrap-label*` and `itemindent`.

```

3687 \cs_new_protected:Npn \__enumext_keyans_default_item:n #1
3688 {
3689   \tl_if_novalue:nTF { #1 }
3690   {
3691     \bool_set_true:N \__enumext_wrap_label_v_bool
3692     \__enumext_item_std:w \tl_use:N \__enumext_fake_item_indent_v_tl
3693   }
3694   {
3695     \bool_set_eq:NN \__enumext_wrap_label_v_bool \__enumext_wrap_label_opt_v_bool
3696     \__enumext_item_std:w [#1] \tl_use:N \__enumext_fake_item_indent_v_tl
3697   }
3698 }

```

(End of definition for `\__enumext_keyans_default_item:n`.)

`\__enumext_keyans_starred_item:n` The function `\__enumext_keyans_starred_item:n` will take as argument `#1` the *optional argument* `[⟨content⟩]` passed to `\item*` and save it via the `\__enumext_keyans_save_item_opt:n` function, then activate the `wrap-label` key, execute `\item` using `\__enumext_item_std:w`, the `itemindent` key and print the *optional argument* using the `\__enumext_keyans_show_item_opt:` function handled by the `wrap-opt` key.

```

3699 \cs_new_protected:Npn \__enumext_keyans_starred_item:n #1
3700 {
3701   \__enumext_keyans_save_item_opt:n { #1 }
3702   \bool_set_true:N \__enumext_wrap_label_v_bool
3703   \__enumext_item_std:w \tl_use:N \__enumext_fake_item_indent_v_tl
3704   \__enumext_keyans_show_item_opt:

```

Now *store* the current `⟨label⟩` first in the *prop list* (including the *optional argument*), run the internal “*label and ref*” system if the `save-ref` key is active, then *store* in the *sequence* and finally increments `\g__enumext_check_starred_cmd_int` for internal check system.

```

3705   \__enumext_keyans_addto_prop:n { #1 }

```

```

3706     \__enumext_keyans_store_ref:
3707     \__enumext_keyans_addto_seq:n { #1 }
3708     \int_gincr:N \__enumext_check_starred_cmd_int
3709 }

```

(End of definition for __enumext_keyans_starred_item:n)

\item*
 __enumext_keyans_redefine_item:

The function __enumext_keyans_redefine_item: is responsible for adding the *starred argument* and *optional argument* by the __enumext_list_arg_two_v: function in the definition of the *keyans* environment. Here we will set to true the variable \l__enumext_item_wrap_key_bool used by the *wrap-ans** key only when \item* is executed and additionally we need to use \peek_remove_spaces:n to avoid an unwanted space when using \item* together with the *itemindent* key. This function are passed to __enumext_list_arg_two_v: used in the definition of the *keyans* environment (§13.41).

```

3710 \cs_new_protected:Nn \__enumext_keyans_redefine_item:
3711 {
3712     \RenewDocumentCommand \item { s o }
3713     {
3714         \bool_if:nTF {##1}
3715         {
3716             \bool_set_true:N \l__enumext_item_wrap_key_bool % wrap-ans*
3717             \peek_remove_spaces:n
3718             {
3719                 \__enumext_keyans_starred_item:n {##2}
3720             }
3721         }
3722         {
3723             \bool_set_false:N \l__enumext_item_wrap_key_bool
3724             \__enumext_keyans_default_item:n {##2}
3725         }
3726     }
3727 }

```

(End of definition for \item* and __enumext_keyans_redefine_item:. This function is documented on page 17.)

__enumext_keyans_make_label:
 __enumext_keyans_wrapper_label:n
 __enumext_keyans_make_label_std:
 __enumext_keyans_make_label_box:

The function __enumext_keyans_make_label: redefine \makelabel for the keys *mode-box*, *align*, *font*, *wrap-label*, *wrap-label**, *wrap-ans** and \item* for *keyans* environment. This function are passed to __enumext_list_arg_two_v: used in the definition of the *keyans* environment (§13.41).

```

3728 \cs_new_protected:Nn \__enumext_keyans_make_label:
3729 {
3730     \IfDocumentMetadataTF
3731     {
3732         \__enumext_keyans_make_label_box:
3733     }
3734     {
3735         \bool_if:NTF \l__enumext_mode_box_bool
3736         {
3737             \__enumext_keyans_make_label_box:
3738         }
3739         {
3740             \__enumext_keyans_make_label_std:
3741         }
3742     }
3743 }

```

We added conditionals to the __enumext_keyans_wrapper_label:n function to handle the keys *wrap-ans**, *wrap-label* and *wrap-label**.

```

3744 \cs_new_protected:Npn \__enumext_keyans_wrapper_label:n #1
3745 {
3746     \bool_lazy_all:nT
3747     {
3748         { \bool_if_p:N \l__enumext_wrap_label_v_bool }
3749         { \bool_if_p:N \l__enumext_show_answer_bool }
3750         { \bool_if_p:N \l__enumext_item_wrap_key_bool }
3751         { \cs_if_exist_p:N \__enumext_keyans_wrapper_item_v:n }
3752     }
3753     {
3754         \cs_set_eq:NN \__enumext_wrapper_label_v:n \__enumext_keyans_wrapper_item_v:n
3755     }
3756     \bool_if:NTF \l__enumext_wrap_label_v_bool
3757     {
3758         \__enumext_wrapper_label_v:n { #1 }

```

```

3759     }
3760     { #1 }
3761 }

```

Standard definition when `\DocumentMetadata` is not active.

```

3762 \cs_new_protected:Nn \__enumext_keyans_make_label_std:
3763 {
3764   \RenewDocumentCommand \makeLabel { m }
3765   {
3766     \tl_use:N \l__enumext_label_fill_left_v_tl
3767     \__enumext_keyans_show_ans:
3768     \__enumext_keyans_show_pos:
3769     \tl_use:N \l__enumext_label_font_style_v_tl
3770     \__enumext_keyans_wrapper_label:n { ##1 }
3771     \tl_use:N \l__enumext_label_fill_right_v_tl
3772   }
3773 }

```

Definition using `\makebox` when `\DocumentMetadata` is active or `mode-box` is active.

```

3774 \cs_new_protected:Nn \__enumext_keyans_make_label_box:
3775 {
3776   \RenewDocumentCommand \makeLabel { m }
3777   {
3778     \strut\smash
3779     {
3780       \makebox[ \l__enumext_labelwidth_v_dim ][ \l__enumext_align_label_pos_v_str ]
3781       {
3782         \__enumext_keyans_show_ans:
3783         \__enumext_keyans_show_pos:
3784         \tl_use:N \l__enumext_label_font_style_v_tl
3785         \__enumext_keyans_wrapper_label:n { ##1 }
3786       }
3787     }
3788   }
3789 }

```

(End of definition for `\__enumext_keyans_make_label:` and others.)

13.41 Second argument of the lists

At this point in the code we have already programmed most of the tools needed to create a *custom list* environment, remember that the `\__enumext_start_list:nn` function takes two arguments, we have the “first” one ready, the “second” one we will define for all levels of the `enumext` environment, the `keyans` environment and the `enumext*` and `keyans*` environments.

Here we will implement the `\__enumext_list_arg_two_X:` function, which will be responsible for setting all the list parameters, the counter, the redefinition of `\item`, `\makeLabel` along with the keys `ref`, `itemindent` and `show-length`.

13.41.1 Calculation of `\leftmargin` and `\itemindent`

Consider the figure 9 where the default margins (on the left) of a list are represented.

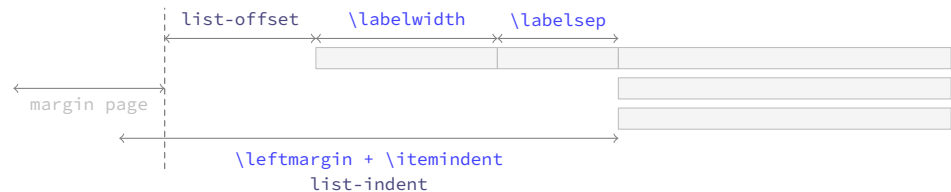


Figure 9: Representation of standard horizontal lengths in `list` environment.

The idea is to have control over these margins so that our list does not overlap the left margin of the page. The *key* relationship is that the “right edge” of the `\labelsep` equals the “right edge” of the `\itemindent`, so that the left edge of the “label box” is at `\leftmargin + \itemindent` minus `\labelwidth + \labelsep`. Thus, the handling of the margins by the package will be as shown in the figure 10.

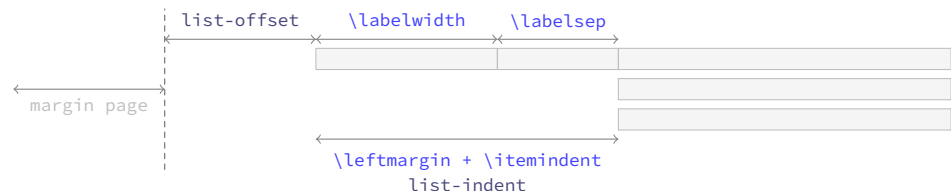


Figure 10: Representation of horizontal lengths concept in list in `enumext`.

Where the default values will look like in the figure 11.

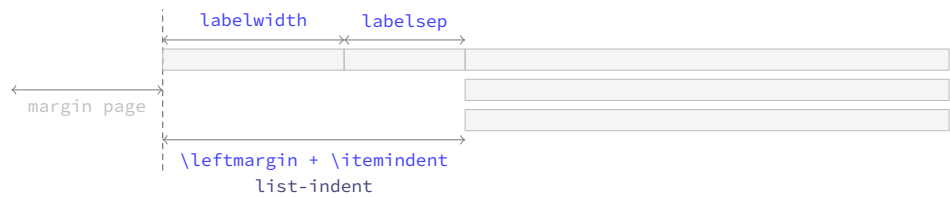


Figure 11: Default horizontal lengths in enumext.

```

\__enumext_calc_hspace:NNNNNNN
\__enumext_calc_hspace:ccccccc

```

The function `\__enumext_calc_hspace:NNNNNNN` takes seven arguments to be able to determine horizontal spaces for all list environment:

```

#1: \__enumext_labelwidth_X_dim      #2: \__enumext_labelsep_X_dim
#3: \__enumext_listoffset_X_dim      #4: \__enumext_leftmargin_tmp_X_dim
#5: \__enumext_leftmargin_X_dim      #6: \__enumext_itemindent_X_dim
#7: \__enumext_leftmargin_tmp_X_bool

```

And returns the “adjusted” values of `\leftmargin` and `\itemindent`.

```

3790 \cs_new_protected:Npn \__enumext_calc_hspace:NNNNNNN #1 #2 #3 #4 #5 #6 #7
3791 {
3792   \dim_compare:nNt { #1 } < { \c_zero_dim }
3793   {
3794     \msg_warning:nnnV { enumext } { width-non-positive } { labelwidth } #1
3795     \dim_set:Nn #1 { \dim_abs:n { #1 } }
3796   }
3797   \dim_compare:nNt { #2 } < { \c_zero_dim }
3798   {
3799     \msg_warning:nnnV { enumext } { width-negative } { labelsep } #2
3800     \dim_set:Nn #2 { \dim_abs:n { #2 } }
3801   }

```

If no value has been passed to the `labelwidth` and `labelsep` keys we set the default values for `\__enumext_leftmargin_tmp_X_dim`.

```

3802   \bool_if:NF #7 { \dim_set:Nn #4 { #1 + #2 } }

```

We now analyze the cases and set the values for `\leftmargin` and `\itemindent`.

```

3803   \dim_compare:nNtF { #4 } < { \c_zero_dim }
3804   {
3805     \dim_set:Nn #6 { #1 + #2 - #4 }
3806     \dim_set:Nn #5 { #1 + #2 + #3 - #6 }
3807   }
3808   {
3809     \dim_compare:nNt { #4 } = { #1 + #2 }
3810     { \dim_set:Nn #6 { \c_zero_dim } }
3811     \dim_compare:nNt { #4 } < { #1 + #2 }
3812     { \dim_set:Nn #6 { #1 + #2 - #4 } }
3813     \dim_compare:nNt { #4 } > { #1 + #2 }
3814     {
3815       \dim_set:Nn #6 { -#1 - #2 + #4 }
3816       \dim_set:Nn #6 { #6*-1 }
3817     }
3818     \dim_set:Nn #5 { #1 + #2 + #3 - #6 }
3819   }
3820 }
3821 \cs_generate_variant:Nn \__enumext_calc_hspace:NNNNNNN { ccccccc }

```

(End of definition for `\__enumext_calc_hspace:NNNNNNN`.)

13.41.2 Setting second argument of the lists

We will “not set” `\leftmargini`, `\leftmarginii`, `\leftmarginiii` or `\leftmarginiv`, in this case, we will directly set the parameters for vertical and horizontal list spacing per level.

```

\__enumext_list_arg_two_i:
\__enumext_list_arg_two_ii:
\__enumext_list_arg_two_iii:
\__enumext_list_arg_two_iv:
\__enumext_list_arg_two_v:
3822 \cs_set_protected:Npn \__enumext_tmp:n #1
3823 {
3824   \cs_new_protected:cpn { __enumext_list_arg_two_#1: }
3825   {
3826     \__enumext_calc_hspace:ccccccc
3827     { \__enumext_labelwidth_#1_dim } { \__enumext_labelsep_#1_dim }
3828     { \__enumext_listoffset_#1_dim } { \__enumext_leftmargin_tmp_#1_dim }
3829     { \__enumext_leftmargin_#1_dim } { \__enumext_itemindent_#1_dim }
3830     { \__enumext_leftmargin_tmp_#1_bool }

```

```

3831 \clist_map_inline:nn
3832 { labelsep, labelwidth, itemindent, leftmargin, rightmargin, listparindent }
3833 { \dim_set_eq:cc {####1} { \__enumext_####1_#1_dim } }
3834 \clist_map_inline:nn { topsep, parsep, partopsep, itemsep }
3835 { \skip_set_eq:cc {####1} { \__enumext_####1_#1_skip } }
3836 \clist_map_inline:nn { beginparpenalty, itempenalty, endparpenalty }
3837 { \int_set_eq:cc {@####1} { \__enumext_####1_#1_int } }
3838 \usecounter { enumX#1 }
3839 \setcounter { enumX#1 } { \int_eval:n { \int_use:c { \__enumext_start_#1_int } - 1 } }
3840 \str_if_eq:nnTF {#1} { v }
3841 {
3842   \__enumext_keyans_redefine_item:
3843   \__enumext_keyans_make_label:
3844   \__enumext_keyans_ref:
3845   \__enumext_keyans_fake_item_indent:
3846   \bool_if:cT { \__enumext_show_length_#1_bool }
3847   {
3848     \msg_term:nnnn { enumext } { list-lengths-not-nested } { v } { keyans }
3849   }
3850 }
3851 {
3852   \__enumext_redefine_item:
3853   \__enumext_make_label:
3854   \__enumext_standar_ref:
3855   \__enumext_fake_item_indent:
3856   \bool_if:cT { \__enumext_show_length_#1_bool }
3857   {
3858     \msg_term:nnne { enumext } { list-lengths } {#1}
3859     { \int_use:N \__enumext_level_int }
3860   }
3861 }
3862 }
3863 }
3864 \clist_map_inline:nn { i, ii, iii, iv, v } { \__enumext_tmp:n {#1} }

```

(End of definition for `\__enumext_list_arg_two_i:` and others.)

```

\__enumext_list_arg_two_vii:
\__enumext_list_arg_two_viii:

```

For the horizontal environments `enumext*` and `keyans*` the implementation is similar, but, the value of `\partopsep` is always `\opt`. At this point we will modify the `parsep` key to make it take the value of the `itemsep` key and later, in the environment definition, we will modify `parindent` to make it set the value of `\listparindent` and `parsep` to set the value of `\parskip` locally.

```

3865 \cs_set_protected:Npn \__enumext_tmp:n #1
3866 {
3867   \cs_new_protected:cpn { \__enumext_list_arg_two_#1: }
3868   {
3869     \bool_set_true:c { \__enumext_leftmargin_tmp_#1_bool }
3870     \dim_zero:c { \__enumext_leftmargin_tmp_#1_dim }
3871     \__enumext_calc_hspace:ccccc
3872     { \__enumext_labelwidth_#1_dim } { \__enumext_labelsep_#1_dim }
3873     { \__enumext_listoffset_#1_dim } { \__enumext_leftmargin_tmp_#1_dim }
3874     { \__enumext_leftmargin_#1_dim } { \__enumext_itemindent_#1_dim }
3875     { \__enumext_leftmargin_tmp_#1_bool }
3876     \clist_map_inline:nn
3877     { labelsep, labelwidth, itemindent, leftmargin, rightmargin, listparindent }
3878     { \dim_set_eq:cc {####1} { \__enumext_####1_#1_dim } }
3879     \clist_map_inline:nn { topsep, parsep, partopsep, itemsep }
3880     { \skip_set_eq:cc {####1} { \__enumext_####1_#1_skip } }
3881     \clist_map_inline:nn { beginparpenalty, itempenalty, endparpenalty }
3882     { \int_set_eq:cc {@####1} { \__enumext_####1_#1_int } }
3883     \skip_set_eq:Nc \parsep { \__enumext_itemsep_#1_skip }
3884     \skip_zero:N \partopsep
3885     \usecounter { enumX#1 }
3886     \setcounter { enumX#1 } { \int_eval:n { \int_use:c { \__enumext_start_#1_int } - 1 } }
3887     \__enumext_starred_ref:
3888     \str_if_eq:nnTF {#1} { vii }
3889     {
3890       \__enumext_fake_item_indent_vii:
3891       \bool_if:cT { \__enumext_show_length_vii_bool }
3892       { \msg_term:nnnn { enumext } { list-lengths-not-nested } { vii } { enumext* } }
3893     }
3894   }

```



```

3895         \__enumext_fake_item_indent_viii:
3896         \bool_if:cT { \__enumext_show_length_#1_bool }
3897         { \msg_term:nnnn { enumext } { list-lengths-not-nested } { #1 } { keyans* } }
3898     }
3899 }
3900 }
3901 \clist_map_inline:nn { vii, viii } { \__enumext_tmp:n {#1} }

```

(End of definition for __enumext_list_arg_two_vii: and __enumext_list_arg_two_viii:.)

13.42 The environment enumext

__enumext_safe_exec: The __enumext_safe_exec: function first call the function __enumext_is_not_nested: which sets \g__enumext_standar_bool to “true” if we are NOT nested within `enumext*`, then call the function __enumext_internal_mini_page: to create the environment `\__enumext_mini_page`, we will increment \l__enumext_level_int to restrict nesting of the environment, set \l__enumext_standar_bool to “true” and finally call the function __enumext_is_on_first_level: which sets \l__enumext_standar_first_bool to “true” only if the environment is NOT nested and we are at the “first level”.

```

3902 \cs_new_protected:Nn \__enumext_safe_exec:
3903 {
3904     \__enumext_is_not_nested:
3905     \__enumext_internal_mini_page:
3906     \int_incr:N \l__enumext_level_int
3907     \int_compare:nNtT { \l__enumext_level_int } > { 4 }
3908     { \msg_fatal:nn { enumext } { list-too-deep } }
3909     \bool_set_true:N \l__enumext_standar_bool
3910     \bool_set_false:N \l__enumext_starred_bool
3911     \__enumext_is_on_first_level:
3912 }

```

(End of definition for __enumext_safe_exec:.)

__enumext_parse_keys:n The __enumext_parse_store_keys:n function first we will clear the variable \l__enumext_series_name_str used by the key `series` and then we check if we are at the “first level”, if so we process the `<keys>` and then execute the function __enumext_parse_series:n used by the key `series` and call the function __enumext_nested_base_line_fix: used by the key `base-fix`, otherwise we will pass the `<keys>` to the inner levels of the environment then we execute the function __enumext_store_active_keys:n and reprocess the `<keys>` to pass them to the `sequence` if the key `save-key` is not active.

```

3913 \cs_new_protected:Npn \__enumext_parse_keys:n #1
3914 {
3915     \tl_if_novalue:nF {#1}
3916     {
3917         \str_clear:N \l__enumext_series_name_str
3918         \int_compare:nNtTF { \l__enumext_level_int } = { 1 }
3919         {
3920             \keys_set:nn { enumext / level-1 } {#1}
3921             \bool_if:NF \l__enumext_print_keyans_cmd_bool
3922             {
3923                 \__enumext_parse_series:n {#1}
3924             }
3925             \__enumext_nested_base_line_fix:
3926         }
3927         {
3928             \exp_args:Ne \keys_set:nn
3929             { enumext / level-\int_use:N \l__enumext_level_int } {#1}
3930             \bool_if:NF \l__enumext_print_keyans_cmd_bool
3931             {
3932                 \__enumext_parse_series:n {#1}
3933             }
3934         }
3935         \__enumext_store_active_keys:n {#1}
3936     }
3937 }

```

(End of definition for __enumext_parse_keys:n.)

__enumext_start_store_level: The __enumext_start_store_level: function activate the “storing structure” mechanism in the `sequence` for the command `\anskey` and the environment `anskey*`.

```

3938 \cs_new_protected:Nn \__enumext_start_store_level:
3939 {
3940     \bool_lazy_all:nT

```

```

3941     {
3942     { \bool_if_p:N \__enumext_store_active_bool }
3943     { \bool_not_p:n { \__enumext_keyans_env_bool } }
3944     { \bool_if_p:N \g__enumext_standar_bool }
3945     }
3946     {
3947     \int_compare:nNnT { \__enumext_level_int } > { 1 }
3948     {
3949         \bool_set_true:c { \__enumext_store_upper_level_ \__enumext_level: _bool }
3950         \__enumext_store_level_open:
3951     }
3952     }

```

If `enumext` are nested in `enumext*` add `\__enumext_store_level_open:` to preserve the “*storing structure*”.

```

3953     \bool_lazy_all:nT
3954     {
3955     { \bool_if_p:N \__enumext_store_active_bool }
3956     { \bool_not_p:n { \__enumext_keyans_env_bool } }
3957     { \int_compare_p:nNn { \__enumext_level_h_int } = { 1 } }
3958     }
3959     {
3960     \int_compare:nNnT { \__enumext_level_int } > { 0 }
3961     {
3962         \bool_set_true:c { \__enumext_store_upper_level_ \__enumext_level: _bool }
3963         \__enumext_store_level_open:
3964     }
3965     }
3966     }

```

(End of definition for `\__enumext_start_store_level:`)

`\__enumext_stop_store_level:` The `\__enumext_stop_store_level:` function stop the “*storing structure*” mechanism in the *sequence* for the command `\anskey` and the environment `anskey*`.

```

3967 \cs_new_protected:Nn \__enumext_stop_store_level:
3968 {
3969     \bool_if:cT { \__enumext_store_upper_level_ \__enumext_level: _bool }
3970     {
3971         \__enumext_store_level_close:
3972     }
3973     }

```

(End of definition for `\__enumext_stop_store_level:`)

`\__enumext_multicols_start:` The function `\__enumext_multicols_start:` will start the `multicols` environment according to the value passed by the `columns` key, then set the default value for `\columnsep` when `columns-sep=opt` and set the value of `\multicolsep` equal to zero and leave `\columnseprule` equal to zero for inner levels.

```

3974 \cs_new_protected:Nn \__enumext_multicols_start:
3975 {
3976     \int_compare:nNnT
3977     { \int_use:c { \__enumext_columns_ \__enumext_level: _int } } > { 1 }
3978     {
3979         \dim_compare:nNnT
3980         { \dim_use:c { \__enumext_columns_sep_ \__enumext_level: _dim } } = { \c_zero_dim }
3981         {
3982             \dim_set:cn { \__enumext_columns_sep_ \__enumext_level: _dim }
3983             {
3984                 ( \dim_use:c { \__enumext_labelwidth_ \__enumext_level: _dim }
3985                 + \dim_use:c { \__enumext_labelsep_ \__enumext_level: _dim }
3986                 ) / \int_use:c { \__enumext_columns_ \__enumext_level: _int }
3987                 - \dim_use:c { \__enumext_listoffset_ \__enumext_level: _dim }
3988             }
3989         }
3990         \dim_set_eq:Nc \columnsep { \__enumext_columns_sep_ \__enumext_level: _dim }
3991         \int_compare:nNnT { \__enumext_level_int } > { 1 }
3992         {
3993             \dim_zero:N \columnseprule
3994         }

```

We will calculate the *vertical spacing* settings for the `multicols` environment using the function `\__enumext_multi_addvspace:`, apply our “*vertical adjust spacing*”, then start the `multicols` environment.

```

3995     \bool_if:cF { \__enumext_minipage_active_ \__enumext_level: _bool }
3996     {

```

```

3997         \skip_zero:N \multicolsep
3998         \__enumext_multi_addvspace:
3999     }
4000     \raggedcolumns
4001     \begin{multicols}{\int_use:c { \__enumext_columns_ \__enumext_level: _int } }
4002 }
4003 }

```

(End of definition for __enumext_multicols_start:.)

__enumext_multicols_stop: The function __enumext_multicols_stop: will stop the `multicols` environment and apply our “vertical adjust” spacing. For compatibility with *tagged* PDF, the closing of the `list` environment is executed here along with __enumext_stop_store_level:.

```

4004 \cs_new_protected:Nn \__enumext_multicols_stop:
4005 {
4006     \int_compare:nNnTF
4007     { \int_use:c { \__enumext_columns_ \__enumext_level: _int } } > { 1 }
4008     {
4009         \__enumext_stop_list:
4010         \__enumext_stop_store_level:
4011         \end{multicols}
4012         \__enumext_unskip_unkern:
4013         \__enumext_unskip_unkern:
4014         \par\addvspace{ \skip_use:c { \__enumext_multicols_below_ \__enumext_level: _skip } }
4015     }
4016     {
4017         \__enumext_stop_list:
4018         \__enumext_stop_store_level:
4019     }
4020 }

```

(End of definition for __enumext_multicols_stop:.)

__enumext_before_list: The function __enumext_before_list: first calls the function __enumext_vspace_above: used by the keys `above` and `above*`, then calls the function __enumext_before_args_exec: used by the key `before*` and finally execute the function __enumext_check_ans_active: for the check answer mechanism.

```

4021 \cs_new_protected:Nn \__enumext_before_list:
4022 {
4023     \__enumext_vspace_above:
4024     \__enumext_before_args_exec:
4025     \__enumext_check_ans_active:

```

When the `mini-env` key is active it will set the value of the `\__enumext_minipage_right_X_dim` to be the *width* of the `\__enumext_minipage` environment on the “right side”, using this value together with the value of the `\__enumext_minipage_hsep_X_dim` set by the `mini-sep` key, the value of `\__enumext_minipage_left_X_dim` will be set, which will be the *width* of `\__enumext_minipage` environment on the “left side”, always having a current `\linewidth` as *maximum width* between them.

```

4026     \dim_compare:nNnT
4027     { \dim_use:c { \__enumext_minipage_right_ \__enumext_level: _dim } } > { \c_zero_dim }
4028     {
4029         \dim_set:cn { \__enumext_minipage_left_ \__enumext_level: _dim }
4030         {
4031             \linewidth
4032             - \dim_use:c { \__enumext_minipage_right_ \__enumext_level: _dim }
4033             - \dim_use:c { \__enumext_minipage_hsep_ \__enumext_level: _dim }
4034         }

```

The boolean variable `\__enumext_minipage_active_X_bool` will be activated and the integer variable `\g__enumext_minipage_stat_int` used by the `\miniright` command will be incremented, then the function `\__enumext_minipage_add_space:` is called and the `\__enumext_minipage` environment on the “left side” will be initialized followed by the “vertical spacing” applied to preserve the “baseline” between the *left* and *right* side environments. After these actions, the function `\__enumext_multicols_start:` is called to handle the `multicols` environment.

```

4035         \bool_set_true:c { \__enumext_minipage_active_ \__enumext_level: _bool }
4036         \int_gincr:N \g__enumext_minipage_stat_int
4037         \__enumext_minipage_add_space:
4038         \noindent
4039         \__enumext_mini_page{ \dim_use:c { \__enumext_minipage_left_ \__enumext_level: _dim } }
4040     }
4041     \__enumext_multicols_start:
4042 }

```

(End of definition for `\__enumext_before_list:`)

`\__enumext_second_part:` The function `\__enumext_second_part:` first check the state of the boolean variable `\l__enumext_minipage_active_X_bool`, if it is “true” a small test will be executed to check if we have omitted the use of `\miniright` (the `\__enumext_mini_page` environment has not been closed), then close `\__enumext_mini_page` and add the *adjusted vertical space* `\l__enumext_minipage_after_skip`, otherwise we will close the `\multicols` environment.

```

4043 \cs_new_protected:Nn \__enumext_second_part:
4044 {
4045   \bool_if:cTF { \l__enumext_minipage_active_ \__enumext_level: _bool }
4046   {
4047     \int_compare:nNt { \g__enumext_minipage_stat_int } = { 1 }
4048     {
4049       \msg_warning:nn { enumext } { missing-miniright }
4050       \miniright
4051     }
4052     \int_gzero:N \g__enumext_minipage_stat_int
4053     \__enumext_unskip_unkern: % remove topsep + [partopsep]
4054     \end__enumext_mini_page
4055   }
4056   {
4057     \__enumext_multicols_stop:
4058   }

```

Now we will execute the functions `\__enumext_after_stop_list:` used by the key `after`, `\__enumext_check_ans_key_hook:` used by the key `check-ans`, `\__enumext_vspace_below:` used by the keys `below` and `below*`. Finally set `\l__enumext_standar_bool` to false and call the function `\__enumext_resume_save_counter:` used by the `series`, `resume` and `resume*` keys.

```

4059   \__enumext_after_stop_list:
4060   \__enumext_check_ans_key_hook:
4061   \__enumext_vspace_below:
4062   \bool_set_false:N \l__enumext_standar_bool
4063   \bool_if:NF \l__enumext_print_keyans_cmd_bool
4064   {
4065     \__enumext_standar_save_counter:
4066   }
4067 }

```

(End of definition for `\__enumext_second_part:`)

`\__enumext_set_item_width:` The function `\__enumext_set_item_width:` will set the value of `\itemwidth` taking into account the value established by the `list-offset` key for each level of the environment.

```

4068 \cs_new_protected:Nn \__enumext_set_item_width:
4069 {
4070   \dim_set:Nn \itemwidth { \linewidth }
4071   \dim_compare:nT
4072   {
4073     \dim_use:c { \l__enumext_listoffset_ \__enumext_level: _dim } != \c_zero_dim
4074   }
4075   {
4076     \dim_sub:Nn \itemwidth
4077     {
4078       \dim_use:c { \l__enumext_listoffset_ \__enumext_level: _dim }
4079     }
4080   }
4081 }

```

(End of definition for `\__enumext_set_item_width:`)

enumext Now create the `enumext` environment based on `list` environment by levels.

```

4082 \NewDocumentEnvironment{enumext}{0}{}
4083 {
4084   \__enumext_safe_exec:
4085   \__enumext_parse_keys:n {#1}
4086   \__enumext_before_list:
4087   \__enumext_start_store_level:
4088   \__enumext_start_list:nn
4089   { \tl_use:c { \l__enumext_label_ \__enumext_level: _tl } }
4090   {
4091     \use:c { __enumext_list_arg_two_ \__enumext_level: : }
4092     \__enumext_before_keys_exec:

```

```

4093     }
4094     \__enumext_set_item_width:
4095     \__enumext_after_args_exec:
4096   }
4097   {
4098     \__enumext_second_part:
4099   }

```

(End of definition for enumext. This function is documented on page 5.)

As we don't want our check to be executed `check-ans` by levels but on the complete list, we will take it out of the `enumext` environment using the “hook” function `\__enumext_after_env:nn`.

```

4100 \__enumext_after_env:nn {enumext}
4101 {
4102   \__enumext_execute_after_env:
4103 }

```

13.43 The environment keyans

The environment `keyans` also based on lists. The main differences with the `enumext` environment are the *nesting* and the way the *answers* (choice) will be stored and checked, this environment is intended exclusively for “multiple choice questions”.

The `keyans` environment will only be available if the `save-ans` key is active and can only be used at the “first level” within the `enumext` environment. We do not want the environment to be nested, so we will set a maximum at this point. If the conditions are not met, an error message will be returned.

```

4104 \cs_new_protected:Nn \__enumext_keyans_safe_exec:
4105 {
4106   \bool_if:NF \l__enumext_store_active_bool
4107   {
4108     \msg_error:nnnn { enumext } { wrong-place } { keyans } { save-ans }
4109   }
4110   \int_incr:N \l__enumext_keyans_level_int
4111   \bool_set_true:N \l__enumext_keyans_env_bool
4112   \__enumext_keyans_name_and_start:
4113   % Set false for interfering with enumext nested in keyans (yes, its possible and crayze)
4114   \bool_set_false:N \l__enumext_store_active_bool
4115   \int_compare:nNnT { \l__enumext_keyans_level_int } > { 1 }
4116   {
4117     \msg_error:nn { enumext } { keyans-nested }
4118   }
4119   \int_compare:nNnT { \l__enumext_level_int } > { 1 }
4120   {
4121     \msg_error:nn { enumext } { keyans-wrong-level }
4122   }
4123 }

```

(End of definition for __enumext_keyans_safe_exec:.)

`\__enumext_keyans_parse_keys:n` Parse [`⟨key = val⟩`] for `keyans` environment.

```

4124 \cs_new_protected:Npn \__enumext_keyans_parse_keys:n #1
4125 {
4126   \keys_set:nn { enumext / keyans } {#1}
4127 }

```

(End of definition for __enumext_keyans_parse_keys:n.)

`\__enumext_before_list_v:` Same implementation as the one used in the `enumext` environment.

```

\__enumext_keyans_multicols_start:
\__enumext_keyans_multicols_stop:
\__enumext_second_part_v:
4128 \cs_new_protected:Nn \__enumext_before_list_v:
4129 {
4130   \__enumext_vspace_above_v:
4131   \__enumext_before_args_exec_v:
4132   \dim_compare:nNnT { \l__enumext_minipage_right_v_dim } > { \c_zero_dim }
4133   {
4134     \dim_set:Nn \l__enumext_minipage_left_v_dim
4135     {
4136       \linewidth - \l__enumext_minipage_right_v_dim - \l__enumext_minipage_hsep_v_dim
4137     }
4138     \bool_set_true:N \l__enumext_minipage_active_v_bool
4139     \int_gincr:N \g__enumext_minipage_stat_int
4140     \__enumext_keyans_minipage_add_space:
4141     \__enumext_mini_page{ \l__enumext_minipage_left_v_dim }

```

```

4142     }
4143     \__enumext_keyans_multicols_start:
4144 }
4145 \cs_new_protected:Nn \__enumext_keyans_multicols_start:
4146 {
4147     \int_compare:nNnT { \l__enumext_columns_v_int } > { 1 }
4148     {
4149         \dim_compare:nNnT { \l__enumext_columns_sep_v_dim } = { \c_zero_dim }
4150         {
4151             \dim_set:Nn \l__enumext_columns_sep_v_dim
4152             {
4153                 (
4154                     \l__enumext_labelwidth_v_dim + \l__enumext_labelsep_v_dim
4155                 ) / \l__enumext_columns_v_int
4156                 - \l__enumext_listoffset_v_dim
4157             }
4158         }
4159         \dim_set_eq:NN \columnsep \l__enumext_columns_sep_v_dim
4160         \dim_zero:N \columnseprule % no rule here
4161         \bool_if:NF \l__enumext_minipage_active_v_bool
4162         {
4163             \skip_zero:N \multicolsep
4164             \__enumext_keyans_multi_advspace:
4165         }
4166         \raggedcolumns
4167         \begin{multicols}{\l__enumext_columns_v_int}
4168     }
4169 }
4170 \cs_new_protected:Nn \__enumext_keyans_multicols_stop:
4171 {
4172     \int_compare:nNnTF { \l__enumext_columns_v_int } > { 1 }
4173     {
4174         \__enumext_stop_list:
4175         \end{multicols}
4176         \__enumext_unskip_unkern:
4177         \__enumext_unskip_unkern:
4178         \par\addvspace{ \l__enumext_multicols_below_v_skip }
4179     }
4180     {
4181         \__enumext_stop_list:
4182     }
4183 }
4184 \cs_new_protected:Nn \__enumext_second_part_v:
4185 {
4186     \bool_if:NTF \l__enumext_minipage_active_v_bool
4187     {
4188         \int_compare:nNnT { \g__enumext_minipage_stat_int } = { 1 }
4189         {
4190             \msg_warning:nn { enumext } { missing-miniright }
4191             \miniright
4192         }
4193         \int_gzero:N \g__enumext_minipage_stat_int
4194         \__enumext_unskip_unkern: % remove \topsep + [\partopsep]
4195         \end__enumext_mini_page
4196         \par\addvspace{ \l__enumext_minipage_after_skip }
4197     }
4198     {
4199         \__enumext_keyans_multicols_stop:
4200     }
4201     \bool_set_false:N \l__enumext_keyans_env_bool
4202     \__enumext_after_stop_list_v:
4203     \__enumext_vspace_below_v:
4204 }

```

(End of definition for __enumext_before_list_v: and others.)

__enumext_keyans_set_item_width:

The function __enumext_keyans_set_item_width: will set the value of \itemwidth taking into account the value established by the list-offset key.

```

4205 \cs_new_protected:Nn \__enumext_keyans_set_item_width:
4206 {
4207     \dim_set:Nn \itemwidth { \linewidth }

```

```

4208 \dim_compare:nT
4209 {
4210   \l__enumext_listoffset_v_dim != \c_zero_dim
4211 }
4212 {
4213   \dim_sub:Nn \itemwidth { \l__enumext_listoffset_v_dim }
4214 }
4215 }

```

(End of definition for `\__enumext_keyans_set_item_width:`)

keyans Now we define the environment **keyans** also based on lists.

```

4216 \NewDocumentEnvironment{keyans}{0}{}
4217 {
4218   \__enumext_keyans_safe_exec:
4219   \__enumext_keyans_parse_keys:n {#1}
4220   \__enumext_before_list_v:
4221   \__enumext_start_list:nn
4222   { \tl_use:N \l__enumext_label_v_tl }
4223   {
4224     \__enumext_list_arg_two_v:
4225     \__enumext_before_keys_exec_v:
4226   }
4227   \__enumext_keyans_set_item_width:
4228   \__enumext_after_args_exec_v:
4229 }
4230 {
4231   \__enumext_check_starred_cmd:n { item }
4232   \__enumext_second_part_v:
4233 }

```

(End of definition for `keyans`. This function is documented on page 16.)

13.44 Tagging PDF support for non-standart list environments

The $\TeX$ release 2022-06-01 brings automatic support for *tagged* PDF in several aspects, including the standard *list environments* and the `list` environment. Unfortunately non-standard *list environments* like `keyanspic` or the horizontal list environments `enumext*` and `keyans*` are not structured in a nice way, i.e. the expected result in the PDF file is the expected one, but the underlying structure is not correct. In simple terms, for *tagged* PDF a `list` environment is a `list` environment, no matter what it looks like in the PDF file.

To maintain a correct `list` structure when `\DocumentMetadata` is active, it is necessary to do some things manually using `tagpdf`[18] and `ltsockets`[20]. This implementation is an adaptation of my answer thanks to Ulrike Fischer's comments in [How can I modify my `\item` redefinition to be compatible with tagging-pdf.](#)

13.44.1 Socket for tagging support in `enumext*` and `keyans*`

We will first define the necessary sockets and their behavior for `enumext*` and `keyans*`.

```

start-list-tags
stop-start-tags
stop-list-tags
__enumext_start_list_tag:n
__enumext_stop_start_list_tag:
__enumext_stop_list_tag:n
4234 \socket_new:nn {tagsupport/__enumext/starred}{1}
4235 \socket_new_plug:nnn {tagsupport/__enumext/starred} {start-list-tags}
4236 {
4237   \tag_resume:n {#1}
4238   \tag_struct_begin:n {tag=LI}
4239   \tag_struct_begin:n {tag=Lbl}
4240   \tag_mc_begin:n {tag=Lbl}
4241 }
4242 \socket_new_plug:nnn {tagsupport/__enumext/starred} {stop-start-tags}
4243 {
4244   \tag_mc_end:
4245   \tag_struct_end:n {tag=Lbl}
4246   \tag_struct_begin:n {tag=LBody}
4247   \tag_struct_begin:n {tag=text-unit}
4248   \tag_struct_begin:n {tag=text}
4249 }
4250 \socket_new_plug:nnn {tagsupport/__enumext/starred} {stop-list-tags}
4251 {
4252   \tag_struct_end:n {tag=text}
4253   \tag_struct_end:n {tag=text-unit}
4254   \tag_struct_end:n {tag=LBody}
4255   \tag_struct_end:n {tag=LI}
4256   \tag_suspend:n {#1}
4257 }

```


And now we'll wrap them so that they're only active when `\DocumentMetadata` is present.

```

4258 \cs_new_protected_nopar:Npn \__enumext_start_list_tag:n #1
4259 {
4260   \IfDocumentMetadataT
4261   {
4262     \socket_assign_plug:nn {tagsupport/__enumext/starred} {start-list-tags}
4263     \socket_use:nn {tagsupport/__enumext/starred} {#1}
4264   }
4265 }
4266 \cs_new_protected_nopar:Nn \__enumext_stop_start_list_tag:
4267 {
4268   \IfDocumentMetadataT
4269   {
4270     \socket_assign_plug:nn {tagsupport/__enumext/starred} {stop-start-tags}
4271     \socket_use:nn {tagsupport/__enumext/starred} { }
4272   }
4273 }
4274 \cs_new_protected_nopar:Npn \__enumext_stop_list_tag:n #1
4275 {
4276   \IfDocumentMetadataT
4277   {
4278     \socket_assign_plug:nn {tagsupport/__enumext/starred} {stop-list-tags}
4279     \socket_use:nn {tagsupport/__enumext/starred} {#1}
4280   }
4281 }

```

(End of definition for start-list-tags and others.)

13.44.2 Socket for tagging support in keyanspic

We will first define the necessary sockets and their behavior for `keyanspic` environment.

```

start-list-tags
stop-start-tags
stop-list-tags
\__enumext_anspic_start_list_tag:
\__enumext_anspic_stop_start_list_tag:
\__enumext_anspic_stop_list_tag:
4282 \socket_new:nn {tagsupport/__enumext/keyanspic}{ 0 }
4283 \socket_new_plug:nnn {tagsupport/__enumext/keyanspic} {start-list-tags}
4284 {
4285   \tag_resume:n {keyanspic}
4286   \tag_struct_begin:n {tag=LI}
4287   \tag_struct_begin:n {tag=Lbl}
4288   \tag_mc_begin:n {tag=Lbl}
4289 }
4290 \socket_new_plug:nnn {tagsupport/__enumext/keyanspic} {stop-start-tags}
4291 {
4292   \tag_mc_end:
4293   \tag_struct_end:n {tag=Lbl}
4294   \tag_struct_begin:n {tag=LBody}
4295   \tag_struct_begin:n {tag=text-unit}
4296   \tag_struct_begin:n {tag=text}
4297   \tag_mc_begin:n {tag=text}
4298 }
4299 \socket_new_plug:nnn {tagsupport/__enumext/keyanspic} {stop-list-tags}
4300 {
4301   \tag_mc_end:
4302   \tag_struct_end:n {tag=text}
4303   \tag_struct_end:n {tag=text-unit}
4304   \tag_struct_end:n {tag=LBody}
4305   \tag_struct_end:n {tag=LI}
4306   \tag_suspend:n {keyanspic}
4307 }

```

And now we'll wrap them so that they're only active when `\DocumentMetadata` is present.

```

4308 \cs_new_protected_nopar:Nn \__enumext_anspic_start_list_tag:
4309 {
4310   \IfDocumentMetadataT
4311   {
4312     \socket_assign_plug:nn {tagsupport/__enumext/keyanspic} {start-list-tags}
4313     \socket_use:n {tagsupport/__enumext/keyanspic}
4314   }
4315 }
4316 \cs_new_protected_nopar:Nn \__enumext_anspic_stop_start_list_tag:
4317 {
4318   \IfDocumentMetadataT
4319   {
4320     \socket_assign_plug:nn {tagsupport/__enumext/keyanspic} {stop-start-tags}

```

```

4321     \socket_use:n {tagsupport/__enumext/keyanspic}
4322   }
4323 }
4324 \cs_new_protected_nopar:Nn \__enumext_anspic_stop_list_tag:
4325 {
4326   \IfDocumentMetadataT
4327   {
4328     \socket_assign_plug:nn {tagsupport/__enumext/keyanspic} {stop-list-tags}
4329     \socket_use:n {tagsupport/__enumext/keyanspic}
4330   }
4331 }

```

(End of definition for `start-list-tags` and others.)

13.45 The environment `keyanspic` and `\anspic`

The `keyanspic` environment is a `list` based environment that uses the same configuration for “*spacing*” and `<label>` as the `keyans` environment, but it does not use `\item`. The `<contents>` are passed to the environment by means of the `\anspic` command as replacement for `\item` command and placed inside `minipage` environments, with the `<label>` centered “*above*” or “*below*”, adjusting *widths* and *position* according to the options passed to the environment.

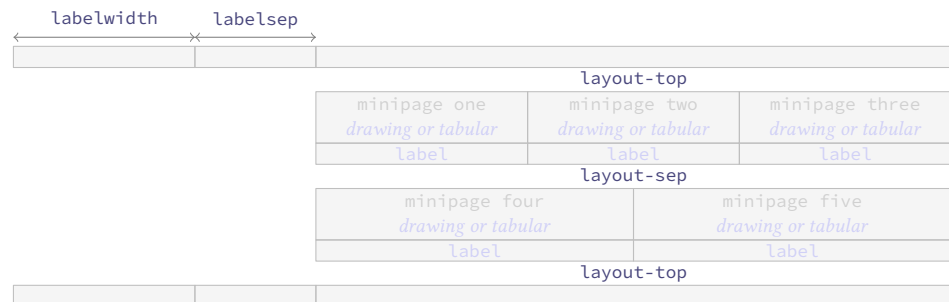


Figure 12: Representation of the `keyanspic` spacing in `enumext`.

In order for the `keyanspic` environment and the `\anspic` command to work correctly, we need to set and export some variables in the first part of the environment definition and pass them to `\anspic` which is executed in the second part of the environment. This implementation is adapted from the answer given by Enrico Gregorio (@egreg) in [How to process the body of an environment and divide it by a \macro?](#).

13.45.1 The environment `keyanspic`

First we define the keys that allows us to process the position of the `<label>` centered “*above*” or “*below*” which will be `label-pos`, the vertical separation of these from `drawing or tabular` will be handled with the key `label-sep`. The “*layout style*” will be handled with the key `layout-sty` will take two values separated by comma `{<n° upper, n° lower>}` and will determine the number of `minipage` environments in which all arguments of `\anspic` will be printed at the “*upper*” and “*lower*” within the environments separated by the value of the key `layout-sep`. The vertical space “*top*” and “*bottom*” of the environment will be handled with the key `layout-top`.

```

mark-ans 4332 \keys_define:nn { enumext / keyanspic }
mark-pos 4333 {
mark-sep 4334   label-pos .choice:,
save-sep 4335   label-pos / above .code:n =
wrap-opt 4336     \bool_set_true:N \l__enumext_anspic_label_above_bool
wrap-ans* 4337     \str_set:Nn \l__enumext_anspic_mini_pos_str { t },
show-ans 4338   label-pos / below .code:n =
show-pos 4339     \bool_set_false:N \l__enumext_anspic_label_above_bool
4340     \str_set:Nn \l__enumext_anspic_mini_pos_str { b },
4341   label-pos / unknown .code:n =
4342     \msg_error:nnee { enumext } { unknown-choice }
4343     { label-pos } { above,~ below } { \exp_not:n {#1} },
4344   label-pos .initial:n = below,
4345   label-pos .value_required:n = true,
4346   label-sep .skip_set:N = \l__enumext_anspic_label_sep_skip,
4347   label-sep .value_required:n = true,
4348   layout-sty .tl_set:N = \l__enumext_anspic_layout_style_tl,
4349   layout-sty .value_required:n = true,
4350   layout-sep .code:n = \keys_set:nn { enumext / keyans } { parsep = #1 },
4351   layout-sep .value_required:n = true,
4352   layout-top .code:n = \keys_set:nn { enumext / keyans } { topsep = #1 },
4353   layout-top .value_required:n = true,
4354   mark-ans .code:n = \keys_set:nn { enumext / keyans } { mark-ans = #1 },
4355   mark-ans .value_required:n = true,

```

```

4356 mark-pos .code:n = \keys_set:nn { enumext / keyans } { mark-pos = #1 },
4357 mark-pos .value_required:n = true,
4358 mark-sep .code:n = \keys_set:nn { enumext / keyans } { mark-sep = #1 },
4359 mark-sep .value_required:n = true,
4360 save-sep .code:n = \keys_set:nn { enumext / keyans } { save-sep = #1 },
4361 save-sep .value_required:n = true,
4362 wrap-opt .code:n = \keys_set:nn { enumext / keyans } { wrap-opt = #1 },
4363 wrap-opt .value_required:n = true,
4364 wrap-ans* .code:n = \keys_set:nn { enumext / keyans } { wrap-ans* = #1 },
4365 wrap-ans* .value_required:n = true,
4366 show-ans .code:n = \keys_set:nn { enumext / keyans } { show-ans = #1 },
4367 show-ans .value_required:n = true,
4368 show-pos .code:n = \keys_set:nn { enumext / keyans } { show-pos = #1 },
4369 show-pos .value_required:n = true,
4370 unknown .code:n = {
4371     \tl_set:Nn \l__enumext_envir_name_tl { keyanspic }
4372     \__enumext_keyans_unknown_keys:n {#1}
4373 },
4374 }

```

(End of definition for label-pos and others.)

```

\__enumext_keyans_pic_safe_exec:
\__enumext_keyans_pic_parse_keys:n
\__enumext_keyans_pic_skip_abs:N
\__enumext_keyans_pic_arg_two:

```

The function `\__enumext_keyans_pic_safe_exec:` check the nested level position inside the `enumext` environment.

```

4375 \cs_new_protected:Nn \__enumext_keyans_pic_safe_exec:
4376 {
4377     \int_incr:N \l__enumext_keyans_pic_level_int
4378     \int_compare:nNnT { \l__enumext_keyans_pic_level_int } > { 1 }
4379     {
4380         \msg_error:nn { enumext } { keyanspic-nested }
4381     }
4382     \__enumext_keyans_name_and_start:
4383 }

```

Parse [`key = val`] for `keyanspic` environment.

```

4384 \cs_new_protected:Npn \__enumext_keyans_pic_parse_keys:n #1
4385 {
4386     \tl_if_novalue:nF {#1}
4387     {
4388         \keys_set:nn { enumext / keyanspic } {#1}
4389     }
4390 }

```

The function `\__enumext_keyans_pic_skip_abs:N` will return a positive value `\parsep` from `keyans` environment.

```

4391 \cs_new_protected:Npn \__enumext_keyans_pic_skip_abs:N #1
4392 {
4393     \dim_compare:nNnT { #1 } < { \c_zero_dim }
4394     {
4395         \skip_set:Nn #1 { -#1 }
4396     }
4397 }

```

The `\__enumext_keyans_pic_arg_two:` function will be used in the *second argument* of the `list` environment that defines the `keyanspic` environment, with this we will take the configuration of the “spaces” and the keys `label`, `wrap-label`, `parsep` and `topsep` from the `keyans` environment. The first thing we need to do is set the boolean variable `\l__enumext_leftmargin_tmp_v_bool` handled by the `list-indent` key to “false”, then copy the definition of the second list argument from the `keyans` environment definition and make sure that `\parsep` does not have a negative value.

```

4398 \cs_new_protected:Npn \__enumext_keyans_pic_arg_two:
4399 {
4400     \bool_set_false:N \l__enumext_leftmargin_tmp_v_bool
4401     \__enumext_list_arg_two_v:
4402     \__enumext_keyans_pic_skip_abs:N \parsep

```

Now we increment the counter `enumXv` of the `keyans` environment and save the *total height* of the (*label*) in `\l__enumext_anspic_label_htdp_dim` used by `\anspic` and we will adjust the values of `\parsep` only if the key `label-pos` is set to *below*.

```

4403     \bool_if:NF \l__enumext_anspic_label_above_bool
4404     {
4405         \stepcounter { enumXv }
4406         \hbox_set:Nn \l__enumext_anspic_label_box { \l__enumext_label_v_tl }

```

```

4407     \dim_set:Nn \l__enumext_anspic_label_htdp_dim
4408     {
4409         \box_ht_plus_dp:N \l__enumext_anspic_label_box
4410     }
4411     \skip_add:Nn \parsep
4412     {
4413         \l__enumext_anspic_label_htdp_dim
4414         + \box_dp:N \strutbox
4415         + \l__enumext_anspic_label_sep_skip
4416     }
4417 }

```

Finally we *adjust* the value of `\leftmargin` and `\topsep` then set `\listparindent`, `\partopsep` and `\itemsep` to zero so that the *horizontal* and *vertical* space is not affected.

```

4418     \dim_add:Nn \leftmargin { -\l__enumext_labelwidth_v_dim - \l__enumext_labelsep_v_dim }
4419     \ignorespaces
4420     \skip_add:Nn \topsep { 0.5\box_dp:N \strutbox }
4421     \dim_zero:N \listparindent
4422     \skip_zero:N \partopsep
4423     \skip_zero:N \itemsep
4424 }

```

(End of definition for `\__enumext_keyans_pic_safe_exec`: and others.)

keyanspic Now we define the environment `keyanspic`. For compatibility with *tagged* PDF we must use the `\begin{list}` form and a lot of conditional code using `\IfDocumentMetadataTF`. We will first stop the code for automatic *tagged* PDF for `list` environments, redefine `\item` so that it cannot be used, and stop the code for automatic *tagged* PDF for the `keyanspic` environment.

```

4425 \NewDocumentEnvironment{keyanspic}{o }
4426 {
4427     \__enumext_keyans_pic_safe_exec:
4428     \__enumext_keyans_pic_parse_keys:n {#1}
4429     \begin{list} { } { \__enumext_keyans_pic_arg_two: }
4430     \IfDocumentMetadataT
4431     {
4432         \tag_suspend:n {list}
4433     }
4434     \item[] \scan_stop:
4435     \RenewDocumentCommand \item {}
4436     {
4437         \msg_error:nn { enumext } { keyanspic-item-cmd }
4438     }
4439     \IfDocumentMetadataT
4440     {
4441         \tag_resume:n {keyanspic}
4442         \tag_tool:n {para/tagging=false} % no socket for this now
4443         \tag_suspend:n {keyanspic}
4444     }
4445 }

```

In the second part of the environment definition we will manually place our code for *tagged* PDF and execute the command `\anspic` using the `\__enumext_anspic_exec`: function.

```

4446 {
4447     \IfDocumentMetadataT
4448     {
4449         \tag_resume:n {keyanspic}
4450     }
4451     \__enumext_anspic_exec:
4452     \IfDocumentMetadataT
4453     {
4454         \tag_suspend:n {keyanspic}
4455     }
4456     \end{list}
4457     \IfDocumentMetadataT
4458     {
4459         \tag_struct_end:n {tag=L}
4460     }

```

Finally we check if `\anspic*` has been used, set the counter `enumXvi` to zero and apply our “adjusted” vertical space bottom.

```

4461     \__enumext_check_starred_cmd:n { anspic }
4462     \setcounter { enumXvi } { 0 }

```

```

4463 \bool_if:NTF \l__enumext_anspic_label_above_bool
4464 {
4465   \par\addvspace{ 0.5\box_dp:N \strutbox }
4466 }
4467 {
4468   \par
4469   \addvspace
4470   {
4471     \dim_eval:n
4472     {
4473       \l__enumext_anspic_label_htdp_dim + \box_ht_plus_dp:N \strutbox
4474       + \l__enumext_anspic_label_sep_skip + \l__enumext_topsep_v_skip
4475     }
4476   }
4477 }
4478 }

```

(End of definition for `keyanspic`. This function is documented on page 17.)

13.45.2 The command `\anspic`

The `\anspic` command take three arguments, the *starred versions* `\anspic*[\langle content \rangle]` store the current `\label` next to the *optional argument* `[\langle content \rangle]` in the *sequence* and *prop list* defined by `save-ans` key. The third *mandatory argument* `{\langle drawing or tabular \rangle}` is NOT stored in the *sequence* or *prop list*.

- One of the complications here to make the `keyanspic` environment compatible with *tagged* PDF is the position of `\label`, the `\anspic` command processes the arguments in order, where #1 and #2 correspond to `\label` and #3 to the mandatory argument and puts all this inside a `minipage` environment. If #1 and #2, that is `\label`, is above #3 there are no problems with *tagged* PDF, but if #3 comes first the list created with *tagged* PDF will not be correct.

`\anspic`

We check that the command is active in the `keyanspic` environment only if the `save-ans` key is present, otherwise we return an error. The three arguments are handled by the function `\__enumext_anspic_args:nnn` and stored in the sequence `\l__enumext_anspic_args_seq` which is processed by the `keyanspic` environment.

```

4479 \NewDocumentCommand \anspic { s o +m }
4480 {
4481   \bool_if:NF \l__enumext_store_active_bool
4482   {
4483     \msg_error:nnnn { enumext } { wrong-place }{ keyanspic }{ save-ans }
4484   }
4485   \int_compare:nNnT { \l__enumext_level_int } > { 1 }
4486   {
4487     \msg_error:nn { enumext } { keyanspic-wrong-level }
4488   }
4489   \int_compare:nNnT { \l__enumext_keyans_level_int } = { 1 }
4490   {
4491     \msg_error:nnnn { enumext } { command-wrong-place }{ anspic }{ keyans }
4492   }
4493   \seq_put_right:Nn \l__enumext_anspic_args_seq
4494   {
4495     \__enumext_anspic_args:nnn { #1 } { #2 } { #3 }
4496   }
4497 }

```

The `\__enumext_anspic_body_dim:n` function will set the value of `\l__enumext_anspic_body_htdp_dim` equal to the “height plus depth” of the *mandatory argument* if the key `label-pos` is set “below”.

```

4498 \cs_new_protected:Npn \__enumext_anspic_body_dim:n #1
4499 {
4500   \bool_if:NF \l__enumext_anspic_label_above_bool
4501   {
4502     \IfDocumentMetadataT
4503     {
4504       \tag_suspend:n {keyanspic}
4505     }
4506     \vbox_set:Nn \l__enumext_anspic_body_box { #1 }
4507     \dim_set:Nn \l__enumext_anspic_body_htdp_dim
4508     {
4509       \box_ht_plus_dp:N \l__enumext_anspic_body_box
4510     }
4511     \IfDocumentMetadataT
4512     {
4513       \tag_resume:n {keyanspic}
4514     }

```

```

4515     }
4516 }

```

The `\__enumext_anspic_label:nn` function will process inside `\makebox` the *starred argument* ‘*’ and *optional argument* passed to the command. Here we will store the `<label>` and *optional argument* in *prop list* and *sequence* and execute the `show-ans`, `show-pos`, `font`, `wrap-label`, `wrap-ans*` and `wrap-opt` keys.

```

4517 \cs_new_protected:Npn \__enumext_anspic_label:nn #1 #2
4518 {
4519   \makebox[ \l__enumext_anspic_mini_width_dim ][ c ]
4520   {
4521     \bool_if:NTF { #1 }
4522     {
4523       \bool_set_true:N \l__enumext_item_wrap_key_bool
4524       \bool_set_true:N \l__enumext_wrap_label_v_bool
4525       \__enumext_keyans_save_item_opt:n { #2 }
4526       \__enumext_keyans_addto_prop:n { #2 }
4527       \__enumext_keyans_store_ref:
4528       \__enumext_keyans_addto_seq:n { #2 }
4529       \int_gincr:N \g__enumext_check_starred_cmd_int
4530       \__enumext_keyans_show_ans:
4531       \__enumext_keyans_show_pos:
4532       \makebox[ \l__enumext_labelwidth_v_dim ][ c ]
4533       {
4534         \tl_use:N \l__enumext_label_font_style_v_tl
4535         \__enumext_keyans_wrapper_label:n { \l__enumext_label_vi_tl }
4536       }
4537       \skip_horizontal:n { \l__enumext_labelsep_v_dim }
4538       \__enumext_keyans_show_item_opt:
4539     }
4540     {
4541       \bool_set_false:N \l__enumext_item_wrap_key_bool
4542       \tl_use:N \l__enumext_label_font_style_v_tl
4543       \__enumext_wrapper_label_v:n { \l__enumext_label_vi_tl }
4544     }
4545   }
4546 }

```

The function `\__enumext_anspic_label_pos:nnn` will be in charge of handling the “*counter*” and the position of the `<label>`, set by `label-pos` key which will have the same configuration as the `keyans` environment.

```

4547 \cs_new_protected:Npn \__enumext_anspic_label_pos:nnn #1 #2 #3
4548 {
4549   \stepcounter { enumXvi }
4550   \__enumext_anspic_body_dim:n { #3 }
4551   \bool_if:NTF \l__enumext_anspic_label_above_bool
4552   {
4553     \__enumext_anspic_label:nn { #1 } { #2 }
4554   }
4555   {
4556     \raisebox
4557     {
4558       -\dim_eval:n
4559       {
4560         \l__enumext_anspic_label_htdp_dim
4561         + \l__enumext_anspic_body_htdp_dim
4562         + \box_dp:N \strutbox
4563         + \l__enumext_anspic_label_sep_skip
4564       }
4565     }
4566     [ opt ] [ opt ]
4567     {
4568       \__enumext_anspic_label:nn { #1 } { #2 }
4569     }
4570   }
4571 }
4572 %

```

The `\__enumext_anspic_args:nnn` function will be responsible for placing the code compatible with *tagged* PDF and the arguments within the `\l__enumext_anspic_args_seq` sequence which will be processed by the `\__enumext_anspic_print:n` function in the second part of the definition of the `keyanspic` environment.

```

4573 \cs_new_protected:Nn \__enumext_anspic_args:nnn
4574 {
4575   \__enumext_anspic_start_list_tag:

```

```

4576   \__enumext_anspic_label_pos:nnn { #1 } { #2 } { #3 }
4577   \__enumext_anspic_stop_start_list_tag:
4578   \bool_if:NTF \l__enumext_anspic_label_above_bool
4579   {
4580     \[\l__enumext_anspic_label_sep_skip] #3
4581   }
4582   {
4583     \[ #3
4584   }
4585   \__enumext_anspic_stop_list_tag:
4586 }

```

The value $\langle n^{\circ upper}, n^{\circ lower} \rangle$ passed to the `layout-sty` key is split by comma and is handled directly by the function `\__enumext_anspic_print:n` and passed to the function `\__enumext_anspic_row:n`.

```

4587 \cs_new_protected:Nn \__enumext_anspic_print:n
4588 {
4589   \clist_map_function:nN { #1 } \__enumext_anspic_row:n
4590 }
4591 \cs_generate_variant:Nn \__enumext_anspic_print:n { e, V }

```

The function `\__enumext_anspic_row:n` will set the *widths* for the `minipage` environments and place *all arguments* passed to `\anspic` saved in the `\l__enumext_anspic_args_seq` sequence inside them.

```

4592 \cs_new_protected:Nn \__enumext_anspic_row:n
4593 {
4594   \dim_set:Nn \l__enumext_anspic_mini_width_dim { \linewidth / #1 }
4595   \int_set:Nn \l__enumext_anspic_above_int { \l__enumext_anspic_below_int }
4596   \int_set:Nn \l__enumext_anspic_below_int { \l__enumext_anspic_above_int + #1 }
4597   \int_step_inline:nnn
4598   { \l__enumext_anspic_above_int + 1 }
4599   { \l__enumext_anspic_below_int }
4600   {
4601     \IfDocumentMetadataT
4602     {
4603       \tag_suspend:n {minipage}
4604     }
4605     \begin{minipage}[ \l__enumext_anspic_mini_pos_str ]{ \l__enumext_anspic_mini_width_dim }
4606       \centering
4607       \seq_item:Nn \l__enumext_anspic_args_seq { ##1 }
4608     \end{minipage}
4609     \IfDocumentMetadataT
4610     {
4611       \tag_resume:n {minipage}
4612     }
4613   }
4614   \par
4615 }

```

The `\__enumext_anspic_exec:` function will execute all the code in the `\anspic` command in the second argument of the `keyanspic` environment definition. If the key `layout-sty` is not set, everything will be printed on a *single line*.

```

4616 \cs_new_protected:Nn \__enumext_anspic_exec:
4617 {
4618   \tl_if_empty:NTF \l__enumext_anspic_layout_style_tl
4619   {
4620     \__enumext_anspic_print:e { \seq_count:N \l__enumext_anspic_args_seq }
4621   }
4622   {
4623     \__enumext_anspic_print:V \l__enumext_anspic_layout_style_tl
4624   }
4625 }

```

(End of definition for `\anspic` and others. This function is documented on page 18.)

13.46 The horizontal environments

Generating *horizontal list environments* is NOT as simple as standard $\text{\LaTeX}$ list environments. The fundamental part of the code is adapted from the `shortlst` package to a more modern version using `expl3`. It is not possible to redefine `\item` and `\makeLabel` using `\RenewDocumentCommand` as in the vertical *non starred* versions.

To achieve the *horizontal list environments* we will capture the `\item` command and the $\langle content \rangle$ of this in *horizontal box* using `\makebox` for the `label` and a `minipage` environment for the $\langle content \rangle$ passed to `\item`, we will also add the *optional argument* $\langle number \rangle$ to `\item` to be able to *join columns* horizontally, in simple

terms, we want `\item` to behave in the same way as in the `enumext` environment but adding an *first optional argument* (`(number)`).

A side effect is the limitation of using `\item` in this way *without* using `\RenewDocumentCommand`, which loses the original definition and affects the *standard list environments* provided by $\TeX$ and any environment defined using base `list` environment, including: `itemize`, `enumerate`, `description`, `quote`, `quotation`, `verse`, `center`, `flushleft`, `flushright`, `verbatim`, `tabbing`, `trivlist`, `list` and all environments created with `\newtheorem`.

- One way to get around this is to use something like:

```
\AddToHook{env/enumerate/before}{recover original \item definition}
```

inside `minipage`, but in my partial tests this does not have the desired effect and the vertical and horizontal spacing is distorted. For now this will remain as a limitation and I will see if it is feasible to implement it in the future.

- For compatibility with the *tagged* PDF we close the environments according to the presence or not of the `mini-env` key.

13.46.1 Functions for item box width

We set the default value for the *width of the box* containing the *content* of the items for `enumext*` environment.

```

__enumext_starred_columns_set_vii:
__enumext_starred_columns_set_viii:
4626 \cs_new_protected:Nn __enumext_starred_columns_set_vii:
4627 {
4628   \dim_compare:nNnT { \__enumext_columns_sep_vii_dim } = { \c_zero_dim }
4629   {
4630     \dim_set:Nn \__enumext_columns_sep_vii_dim
4631     {
4632       ( \__enumext_labelwidth_vii_dim + \__enumext_labelsep_vii_dim )
4633       / \__enumext_columns_vii_int
4634     }
4635   }
4636   \int_set:Nn \__enumext_tmpa_vii_int { \__enumext_columns_vii_int - 1 }
4637   \dim_set:Nn \__enumext_item_width_vii_dim
4638   {
4639     ( \linewidth - \__enumext_columns_sep_vii_dim * \__enumext_tmpa_vii_int )
4640     / \__enumext_columns_vii_int
4641     - \__enumext_labelwidth_vii_dim
4642     - \__enumext_labelsep_vii_dim
4643   }

```

When the key `rightmargin` is active we must adjust the values.

```

4644 \dim_compare:nNnT { \__enumext_rightmargin_vii_dim } > { \c_zero_dim }
4645 {
4646   \dim_sub:Nn \__enumext_item_width_vii_dim
4647   {
4648     ( \__enumext_rightmargin_vii_dim * \__enumext_tmpa_vii_int )
4649     / \__enumext_columns_vii_int
4650   }
4651   \dim_add:Nn \__enumext_columns_sep_vii_dim
4652   {
4653     \__enumext_rightmargin_vii_dim
4654   }
4655 }
4656 }

```

Same implementation for the `keyans*` environment.

```

4657 \cs_new_protected:Nn __enumext_starred_columns_set_viii:
4658 {
4659   \dim_compare:nNnT { \__enumext_columns_sep_viii_dim } = { \c_zero_dim }
4660   {
4661     \dim_set:Nn \__enumext_columns_sep_viii_dim
4662     {
4663       ( \__enumext_labelwidth_viii_dim + \__enumext_labelsep_viii_dim )
4664       / \__enumext_columns_viii_int
4665     }
4666   }
4667   \int_set:Nn \__enumext_tmpa_viii_int { \__enumext_columns_viii_int - 1 }
4668   \dim_set:Nn \__enumext_item_width_viii_dim
4669   {
4670     ( \linewidth - \__enumext_columns_sep_viii_dim * \__enumext_tmpa_viii_int )
4671     / \__enumext_columns_viii_int
4672     - \__enumext_labelwidth_viii_dim
4673     - \__enumext_labelsep_viii_dim
4674   }
4675   \dim_compare:nNnT { \__enumext_rightmargin_viii_dim } > { \c_zero_dim }

```

```

4676     {
4677         \dim_sub:Nn \l__enumext_item_width_viii_dim
4678         {
4679             ( \l__enumext_rightmargin_viii_dim * \l__enumext_tmpa_vii_int )
4680             / \l__enumext_columns_viii_int
4681         }
4682         \dim_add:Nn \l__enumext_columns_sep_viii_dim
4683         {
4684             \l__enumext_rightmargin_viii_dim
4685         }
4686     }
4687 }

```

(End of definition for `\__enumext_starred_columns_set_vii:` and `\__enumext_starred_columns_set_viii:`.)

13.46.2 Functions for join item columns

```

\__enumext_starred_joined_item_vii:n
\__enumext_starred_joined_item_viii:n

```

The functions `\__enumext_starred_joined_item_vii:n` and `\__enumext_starred_joined_item_viii:n` will set the *width* of the box in which the *content* passed to `\item(<columns>)` will be stored together with the value of `\itemwidth` for the `enumext*` environment.

```

4688 \cs_new_protected:Npn \__enumext_starred_joined_item_vii:n #1
4689 {
4690     \int_set:Nn \l__enumext_joined_item_vii_int {#1}
4691     \int_compare:nNnT { \l__enumext_joined_item_vii_int } > { \l__enumext_columns_vii_int }
4692     {
4693         \msg_warning:nnee { enumext } { item-joined }
4694         { \int_use:N \l__enumext_joined_item_vii_int }
4695         { \int_use:N \l__enumext_columns_vii_int }
4696         \int_set:Nn \l__enumext_joined_item_vii_int
4697         {
4698             \l__enumext_columns_vii_int - \l__enumext_item_column_pos_vii_int + 1
4699         }
4700     }
4701     \int_compare:nNnT
4702     { \l__enumext_joined_item_vii_int }
4703     >
4704     { \l__enumext_columns_vii_int - \l__enumext_item_column_pos_vii_int + 1 }
4705     {
4706         \msg_warning:nnee { enumext } { item-joined-columns }
4707         { \int_use:N \l__enumext_joined_item_vii_int }
4708         {
4709             \int_eval:n
4710             { \l__enumext_columns_vii_int - \l__enumext_item_column_pos_vii_int + 1 }
4711         }
4712         \int_set:Nn \l__enumext_joined_item_vii_int
4713         {
4714             \l__enumext_columns_vii_int - \l__enumext_item_column_pos_vii_int + 1
4715         }
4716     }
4717     \int_compare:nNnTF { \l__enumext_joined_item_vii_int } > { 1 }
4718     {
4719         \int_set_eq:NN \l__enumext_joined_item_aux_vii_int \l__enumext_joined_item_vii_int
4720         \int_decr:N \l__enumext_joined_item_aux_vii_int
4721         \int_add:Nn \l__enumext_item_column_pos_vii_int { \l__enumext_joined_item_aux_vii_int }
4722         \int_gadd:Nn \g__enumext_item_count_all_vii_int { \l__enumext_joined_item_aux_vii_int }
4723         \dim_set:Nn \l__enumext_joined_width_vii_dim
4724         {
4725             \l__enumext_item_width_vii_dim * \l__enumext_joined_item_vii_int
4726             + ( \l__enumext_labelwidth_vii_dim + \l__enumext_labelsep_vii_dim
4727               + \l__enumext_columns_sep_vii_dim
4728               ) * \l__enumext_joined_item_aux_vii_int
4729         }
4730         \dim_set_eq:NN \itemwidth \l__enumext_joined_width_vii_dim
4731     }
4732     {
4733         \dim_set_eq:NN \l__enumext_joined_width_vii_dim \l__enumext_item_width_vii_dim
4734         \dim_set_eq:NN \itemwidth \l__enumext_item_width_vii_dim
4735     }
4736 }

```

Same implementation for the `keyans*` environment.

```

4737 \cs_new_protected:Npn \__enumext_starred_joined_item_viii:n #1

```

```

4738 {
4739   \int_set:Nn \l__enumext_joined_item_viii_int {#1}
4740   \int_compare:nNnT { \l__enumext_joined_item_viii_int } > { \l__enumext_columns_viii_int }
4741   {
4742     \msg_warning:nnee { enumext } { item-joined }
4743     { \int_use:N \l__enumext_joined_item_viii_int }
4744     { \int_use:N \l__enumext_columns_viii_int }
4745     \int_set:Nn \l__enumext_joined_item_viii_int
4746     {
4747       \l__enumext_columns_viii_int - \l__enumext_item_column_pos_viii_int + 1
4748     }
4749   }
4750   \int_compare:nNnT
4751   { \l__enumext_joined_item_viii_int }
4752   >
4753   { \l__enumext_columns_viii_int - \l__enumext_item_column_pos_viii_int + 1 }
4754   {
4755     \msg_warning:nnee { enumext } { item-joined-columns }
4756     { \int_use:N \l__enumext_joined_item_viii_int }
4757     {
4758       \int_eval:n
4759       { \l__enumext_columns_viii_int - \l__enumext_item_column_pos_viii_int + 1 }
4760     }
4761     \int_set:Nn \l__enumext_joined_item_viii_int
4762     {
4763       \l__enumext_columns_viii_int - \l__enumext_item_column_pos_viii_int + 1
4764     }
4765   }
4766   \int_compare:nNnTF { \l__enumext_joined_item_viii_int } > { 1 }
4767   {
4768     \int_set_eq:NN \l__enumext_joined_item_aux_viii_int \l__enumext_joined_item_viii_int
4769     \int_decr:N \l__enumext_joined_item_aux_viii_int
4770     \int_add:Nn \l__enumext_item_column_pos_viii_int { \l__enumext_joined_item_aux_viii_int }
4771     \int_gadd:Nn \g__enumext_item_count_all_viii_int { \l__enumext_joined_item_aux_viii_int }
4772     \dim_set:Nn \l__enumext_joined_width_viii_dim
4773     {
4774       \l__enumext_item_width_viii_dim * \l__enumext_joined_item_viii_int
4775       + ( \l__enumext_labelwidth_viii_dim + \l__enumext_labelsep_viii_dim
4776         + \l__enumext_columns_sep_viii_dim
4777       ) * \l__enumext_joined_item_aux_viii_int
4778     }
4779     \dim_set_eq:NN \itemwidth \l__enumext_joined_width_viii_dim
4780   }
4781   {
4782     \dim_set_eq:NN \l__enumext_joined_width_viii_dim \l__enumext_item_width_viii_dim
4783     \dim_set_eq:NN \itemwidth \l__enumext_item_width_viii_dim
4784   }
4785 }

```

(End of definition for `\__enumext_starred_joined_item_vii:n` and `\__enumext_starred_joined_item_viii:n`)

13.46.3 Functions for mini-env, mini-right and mini-right* keys

`\__enumext_start_mini_vii:` The implementation of the `mini-env` key support is almost identical to the one used in the `enumext` and `keyans` environments, the difference is that the `\__enumext_mini_page` environment on the “right side” is executed “after” closing the environment, so it is necessary to make a global copy of the variable `\l__enumext_minipage_right_vii_dim` in the variable `\g__enumext_minipage_right_vii_dim`.

```

4786 \cs_new_protected:Nn \__enumext_start_mini_vii:
4787 {
4788   \dim_compare:nNnT { \l__enumext_minipage_right_vii_dim } > { \c_zero_dim }
4789   {
4790     \dim_set:Nn \l__enumext_minipage_left_vii_dim
4791     {
4792       \linewidth
4793       - \l__enumext_minipage_right_vii_dim
4794       - \l__enumext_minipage_hsep_vii_dim
4795     }
4796     \bool_set_true:N \l__enumext_minipage_active_vii_bool
4797     \dim_gset_eq:NN
4798     \g__enumext_minipage_right_vii_dim
4799     \l__enumext_minipage_right_vii_dim
4800     \__enumext_mini_addvspace_vii:

```

```

4801         \nointerlineskip\noindent
4802         \__enumext_mini_page{ \l__enumext_minipage_left_vii_dim }
4803     }
4804 }

```

The function `\__enumext_stop_mini_vii:` closes the `\__enumext_mini_page` environment on the “*left side*”, applies `\hfill` and set the variable `\g__enumext_minipage_active_vii_bool` to “*true*” which will be used in the function `\__enumext_after_env:nn` to execute the `minipage` on the “*right side*”. At this point we will execute the `\__enumext_stop_list:` and `\__enumext_stop_store_level_vii:` functions stopping the `list` environment and the level saving mechanism for storage in *sequence* of the `\anskey` command and `anskey*` environment. This function is passed to the `\__enumext_after_list_vii:` function in the second part of the `enumext*` environment definition (§13.47).

```

4805 \cs_new_protected:Nn \__enumext_stop_mini_vii:
4806 {
4807     \bool_if:NTF \l__enumext_minipage_active_vii_bool
4808     {
4809         \__enumext_stop_list:
4810         \__enumext_stop_store_level_vii:
4811         \IfDocumentMetadataT { \tag_resume:n {enumext*} }
4812         \end__enumext_mini_page
4813         \hfill
4814         \bool_gset_true:N \g__enumext_minipage_active_vii_bool
4815     }
4816     {
4817         \__enumext_stop_list:
4818         \__enumext_stop_store_level_vii:
4819     }
4820 }

```

(End of definition for `\__enumext_start_mini_vii:` and `\__enumext_stop_mini_vii:`.)

Finally we execute the `{\code}` passed to the `mini-right` or `mini-right*` keys stored in the variable `\g__enumext_miniright_code_vii_tl` in the `minipage` environment on the “*right side*”. For compatibility with the `caption` package and possibly other `{\code}` passed to this key, we will pass it to a box and then print it.

```

4821 \__enumext_after_env:nn {enumext*}
4822 {
4823     \bool_if:NT \g__enumext_minipage_active_vii_bool
4824     {
4825         \__enumext_minipage:w [ t ] { \g__enumext_minipage_right_vii_dim }
4826         \legacy_if_gset_false:n { @minipage }
4827         \skip_vertical:N \c_zero_skip
4828         \par\addvspace { \g__enumext_minipage_right_skip }
4829         \bool_if:NF \g__enumext_minipage_center_vii_bool
4830         {
4831             \tl_put_left:Nn \g__enumext_miniright_code_vii_tl
4832             {
4833                 \centering
4834             }
4835         }
4836         \vbox_set_top:Nn \l__enumext_miniright_code_vii_box
4837         {
4838             \tl_use:N \g__enumext_miniright_code_vii_tl
4839         }
4840         \box_use_drop:N \l__enumext_miniright_code_vii_box
4841         \skip_vertical:N \c_zero_skip
4842         \__enumext_endminipage:
4843         \par\addvspace{ \g__enumext_minipage_after_skip }
4844     }
4845     \bool_gset_false:N \g__enumext_minipage_active_vii_bool
4846     \bool_gset_true:N \g__enumext_minipage_center_vii_bool
4847     \tl_gclear:N \g__enumext_miniright_code_vii_tl
4848     \dim_gzero:N \g__enumext_minipage_right_vii_dim
4849     \bool_gset_false:N \g__enumext_starred_bool
4850 }

```

`\__enumext_start_mini_viii:` The implementation of the `mini-env`, `mini-right` and `mini-right*` keys is identical to the one used in the `enumext*` environment.

```

4851 \cs_new_protected:Nn \__enumext_start_mini_viii:
4852 {
4853     \dim_compare:nNtT { \l__enumext_minipage_right_viii_dim } > { \c_zero_dim }

```

```

4854     {
4855         \dim_set:Nn \l__enumext_minipage_left_viii_dim
4856         {
4857             \linewidth
4858             - \l__enumext_minipage_right_viii_dim
4859             - \l__enumext_minipage_hsep_viii_dim
4860         }
4861         \bool_set_true:N \l__enumext_minipage_active_viii_bool
4862         \dim_gset_eq:NN
4863             \g__enumext_minipage_right_viii_dim
4864             \l__enumext_minipage_right_viii_dim
4865         \__enumext_mini_addvspace_viii:
4866         \nointerlineskip\noindent
4867         \__enumext_mini_page{ \l__enumext_minipage_left_viii_dim }
4868     }
4869 }
4870 \cs_new_protected:Nn \__enumext_stop_mini_viii:
4871 {
4872     \bool_if:NTF \l__enumext_minipage_active_viii_bool
4873     {
4874         \__enumext_stop_list:
4875         \IfDocumentMetadataT { \tag_resume:n {keyans*} }
4876         \end__enumext_mini_page
4877         \hfill
4878         \bool_gset_true:N \g__enumext_minipage_active_viii_bool
4879     }
4880     {
4881         \__enumext_stop_list:
4882     }
4883 }
4884 \__enumext_after_env:nn {keyans*}
4885 {
4886     \bool_if:NT \g__enumext_minipage_active_viii_bool
4887     {
4888         \__enumext_mini_page{ \g__enumext_minipage_right_viii_dim }
4889         \par\addvspace { \g__enumext_minipage_right_skip }
4890         \bool_if:NF \g__enumext_minipage_center_viii_bool
4891         {
4892             \tl_put_left:Nn \g__enumext_miniright_code_viii_tl
4893             {
4894                 \centering
4895             }
4896         }
4897         \vbox_set_top:Nn \l__enumext_miniright_code_viii_box
4898         {
4899             \tl_use:N \g__enumext_miniright_code_viii_tl
4900         }
4901         \box_use_drop:N \l__enumext_miniright_code_viii_box
4902         \end__enumext_mini_page
4903         \par\addvspace{ \g__enumext_minipage_after_skip }
4904     }
4905     \bool_gset_false:N \g__enumext_minipage_active_viii_bool
4906     \bool_gset_true:N \g__enumext_minipage_center_viii_bool
4907     \tl_gclear:N \g__enumext_miniright_code_viii_tl
4908     \dim_gzero:N \g__enumext_minipage_right_viii_dim
4909 }

```

(End of definition for __enumext_start_mini_viii: and __enumext_stop_mini_viii:.)

13.47 The environment enumext*

enumext* First we will generate the environment and we will give a temporary definition to __enumext_stop_item_tmp_vii: equal to __enumext_first_item_tmp_vii: and next to \item equal to __enumext_start_item_tmp_vii: which we will redefine later. Unlike the implementation used by the **shortlst** package, we will not set the values of \rightskip and \@rightskip equal to \@flushglue whose value is 0.0pt plus 1.0 fil, in the tests I have performed this fails in some circumstances and different results are obtained when using pdfTeX and LuaTeX.

```

4910 \NewDocumentEnvironment{enumext*}{o}
4911 {
4912     \__enumext_safe_exec_vii:
4913     \__enumext_parse_keys_vii:n {#1}

```

```

4914     \__enumext_before_list_vii:
4915     \__enumext_start_store_level_vii:
4916     \__enumext_start_list:nn { }
4917     {
4918         \__enumext_list_arg_two_vii:
4919         \__enumext_before_keys_exec_vii:
4920     }
4921     \IfDocumentMetadataT { \tag_suspend:n {enumext*} }
4922     \__enumext_starred_columns_set_vii:
4923     \item[] \scan_stop:
4924     \cs_set_eq:NN \__enumext_stop_item_tmp_vii: \__enumext_first_item_tmp_vii:
4925     \cs_set_eq:NN \item \__enumext_start_item_tmp_vii:
4926     \ignorespaces
4927 }
4928 {
4929     \IfDocumentMetadataT { \tag_struct_end:n {tag=text-unit} }
4930     \__enumext_stop_item_tmp_vii:
4931     \__enumext_remove_extra_parsep_vii:
4932     \__enumext_after_list_vii:
4933 }

```

(End of definition for enumext*. This function is documented on page 5.)

`\__enumext_safe_exec_vii:` We will first call the function `\__enumext_is_not_nested:` which sets `\g__enumext_starred_bool` to true if we are NOT nested within `enumext`, then call the function `\__enumext_internal_mini_page:` to create the environment `__enumext_mini_page`, we will increment `\l__enumext_level_h_int` to restrict nesting of the environment, set `\l__enumext_starred_bool` to true and finally call the function `\__enumext_is_on_first_level:` which sets `\l__enumext_starred_first_bool` to true if we are not nested, allowing the “storage system” to be used.

```

4934 \cs_new_protected:Nn \__enumext_safe_exec_vii:
4935 {
4936     \__enumext_is_not_nested:
4937     \__enumext_internal_mini_page:
4938     \int_incr:N \l__enumext_level_h_int
4939     \int_compare:nNnT { \l__enumext_level_h_int } > { 1 }
4940     {
4941         \msg_error:nn { enumext } { nested }
4942     }
4943     \int_compare:nNnT { \l__enumext_keyans_level_h_int } = { 1 }
4944     {
4945         \msg_error:nnn { enumext } { nested-horizontal } { keyans*}
4946     }
4947     \bool_set_true:N \l__enumext_starred_bool
4948     \bool_set_false:N \l__enumext_standar_bool
4949     \__enumext_is_on_first_level:
4950 }

```

(End of definition for `\__enumext_safe_exec_vii:`.)

`\__enumext_parse_keys_vii:n` We will first check the state of the variable `\l__enumext_resume_count_vii_bool` set by the key `resume` and call the function `\__enumext_resume_last_counter:` if it is “true”, then we will clear the variable `\l__enumext_series_name_str` used by the key `series`, process the environment `[⟨key = val⟩]` and execute the function `\__enumext_parse_series:n` used by the key `series`, then we execute the function `\__enumext_store_active_keys_vii:n` and reprocess the `⟨keys⟩` to pass them to the storage *sequence* if the key `save-key` is not active.

Here it is necessary to check the status of `\l__enumext_resume_count_vii_bool` in case the key `resume` is set using `\setenumext{enumext*}{resume}`.

```

4951 \cs_new_protected:Npn \__enumext_parse_keys_vii:n #1
4952 {
4953     \bool_if:NT \l__enumext_resume_count_vii_bool
4954     {
4955         \__enumext_resume_last_counter:
4956     }
4957     \tl_if_novalue:nF {#1}
4958     {
4959         \str_clear:N \l__enumext_series_name_str
4960         \keys_set:nn { enumext / enumext* } {#1}
4961         \bool_if:NF \l__enumext_print_keyans_cmd_bool
4962         {

```

```

4963         \__enumext_parse_series:n {#1}
4964     }
4965     \__enumext_store_active_keys_vii:n {#1}
4966 }
4967 }

```

(End of definition for __enumext_parse_keys_vii:n.)

`\__enumext_before_list_vii:` The function `\__enumext_before_list_vii:` first calls the function `\__enumext_vspace_above_vii:` used by the keys `above` and `above*`, then calls the function `\__enumext_check_ans_active:` for the check answer mechanism and finally calls the functions `\__enumext_before_args_exec:` and `\__enumext_start_mini_vii:` used by the keys `before*`, `mini-env`, `mini-right` and `mini-right*`.

```

4968 \cs_new_protected:Nn \__enumext_before_list_vii:
4969 {
4970     \__enumext_vspace_above_vii:
4971     \__enumext_check_ans_active:
4972     \__enumext_before_args_exec_vii:
4973     \__enumext_start_mini_vii:
4974 }

```

(End of definition for __enumext_before_list_vii:.)

`\__enumext_after_list_vii:` The function `\__enumext_after_list_vii:` first calls the function `\__enumext_stop_mini_vii:` which internally calls `\__enumext_stop_list:` and `\__enumext_stop_store_level_vii:` (§13.46.3) used by the keys `mini-env`, `mini-right` and `mini-right*`, then to the functions `\__enumext_after_stop_list_vii:` used by the key `after`, `\__enumext_check_ans_key_hook:` used by the key `check-ans`, `\__enumext_vspace_below_vii:` used by the keys `below` and `below*`. Finally set `\__enumext_starred_bool` to false and call the `\__enumext_resume_save_counter:` function used by the `series`, `resume` and `resume*` keys.

```

4975 \cs_new_protected:Nn \__enumext_after_list_vii:
4976 {
4977     \__enumext_stop_mini_vii:
4978     \__enumext_after_stop_list_vii:
4979     \__enumext_check_ans_key_hook:
4980     \__enumext_vspace_below_vii:
4981     \bool_set_false:N \__enumext_starred_bool
4982     \bool_if:NF \__enumext_print_keyans_cmd_bool
4983     {
4984         \__enumext_starred_save_counter:
4985     }
4986 }

```

(End of definition for __enumext_after_list_vii:.)

`\__enumext_start_store_level_vii:` and `\__enumext_stop_store_level_vii:` activate the “storing structure” mechanism in *sequence* for `\anskey` command and `anskey*` environment if `enumext*` are nested in `enumext`.

```

4987 \cs_new_protected:Nn \__enumext_start_store_level_vii:
4988 {
4989     \bool_if:NT \__enumext_store_active_bool
4990     {
4991         \int_compare:nNnT { \__enumext_level_int } > { 0 }
4992         {
4993             \__enumext_store_level_open_vii:
4994         }
4995     }
4996 }
4997 \cs_new_protected:Nn \__enumext_stop_store_level_vii:
4998 {
4999     \bool_if:NT \__enumext_store_active_bool
5000     {
5001         \int_compare:nNnT { \__enumext_level_int } > { 0 }
5002         {
5003             \__enumext_store_level_close_vii:
5004         }
5005     }
5006 }

```

(End of definition for __enumext_start_store_level_vii: and __enumext_stop_store_level_vii:.)

13.47.1 The command `\item` in `enumext*`

`\__enumext_first_item_tmp_vii:`

The `\__enumext_first_item_tmp_vii:` function will remove horizontal space equal to `\labelwidth` plus `\labelsep` to the left of the “*first*” `\item` in the environment at the point of execution of this function, where it is equal to the `\__enumext_stop_item_tmp_vii:` function inside the environment body definition.

```
5007 \cs_new_protected_nopar:Nn \__enumext_first_item_tmp_vii:
5008 {
5009     \skip_horizontal:n
5010     {
5011         -\__enumext_labelwidth_vii_dim - \__enumext_labelsep_vii_dim
5012     }
5013     \ignorespaces
5014 }
```

(End of definition for `\__enumext_first_item_tmp_vii:`.)

`\__enumext_start_item_tmp_vii:`

`\__enumext_item_peek_args_vii:`

`\__enumext_joined_item_vii:w`

`\__enumext_standar_item_vii:w`

`\__enumext_starred_item_vii:w`

First we will call the function `\__enumext_stop_item_tmp_vii:` that we will redefine later, we will increment the value of `\__enumext_item_column_pos_vii_int` that will count the item’s by rows and the value of `\g__enumext_item_count_all_vii_int` that will count the total of item’s in the environment. After that we will call the function `\__enumext_item_peek_args_vii:` that will handle the arguments passed to `\item`.

```
5015 \cs_new_protected_nopar:Nn \__enumext_start_item_tmp_vii:
5016 {
5017     \__enumext_stop_item_tmp_vii:
5018     \int_incr:N \__enumext_item_column_pos_vii_int
5019     \int_gincr:N \g__enumext_item_count_all_vii_int
5020     \__enumext_item_peek_args_vii:
5021 }
```

The function `\__enumext_item_peek_args_vii:` will handle the `\item(<number>)`. Look for the argument “(”, if it is present we will call the function `\__enumext_joined_item_vii:w (<number>)`, which is in charge of joining the item’s in the same row, in case they are not present we will set the default value (1).

```
5022 \cs_new_protected:Nn \__enumext_item_peek_args_vii:
5023 {
5024     \peek_meaning:NTF (
5025     { \__enumext_joined_item_vii:w }
5026     { \__enumext_joined_item_vii:w (1) }
5027 }
```

The function `\__enumext_joined_item_vii:w` will first call the function `\__enumext_starred_joined_item_vii:n` in charge of setting the *width* of the box that will store the content passed to `\item`. Then we will look for the argument “*”, if it is present we will call the function `\__enumext_starred_item_vii:w` otherwise we will call the function `\__enumext_standar_item_vii:w`.

```
5028 \cs_new_protected:Npn \__enumext_joined_item_vii:w (#1)
5029 {
5030     \__enumext_starred_joined_item_vii:n {#1}
5031     \peek_meaning_remove:NTF *
5032     { \__enumext_starred_item_vii:w }
5033     { \__enumext_standar_item_vii:w }
5034 }
```

The function `\__enumext_standar_item_vii:w` will first look for the argument “[”, if present it will set the state of the variable `\__enumext_wrap_label_opt_vii_bool` equal to the state of the variable `\__enumext_wrap_label_opt_vii_bool` handled by the key `wrap-label*` and finally execute the *non-enumerated* version `\item[<custom>]` by means of the function `\__enumext_start_item_vii:w`, otherwise we will set the value of the variable `\__enumext_wrap_label_vii_bool` handled by the `wrap-label` key to true and set the switch `\if@noitemarg` to true to execute the enumerated version of `\item` by means of the function `\__enumext_start_item_vii:w [ \__enumext_label_vii_tl ]`.

```
5035 \cs_new_protected:Npn \__enumext_standar_item_vii:w
5036 {
5037     \bool_set_false:N \__enumext_item_starred_vii_bool
5038     \peek_meaning:NTF [
5039     {
5040         \bool_set_eq:NN \__enumext_wrap_label_vii_bool \__enumext_wrap_label_opt_vii_bool
5041         \__enumext_start_item_vii:w
5042     }
5043     {
5044         \bool_set_true:N \__enumext_wrap_label_vii_bool
5045         \legacy_if_set_true:n { @noitemarg }
5046         \__enumext_start_item_vii:w [ \__enumext_label_vii_tl ] \ignorespaces
5047     }
5048 }
```

The function `\__enumext_starred_item_vii:w` together with the specified auxiliary functions `aux_i:w`, `aux_ii:w`, and `aux_iii:w` execute `\item*`, `\item*[\langle symbol \rangle]` and `\item*[\langle symbol \rangle][\langle offset \rangle]`.

```

5049 \cs_new_protected:Npn \__enumext_starred_item_vii:w
5050 {
5051   \bool_set_true:N \l__enumext_item_starred_vii_bool
5052   \bool_set_true:N \l__enumext_wrap_label_vii_bool
5053   \peek_meaning:NTF [
5054     { \__enumext_starred_item_vii_aux_i:w }
5055     { \__enumext_starred_item_vii_aux_ii:w }
5056   }
5057   \cs_new_protected:Npn \__enumext_starred_item_vii_aux_i:w [#1]
5058   {
5059     \tl_gset:Nn \g__enumext_item_symbol_aux_vii_tl {#1}
5060     \__enumext_starred_item_vii_aux_ii:w
5061   }
5062   \cs_new_protected:Npn \__enumext_starred_item_vii_aux_ii:w
5063   {
5064     \peek_meaning:NTF [
5065       { \__enumext_starred_item_vii_aux_iii:w }
5066       {
5067         \dim_set_eq:NN \l__enumext_item_symbol_sep_vii_dim \l__enumext_labelsep_vii_dim
5068         \legacy_if_set_true:n { @noitemarg }
5069         \__enumext_start_item_vii:w [ \l__enumext_label_vii_tl ] \ignorespaces
5070       }
5071     }
5072     \cs_new_protected:Npn \__enumext_starred_item_vii_aux_iii:w [#1]
5073     {
5074       \dim_set:Nn \l__enumext_item_symbol_sep_vii_dim {#1}
5075       \legacy_if_set_true:n { @noitemarg }
5076       \__enumext_start_item_vii:w [ \l__enumext_label_vii_tl ] \ignorespaces
5077     }

```

(End of definition for `\__enumext_start_item_tmp_vii:` and others.)

`\__enumext_fake_make_label_vii:n`

The `\__enumext_fake_make_label_vii:n` function will be in charge of handling our definition of `\item`. First we increment the counter `enumXvii` for the enumerated items and activate support for the *check answers* mechanism, followed by support for `\item*[\langle symbol \rangle][\langle offset \rangle]` if present, then the `wrap-label` and `wrap-label*` keys which we execute using `\makebox` whose width will be given by the `labelwidth` key and position by the `align` key, inside the argument of this we will execute the `font` key together with the function defined by the `wrap-label` or `wrap-label*` keys. Finally we execute the `labelsep` key applying a `\skip_horizontal:N` and `\ignorespaces`.

- For compatibility with *tagged* PDF and `hyperref` when an environment `enumext` is nested in `enumext*` and the key `save-ans` is not active need setting the `\if@hyper@item` switch to “true”. The explanation for this is given by the master Heiko Oberdiek on `\refstepcounter{enumi} twice (or more) creates destination with the same identifier`. This patch is only needed if you are running `pdflatex` and not if you are running `lualatex`

```

5078 \cs_new_protected_nopar:Npn \__enumext_fake_make_label_vii:n #1
5079 {
5080   \legacy_if:nT { @noitemarg }
5081   {
5082     \legacy_if_set_false:n { @noitemarg }
5083     \legacy_if:nT { @nmbrrlist }
5084     {
5085       \IfDocumentMetadataT
5086       {
5087         \bool_if:NT \l__enumext_hyperref_bool
5088         {
5089           \legacy_if_set_true:n { @hyper@item }
5090         }
5091       }
5092       \refstepcounter{enumXvii}
5093       \bool_if:NT \l__enumext_check_answers_bool
5094       {
5095         \int_gincr:N \g__enumext_item_number_int
5096         \bool_set_true:N \l__enumext_item_number_bool
5097       }
5098     }
5099   }
5100   \bool_if:NT \l__enumext_item_starred_vii_bool
5101   {
5102     \tl_if_blank:VT \g__enumext_item_symbol_aux_vii_tl

```

```

5103     {
5104         \tl_gset_eq:NN
5105         \g__enumext_item_symbol_aux_vii_tl \l__enumext_item_symbol_vii_tl
5106     }
5107     \mode_leave_vertical:
5108     \skip_horizontal:n { -\l__enumext_item_symbol_sep_vii_dim }
5109     \hbox_overlap_left:n { \g__enumext_item_symbol_aux_vii_tl }
5110     \skip_horizontal:N \l__enumext_item_symbol_sep_vii_dim
5111     \tl_gclear:N \g__enumext_item_symbol_aux_vii_tl
5112 }
5113 \makebox[ \l__enumext_labelwidth_vii_dim ][ \l__enumext_align_label_vii_str ]
5114 {
5115     \tl_use:N \l__enumext_label_font_style_vii_tl
5116     \bool_if:NTF \l__enumext_wrap_label_vii_bool
5117     {
5118         \__enumext_wrapper_label_vii:n {#1}
5119     }
5120     { #1 }
5121 }
5122 \skip_horizontal:N \l__enumext_labelsep_vii_dim \ignorespaces
5123 }

```

(End of definition for `\__enumext_fake_make_label_vii:n`.)

13.47.2 Real definition of `\item` in `enumext*`

The functions `\__enumext_start_item_vii:w` and `\__enumext_stop_item_vii:` executing the true definition of `\item` inside the `enumext*` environment, unlike the implementation in `shortlst` we will NOT use an extra group and the plain form of the `lrbox` environment.

`\__enumext_start_item_vii:w` The first thing we will do is set the value of `\__enumext_stop_item_tmp_vii:` equal to `\__enumext_stop_item_vii:` which we will define later, after that we will start capturing `\item` and “item content” in a *horizontal box* where the width will be `\itemwidth` plus `\labelwidth` plus `\labelsep`.

```

5124 \cs_new_protected_nopar:Npn \__enumext_start_item_vii:w [#1]
5125 {
5126     \cs_set_eq:NN \__enumext_stop_item_tmp_vii: \__enumext_stop_item_vii:
5127     \hbox_set_to_wd:Nnw \l__enumext_item_text_vii_box
5128     {
5129         \l__enumext_joined_width_vii_dim
5130         + \l__enumext_labelwidth_vii_dim
5131         + \l__enumext_labelsep_vii_dim
5132     }

```

Redefine the `\footnote` command.

```
5133 \__enumext_renew_footnote_starred:
```

Now we insert our *sockets* for *tagging* PDF support and run `\item`.

```

5134 \__enumext_start_list_tag:n {enumext*}
5135 \__enumext_fake_make_label_vii:n {#1}
5136 \__enumext_stop_start_list_tag:

```

Finally we open the `minipage` environment, capture the “item content”, make `\parindent` take the value of the key `listparindent` and `\parskip` take the value of the key `parsep`, then execute the keys `itemindent` and `first`.

- Here the use of `\unskip` and `\skip_horizontal:n` with the value of `listparindent` is necessary, otherwise an unwanted space is created when using `\item[⟨opt⟩]` and the value passed to the key `itemindent` is incremented.

```

5137 \__enumext_minipage:w [ t ]{ \l__enumext_joined_width_vii_dim }
5138 \dim_set_eq:NN \parindent \l__enumext_listparindent_vii_dim
5139 \skip_set_eq:NN \parskip \l__enumext_parsep_vii_skip
5140 \__enumext_unskip_unkern:
5141 \__enumext_unskip_unkern:
5142 \skip_horizontal:n { -\l__enumext_listparindent_vii_dim } \ignorespaces
5143 \tl_use:N \l__enumext_fake_item_indent_vii_tl
5144 \tl_use:N \l__enumext_after_list_args_vii_tl
5145 }

```

The `\__enumext_stop_item_vii:` function will finish the fetching `\item` and “item content” by closing the `minipage` environment, the *sockets* for *tagging* PDF and the *horizontal box*.

```

5146 \cs_new_protected_nopar:Nn \__enumext_stop_item_vii:
5147 {
5148     \__enumext_endminipage:
5149     \__enumext_stop_list_tag:n {enumext*}
5150     \hbox_set_end:

```

Here we will reduce the *warnings* a bit by setting the value of `\hbadness` to `10000`, print `\item` and “*item content*” from the *horizontal box*.,

```
5151 \int_set:Nn \hbadness { 10000 }
5152 \box_use_drop:N \l__enumext_item_text_vii_box
```

Finally apply the *vertical space* between rows set by `itemsep` key passed to `\parsep` using `\par\noindent` and *horizontal space* between columns set by `columns-sep` key using `\skip_horizontal:N`.

```
5153 \int_compare:nNnTF
5154 { \l__enumext_item_column_pos_vii_int } = { \l__enumext_columns_vii_int }
5155 {
5156   \par\noindent
5157   \int_zero:N \l__enumext_item_column_pos_vii_int
5158 }
5159 {
5160   \skip_horizontal:N \l__enumext_columns_sep_vii_dim
5161 }
5162 }
```

(End of definition for `\__enumext_start_item_vii:w` and `\__enumext_stop_item_vii:.`)

`\__enumext_remove_extra_parsep_vii:`

Remove the extra *vertical space* equal to `\parsep=itemsep` when the total number of `\item` is divisible by the number of `\item` in the last row of the environment. Here the use of `\unskip` or `\removeatlastskip` fails and does not obtain the expected result, using `\vspace` is the option and in this case, we can use a simplified version since we are always in *(vertical mode)*.

```
5163 \cs_new_protected:Nn \__enumext_remove_extra_parsep_vii:
5164 {
5165   \int_compare:nNnT
5166   {
5167     \int_mod:nn
5168     { \g__enumext_item_count_all_vii_int } { \l__enumext_columns_vii_int }
5169   }
5170   =
5171   { 0 }
5172   {
5173     \para_end:
5174     \skip_vertical:n { -\l__enumext_itemsep_vii_skip }
5175     \skip_vertical:N \c_zero_skip
5176     \int_gzero:N \g__enumext_item_count_all_vii_int
5177   }
5178 }
```

(End of definition for `\__enumext_remove_extra_parsep_vii:.`)

As we don’t want our check to be executed `check-ans` by levels but on the complete list, we will take it out of the `enumext*` environment using the “hook” function `\__enumext_after_env:nn`.

```
5179 \__enumext_after_env:nn {enumext*}
5180 {
5181   \__enumext_execute_after_env:
5182 }
```

13.48 The environment `keyans*`

`keyans*`

The implementation of `keyans*` environment is the similar as that used by the `enumext*` environment except for the `\__enumext_check_starred_cmd:n` function added in the second part.

```
5183 \NewDocumentEnvironment{keyans*}{ o }
5184 {
5185   \__enumext_safe_exec_viii:
5186   \__enumext_parse_keys_viii:n {#1}
5187   \__enumext_before_list_viii:
5188   \__enumext_start_list:nn { }
5189   {
5190     \__enumext_list_arg_two_viii:
5191     \__enumext_before_keys_exec_viii:
5192   }
5193   \IfDocumentMetadataT { \tag_suspend:n {keyans*} }
5194   \__enumext_starred_columns_set_viii:
5195   \item[] \scan_stop:
5196   \cs_set_eq:NN \__enumext_stop_item_tmp_viii: \__enumext_first_item_tmp_viii:
5197   \cs_set_eq:NN \item \__enumext_start_item_tmp_viii:
5198   \ignorespaces
5199 }
```

```

5200 {
5201   \IfDocumentMetadataT { \tag_struct_end:n {tag=text-unit} }
5202   \__enumext_stop_item_tmp_viii:
5203   \__enumext_remove_extra_parsep_viii:
5204   \__enumext_check_starred_cmd:n { item }
5205   \__enumext_after_list_viii:
5206 }

```

(End of definition for `keyans*`. This function is documented on page 16.)

`\__enumext_safe_exec_viii:` The `\__enumext_safe_exec_viii:` function will first check if the `save-ans` key is active and only when this is true the environment will be available, it will increment the value of `\l__enumext_keyans_level_h_int` and return an error message when we are nesting the environment, then it will call the `\__enumext_keyans_name_and_start:` function in charge of saving the name of the environment and the line it is running on, then it will check if we are trying to nest `keyans*` in `enumext*` returning an error and we will set `\l__enumext_starred_bool` to true, finally we will check if we are within the appropriate level within the `enumext` environment.

```

5207 \cs_new_protected:Nn \__enumext_safe_exec_viii:
5208 {
5209   \bool_if:NF \l__enumext_store_active_bool
5210   {
5211     \msg_error:nnnn { enumext } { wrong-place } { keyans* } { save-ans }
5212   }
5213   \int_incr:N \l__enumext_keyans_level_h_int
5214   \int_compare:nNnT { \l__enumext_keyans_level_h_int } > { 1 }
5215   {
5216     \msg_error:nn { enumext } { nested }
5217   }
5218   \__enumext_keyans_name_and_start:
5219   \bool_if:NT \l__enumext_starred_bool
5220   {
5221     \msg_error:nnn { enumext } { nested-horizontal } { enumext* }
5222   }
5223   \bool_set_true:N \l__enumext_starred_bool
5224   % Set false for interfering with enumext nested in keyans* (yes, its possible and crayze)
5225   \bool_set_false:N \l__enumext_store_active_bool
5226   \int_compare:nNnT { \l__enumext_level_int } > { 1 }
5227   {
5228     \msg_error:nn { enumext } { keyans-wrong-level }
5229   }
5230 }

```

(End of definition for `\__enumext_safe_exec_viii:`.)

`\__enumext_parse_keys_viii:n` Parse [`<key = val>`] for `keyans*`.

```

5231 \cs_new_protected:Npn \__enumext_parse_keys_viii:n #1
5232 {
5233   \tl_if_novalue:nF {#1}
5234   {
5235     \keys_set:nn { enumext / keyans* } {#1}
5236   }
5237 }

```

(End of definition for `\__enumext_parse_keys_viii:n`.)

`\__enumext_before_list_viii:` The function `\__enumext_before_list_viii:` will add the vertical spacing on the environment if the `above` key is active next to the `{<code>}` defined by the `before*` key if it is active, the call the function `\__enumext_start_mini_viii:` handle by `mini-env`.

```

5238 \cs_new_protected:Nn \__enumext_before_list_viii:
5239 {
5240   \__enumext_vspace_above_viii:
5241   \__enumext_before_args_exec_viii:
5242   \__enumext_start_mini_viii:
5243 }

```

(End of definition for `\__enumext_before_list_viii:`.)

`\__enumext_after_list_viii:` The function `\__enumext_after_list_viii:` first call the function `\__enumext_stop_mini_viii:`, then apply the `{\code}` handled by the `after` key together with the *vertical space* handled by the `below` key if they are present.

```

5244 \cs_new_protected:Nn \__enumext_after_list_viii:
5245 {
5246     \__enumext_stop_mini_viii:
5247     \__enumext_after_stop_list_viii:
5248     \__enumext_vspace_below_viii:
5249 }
```

(End of definition for `\__enumext_after_list_viii:`.)

13.48.1 The command `\item` in `keyans*`

The idea here is to make the `\item` command behave in the same way as in the `keyans` environment with the difference of the *optional argument* (`\langle number \rangle`) which works in the same way as in the `enumext*` environment. In simple terms we want to store the `\langle label \rangle` next to the `[\langle content \rangle]` if it is present in the *sequence* and *prop list* defined by `save-ans` key for `\item*`, `\item*[\langle content \rangle]`, `\item(\langle number \rangle)*` and `\item(\langle number \rangle)*[\langle content \rangle]` commands.

`\__enumext_first_item_tmp_viii:` The `\__enumext_first_item_tmp_viii:` function will remove horizontal space equal to `\labelwidth` plus `\labelsep` to the left of the “*first*” `\item` in the environment at the point of execution of this function, where it is equal to the `\__enumext_stop_item_tmp_viii:` function inside the environment body definition.

```

5250 \cs_new_protected_nopar:Nn \__enumext_first_item_tmp_viii:
5251 {
5252     \skip_horizontal:n
5253     {
5254         -\__enumext_labelwidth_viii_dim - \__enumext_labelsep_viii_dim
5255     }
5256     \ignorespaces
5257 }
```

(End of definition for `\__enumext_first_item_tmp_viii:`.)

`\__enumext_start_item_tmp_viii:` First we will call the function `\__enumext_stop_item_tmp_viii:` that we will redefine later, we will increment the value of `\l__enumext_item_column_pos_viii_int` that will count the item’s by rows and the value of `\g__enumext_item_count_all_viii_int` that will count the total of item’s in the environment. `\__enumext_item_peek_args_viii:` After that we will call the function `\__enumext_item_peek_args_viii:` that will handle the arguments passed to `\item`.

```

5258 \cs_new_protected_nopar:Nn \__enumext_start_item_tmp_viii:
5259 {
5260     \__enumext_stop_item_tmp_viii:
5261     \int_incr:N \l__enumext_item_column_pos_viii_int
5262     \int_gincr:N \g__enumext_item_count_all_viii_int
5263     \__enumext_item_peek_args_viii:
5264 }
```

The function `\__enumext_item_peek_args_viii:` will handle the `\item(\langle number \rangle)`. Look for the argument “`(`”, if it is present we will call the function `\__enumext_joined_item_viii:w` (`\langle number \rangle`), which is in charge of joining the item’s in the same row, in case they are not present we will set the default value `(1)`.

```

5265 \cs_new_protected:Nn \__enumext_item_peek_args_viii:
5266 {
5267     \peek_meaning:NTF (
5268         { \__enumext_joined_item_viii:w }
5269         { \__enumext_joined_item_viii:w (1) }
5270 }
```

The function `\__enumext_joined_item_viii:w` will first call the function `\__enumext_starred_joined_item_viii:n` in charge of setting the *width* of the box that will store the content passed to `\item`. Then we will look for the argument “`*`”, if it is present we will call the function `\__enumext_starred_item_viii:w` otherwise we will call the function `\__enumext_standar_item_viii:w`.

```

5271 \cs_new_protected:Npn \__enumext_joined_item_viii:w (#1)
5272 {
5273     \__enumext_starred_joined_item_viii:n {#1}
5274     \peek_meaning_remove:NTF *
5275     { \__enumext_starred_item_viii:w }
5276     { \__enumext_standar_item_viii:w }
5277 }
```

The function `\__enumext_standar_item_viii:w` will first look for the argument “[”, if present it will set the state of the variable `\l__enumext_wrap_label_opt_viii_bool` equal to the state of the variable `\l__enumext_wrap_label_opt_viii_bool` handled by the key `wrap-label*` and finally execute the *non-enumerated* version `\item[⟨custom⟩]` by means of the function `\__enumext_start_item_viii:w`, otherwise we will set the value of the variable `\l__enumext_wrap_label_viii_bool` handled by the `wrap-label` key to true and set the switch `\if@noitemarg` to true to execute the enumerated version of `\item` by means of the function `\__enumext_start_item_viii:w [ \l__enumext_label_viii_tl ]`.

```

5278 \cs_new_protected:Npn \__enumext_standar_item_viii:w
5279 {
5280   \bool_set_false:N \l__enumext_item_starred_viii_bool
5281   \bool_set_false:N \l__enumext_item_wrap_key_bool
5282   \peek_meaning:NTF [
5283     {
5284       \bool_set_eq:NN \l__enumext_wrap_label_viii_bool \l__enumext_wrap_label_opt_viii_bool
5285       \__enumext_start_item_viii:w
5286     }
5287     {
5288       \bool_set_true:N \l__enumext_wrap_label_viii_bool
5289       \legacy_if_set_true:n { @noitemarg }
5290       \__enumext_start_item_viii:w [ \l__enumext_label_viii_tl ] \ignorespaces
5291     }
5292   }

```

(End of definition for `\__enumext_start_item_tmp_viii:` and others.)

```

\__enumext_starred_item_viii:w
\__enumext_starred_item_viii_aux_i:w
\__enumext_starred_item_viii_aux_ii:w
\__enumext_keyans_starred_item_star:

```

The function `\__enumext_starred_item_viii:w` together with the specified auxiliary functions `aux_i:w` and `aux_ii:w` execute `\item*` and `\item*[⟨content⟩]`.

```

5293 \cs_new_protected:Npn \__enumext_starred_item_viii:w
5294 {
5295   \bool_set_true:N \l__enumext_item_starred_viii_bool
5296   \bool_set_true:N \l__enumext_item_wrap_key_bool
5297   \bool_set_true:N \l__enumext_wrap_label_viii_bool
5298   \peek_meaning:NTF [
5299     { \__enumext_starred_item_viii_aux_i:w }
5300     { \__enumext_starred_item_viii_aux_ii:w }
5301   }

```

The function `\__enumext_starred_item_viii_aux_i:w` will save the *optional argument* to `\item*` in `\l__enumext_store_current_opt_arg_tl` and will save this argument along with the spacing set by the key `save-sep` in variable `\l__enumext_store_current_label_tl` if present, then call the function `\__enumext_starred_item_viii_aux_ii:w`.

```

5302 \cs_new_protected:Npn \__enumext_starred_item_viii_aux_i:w [#1]
5303 {
5304   \tl_clear:N \l__enumext_store_current_label_tl
5305   \tl_if_no_value:nF { #1 }
5306   {
5307     \tl_if_empty:NF \l__enumext_store_keyans_item_opt_sep_viii_tl
5308     {
5309       \tl_put_right:NV \l__enumext_store_current_label_tl \l__enumext_store_keyans_item_opt_sep_viii_tl
5310       \tl_put_right:Nn \l__enumext_store_current_label_tl { #1 }
5311     }
5312     \tl_set:Nn \l__enumext_store_current_opt_arg_tl { #1 }
5313   }
5314   \__enumext_starred_item_viii_aux_ii:w
5315 }
5316 \cs_new_protected:Npn \__enumext_starred_item_viii_aux_ii:w
5317 {
5318   \legacy_if_set_true:n { @noitemarg }
5319   \__enumext_start_item_viii:w [ \l__enumext_label_viii_tl ] \ignorespaces
5320 }

```

The function `\__enumext_keyans_starred_item_star:` will be in charge of storing the current *⟨label⟩* for `\item*` followed by the *⟨content⟩* for `\item*[⟨content⟩]` if present in the *sequence* and *prop list* set by the `save-ans` key. In this same function the keys `show-ans`, `show-pos`, `mark-sep` and `save-ref` are implemented.

```

5321 \cs_new_protected:Nn \__enumext_keyans_starred_item_star:
5322 {
5323   \tl_put_left:Ne \l__enumext_store_current_label_tl { \l__enumext_label_viii_tl }
5324   \__enumext_store_addto_prop:V \l__enumext_store_current_label_tl
5325   \__enumext_keyans_store_ref:
5326   \tl_put_left:Nn \l__enumext_store_current_label_tl { \item }

```



```

5327 \__enumext_keyans_addto_seq_link:
5328 \int_gincr:N \g__enumext_check_starred_cmd_int
5329 \dim_compare:nNt { \l__enumext_mark_sym_sep_viii_dim } = { \c_zero_dim }
5330 {
5331   \dim_set:Nn \l__enumext_mark_sym_sep_viii_dim { \l__enumext_labelsep_viii_dim }
5332 }
5333 \bool_if:NT \l__enumext_show_answer_bool
5334 {
5335   \tl_set_eq:NN \l__enumext_mark_answer_sym_tl \l__enumext_mark_answer_sym_viii_tl
5336   \str_set_eq:NN \l__enumext_mark_position_str \l__enumext_mark_position_viii_str
5337   \__enumext_print_keyans_box:NN
5338   \l__enumext_labelwidth_viii_dim \l__enumext_mark_sym_sep_viii_dim
5339 }
5340 \bool_if:NT \l__enumext_show_position_bool
5341 {
5342   \tl_set:Nx \l__enumext_mark_answer_sym_tl
5343   {
5344     \group_begin:
5345     \exp_not:N \normalfont
5346     \exp_not:N \footnotesize [ \int_eval:n
5347       {
5348         \prop_count:c { g__enumext_ \l__enumext_store_name_tl _prop }
5349       }
5350     ]
5351     \group_end:
5352   }
5353   \str_set_eq:NN \l__enumext_mark_position_str \l__enumext_mark_position_viii_str
5354   \__enumext_print_keyans_box:NN
5355   \l__enumext_labelwidth_viii_dim \l__enumext_mark_sym_sep_viii_dim
5356 }
5357 }

```

(End of definition for `\__enumext_starred_item_viii:w` and others.)

```

\__enumext_keyans_wrapper_label_viii:n
\__enumext_fake_make_label_viii:n

```

The implementation at this is very similar to that of the `enumext*` environment.

```

5358 \cs_new_protected:Npn \__enumext_keyans_wrapper_label_viii:n #1
5359 {
5360   \bool_lazy_all:nT
5361   {
5362     { \bool_if_p:N \l__enumext_wrap_label_viii_bool }
5363     { \bool_if_p:N \l__enumext_show_answer_bool }
5364     { \bool_if_p:N \l__enumext_item_wrap_key_bool }
5365     { \cs_if_exist_p:N \__enumext_keyans_wrapper_item_viii:n }
5366   }
5367   {
5368     \cs_set_eq:NN
5369     \__enumext_wrapper_label_viii:n \__enumext_keyans_wrapper_item_viii:n
5370   }
5371   \bool_if:NTF \l__enumext_wrap_label_viii_bool
5372   {
5373     \__enumext_wrapper_label_viii:n {#1}
5374   }
5375   { #1 }
5376 }
5377 \cs_new_protected_nopar:Npn \__enumext_fake_make_label_viii:n #1
5378 {
5379   \legacy_if:nT { @noitemarg }
5380   {
5381     \legacy_if_set_false:n { @noitemarg }
5382     \legacy_if:nT { @nmbrrlist }
5383     {
5384       \refstepcounter{enumXviii}
5385     }
5386   }
5387   \bool_if:NT \l__enumext_item_starred_viii_bool
5388   {
5389     \__enumext_keyans_starred_item_star:
5390   }
5391   \makebox[ \l__enumext_labelwidth_viii_dim ][ \l__enumext_align_label_viii_str ]
5392   {
5393     \tl_use:N \l__enumext_label_font_style_viii_tl

```

```

5394     \__enumext_keyans_wrapper_label_viii:n {#1}
5395   }
5396   \skip_horizontal:N \__enumext_labelsep_viii_dim \ignorespaces
5397 }

```

(End of definition for __enumext_keyans_wrapper_label_viii:n and __enumext_fake_make_label_viii:n)

13.48.2 Real definition of \item in keyans*

The implementation at this is very similar to that of the `enumext*` environment.

```

\__enumext_start_item_viii:w
\__enumext_stop_item_viii:
5398 \cs_new_protected_nopar:Npn \__enumext_start_item_viii:w [#1]
5399 {
5400   \cs_set_eq:NN \__enumext_stop_item_tmp_viii: \__enumext_stop_item_viii:
5401   \hbox_set_to_wd:Nnw \__enumext_item_text_viii_box
5402   {
5403     \__enumext_joined_width_viii_dim
5404     + \__enumext_labelwidth_viii_dim
5405     + \__enumext_labelsep_viii_dim
5406   }
5407   \__enumext_renew_footnote_starred:
5408   \__enumext_start_list_tag:n {keyans*}
5409   \__enumext_fake_make_label_viii:n {#1}
5410   \__enumext_stop_start_list_tag:
5411   \__enumext_minipage:w [ t ]{ \__enumext_joined_width_viii_dim }
5412   \dim_set_eq:NN \parindent \__enumext_listparindent_viii_dim
5413   \skip_set_eq:NN \parskip \__enumext_parsep_viii_skip
5414   \__enumext_unskip_unkern:
5415   \__enumext_unskip_unkern:
5416   \skip_horizontal:n { -\__enumext_listparindent_viii_dim } \ignorespaces
5417   \tl_use:N \__enumext_fake_item_indent_viii_tl
5418   \bool_if:NT \__enumext_item_starred_viii_bool
5419   {
5420     \__enumext_keyans_show_item_opt_viii:
5421   }
5422   \tl_use:N \__enumext_after_list_args_viii_tl
5423 }
5424 \cs_new_protected_nopar:Nn \__enumext_stop_item_viii:
5425 {
5426   \__enumext_endminipage:
5427   \__enumext_stop_list_tag:n {keyans*}
5428   \hbox_set_end:
5429   \int_set:Nn \hbadness { 10000 }
5430   \box_use_drop:N \__enumext_item_text_viii_box
5431   \int_compare:nNnTF
5432   { \__enumext_item_column_pos_viii_int } = { \__enumext_columns_viii_int }
5433   {
5434     \par\noindent
5435     \int_zero:N \__enumext_item_column_pos_viii_int
5436   }
5437   {
5438     \skip_horizontal:N \__enumext_columns_sep_viii_dim
5439   }
5440 }

```

(End of definition for __enumext_start_item_viii:w and __enumext_stop_item_viii:.)

__enumext_remove_extra_parsep_viii: The implementation at this is very similar to that of the `enumext*` environment.

```

5441 \cs_new_protected:Nn \__enumext_remove_extra_parsep_viii:
5442 {
5443   \int_compare:nNnT
5444   {
5445     \int_mod:nn
5446     { \g__enumext_item_count_all_viii_int }
5447     { \__enumext_columns_viii_int }
5448   }
5449   =
5450   { 0 }
5451   {
5452     \para_end:
5453     \skip_vertical:n { -\__enumext_itemsep_viii_skip }
5454     \skip_vertical:N \c_zero_skip
5455     \int_gzero:N \g__enumext_item_count_all_viii_int

```

```

5456     }
5457 }

```

(End of definition for `\__enumext_remove_extra_parsep_viii:`.)

13.49 The command `\getkeyans`

`\getkeyans` The `\getkeyans` command takes a *mandatory argument* of the form $\langle \textit{store name} : \textit{position} \rangle$. Retrieve a “*single content*” stored by `\anskey`, `\anspic*` and `\item*` and `anskey*` from *prop list* defined by `save-ans` key.

```

5458 \NewDocumentCommand \getkeyans { m }
5459 {
5460   \exp_args:Ne \__enumext_getkeyans_aux:n
5461   { \tl_to_str:e { \text_expand:n {#1} } }
5462 }

```

The internal function `\__enumext_getkeyans_aux:n` is in charge of *splitting* the *mandatory argument* using “:”. If “:” is omitted it will return an error.

```

5463 \cs_new_protected:Npn \__enumext_getkeyans_aux:n #1
5464 {
5465   \str_if_in:nnTF {#1} { : }
5466   {
5467     \use:e
5468     {
5469       \cs_set:Npn \exp_not:N \__enumext_tmp:w ##1 \c_colon_str ##2 \scan_stop:
5470       { {##1} {##2} }
5471     }
5472     \exp_after:wN \__enumext_getkeyans:nn \__enumext_tmp:w #1 \scan_stop:
5473   }
5474   { \msg_error:nnn { enumext } { missing-colon } {#1} }
5475 }

```

The internal function `\__enumext_getkeyans:nn` will check for the existence of the *prop list*, if it does not exist it will return an error message, then it will fetch the content specified by the *second argument* from *prop list*.

```

5476 \cs_new_protected:Npn \__enumext_getkeyans:nn #1 #2
5477 {
5478   \prop_if_exist:cTF { g__enumext_#1_prop }
5479   {
5480     \prop_item:cn { g__enumext_#1_prop } {#2}
5481   }
5482   {
5483     \msg_error:nnn { enumext } { undefined-storage-anskey } {#1}
5484   }
5485 }

```

(End of definition for `\getkeyans`, `\__enumext_getkeyans_aux:n`, and `\__enumext_getkeyans:nn`. This function is documented on page 19.)

13.50 The command `\printkeyans`

The `\printkeyans` command prints “*all stored content*” in the *sequence* defined by the `save-ans` key. The first thing we will do is define a set of $\langle \textit{filtered keys} \rangle$ with which we will control the options of the different nesting levels for the environment `enumext` and `enumext*` by storing their values in the list of tokens `\l__enumext_print_keyans_X_tl`.

The variable `\l__enumext_print_keyans_starred_tl` will have the default $\langle \textit{keys} \rangle$ for `\printkeyans*` and will be set by `\setenumext[⟨print*⟩]` and the variable `\l__enumext_print_keyans_vii_tl` will have the default keys for the environment `enumext*` nested within the *sequence* and will be set by `\setenumext[⟨print,*⟩]`, the rest of the variables will be for the environment `enumext` and will be set by `\setenumext[⟨print,level⟩]`.

```

5486 \keys_define:nn { enumext / print }
5487 {
5488   print* .code:n      = \keys_precompile:neN { enumext / enumext* }
5489               { \__enumext_filter_save_key:n {#1} }
5490               \l__enumext_print_keyans_starred_tl, % starred cmd
5491   print* .initial:n   = { labelwidth=0pt, labelsep=0.3333em, itemindent=0pt, list-offset=0pt,
5492               rightmargin=0pt, listparindent=0pt, nosepl, label=\arabic*.,
5493               columns=2, first=\small, font=\small },
5494   print-1 .code:n     = \keys_precompile:neN { enumext / level-1 }
5495               { \__enumext_filter_save_key:n {#1} }
5496               \l__enumext_print_keyans_i_tl,
5497   print-1 .initial:n  = { labelwidth=0pt, labelsep=0.3333em, itemindent=0pt, list-offset=0pt,

```

```

5498         rightmargin=0pt, listparindent=0pt, nosep, label=\arabic*.,
5499         columns=2, first=\small, font=\small },
5500     print-2 .code:n = \keys_precompile:neN { enumext / level-2 }
5501         { \__enumext_filter_save_key:n {#1} }
5502         \l__enumext_print_keyans_ii_tl,
5503     print-2 .initial:n = { labelwidth=0pt, labelsep=0.3333em, itemindent=0pt, list-offset=0pt,
5504         rightmargin=0pt, listparindent=0pt, nosep, label=(\alph*),
5505         first=\small, font=\small },
5506     print-3 .code:n = \keys_precompile:neN { enumext / level-3 }
5507         { \__enumext_filter_save_key:n {#1} }
5508         \l__enumext_print_keyans_iii_tl,
5509     print-3 .initial:n = { labelwidth=0pt, labelsep=0.3333em, itemindent=0pt, list-offset=0pt,
5510         rightmargin=0pt, listparindent=0pt, nosep, label=\roman*.,
5511         first=\small, font=\small },
5512     print-4 .code:n = \keys_precompile:neN { enumext / level-4 }
5513         { \__enumext_filter_save_key:n {#1} }
5514         \l__enumext_print_keyans_iv_tl,
5515     print-4 .initial:n = { labelwidth=0pt, labelsep=0.3333em, itemindent=0pt, list-offset=0pt,
5516         rightmargin=0pt, listparindent=0pt, nosep, label=\Alph*.,
5517         first=\small, font=\small },
5518     print-* .code:n = \keys_precompile:neN { enumext / enumext* }
5519         { \__enumext_filter_save_key:n {#1} }
5520         \l__enumext_print_keyans_vii_tl, % starred nested
5521     print-* .initial:n = { labelwidth=0pt, labelsep=0.3333em, itemindent=0pt, list-offset=0pt,
5522         rightmargin=0pt, listparindent=0pt, nosep, label=\arabic*.,
5523         first=\small, font=\small },
5524 }

```

- The reason for storing $\langle keys \rangle$ in token lists using `\keys_precompile:neN` is because the keys are set via `\setenumext` but are later executed by running the command `\printkeyans` and they are not handled directly by its *optional argument*, except those related to the *first* opening level.

`\printkeyans`

`\__enumext_printkeyans:nnn`

Create a user command to print “*all stored content*” in *sequence* for `\anskey`, `anskey*`, `\item*` and `\anspic*`. Within a group we will run our “*precompiled keys*” and then call the internal function `\__enumext_printkeyans:nnn`.

```

5525 \NewDocumentCommand \printkeyans { s O{} m }
5526 {
5527     \group_begin:
5528         \bool_set_true:N \l__enumext_print_keyans_cmd_bool
5529         \tl_use:N \l__enumext_print_keyans_i_tl
5530         \tl_use:N \l__enumext_print_keyans_ii_tl
5531         \tl_use:N \l__enumext_print_keyans_iii_tl
5532         \tl_use:N \l__enumext_print_keyans_iv_tl
5533         \tl_use:N \l__enumext_print_keyans_vii_tl
5534         \__enumext_printkeyans:nnn { #1 } { #2 } { #3 }
5535         \bool_set_false:N \l__enumext_print_keyans_cmd_bool
5536     \group_end:
5537 }

```

The internal function `\__enumext_printkeyans:nnn` will check for the existence of the *sequence*, if it does not exist it will return an error message, then it will check if not empty.

```

5538 \cs_new_protected:Npn \__enumext_printkeyans:nnn #1 #2 #3
5539 {
5540     \seq_if_exist:cTF { g__enumext_#3_seq }
5541     {
5542         \seq_if_empty:cF { g__enumext_#3_seq }
5543     }

```

If the *starred argument* `*` is present we will check that the environment `enumext*` is not saved in the *sequence*, then execute the variable `\l__enumext_print_keyans_starred_tl` that contains the default $\langle keys \rangle$ for the environment `enumext*`, we set `\l__enumext_base_line_fix_bool` and `\l__enumext_print_keyans_star_bool` to true for *baseline correction*, open the `enumext*` environment passing the *optional argument* and map the *sequence*, then set `\l__enumext_base_line_fix_bool` and `\l__enumext_print_keyans_star_bool` to false.

```

5544         \bool_if:nTF {#1}
5545         {
5546             \seq_if_in:cnTF { g__enumext_#3_seq } { \end{enumext*} }
5547             {
5548                 \msg_error:nnnn { enumext } { print-starred } {#3} { enumext* }
5549             }
5550         }

```

```

5551         \tl_use:N \l__enumext_print_keyans_starred_tl
5552         \bool_set_true:N \l__enumext_base_line_fix_bool
5553         \bool_set_true:N \l__enumext_print_keyans_star_bool
5554         \begin{enumext*}[#2]
5555             \seq_map_inline:cn { g__enumext_#3_seq } { ##1 }
5556         \end{enumext*}
5557         \bool_set_false:N \l__enumext_base_line_fix_bool
5558         \bool_set_false:N \l__enumext_print_keyans_star_bool
5559     }
5560 }

```

Otherwise it will open the environment `enumext` passing the *optional argument* to the “first level” then map the *sequence*.

```

5561     {
5562         \begin{enumext}[#2]
5563             \seq_map_inline:cn { g__enumext_#3_seq } { ##1 }
5564         \end{enumext}
5565     }
5566 }
5567 }
5568 {
5569     \msg_error:nnn { enumext } { undefined-storage-anskey } {#3}
5570 }
5571 }

```

(End of definition for `\printkeyans` and `\__enumext_printkeyans:nnn`. This function is documented on page 20.)

13.51 The command `\setenumext`

The command `\setenumext` will be in charge of managing the `<keys>` passed to all environments and to the `\printkeyans` command. We must take precautions with the `enumext*` and `enumext` environments so as not to capture `<keys>` that complicate us.

```

\__enumext_filter_level:n
  \__enumext_filter_level_key:n
  \__enumext_filter_level_pair:nn

```

The function `\__enumext_filter_level:n` will be in charge of filtering the `<keys>` passed to the `enumext` and `enumext*` environments.

```

5572 \cs_new:Npn \__enumext_filter_level:n #1
5573 {
5574     \use:e
5575     {
5576         \keyval_parse:NNn
5577         \__enumext_filter_level_key:n
5578         \__enumext_filter_level_pair:nn {#1}
5579     }
5580 }

```

The function `\__enumext_filter_level_key:n` will be responsible for filtering the `<keys>` that are passed “without value” by excluding the keys `resume*`, `reset` and `reset*` passed to the `enumext` and `enumext*` environments.

```

5581 \cs_new:Npn \__enumext_filter_level_key:n #1
5582 {
5583     \str_case:nnF {#1}
5584     {
5585         { resume* } {} { reset } {} { reset* } {}
5586     }
5587     { , { \exp_not:n {#1} } }
5588 }

```

The function `\__enumext_filter_level_pair:nn` will be responsible for filtering the `<keys>` that are passed “with value” by excluding the `series`, `resume` and `save-ans` keys passed to the `enumext` and `enumext*` environments.

```

5589 \cs_new:Npn \__enumext_filter_level_pair:nn #1#2
5590 {
5591     \str_case:nnF {#1}
5592     {
5593         { series } {} { save-ans } {} { resume } {}
5594     }
5595     { , { \exp_not:n {#1} } = { \exp_not:n {#2} } }
5596 }

```

(End of definition for `\__enumext_filter_level:n`, `\__enumext_filter_level_key:n`, and `\__enumext_filter_level_pair:nn`.)

Now define a “meta families” of `<keys>` to access from `\setenumext`.

```

5597 \keys_define:nn { enumext / meta-families }
5598 {
5599   enumext-1 .code:n = {
5600     \keys_set:ne { enumext / level-1 }
5601     {
5602       \__enumext_filter_level:n {#1}
5603     }
5604   },
5605   enumext-2 .code:n = {
5606     \keys_set:ne { enumext / level-2 }
5607     {
5608       \__enumext_filter_level:n {#1}
5609     }
5610   },
5611   enumext-3 .code:n = {
5612     \keys_set:ne { enumext / level-3 }
5613     {
5614       \__enumext_filter_level:n {#1}
5615     }
5616   },
5617   enumext-4 .code:n = {
5618     \keys_set:ne { enumext / level-4 }
5619     {
5620       \__enumext_filter_level:n {#1}
5621     }
5622   },
5623   enumext* .code:n = {
5624     \keys_set:ne { enumext / enumext* }
5625     {
5626       \__enumext_filter_level:n {#1}
5627     }
5628   },
5629   keyans .code:n = { \keys_set:nn { enumext / keyans } {#1} },
5630   keyans* .code:n = { \keys_set:nn { enumext / keyans* } {#1} },
5631   print* .code:n = { \keys_set:nn { enumext / print } { print* = {#1} } },
5632   print-1 .code:n = { \keys_set:nn { enumext / print } { print-1 = {#1} } },
5633   print-2 .code:n = { \keys_set:nn { enumext / print } { print-2 = {#1} } },
5634   print-3 .code:n = { \keys_set:nn { enumext / print } { print-3 = {#1} } },
5635   print-4 .code:n = { \keys_set:nn { enumext / print } { print-4 = {#1} } },
5636   print-* .code:n = { \keys_set:nn { enumext / print } { print-* = {#1} } },
5637   unknown .code:n = { \msg_error:nn { enumext } { unknown-key-family } },
5638 }

```

We store them in the constant sequence `\c__enumext_all_families_seq` separated by commas.

```

5639 \seq_const_from_clist:Nn \c__enumext_all_families_seq
5640 {
5641   enumext-1, enumext-2, enumext-3, enumext-4, keyans, enumext*,
5642   keyans*, print-1, print-2, print-3, print-4, print-*, print*,
5643 }

```

\setenumext

Now we define the user command `\setenumext`.

```

\__enumext_set_parse:n
\__enumext_set_error:nn
5644 \NewDocumentCommand \setenumext { 0{enumext,1} +m }
5645 {
5646   \seq_clear:N \l__enumext_setkey_tmpa_seq
5647   \seq_set_from_clist:Nn \l__enumext_setkey_tmpb_seq {#1}
5648   \int_set:Nn \l__enumext_setkey_tmpa_int
5649   {
5650     \seq_count:N \l__enumext_setkey_tmpb_seq
5651   }
5652   \int_compare:nNnTF { \l__enumext_setkey_tmpa_int } > { 1 }
5653   {
5654     \seq_pop_left:NN \l__enumext_setkey_tmpb_seq \l__enumext_setkey_tmpa_tl
5655     \seq_map_function:NN \l__enumext_setkey_tmpb_seq \__enumext_set_parse:n
5656     \seq_set_map_e:NNn \l__enumext_setkey_tmpa_seq \l__enumext_setkey_tmpa_seq
5657     {
5658       \tl_use:N \l__enumext_setkey_tmpa_tl - #1
5659     }
5660   }
5661   {
5662     \seq_put_right:Ne \l__enumext_setkey_tmpa_seq { \tl_trim_spaces:n {#1} }
5663   }

```

```

5664 \seq_if_empty:NTF \l__enumext_setkey_tmpa_seq
5665 { \seq_map_inline:Nn \c__enumext_all_families_seq }
5666 { \seq_map_inline:Nn \l__enumext_setkey_tmpa_seq }
5667 {
5668   \keys_set:nn { enumext / meta-families } { ##1 = {#2} }
5669 }
5670 }

```

Internal functions used by the `\setenumext` command.

```

5671 \cs_new_protected:Npn \__enumext_set_parse:n #1
5672 {
5673   \tl_set:Nc \l__enumext_setkey_tmpb_tl { \tl_trim_spaces:n {#1} }
5674   \clist_map_inline:nn { 0, 1, 2, 3, 4, * } % <- max level
5675   { \tl_remove_all:Nn \l__enumext_setkey_tmpb_tl {##1} }
5676   \tl_if_empty:NTF \l__enumext_setkey_tmpb_tl
5677   {
5678     \seq_put_right:Nc \l__enumext_setkey_tmpa_seq
5679     { \tl_trim_spaces:n {#1} }
5680   }
5681   { \__enumext_set_error:nn {#1} { } }
5682 }
5683 \cs_new_protected:Npn \__enumext_set_error:nn #1 #2
5684 { \msg_error:nnnn { enumext } { invalid-key } {#1} {#2} }

```

(End of definition for `\setenumext`, `\__enumext_set_parse:n`, and `\__enumext_set_error:nn`. This function is documented on page 6.)

13.52 The command `\setenumextmeta`

The command `\setenumextmeta` will be responsible for adding new “meta-keys” for the `enumext` and `enumext*` environments. The implementation code was given by Jonathan P. Spratte (@Skillmon) answer in Simplify syntax for command that adds .meta key to existing keys (l3keys).

`\setenumextmeta`

First we will create a `<keys>` of type `.code:n` for “all levels” of the `enumext` environment.

```

5685 \int_step_inline:nn { 4 }
5686 {
5687   \keys_define:nn { enumext }
5688   {
5689     #1 .code:n = \str_set:Nn \l__enumext_meta_path_str { level-#1 }
5690     ,#1 .value_forbidden:n = true
5691   }
5692 }

```

And now we define the `<keys>` for the environments using `.code:n` for the `enumext` environment and `.meta:n` for the `enumext*` environment.

```

5693 \clist_map_inline:nn { enumext }
5694 {
5695   \keys_define:nn { enumext }
5696   {
5697     #1 .code:n = % ignored for now, might do something useful in the future
5698     ,#1 .value_forbidden:n = true
5699     ,#1* .code:n = \str_set:Nn \l__enumext_meta_path_str { #1* }
5700     ,#1* .value_forbidden:n = true
5701   }
5702 }
5703 \keys_define:nn { enumext }
5704 {
5705   * .meta:n = enumext*
5706   ,* .value_forbidden:n = true
5707 }

```

Now we create the user command taking care that unknown cannot be passed as an argument.

```

5708 \NewDocumentCommand \setenumextmeta { s O{enumext,1} m +m }
5709 {
5710   \str_if_eq:eeTF { \tl_trim_spaces:n {#3} } { unknown }
5711   { \msg_error:nn { enumext } { prohibited-unknown } }
5712   {
5713     \bool_if:nTF {#1}
5714     {
5715       \int_step_inline:nn { 4 }
5716       { \__enumext_key_set_meta:nnn { enumext, ##1 } {#3} {#4} }
5717       \__enumext_key_set_meta:nnn { enumext* } {#3} {#4}
5718     }

```



```

5719         { \__enumext_key_set_meta:nnn {#2} {#3} {#4} }
5720     }
5721 }

```

The internal functions `\__enumext_key_set_meta:nnn` and `\__enumext_key_def_meta:nnn` will check the *optional argument* and create the “*meta-key*”.

```

5722 \cs_new_protected:Npn \__enumext_key_set_meta:nnn #1
5723 {
5724     \keys_set:nn { enumext } {#1}
5725     \__enumext_key_def_meta:Vnn \l__enumext_meta_path_str
5726 }
5727 \cs_new_protected:Npn \__enumext_key_def_meta:nnn #1#2#3
5728 {
5729     \bool_lazy_or:nnTF
5730     { \keys_if_exist_p:nn { enumext / #1} {#2} }
5731     { \keys_if_exist_p:nn { enumext / enumext* } {#2} }
5732     { \msg_error:nnn { enumext } { already-defined } {#2} }
5733     {
5734         \keys_define:nn { enumext / #1 }
5735         {
5736             #2 .meta:n = {#3},
5737             #2 .value_forbidden:n = true
5738         }
5739     }
5740 }
5741 \cs_generate_variant:Nn \__enumext_key_def_meta:nnn { V }

```

(End of definition for `\setenumextmeta`, `\__enumext_key_set_meta:nnn`, and `\__enumext_key_def_meta:nnn`. This function is documented on page 6.)

13.53 The command `\foreachkeyans`

The command `\foreachkeyans` will execute a *loop* over the *prop list* and return its contents. The implementation code is adapted from the answer provided by Enrico Gregorio (@egreg) in [Expand a .cs defined by key inside the function](#).

`\foreachkeyans`

```

\__enumext_parse_foreach_keys:nn
\__enumext_parse_foreach_keys:n
\__enumext_foreach_keyans:nn
\__enumext_foreach_add_body:n

```

We define a set of *⟨keys⟩* for command and we will save the default values of these in `\g__enumext_foreach_default_keys_tl` to avoid the use of group.

```

5742 \keys_define:nn { enumext / foreach }
5743 {
5744     before .tl_set:N = \l__enumext_foreach_before_tl,
5745     before .value_required:n = true,
5746     after .tl_set:N = \l__enumext_foreach_after_tl,
5747     after .value_required:n = true,
5748     start .int_set:N = \l__enumext_foreach_start_int,
5749     start .value_required:n = true,
5750     stop .int_set:N = \l__enumext_foreach_stop_int,
5751     stop .value_required:n = true,
5752     step .int_set:N = \l__enumext_foreach_step_int,
5753     step .value_required:n = true,
5754     wrapper .cs_set_protected:Np = \__enumext_foreach_wrapper:n #1,
5755     wrapper .value_required:n = true,
5756     sep .tl_set:N = \l__enumext_foreach_sep_tl,
5757     sep .value_required:n = true,
5758     unknown .code:n = { \__enumext_parse_foreach_keys:n {#1} }
5759 }
5760 \keys_precompile:nnN { enumext / foreach }
5761 {
5762     before={},after={},start=1,step=1,stop=0,wrapper=#1,sep={; }
5763 }
5764 \l__enumext_foreach_default_keys_tl

```

Functions for handling unknown *⟨keys⟩*.

```

5765 \cs_new_protected:Npn \__enumext_parse_foreach_keys:nn #1#2
5766 {
5767     \tl_if_blank:nTF {#2}
5768     {
5769         \msg_error:nnn { enumext } { for-key-unknown } {#1}
5770     }
5771     {
5772         \msg_error:nnnn { enumext } { for-key-value-unknown } {#1} {#2}
5773     }
5774 }

```

```

5775 \cs_new_protected:Npn \__enumext_parse_foreach_keys:n #1
5776 {
5777   \exp_args:NV \__enumext_parse_foreach_keys:nn \l_keys_key_str {#1}
5778 }

```

We create the command.

```

5779 \NewDocumentCommand \foreachkeyans { +0{} m }
5780 {
5781   \__enumext_foreach_keyans:nn {#1} {#2}
5782 }

```

Finally the internal functions `\__enumext_foreach_keyans:nn` and `\__enumext_foreach_add_body:n` will loop through the prop list and print the contents.

```

5783 \cs_new_protected:Npn \__enumext_foreach_keyans:nn #1 #2
5784 {
5785   \tl_use:N \l__enumext_foreach_default_keys_tl
5786   \keys_set:nn { enumext / foreach } {#1}
5787   \tl_set:Nn \l__enumext_foreach_name_prop_tl {#2}
5788   \prop_if_exist:cF { g__enumext_#2_prop }
5789   {
5790     \msg_error:nnn { enumext } { undefined-storage-anskey } {#2}
5791   }
5792   \int_compare:nNnT { \l__enumext_foreach_stop_int } = { 0 }
5793   {
5794     \int_set:Nn \l__enumext_foreach_stop_int
5795     { \prop_count:c { g__enumext_#2_prop } }
5796   }
5797   \seq_clear:N \l__enumext_foreach_print_seq
5798   \int_step_function:nnnN
5799   { \l__enumext_foreach_start_int }
5800   { \l__enumext_foreach_step_int }
5801   { \l__enumext_foreach_stop_int }
5802   \__enumext_foreach_add_body:n
5803   \seq_use:NV \l__enumext_foreach_print_seq \l__enumext_foreach_sep_tl
5804 }
5805 \cs_new_protected:Npn \__enumext_foreach_add_body:n #1
5806 {
5807   \seq_put_right:Ne \l__enumext_foreach_print_seq
5808   {
5809     \exp_not:V \l__enumext_foreach_before_tl
5810     \__enumext_foreach_wrapper:n
5811     {
5812       \prop_item:cn { g__enumext_ \l__enumext_foreach_name_prop_tl _prop }{#1}
5813     }
5814     \exp_not:V \l__enumext_foreach_after_tl
5815   }
5816 }

```

(End of definition for `\foreachkeyans` and others. This function is documented on page 19.)

13.54 Messages

Message used by package-load for `multicol` and `hyperref` packages.

```

5817 \msg_new:nnn { enumext } { package-load }
5818 {
5819   The~'#1'~package~is~already~loaded.
5820 }
5821 \msg_new:nnn { enumext } { package-not-load }
5822 {
5823   The~'#1'~package~will~be~loaded~as~a~dependency.
5824 }

```

Message used in the creation of counters by `enumext` package.

```

5825 \msg_new:nnn { enumext } { counters }
5826 {
5827   The~counter~'#1'~is~already~defined~by~some~\\
5828   package~or~macro,~it~cannot~be~continued.
5829 }

```

Message used by `align` and `mark-pos` keys.

```

5830 \msg_new:nnn { enumext } { unknown-choice }
5831 {
5832   The~value~'#3'~for~'#1'~key~is~invalid~use~('#2').
5833 }

```

Message used by reserved `anskey*` environment by `enumext` package.

```
5834 \msg_new:nnn { enumext } { anskey-env-error }
5835 {
5836   The~environment~'#1'~is~reserved~by ~\
5837   'enumext'~package,~It~is~already~defined.
5838 }
5839 {
5840   The~environment~'#1'~is~defined~internally ~
5841   for~the~'save-ans'~key~with~save-ans~key~active.~See~documentation.\
5842 }
```

Message used in the creation of *prop list* by `enumext` package.

```
5843 \msg_new:nnn { enumext } { store-prop }
5844 {
5845   *~Package~enumext:~Creating ~
5846   \c_backslash_str g__enumext_#1_prop~\msg_line_context:.
5847 }
5848 \msg_new:nnn { enumext } { store-seq }
5849 {
5850   *~Package~enumext:~Creating ~
5851   \c_backslash_str g__enumext_#1_seq~\msg_line_context:.
5852 }
5853 \msg_new:nnn { enumext } { store-int }
5854 {
5855   *~Package~enumext:~Creating ~
5856   \c_backslash_str g__enumext_resume_#1_int~\msg_line_context:.
5857 }
5858 \msg_new:nnn { enumext } { prop-seq-int-hook }
5859 {
5860   *~Package~enumext:~Elements~in ~
5861   \c_backslash_str g__enumext_#1_prop~=#2.\
5862   *~Package~enumext:~Elements~in ~
5863   \c_backslash_str g__enumext_#1_seq~=#3.\
5864   *~Package~enumext:~Value~off ~
5865   \c_backslash_str g__enumext_resume_#1_int~=#4.
5866 }
5867 \msg_new:nnn { enumext } { item-answer-hook }
5868 {
5869   *~Package~enumext:~Value~off ~
5870   \c_backslash_str g__enumext_item_number_int~=#1.\
5871   *~Package~enumext:~Value~off ~
5872   \c_backslash_str g__enumext_item_anskey_int~=#2.\
5873   *~Package~enumext:~Difference~item_number_int~-~item_anskey_int~=#3.
5874 }
```

Message used by `[⟨key = val⟩]` system and `\setenumext` command.

```
5875 \msg_new:nnn { enumext } { invalid-key }
5876 {
5877   The~key~'#1'~is~not~know~the~level~#2.
5878 }
5879 \msg_new:nnn { enumext } { unknown-key-family }
5880 {
5881   Unknown~key~family~`\l_keys_key_str'~for~enumext.
5882 }
```

Messages used in length calculation.

```
5883 \msg_new:nnn { enumext } { width-negative }
5884 {
5885   Ignoring~negative~value~'#1=#2'~\msg_line_context:.\
5886   The~key~'#1'~ accepts~values ~>=~0pt.
5887 }
```

Messages used by `show-length` key in `enumext`.

```
5888 \msg_new:nnn { enumext } { list-lengths }
5889 {
5890   ****~Lengths~used~by~'enumext'~level~'#2'~\msg_line_context:~\c_space_tl ****\
5891   \__enumext_show_length:nnn { dim } { labelsep } {#1}
5892   \__enumext_show_length:nnn { dim } { labelwidth } {#1}
5893   \__enumext_show_length:nnn { dim } { itemindent } {#1}
5894   \__enumext_show_length:nnn { dim } { leftmargin } {#1}
5895   \__enumext_show_length:nnn { dim } { rightmargin } {#1}
5896   \__enumext_show_length:nnn { dim } { listparindent } {#1}
5897   \__enumext_show_length:nnn { skip } { topsep } {#1}
```

```

5898     \__enumext_show_length:nnn { skip } { parsep } {#1}
5899     \__enumext_show_length:nnn { skip } { partopsep } {#1}
5900     \__enumext_show_length:nnn { skip } { itemsep } {#1}
5901     *****
5902 }

```

Messages used by `show-length` key in `enumext*`, `keyans*` and `keyans`.

```

5903 \msg_new:nnn { enumext } { list-lengths-not-nested }
5904 {
5905     ****~Lengths~used~by~'#2'~environment~\msg_line_context:~\c_space_tl ****\\
5906     \__enumext_show_length:nnn { dim } { labelsep } {#1}
5907     \__enumext_show_length:nnn { dim } { labelwidth } {#1}
5908     \__enumext_show_length:nnn { dim } { itemindent } {#1}
5909     \__enumext_show_length:nnn { dim } { leftmargin } {#1}
5910     \__enumext_show_length:nnn { dim } { rightmargin } {#1}
5911     \__enumext_show_length:nnn { dim } { listparindent } {#1}
5912     \__enumext_show_length:nnn { skip } { topsep } {#1}
5913     \__enumext_show_length:nnn { skip } { parsep } {#1}
5914     \__enumext_show_length:nnn { skip } { partopsep } {#1}
5915     \__enumext_show_length:nnn { skip } { itemsep } {#1}
5916     *****
5917 }

```

Messages used by `ref` key.

```

5918 \msg_new:nnn { enumext } { key-ref-empty }
5919 {
5920     Key~'ref'~need~a~value~in~'#1'~ \msg_line_context:.
5921 }

```

Messages used by `save-ans` key.

```

5922 \msg_new:nnn { enumext } { save-ans-empty }
5923 {
5924     Key~'save-ans'~need~a~value~in~'#1'~ \msg_line_context:.
5925 }
5926 \msg_new:nnn { enumext } { save-ans-log }
5927 {
5928     *~Package~enumext:~Start~#1\c_space_tl with~save-ans=#2~\msg_line_context:.
5929 }
5930 \msg_new:nnn { enumext } { save-ans-log-hook }
5931 {
5932     *~Package~enumext:~Stop~#1\c_space_tl with~save-ans=#2~\msg_line_context:.
5933 }

```

Messages used by the internal system to check answer used by `check-ans` key.

```

5934 \msg_new:nnn { enumext } { items-same-answer }
5935 {
5936     *****\\
5937     *~Package~enumext:~Checking~answers~in~'#1' ~
5938     for~\c_left_brace_str #2 \c_right_brace_str\\
5939     *~started~#3~and~close~\msg_line_context: : ~
5940     'OK',~all~items~with~answer.\\
5941     *****
5942 }
5943 \msg_new:nnn { enumext } { item-greater-answer }
5944 {
5945     Checking~answers~in~'#1'~for~\c_left_brace_str #2 \c_right_brace_str\\
5946     started~#3~and~close~\msg_line_context: : ~'NOT~OK'\\
5947     Items~>~Answers.
5948 }
5949 \msg_new:nnn { enumext } { item-less-answer }
5950 {
5951     Checking~answers~in~'#1'~for~\c_left_brace_str #2 \c_right_brace_str\\
5952     started~#3~and~close~\msg_line_context: : ~'NOT~OK'\\
5953     Items~<~Answers.
5954 }

```

Messages used by the internal system to check for “starred” `\item*` and `\anspic*` commands.

```

5955 \msg_new:nnn { enumext } { missing-starred }
5956 {
5957     Missing~'\c_backslash_str #1'~#2.
5958 }
5959 \msg_new:nnn { enumext } { many-starred }
5960 {

```

```

5961     Many~'\c_backslash_str #1'~#2.
5962 }

```

Messages used by `\printkeyans*` command.

```

5963 \msg_new:nnn { enumext } { print-starred }
5964 {
5965     \c_backslash_str printkeyans*:~ The~sequence~'#1'~already~contains ~
5966     #2~environment~ \msg_line_context:.
5967 }

```

Message for the nesting depth of the environment `enumext`.

```

5968 \msg_new:nnn { enumext } { list-too-deep }
5969 {
5970     Too~deep~nesting ~for~'enumext'~\msg_line_context:~ \\
5971     The~maximum ~level ~of ~nesting ~is~4.
5972 }

```

Messages used by `\anskey`, `anskey*` and `\anspic` commands.

```

5973 \msg_new:nnn { enumext } { anskey-unnumber-item }
5974 {
5975     Can't~store~with~a~unnumbered~\c_backslash_str item~\msg_line_context:.
5976 }
5977 \msg_new:nnn { enumext } { anskey-empty-arg }
5978 {
5979     Can't~store~empty~content~\msg_line_context:.
5980 }
5981 \msg_new:nnn { enumext } { anskey-wrong-place }
5982 {
5983     Wrong~place~for~command~'\c_backslash_str #1'~\msg_line_context:~ \\
5984     '\c_backslash_str #1'~works~in~the~environment~'#2'.
5985 }
5986 \msg_new:nnn { enumext } { anskey-nested }
5987 {
5988     The~command~\c_backslash_str anskey~ can't~be~nested~\msg_line_context:.
5989 }
5990 \msg_new:nnn { enumext } { anskey-math-mode }
5991 {
5992     #1~can't~work~in~math~mode~\msg_line_context:.
5993 }
5994 \msg_new:nnn { enumext } { anskey-env-wrong }
5995 {
5996     The~environment~anskey*~cannot~use~in~'#1'~\msg_line_context:.
5997 }
5998 \msg_new:nnn { enumext } { command-wrong-place }
5999 {
6000     Wrong~place~for~command~'\c_backslash_str #1'~\msg_line_context:~ \\
6001     '\c_backslash_str #1'~works~outside~the~environment~'#2'.
6002 }
6003 \msg_new:nnnn { enumext } { anskey-env-key-unknown }
6004 {
6005     The~key~'#1'~is~unknown~by~environment~
6006     'anskey*'~and~is~being~ignored.
6007 }
6008 {
6009     The~environment~'anskey*'~does~not~have~a~key~called ~'#1'.\\
6010     Check~that~you~have~spelled~the~key~name~correctly.
6011 }
6012 \msg_new:nnnn { enumext } { anskey-env-key-value-unknown }
6013 {
6014     The~key~'#1=#2'~is~unknown~by~environment ~
6015     'anskey*'~and~is~being~ignored.
6016 }
6017 {
6018     The~environment~'anskey*'~does~not~have~a~key~called ~'#1'.\\
6019     Check~that~you~have~spelled~the~key~name~correctly.
6020 }
6021 \msg_new:nnnn { enumext } { anskey-cmd-key-unknown }
6022 { The~key~'#1'~is~unknown~by~'\c_backslash_str anskey'~and~is~being~ignored.}
6023 {
6024     The~command ~'\c_backslash_str anskey'~does~not~have~a~key~called ~'#1'.\\
6025     Check~that~you~have~spelled~the~key~name~correctly.
6026 }

```

```

6027 \msg_new:nnnn { enumext } { anskey-cmd-key-value-unknown }
6028 { The~key~'#1=#2'~is~unknown~by~'\c_backslash_str anskey'~and~is~being~ignored. }
6029 {
6030   The~command~'\c_backslash_str anskey'~does~not~have~a~key~called ~'#1'.\\
6031   Check~that~you~have~spelled~the~key~name~correctly.
6032 }
6033 \msg_new:nnn { enumext } { overwrite-file }
6034 {
6035   Overwriting~file~'#1'.
6036 }
6037 \msg_new:nnn { enumext } { writing-file }
6038 {
6039   Writing~file~'#1'.
6040 }
6041 \msg_new:nnn { enumext } { not-writing }
6042 {
6043   File~'#1'~already~exists.~Not~writing.
6044 }

```

Messages used by **keyans**, **keyans*** and **keyanspic** environment.

```

6045 \msg_new:nnn { enumext } { keyans-nested }
6046 {
6047   The~environment~'keyans'~can't~be ~nested ~\msg_line_context:.
6048 }
6049 \msg_new:nnn { enumext } { keyans-wrong-level }
6050 {
6051   Wrong~level~position~for~'keyans'~\msg_line_context:.~ \\
6052   The~environment~'keyans'~can~only~be~in~the~first~level.
6053 }
6054 \msg_new:nnn { enumext } { wrong-place }
6055 {
6056   Wrong~place~for~'#1'~environment ~\msg_line_context:.~ \\
6057   '#1'~is~only~found~with~'#2'~ in ~ 'enumext.
6058 }
6059 \msg_new:nnn { enumext } { keyanspic-nested }
6060 {
6061   The~environment~'keyanspic'~can't~be ~nested~ \msg_line_context:.~.
6062 }
6063 \msg_new:nnn { enumext } { keyanspic-wrong-level }
6064 {
6065   Wrong~level~position~for~'keyanspic'~\msg_line_context:.~ \\
6066   The~environment~'keyans'~can~only~be~in~the~first~level.
6067 }
6068 \msg_new:nnn { enumext } { keyanspic-item-cmd }
6069 {
6070   Can't~use ~\c_backslash_str item~in~keyanspic~\msg_line_context:.
6071 }
6072 \msg_new:nnnn { enumext } { keyans-unknown-key }
6073 {
6074   The~key~'#1'~is~unknown~by~environment~
6075   '\l__enumext_envir_name_tl'~and~is~being~ignored.
6076 }
6077 {
6078   The~environment~'\l__enumext_envir_name_tl'~does~not
6079   ~have~a~key~called ~'#1'.\\
6080   Check~that~you~have~spelled~the~key~name~correctly.
6081 }
6082 \msg_new:nnnn { enumext } { keyans-unknown-key-value }
6083 {
6084   The~key~'#1=#2'~is~unknown~by~environment ~
6085   '\l__enumext_envir_name_tl'~and~is~being~ignored.
6086 }
6087 {
6088   The~environment~'\l__enumext_envir_name_tl'~does~not
6089   ~have~a~key~called ~'#1'.\\
6090   Check~that~you~have~spelled~the~key~name~correctly.
6091 }

```

Message used by unknown **⟨keys⟩** in **enumext***. environment.

```

6092 \msg_new:nnnn { enumext } { starred-unknown-key }
6093 {
6094   The~key~'#1'~is~unknown~by~environment~

```

```

6095     '\l__enumext_envir_name_tl'~and~is~being~ignored.
6096   }
6097   {
6098     The~environment~'\l__enumext_envir_name_tl'~does~not
6099     ~have~a~key~called ~'#1'.\\
6100     Check~that~you~have~spelled~the~key~name~correctly.
6101   }
6102   \msg_new:nnnn { enumext } { starred-unknown-key-value }
6103   {
6104     The~key~'#1=#2'~is~unknown~by~environment ~
6105     '\l__enumext_envir_name_tl'~and~is~being~ignored.
6106   }
6107   {
6108     The~environment~'\l__enumext_envir_name_tl'~does~not
6109     ~have~a~key~called ~'#1'.\\
6110     Check~that~you~have~spelled~the~key~name~correctly.
6111   }

```

Message used by unknown *⟨keys⟩* in `enumext` environment.

```

6112   \msg_new:nnnn { enumext } { standar-unknown-key }
6113   {
6114     The~key~'#1'~is~unknown~by~environment~'\l__enumext_envir_name_tl' \c_space_tl
6115     ~on~level~\int_use:N \l__enumext_level_int \c_space_tl and~is~being~ignored.
6116   }
6117   {
6118     The~environment~'\l__enumext_envir_name_tl'~does~not
6119     ~have~a~key~called ~'#1'~on~level~\int_use:N \l__enumext_level_int.\\
6120     Check~that~you~have~spelled~the~key~name~correctly.
6121   }
6122   \msg_new:nnnn { enumext } { standar-unknown-key-value }
6123   {
6124     The~key~'#1=#2'~is~unknown~by~environment~'\l__enumext_envir_name_tl' \c_space_tl
6125     ~on~level~\int_use:N \l__enumext_level_int \c_space_tl and~is~being~ignored.
6126   }
6127   {
6128     The~environment~'\l__enumext_envir_name_tl'~does~not
6129     ~have~a~key~called ~'#1'~on~level~\int_use:N \l__enumext_level_int.\\
6130     Check~that~you~have~spelled~the~key~name~correctly.
6131   }

```

Message used by unknown *⟨keys⟩* in `\foreachkeyans`.

```

6132   \msg_new:nnnn { enumext } { for-key-unknown }
6133   { The~key~'#1'~is~unknown~by~'\c_backslash_str foreachkeyans'~and~is~being~ignored. }
6134   {
6135     The~command~'\c_backslash_str foreachkeyans'~does~not~have~a~key~called~'#1'.\\
6136     Check~that~you~have~spelled~the~key~name~correctly.
6137   }
6138   \msg_new:nnnn { enumext } { for-key-value-unknown }
6139   { The~key~'#1=#2'~is~unknown~by~'\c_backslash_str foreachkeyans'~and~is~being~ignored. }
6140   {
6141     The~command~'\c_backslash_str foreachkeyans'~does~not~have~a~key~called~'#1'.\\
6142     Check~that~you~have~spelled~the~key~name~correctly.
6143   }

```

Messages used by `\getkeyans` command.

```

6144   \msg_new:nnn { enumext } { undefined-storage-anskey }
6145   {
6146     Storage~named~'#1'~is~not~defined~\msg_line_context:.
6147   }

```

Messages used by `\miniright` command.

```

6148   \msg_new:nnn { enumext } { missing-miniright }
6149   {
6150     Missing~'\c_backslash_str miniright'~in~\msg_line_context:.\\
6151     The~key~'mini-env'~need~'\c_backslash_str miniright'.
6152   }
6153   \msg_new:nnn { enumext } { wrong-miniright-place }
6154   {
6155     Wrong~place~for~'\c_backslash_str miniright'~\msg_line_context:.~ \\
6156     Works~in~'enumext'~and~'keyans'~with~key~'mini-env'.
6157   }
6158   \msg_new:nnn { enumext } { wrong-miniright-use }
6159   {

```



```

6160     Wrong-use-for~'\c_backslash_str miniright'~\msg_line_context:~ \\
6161     '\c_backslash_str miniright'~need~a~key~'mini-env'.
6162 }
6163 \msg_new:nnn { enumext } { wrong-miniright-starred }
6164 {
6165     Can't-use ~\c_backslash_str miniright~in~starred~environments~\msg_line_context:.
6166 }
6167 \msg_new:nnn { enumext } { many-miniright-used }
6168 {
6169     Can't-use ~\c_backslash_str miniright~more~than~once~ \msg_line_context:.
6170 }

```

Messages used by `\setenumextmeta` command.

```

6171 \msg_new:nnn { enumext } { already-defined }
6172 {
6173     The~key~'#1'~is~already~defined~\msg_line_context:.
6174 }
6175 \msg_new:nnn { enumext } { prohibited-unknown }
6176 {
6177     The~name~'unknown'~can't~be~chosen~ for~a~meta~key~\msg_line_context:.
6178 }

```

Messages used by `enumext*` and `keyans*` environments.

```

6179 \msg_new:nnn { enumext } { nested }
6180 {
6181     The~environment~\l__enumext_envir_name_tl \c_space_tl can't~be~nested~\msg_line_context:.
6182 }
6183 \msg_new:nnn { enumext } { nested-horizontal }
6184 {
6185     The~environment~\l__enumext_envir_name_tl \c_space_tl can't~be~nested~in~'#1'~ \msg_line_context:.
6186 }
6187 \msg_new:nnn { enumext } { item-joined }
6188 {
6189     Items~joined~(#1)~>~#2 ~columns ~\msg_line_context:.
6190 }
6191 \msg_new:nnn { enumext } { item-joined-columns }
6192 {
6193     Not~space~to~join~items~(#1)~>~#2 ~\msg_line_context:.
6194 }

```

Messages used by `resume` key.

```

6195 \msg_new:nnn { enumext } { unknown-series-starred }
6196 {
6197     The~series~'#1'~for~the~resume~key~does~not~exist~in~the~
6198     ~enumext*~environment~ \msg_line_context:.
6199 }
6200 \msg_new:nnn { enumext } { unknown-series-standar }
6201 {
6202     The~series~'#1'~for~the~resume~key~does~not~exist~at~level~\int_use:N \l__enumext_level_int
6203     \c_space_tl of~enumext~environment~ \msg_line_context:.
6204 }
6205 \msg_new:nnnn { enumext } { out-of-range }
6206 { The~number~must~be~exactly~1,~2,~3~or~4. }
6207 { Received:~'#1'. }

```

13.55 Finish package

Finish package implementation.

```

6208 \file_input_stop:
6209 </package>

```

14 Index of Implementation

The *italic* numbers denote the pages where the corresponding entry is described, the numbers underlined and all others indicate the line on which they are implemented in the package code.

Symbols	
<code>\+</code>	221
<code>\-</code>	221
<code>\\</code>	229, 4580, 4583, 5827, 5836, 5841, 5861, 5863, 5870, 5872, 5885, 5890, 5905, 5936, 5938, 5940, 5945, 5946, 5951, 5952, 5970, 5983, 6000, 6009, 6018, 6024, 6030, 6051, 6056, 6065, 6079, 6089, 6099, 6109, 6119, 6129, 6135, 6141, 6150, 6155, 6160
A	
<code>above</code>	<u>1724</u>
<code>above*</code>	<u>1724</u>
<code>\addvspace</code>	1291, 1319, 1362, 1365, 1533, 1536, 1633, 1639, 1677, 1683, 1704, 1710, 4014, 4178, 4196, 4465, 4469, 4828, 4843, 4889, 4903
<code>after</code>	<u>1121</u>
<code>align</code>	<u>664</u>
<code>\Alph</code>	44, 48, 49
<code>\Alph</code>	606, 734, 775, 835, 5516
<code>\alph</code>	44, 48, 49
<code>\alph</code>	607, 732, 5504
<code>\anskey</code>	14, 88, 90, <u>3039</u>
<code>anskey*</code>	15, <u>3169</u>
<code>\anspic</code>	18, <u>116</u> , <u>119</u> , <u>4479</u>
<code>\anspic*</code>	81
<code>\arabic</code>	44
<code>\arabic</code>	605, 731, 774, 5492, 5498, 5522
B	
<code>base-fix</code>	<u>979</u>
<code>\baselineskip</code>	<u>58</u>
<code>\baselineskip</code>	995, 1006
<code>before</code>	<u>1121</u>
<code>before*</code>	<u>1121</u>
<code>beginpenalty</code>	<u>919</u>
<code>below</code>	<u>1724</u>
<code>below*</code>	<u>1724</u>
bool commands:	
<code>\bool_gset_false:N</code>	340, 341, 342, 4845, 4849, 4905
<code>\bool_gset_true:N</code>	250, 260, 1224, 2472, 2478, 4814, 4846, 4878, 4906
<code>\bool_if:NTF</code>	390, 400, 417, 491, 498, 507, 514, 528, 541, 1746, 1760, 1773, 1784, 1795, 1806, 1817, 1828, 1842, 1858, 1877, 1924, 1966, 2001, 2041, 2043, 2054, 2417, 2660, 2670, 2750, 2774, 2781, 2805, 2903, 2925, 2965, 2989, 2993, 3043, 3062, 3086, 3138, 3142, 3172, 3190, 3209, 3225, 3248, 3279, 3294, 3366, 3482, 3516, 3552, 3568, 3589, 3735, 3756, 3802, 3846, 3856, 3891, 3896, 3921, 3930, 3969, 3995, 4045, 4063, 4106, 4161, 4186, 4403, 4463, 4481, 4500, 4551, 4578, 4807, 4823, 4829, 4872, 4886, 4890, 4953, 4961, 4982, 4989, 4999, 5087, 5093, 5100, 5116, 5209, 5219, 5333, 5340, 5371, 5387, 5418
<code>\bool_if:nTF</code>	1684, 1711, 2236, 3538, 3714, 4521, 5544, 5713
<code>\bool_if_p:N</code>	269, 283, 989, 990, 1002, 1003, 1656, 1977, 1978, 1992, 1993, 2064, 2172, 2173, 2187, 2188, 2430, 2456, 2469, 2470, 2475, 2476, 2838, 2848, 2860, 2875, 2876, 2910, 2951, 2952, 3353, 3354, 3383, 3384, 3396, 3397, 3437, 3438, 3457, 3458, 3748, 3749, 3750, 3942, 3944, 3955, 5362, 5363, 5364
<code>\bool_lazy_all:nTF</code>	267, 281, 987, 2428, 2454, 2836, 2845, 2858, 2873, 3435, 3455, 3746, 3940, 3953, 5360
<code>\bool_lazy_and:nnTF</code>	246, 256, 1001, 1648, 1655, 1976, 1991, 2063, 2171, 2186, 2249, 2468, 2474, 2909, 2916, 2950, 3352
<code>\bool_lazy_or:nnTF</code>	2358, 2365, 3382, 3395, 5729
<code>\bool_new:N</code>	22, 23, 24, 25, 26, 27, 28, 47, 50, 51, 52, 62, 86, 91, 92, 97, 98, 101, 108, 123, 124, 136, 137, 144, 150, 151, 153, 157, 159, 160, 177, 189, 191
<code>\bool_not_p:n</code>	247, 257, 991, 1657, 2847, 2911, 2917, 3943, 3956
<code>\bool_set_eq:NN</code>	1848, 1861, 3491, 3695, 5040, 5284
<code>\bool_set_false:N</code>	397, 864, 1013, 2402, 2403, 2435, 2440, 2444, 2448, 2461, 3723, 3910, 4062, 4114, 4201, 4339, 4400, 4541, 4948, 4981, 5037, 5225, 5280, 5281, 5535, 5557, 5558
<code>\bool_set_true:N</code>	274, 288, 383, 386, 657, 1028, 1730, 1735, 1847, 1860, 1863, 2144, 2151, 2375, 2376, 2692, 2700, 3113, 3485, 3487, 3519, 3521, 3691, 3702, 3716, 3869, 3909, 3949, 3962, 4035, 4111, 4138, 4336, 4523, 4524, 4796, 4861, 4947, 5044, 5051, 5052, 5096, 5223, 5288, 5295, 5296, 5297, 5528, 5552, 5553
box commands:	
<code>\box_dp:N</code>	1579, 1580, 1583, 1590, 1603, 1611, 1617, 1625, 4414, 4420, 4465, 4562
<code>\box_ht:N</code>	1362, 1365, 1376, 1377, 1388, 1390, 1405, 1408, 1416, 1417, 1428, 1430, 1445, 1448, 1455, 1456, 1467, 1469, 1484, 1487, 1533, 1536, 1544, 1545, 1553, 1554, 1566, 1568
<code>\box_ht_plus_dp:N</code>	4409, 4473, 4509
<code>\box_new:N</code>	59, 146, 147, 184, 190
<code>\box_use_drop:N</code>	4840, 4901, 5152, 5430
<code>\box_wd:N</code>	613
<code>break-col</code>	<u>3009</u> , <u>3095</u>
C	
<code>\c</code>	870, 872, 884, 886
c@ internal commands:	
<code>\c@__enumext_resume_i_int</code>	<u>591</u>
<code>\c@__enumext_resume_ii_int</code>	<u>591</u>
<code>\c@__enumext_resume_iii_int</code>	<u>591</u>
<code>\c@__enumext_resume_iv_int</code>	<u>591</u>
<code>\c@__enumext_resume_vii_int</code>	<u>591</u>
<code>\centering</code>	1686, 1713, 4606, 4833, 4894
<code>check-ans</code>	<u>2394</u>
Document class:	
<code>article</code>	50
clist commands:	
<code>\clist_const:Nn</code>	196
<code>\clist_map_function:nN</code>	4589
<code>\clist_map_inline:Nn</code>	663, 918, 934, 1120, 1135, 1216, 1740
<code>\clist_map_inline:nn</code>	36, 45, 55, 67, 75, 88, 100, 139, 168, 195, 599, 641, 694, 714, 1033, 1054, 1230, 1870, 2276, 2283, 2298, 2342, 2408, 2587, 2657, 2689, 2833, 3288, 3610, 3625, 3672, 3831, 3834, 3836, 3864, 3876, 3879, 3881, 3901, 5674, 5693
<code>\columnbreak</code>	88

<code>\columnbreak</code>	2913
<code>columns</code>	1200
<code>columns-sep</code>	1200
<code>\columnsep</code>	109
<code>\columnsep</code>	3990, 4159
<code>\columnseprule</code>	109
<code>\columnseprule</code>	3993, 4160
Commands provide by enumext :	
<code>\anskey</code> 33, 77, 78, 83–87, 89, 90, 95, 108, 109, 128, 138, 139, 147	
<code>\anspic*</code> 33, 34, 81, 84, 95, 96, 118, 119, 138, 139	
<code>\anspic</code>	34, 85, 116, 119, 147
<code>\foreachkeyans</code>	143, 149
<code>\getkeyans</code>	84, 138, 149
<code>\item*</code> 33, 34, 81, 84, 85, 95, 96, 99, 103, 130, 135, 138, 139	
<code>\item</code>	99, 103, 123, 129, 131, 134, 135
<code>\miniright</code>	32, 55, 63, 64, 110, 111, 149
<code>\printkeyans*</code>	138
<code>\printkeyans</code>	33, 85, 138, 139
<code>\resetenumext</code>	74
<code>\setenumextmeta</code>	142, 150
<code>\setenumext</code>	33, 139–142, 145
Counters defined by enumext :	
<code>enumXiii</code>	31, 43
<code>enumXii</code>	31, 43
<code>enumXiv</code>	31, 43
<code>enumXi</code>	31, 43
<code>enumXviii</code>	31, 43
<code>enumXvii</code>	31, 43, 130
<code>enumXvi</code>	31, 43
<code>enumXv</code>	31, 43
<code>\counterwithin</code>	2270, 2271
cs commands:	
<code>\cs_generate_variant:Nn</code> . 201, 202, 615, 631, 876, 892, 2273, 2742, 2747, 2823, 3168, 3821, 4591, 5741	
<code>\cs_if_exist:NTF</code>	576, 593
<code>\cs_if_exist_p:N</code>	3751, 5365
<code>\cs_new:Nn</code>	215
<code>\cs_new:Npn</code> . 225, 2011, 2020, 2028, 2704, 2713, 2721, 5572, 5581, 5589	
<code>\cs_new_eq:NN</code> . 367, 368, 373, 374, 402, 403, 406, 407	
<code>\cs_new_protected:Nn</code> . 231, 239, 265, 296, 326, 332, 338, 344, 350, 358, 378, 425, 429, 447, 459, 477, 489, 505, 521, 534, 555, 751, 808, 855, 985, 1136, 1140, 1144, 1148, 1152, 1156, 1160, 1164, 1168, 1172, 1176, 1180, 1184, 1188, 1192, 1196, 1231, 1243, 1276, 1293, 1304, 1321, 1347, 1368, 1493, 1519, 1539, 1572, 1594, 1629, 1635, 1741, 1755, 1769, 1780, 1791, 1802, 1813, 1824, 1875, 1893, 1922, 1937, 1962, 2061, 2157, 2203, 2306, 2316, 2330, 2343, 2348, 2373, 2413, 2423, 2466, 2481, 2488, 2497, 2502, 2507, 2512, 2521, 2526, 2531, 2748, 2772, 2779, 2803, 2810, 2824, 3060, 3079, 3188, 3207, 3238, 3277, 3292, 3320, 3350, 3378, 3391, 3404, 3433, 3446, 3524, 3534, 3545, 3561, 3577, 3710, 3728, 3762, 3774, 3902, 3938, 3967, 3974, 4004, 4021, 4043, 4068, 4104, 4128, 4145, 4170, 4184, 4205, 4375, 4573, 4587, 4592, 4616, 4626, 4657, 4786, 4805, 4851, 4870, 4934, 4968, 4975, 4987, 4997, 5022, 5163, 5207, 5238, 5244, 5265, 5321, 5441	
<code>\cs_new_protected:Npn</code> 203, 207, 211, 410, 574, 600, 610, 616, 735, 776, 840, 862, 877, 1668, 1697, 2037, 2070, 2113, 2140, 2260, 2264, 2268, 2274, 2281, 2353, 2536, 2658, 2668, 2690, 2698, 2734, 2743, 2899, 2962, 2987, 3025, 3029, 3122, 3126, 3159, 3218, 3257, 3330, 3371, 3478, 3497, 3626, 3630, 3647, 3651, 3673, 3677,	

3687, 3699, 3744, 3790, 3824, 3867, 3913, 4124, 4384, 4391, 4398, 4498, 4517, 4547, 4688, 4737, 4951, 5028, 5035, 5049, 5057, 5062, 5072, 5231, 5271, 5278, 5293, 5302, 5316, 5358, 5463, 5476, 5538, 5671, 5683, 5722, 5727, 5765, 5775, 5783, 5805	
<code>\cs_new_protected_nopar:Nn</code> . . . 4266, 4308, 4316, 4324, 5007, 5015, 5146, 5250, 5258, 5424	
<code>\cs_new_protected_nopar:Npn</code> . . 4258, 4274, 5078, 5124, 5377, 5398	
<code>\cs_set:Npn</code> 1964, 2159, 2205, 2318, 2834, 2871, 5469	
<code>\cs_set_eq:NN</code> . . 3754, 4924, 4925, 5126, 5196, 5197, 5368, 5400	
<code>\cs_set_protected:Nn</code> 1059, 1075, 1088, 1100	
<code>\cs_set_protected:Npn</code> 32, 39, 48, 60, 68, 83, 89, 132, 164, 175, 591, 632, 642, 664, 699, 715, 758, 893, 919, 935, 1015, 1038, 1112, 1121, 1200, 1217, 1724, 1835, 2288, 2334, 2394, 2553, 2588, 2676, 2826, 3281, 3599, 3615, 3661, 3822, 3865	
<code>\cs_to_str:N</code>	602, 625

D

<code>\d</code>	221
<code>\DeclareDocumentEnvironment</code>	559
dim commands:	
<code>\dim_abs:n</code>	3795, 3800
<code>\dim_add:Nn</code>	3430, 4418, 4651, 4682
<code>\dim_compare:nNnTF</code> . . 1061, 1077, 1090, 1102, 1380, 1392, 1420, 1432, 1459, 1471, 1548, 1556, 1670, 1699, 2967, 2975, 3425, 3792, 3797, 3803, 3809, 3811, 3813, 3979, 4026, 4132, 4149, 4393, 4628, 4644, 4659, 4675, 4788, 4853, 5329	
<code>\dim_compare:nTF</code>	2935, 4071, 4208
<code>\dim_eval:n</code>	995, 4471, 4558
<code>\dim_gset_eq:NN</code>	4797, 4862
<code>\dim_gzero:N</code>	4848, 4908
<code>\dim_new:N</code> . 56, 63, 64, 65, 85, 128, 129, 141, 148, 149, 183, 185, 186, 192	
<code>\dim_set:Nn</code> . 613, 1029, 2969, 2977, 3412, 3416, 3421, 3427, 3514, 3795, 3800, 3802, 3805, 3806, 3810, 3812, 3815, 3816, 3818, 3982, 4029, 4070, 4134, 4151, 4207, 4407, 4507, 4594, 4630, 4637, 4661, 4668, 4723, 4772, 4790, 4855, 5074, 5331	
<code>\dim_set_eq:NN</code> 722, 765, 832, 3509, 3833, 3878, 3990, 4159, 4730, 4733, 4734, 4779, 4782, 4783, 5067, 5138, 5412	
<code>\dim_sub:Nn</code>	4076, 4213, 4646, 4677
<code>\dim_use:N</code> . 1062, 1070, 1671, 1681, 2813, 2816, 2821, 2979, 3529, 3531, 3584, 3980, 3984, 3985, 3987, 4027, 4032, 4033, 4039, 4073, 4078	
<code>\dim_zero:N</code>	3870, 3993, 4160, 4421
<code>\dim_zero_new:N</code>	573
<code>\c_zero_dim</code> 1064, 1078, 1091, 1103, 1671, 1699, 2937, 2967, 2975, 3412, 3425, 3792, 3797, 3803, 3810, 3980, 4027, 4073, 4132, 4149, 4210, 4393, 4628, 4644, 4659, 4675, 4788, 4853, 5329	
<code>\dimeval</code>	2622

E

<code>\end</code> . . . 2776, 2807, 4011, 4175, 4456, 4608, 5546, 5556, 5564	
end internal commands:	
<code>\end__enumext_mini_page</code> . 1679, 1706, 4054, 4195, 4812, 4876, 4902	
<code>\endlist</code>	368
<code>\endminipage</code>	374
<code>endpenalty</code>	919
<code>enumext</code>	5, 4082

enumext internal commands:

__enumext_add_pre_parsep: . [56](#), [1241](#), [1243](#), [1243](#)
 __enumext_after_args_exec: [54](#), [1136](#), [1148](#), [4095](#)
 __enumext_after_args_exec_v: [1152](#), [1164](#), [4228](#)
 __enumext_after_args_exec_vii: . . [1168](#), [1192](#)
 __enumext_after_args_exec_viii: [1196](#)
 __enumext_after_env:nn [94](#), [112](#), [125](#), [132](#), [207](#), [207](#),
 [547](#), [551](#), [4100](#), [4821](#), [4884](#), [5179](#)
 __enumext_after_hyperref: . . . [39](#), [376](#), [376](#), [378](#)
 \l__enumext_after_list_args_v_tl [1166](#)
 \l__enumext_after_list_args_vii_tl [1194](#), [5144](#)
 \l__enumext_after_list_args_viii_tl . . [1198](#),
 [5422](#)
 __enumext_after_list_vii: [125](#), [128](#), [4932](#), [4975](#),
 [4975](#)
 __enumext_after_list_viii: . . [134](#), [5205](#), [5244](#),
 [5244](#)
 __enumext_after_stop_list: [54](#), [111](#), [1136](#), [1144](#),
 [4059](#)
 __enumext_after_stop_list_v: [1152](#), [1160](#), [4202](#)
 \l__enumext_after_stop_list_v_tl [1162](#)
 __enumext_after_stop_list_vii: . . [128](#), [1168](#),
 [1184](#), [4978](#)
 \l__enumext_after_stop_list_vii_tl . . . [1186](#)
 __enumext_after_stop_list_viii: . [1188](#), [5247](#)
 \l__enumext_after_stop_list_viii_tl . . . [1190](#)
 \l__enumext_align_label_pos_v_str [3408](#), [3780](#)
 \l__enumext_align_label_pos_X_str [68](#)
 \l__enumext_align_label_vii_str [5113](#)
 \l__enumext_align_label_viii_str [5391](#)
 \l__enumext_align_label_X_str [175](#)
 \c__enumext_all_envs_clist . . [196](#), [663](#), [918](#), [934](#),
 [1120](#), [1135](#), [1216](#), [1740](#)
 \c__enumext_all_families_seq . . [141](#), [5639](#), [5665](#)
 __enumext_anskey_env_file_if_writable:n [92](#),
 [3136](#), [3136](#)
 __enumext_anskey_env_file_if_
 writable:nTF [3136](#), [3161](#)
 __enumext_anskey_env_file_write:nn [92](#), [3159](#),
 [3168](#), [3223](#)
 \l__enumext_anskey_env_force_eol_bool . . [94](#),
 [3109](#), [3225](#)
 \c__enumext_anskey_env_hidden_space_str [33](#),
 [94](#), [111](#), [3229](#)
 \l__enumext_anskey_env_overwrite_bool [3117](#),
 [3142](#)
 __enumext_anskey_env_safe_inner: . [93](#), [3183](#),
 [3188](#), [3207](#)
 __enumext_anskey_env_safe_inner:n [93](#)
 __enumext_anskey_env_safe_outer: . [93](#), [3171](#),
 [3188](#), [3188](#)
 __enumext_anskey_env_unknown:n [92](#), [3120](#), [3122](#),
 [3122](#)
 __enumext_anskey_env_unknown:nn . [3122](#), [3124](#),
 [3126](#)
 \l__enumext_anskey_level_int . . [16](#), [3081](#), [3082](#)
 __enumext_anskey_safe_inner: . [91](#), [3054](#), [3060](#),
 [3079](#)
 __enumext_anskey_safe_inner:n [90](#)
 __enumext_anskey_safe_outer: . [90](#), [3041](#), [3060](#),
 [3060](#)
 __enumext_anskey_show_wrap_arg:n . [89](#), [2962](#),
 [2962](#), [2991](#), [3006](#)
 __enumext_anskey_show_wrap_left:n [89](#), [2907](#),

[2987](#), [2987](#)
 __enumext_anskey_unknown:n [90](#), [3009](#), [3023](#), [3025](#)
 __enumext_anskey_unknown:nn . [3009](#), [3027](#), [3029](#)
 __enumext_anskey_wrapper:n [2619](#), [2985](#)
 \l__enumext_anspic_above_int . [140](#), [4595](#), [4596](#),
 [4598](#)
 __enumext_anspic_args:nnn [119](#), [120](#), [4479](#), [4495](#),
 [4573](#)
 \l__enumext_anspic_args_seq [119–121](#), [140](#), [4493](#),
 [4607](#), [4620](#)
 \l__enumext_anspic_below_int . [140](#), [4595](#), [4596](#),
 [4599](#)
 \l__enumext_anspic_body_box . . . [140](#), [4506](#), [4509](#)
 __enumext_anspic_body_dim:n . . [119](#), [4479](#), [4498](#),
 [4550](#)
 \l__enumext_anspic_body_htdp_dim . . [119](#), [140](#),
 [4507](#), [4561](#)
 __enumext_anspic_exec: [118](#), [121](#), [4451](#), [4479](#), [4616](#)
 __enumext_anspic_label:nn [120](#), [4479](#), [4517](#), [4553](#),
 [4568](#)
 \l__enumext_anspic_label_above_bool . . . [140](#),
 [4336](#), [4339](#), [4403](#), [4463](#), [4500](#), [4551](#), [4578](#)
 \l__enumext_anspic_label_box . . [140](#), [4406](#), [4409](#)
 \l__enumext_anspic_label_htdp_dim . [117](#), [140](#),
 [4407](#), [4413](#), [4473](#), [4560](#)
 __enumext_anspic_label_pos:nnn . . [120](#), [4479](#),
 [4547](#), [4576](#)
 \l__enumext_anspic_label_sep_skip [4346](#), [4415](#),
 [4474](#), [4563](#), [4580](#)
 \l__enumext_anspic_layout_style_tl [4348](#), [4618](#),
 [4623](#)
 \l__enumext_anspic_mini_pos_str . . [140](#), [4337](#),
 [4340](#), [4605](#)
 \l__enumext_anspic_mini_width_dim [140](#), [4519](#),
 [4594](#), [4605](#)
 __enumext_anspic_print:n [120](#), [121](#), [4479](#), [4587](#),
 [4591](#), [4620](#), [4623](#)
 __enumext_anspic_row:n . . [121](#), [4479](#), [4589](#), [4592](#)
 __enumext_anspic_start_list_tag: [4282](#), [4308](#),
 [4575](#)
 __enumext_anspic_stop_list_tag: . [4282](#), [4324](#),
 [4585](#)
 __enumext_anspic_stop_start_list_tag: [4282](#),
 [4316](#), [4577](#)
 __enumext_at_begin_document:n . . [39](#), [203](#), [203](#),
 [365](#), [371](#)
 \l__enumext_base_line_fix_bool [51](#), [139](#), [981](#), [990](#),
 [1013](#), [5552](#), [5557](#)
 __enumext_before_args_exec: [54](#), [110](#), [128](#), [1136](#),
 [1136](#), [4024](#)
 __enumext_before_args_exec_v: [1152](#), [1152](#), [4131](#)
 __enumext_before_args_exec_vii: . [1168](#), [1168](#),
 [4972](#)
 __enumext_before_args_exec_viii: [1172](#), [5241](#)
 __enumext_before_env:nn [207](#), [211](#)
 __enumext_before_keys_exec: . . [54](#), [1136](#), [1140](#),
 [4092](#)
 __enumext_before_keys_exec_v: [1152](#), [1156](#), [4225](#)
 __enumext_before_keys_exec_vii [1168](#)
 __enumext_before_keys_exec_vii: . [1176](#), [4919](#)
 __enumext_before_keys_exec_viii: [1180](#), [5191](#)
 __enumext_before_list: . . [110](#), [4021](#), [4021](#), [4086](#)
 __enumext_before_list_v: . . . [4128](#), [4128](#), [4220](#)
 __enumext_before_list_vii: . . . [128](#), [4914](#), [4968](#),
 [4968](#)

```

\__enumext_before_list_viii: .. 133, 5187, 5238,
    5238
\l__enumext_before_no_starred_key_v_tl 1158
\l__enumext_before_no_starred_key_vii_-
    tl ..... 1178
\l__enumext_before_no_starred_key_viii_-
    tl ..... 1182
\l__enumext_before_starred_key_v_tl ... 1154
\l__enumext_before_starred_key_vii_tl . 1170
\l__enumext_before_starred_key_viii_tl 1174
\__enumext_calc_hspace:NNNNNN 106, 3790, 3790,
    3821, 3826, 3871
\__enumext_check_ans_active: 78, 110, 128, 2413,
    2413, 4025, 4971
\g__enumext_check_ans_item_tl ..... 96
\g__enumext_check_ans_key_bool 79, 80, 150, 340,
    2472, 2478, 3248
\l__enumext_check_ans_key_bool 79, 2398, 2403,
    2469, 2475
\__enumext_check_ans_key_hook: .. 79, 111, 128,
    2466, 2466, 4060, 4979
\__enumext_check_ans_level: . 78, 79, 2413, 2419,
    2423
\__enumext_check_ans_log: 80, 94, 2512, 2512, 3252
\__enumext_check_ans_log_msg_greater: 2512,
    2518, 2531
\__enumext_check_ans_log_msg_less: 2512, 2516,
    2521
\__enumext_check_ans_log_msg_same_ok: 2512,
    2517, 2526
\__enumext_check_ans_msg_greater: 2488, 2494,
    2507
\__enumext_check_ans_msg_less: 2488, 2492, 2497
\__enumext_check_ans_msg_same_ok: 2488, 2493,
    2502
\__enumext_check_ans_show: .. 80, 94, 2488, 2488,
    3250
\l__enumext_check_answers_bool 77, 78, 90, 93, 99,
    150, 2376, 2402, 2417, 2750, 2774, 2781, 2805, 3043,
    3172, 3366, 3482, 3516, 5093
\__enumext_check_starred_cmd:n 37, 81, 96, 132,
    2536, 2536, 4231, 4461, 5204
\g__enumext_check_starred_cmd_int . 103, 150,
    2539, 2545, 2550, 3708, 4529, 5328
\l__enumext_check_start_line_env_tl . 37, 150,
    303, 311, 319, 2542, 2548, 2551
\l__enumext_columns_sep_v_dim 4149, 4151, 4159
\l__enumext_columns_sep_vii_dim .. 4628, 4630,
    4639, 4651, 4727, 5160
\l__enumext_columns_sep_viii_dim . 4659, 4661,
    4670, 4682, 4776, 5438
\l__enumext_columns_v_int 1513, 1531, 1702, 4147,
    4155, 4167, 4172
\l__enumext_columns_vii_int .. 4633, 4636, 4640,
    4649, 4691, 4695, 4698, 4704, 4710, 4714, 5154, 5168
\l__enumext_columns_viii_int . 4664, 4667, 4671,
    4680, 4740, 4744, 4747, 4753, 4759, 4763, 5432, 5447
\l__enumext_counter_i_tl ..... 32, 583
\l__enumext_counter_ii_tl ..... 32, 584
\l__enumext_counter_iii_tl ..... 32, 585
\l__enumext_counter_iv_tl ..... 32, 586
\g__enumext_counter_styles_tl . 31, 44, 56, 603,
    621
\l__enumext_counter_v_tl ..... 32, 587
\l__enumext_counter_vi_tl ..... 32, 588

\l__enumext_counter_vii_tl ..... 32, 589
\l__enumext_counter_viii_tl ..... 32, 590
\l__enumext_current_widest_dim 31, 56, 627, 723,
    766, 833
\__enumext_default_item:n ... 3478, 3478, 3542
\__enumext_define_counter:Nn . 31, 574, 574, 583,
    584, 585, 586, 587, 588, 589, 590
\__enumext_endminipage: . 39, 365, 374, 568, 4842,
    5148, 5426
\g__enumext_envir_name_tl 37, 22, 275, 289, 348,
    2346, 2351, 2361, 2500, 2505, 2510, 2524, 2529, 2534
\l__enumext_envir_name_tl . 36, 37, 102, 22, 245,
    255, 302, 310, 318, 3620, 3643, 3667, 4371, 6075, 6078,
    6085, 6088, 6095, 6098, 6105, 6108, 6114, 6118, 6124,
    6128, 6181, 6185
\__enumext_execute_after_env: .. 38, 76, 80, 94,
    3238, 3238, 4102, 5181
\__enumext_fake_item_indent: . 1059, 1059, 3855
\l__enumext_fake_item_indent_v_dim 1078, 1083
\l__enumext_fake_item_indent_v_tl 1080, 3692,
    3696, 3703
\__enumext_fake_item_indent_vii: . 1059, 1088,
    3890
\l__enumext_fake_item_indent_vii_dim . 1091,
    1095
\l__enumext_fake_item_indent_vii_tl .. 1093,
    5143
\__enumext_fake_item_indent_viii: 1059, 1100,
    3895
\l__enumext_fake_item_indent_viii_dim 1103,
    1107
\l__enumext_fake_item_indent_viii_tl . 1105,
    5417
\l__enumext_fake_item_indent_X_tl ..... 89
\__enumext_fake_make_label_vii:n . 130, 5078,
    5078, 5135
\__enumext_fake_make_label_viii:n 5358, 5377,
    5409
\__enumext_filter_level:n 140, 5572, 5572, 5602,
    5608, 5614, 5620, 5626
\__enumext_filter_level_key:n 140, 5572, 5577,
    5581
\__enumext_filter_level_pair:nn .. 140, 5572,
    5578, 5589
\__enumext_filter_save_key:n .. 84, 2665, 2673,
    2696, 2702, 2704, 2704, 5489, 5495, 5501, 5507, 5513,
    5519
\__enumext_filter_save_key_key:n .. 84, 2704,
    2709, 2713
\__enumext_filter_save_key_pair:nn 84, 2704,
    2710, 2721
\__enumext_filter_series:n 70, 2011, 2011, 2048,
    2057, 2092, 2105
\__enumext_filter_series_key:n 70, 2011, 2016,
    2020
\__enumext_filter_series_pair:nn .. 70, 2011,
    2017, 2028
\__enumext_first_item_tmp_vii: 126, 129, 4924,
    5007, 5007
\__enumext_first_item_tmp_viii: .. 134, 5196,
    5250, 5250
\g__enumext_footnote_standar_arg_seq .. 169,
    442, 453, 456
\g__enumext_footnote_standar_int 169, 436, 439,
    441, 444

```


[\g__enumext_footnote_standar_int_seq .. 169, 444, 449, 452, 457](#)
[\g__enumext_footnote_starred_arg_seq .. 169, 472, 483, 486](#)
[\g__enumext_footnote_starred_int 169, 466, 469, 471, 474](#)
[\g__enumext_footnote_starred_int_seq .. 169, 474, 479, 482, 487](#)
[__enumext_footnotes_key_bool 39](#)
[\l__enumext_footnotes_key_bool 34, 39, 159, 386, 390, 397, 498, 514, 528, 541](#)
[__enumext_footnotetext:nn .. 425, 425, 454, 484](#)
[__enumext_foreach_add_body:n 144, 5742, 5802, 5805](#)
[\l__enumext_foreach_after_tl 5746, 5814](#)
[\l__enumext_foreach_before_tl 5744, 5809](#)
[\g__enumext_foreach_default_keys_tl ... 143](#)
[\l__enumext_foreach_default_keys_tl ... 118, 5764, 5785](#)
[__enumext_foreach_keyans:nn . 144, 5742, 5781, 5783](#)
[\l__enumext_foreach_name_prop_tl . 118, 5787, 5812](#)
[\l__enumext_foreach_print_seq 118, 5797, 5803, 5807](#)
[\l__enumext_foreach_sep_tl 5756, 5803](#)
[\l__enumext_foreach_start_int 5748, 5799](#)
[\l__enumext_foreach_step_int 5752, 5800](#)
[\l__enumext_foreach_stop_int . 5750, 5792, 5794, 5801](#)
[__enumext_foreach_wrapper:n 5754, 5810](#)
[__enumext_getkeyans:nn .. 138, 5458, 5472, 5476](#)
[__enumext_getkeyans_aux:n 138, 5458, 5460, 5463](#)
[\l__enumext_hyperref_bool 34, 39, 159, 383, 400, 417, 2952, 3354, 5087](#)
[__enumext_hypertarget:nn 39, 376, 402, 406, 422](#)
[__enumext_if_is_int:n 219](#)
[__enumext_if_is_int:nTF 219, 865, 879](#)
[__enumext_internal_mini_page: 42, 108, 127, 555, 555, 3905, 4937](#)
[__enumext_is_not_nested: . 31, 36, 108, 127, 239, 239, 3904, 4936](#)
[__enumext_is_on_first_level: . 31, 37, 108, 127, 239, 265, 3911, 4949](#)
[\g__enumext_item_anskey_int 90, 96, 150, 335, 362, 363, 2485, 2901, 3368](#)
[__enumext_item_answer_diff: 80, 94, 2481, 2481, 3245](#)
[\g__enumext_item_answer_diff_int 80, 150, 336, 2483, 2490, 2514](#)
[\l__enumext_item_column_pos_vii_int 129, 4698, 4704, 4710, 4714, 4721, 5018, 5154, 5157](#)
[\l__enumext_item_column_pos_viii_int .. 134, 4747, 4753, 4759, 4763, 4770, 5261, 5432, 5435](#)
[l__enumext_item_column_pos_X_int 175](#)
[\g__enumext_item_count_all_vii_int 129, 4722, 5019, 5168, 5176](#)
[\g__enumext_item_count_all_viii_int 134, 4771, 5262, 5446, 5455](#)
[\g__enumext_item_count_all_X_int 175](#)
[\g__enumext_item_number_bool 150](#)
[\l__enumext_item_number_bool 79, 157, 2435, 2440, 2444, 2448, 2461, 3086, 3209, 3485, 3519, 5096](#)
[\g__enumext_item_number_int .. 79, 150, 334, 361, 363, 2434, 2439, 2443, 2447, 2460, 2485, 3484, 3518, 5095](#)
[__enumext_item_peek_args_vii: 129, 5015, 5020, 5022](#)
[__enumext_item_peek_args_viii: .. 134, 5258, 5263, 5265](#)
[__enumext_item_starred_exec: . 100, 3497, 3524, 3566, 3587](#)
[__enumext_item_starred_exec:nn .. 3497, 3497, 3540](#)
[\l__enumext_item_starred_vii_bool 5037, 5051, 5100](#)
[\l__enumext_item_starred_viii_bool 5280, 5295, 5387, 5418](#)
[\l__enumext_item_starred_X_bool 175](#)
[__enumext_item_std:w . 39, 99, 103, 365, 369, 3488, 3494, 3522, 3692, 3696, 3703](#)
[\g__enumext_item_symbol_aux_tl . 99, 122, 3502, 3505, 3530, 3574, 3594](#)
[\g__enumext_item_symbol_aux_vii_tl 5059, 5102, 5105, 5109, 5111](#)
[\g__enumext_item_symbol_aux_X_tl 175](#)
[\l__enumext_item_symbol_sep_vii_dim .. 5067, 5074, 5108, 5110](#)
[\l__enumext_item_symbol_vii_tl 5105](#)
[\l__enumext_item_text_vii_box 5127, 5152](#)
[\l__enumext_item_text_viii_box ... 5401, 5430](#)
[\l__enumext_item_text_X_box 175](#)
[\l__enumext_item_width_vii_dim ... 4637, 4646, 4725, 4733, 4734](#)
[\l__enumext_item_width_viii_dim .. 4668, 4677, 4774, 4782, 4783](#)
[\l__enumext_item_width_X_dim 175](#)
[\l__enumext_item_wrap_key_bool 104, 150, 3438, 3458, 3716, 3723, 3750, 4523, 4541, 5281, 5296, 5364](#)
[\l__enumext_itemindent_X_dim 60](#)
[\l__enumext_itemsep_i_skip ... 1374, 1381, 1384, 1386, 1393, 1397, 1400, 1402, 1542, 1549, 1551, 1552, 1557, 1561, 1563, 1564](#)
[\l__enumext_itemsep_ii_skip .. 1414, 1421, 1424, 1426, 1433, 1437, 1440, 1442](#)
[\l__enumext_itemsep_iii_skip . 1453, 1460, 1463, 1465, 1472, 1476, 1479, 1481](#)
[\l__enumext_itemsep_vii_skip 5174](#)
[\l__enumext_itemsep_viii_skip 5453](#)
[\l__enumext_joined_item_aux_vii_int .. 4719, 4720, 4721, 4722, 4728](#)
[\l__enumext_joined_item_aux_viii_int . 4768, 4769, 4770, 4771, 4777](#)
[\l__enumext_joined_item_aux_X_int 175](#)
[__enumext_joined_item_vii:w .. 129, 5015, 5025, 5026, 5028](#)
[\l__enumext_joined_item_vii_int .. 4690, 4691, 4694, 4696, 4702, 4707, 4712, 4717, 4719, 4725](#)
[__enumext_joined_item_viii:w 134, 5258, 5268, 5269, 5271](#)
[\l__enumext_joined_item_viii_int . 4739, 4740, 4743, 4745, 4751, 4756, 4761, 4766, 4768, 4774](#)
[\l__enumext_joined_item_X_int 175](#)
[\l__enumext_joined_width_vii_dim . 4723, 4730, 4733, 5129, 5137](#)
[\l__enumext_joined_width_viii_dim 4772, 4779, 4782, 5403, 5411](#)
[\l__enumext_joined_width_X_dim 175](#)

```

\__enumext_key_def_meta:nnn .. 143, 5685, 5725,
    5727, 5741
\__enumext_key_set_meta:nnn .. 143, 5685, 5716,
    5717, 5719, 5722
\__enumext_keyans_addto_prop:n 95, 3257, 3257,
    3705, 4526
\__enumext_keyans_addto_seq:n . 96, 3330, 3330,
    3707, 4528
\__enumext_keyans_addto_seq_link: 3330, 3348,
    3350, 5327
\__enumext_keyans_default_item:n . 103, 3687,
    3687, 3724
\l__enumext_keyans_env_bool 22, 3943, 3956, 4111,
    4201
\__enumext_keyans_fake_item_indent: .. 1059,
    1075, 3845
\l__enumext_keyans_level_h_int .. 133, 16, 793,
    817, 3070, 3198, 3308, 4943, 5213, 5214
\l__enumext_keyans_level_int .. 16, 1662, 3066,
    3194, 3303, 3448, 4110, 4115, 4489
\__enumext_keyans_make_label: 104, 3728, 3728,
    3843
\__enumext_keyans_make_label_box: 3728, 3732,
    3737, 3774
\__enumext_keyans_make_label_std: 3728, 3740,
    3762
\__enumext_keyans_mini_right_cmd:n 64, 1664,
    1697, 1697
\__enumext_keyans_mini_set_vskip: ..... 61
\__enumext_keyans_minipage_add_space: 1493,
    1519, 4140
\__enumext_keyans_minipage_set_skip: . 1493,
    1493, 1521
\__enumext_keyans_multi_addvspace: 1293, 1304,
    4164
\__enumext_keyans_multi_set_vskip: 57, 1293,
    1293, 1306
\__enumext_keyans_multicols_start: 4128, 4143,
    4145
\__enumext_keyans_multicols_stop: 1701, 4128,
    4170, 4199
\__enumext_keyans_name_and_start: 31, 37, 133,
    296, 296, 4112, 4382, 5218
\__enumext_keyans_parse_keys:n 4124, 4124, 4219
\__enumext_keyans_pic_arg_two: 117, 4375, 4398,
    4429
\l__enumext_keyans_pic_level_int .. 16, 1643,
    3074, 3202, 3260, 3298, 3333, 4377, 4378
\__enumext_keyans_pic_parse_keys:n 4375, 4384,
    4428
\__enumext_keyans_pic_safe_exec: . 117, 4375,
    4375, 4427
\__enumext_keyans_pic_skip_abs:N . 117, 4375,
    4391, 4402
\__enumext_keyans_pos_mark_set: 98, 3404, 3404,
    3441, 3473
\__enumext_keyans_pre_itemsep_skip: .. 1493,
    1512, 1539
\__enumext_keyans_redefine_item: . 104, 3710,
    3710, 3842
\__enumext_keyans_ref: ..... 48, 840, 855, 3844
\__enumext_keyans_ref:n ..... 48, 837, 840, 840
\__enumext_keyans_safe_exec: . 4104, 4104, 4218
\__enumext_keyans_save_item_opt:n .. 97, 103,
    3371, 3371, 3701, 4525
\__enumext_keyans_set_item_width: 113, 4205,
    4205, 4227
\__enumext_keyans_show_ans: 98, 3404, 3433, 3767,
    3782, 4530
\__enumext_keyans_show_item_opt: 97, 103, 3371,
    3378, 3704, 4538
\__enumext_keyans_show_item_opt_viii: .. 97,
    3371, 3391, 5420
\__enumext_keyans_show_pos: 98, 3404, 3446, 3768,
    3783, 4531
\__enumext_keyans_starred_item:n . 103, 3699,
    3699, 3719
\__enumext_keyans_starred_item_star: .. 135,
    5293, 5321, 5389
\__enumext_keyans_store_ref: .. 95, 3277, 3277,
    3706, 4527, 5325
\__enumext_keyans_store_ref_aux_i: 95, 3277,
    3289, 3292
\__enumext_keyans_store_ref_aux_ii: 96, 3277,
    3318, 3320
\__enumext_keyans_unknown_keys:n . 3615, 3621,
    3626, 4372
\__enumext_keyans_unknown_keys:nn 3615, 3628,
    3630
\__enumext_keyans_wraper_label:n ..... 104
\__enumext_keyans_wraper_label_viii:n 5358,
    5358, 5394
\__enumext_keyans_wrapper_item_v:n 3751, 3754
\__enumext_keyans_wrapper_item_viii:n 5365,
    5369
\__enumext_keyans_wrapper_label:n 3728, 3744,
    3770, 3785, 4535
\__enumext_keyans_wrapper_opt_v:n .... 3386
\__enumext_keyans_wrapper_opt_viii:n .. 3399
\l__enumext_label_copy_i_tl .. 2867, 3296, 3301,
    3306, 3311
\l__enumext_label_copy_v_tl ..... 3306
\l__enumext_label_copy_vi_tl ..... 3301
\l__enumext_label_copy_vii_tl 2843, 2854, 2883,
    3296
\l__enumext_label_copy_viii_tl ..... 3311
\l__enumext_label_copy_X_tl ..... 161
\l__enumext_label_fill_left_v_tl ..... 3766
\l__enumext_label_fill_left_X_tl ..... 89
\l__enumext_label_fill_right_v_tl .... 3771
\l__enumext_label_fill_right_X_tl ..... 89
\l__enumext_label_font_style_v_tl 3769, 3784,
    4534, 4542
\l__enumext_label_font_style_vii_tl ... 5115
\l__enumext_label_font_style_viii_tl .. 5393
\l__enumext_label_i_tl ..... 715
\l__enumext_label_ii_tl ..... 715
\l__enumext_label_iii_tl ..... 715
\l__enumext_label_iv_tl ..... 715
\__enumext_label_style:Nnn 31, 44, 616, 616, 631,
    720, 763, 828, 830
\l__enumext_label_v_tl 96, 825, 3265, 3338, 3407,
    4222, 4406
\l__enumext_label_vi_tl 96, 825, 3262, 3335, 4535,
    4543
\l__enumext_label_vii_tl . 758, 5046, 5069, 5076
\l__enumext_label_viii_tl 758, 5290, 5319, 5323
\l__enumext_label_width_by_box .. 56, 612, 613
\__enumext_label_width_by_box:Nn 44, 610, 610,

```


615, 627, 889, 3406
 \l__enumext_labelsep_v_dim ... 3427, 4154, 4418, 4537
 \l__enumext_labelsep_vii_dim . 2969, 4632, 4642, 4726, 5011, 5067, 5122, 5131
 \l__enumext_labelsep_viii_dim 4663, 4673, 4775, 5254, 5331, 5396, 5405
 \l__enumext_labelwidth_v_dim . 833, 3417, 3422, 3443, 3475, 3780, 4154, 4418, 4532
 \l__enumext_labelwidth_vii_dim ... 2972, 4632, 4641, 4726, 5011, 5113, 5130
 \l__enumext_labelwidth_viii_dim .. 4663, 4672, 4775, 5254, 5338, 5355, 5391, 5404
 \l__enumext_leftmargin_tmp_v_bool . 117, 4400
 \l__enumext_leftmargin_tmp_X_bool 60
 \l__enumext_leftmargin_tmp_X_dim 60
 \l__enumext_leftmargin_X_dim 60
 __enumext_level: . 215, 215, 753, 755, 1062, 1066, 1070, 1138, 1142, 1146, 1150, 1233, 1235, 1237, 1239, 1281, 1283, 1285, 1287, 1291, 1325, 1331, 1336, 1338, 1341, 1344, 1357, 1360, 1671, 1675, 1681, 1744, 1746, 1748, 1751, 1758, 1760, 1762, 1765, 1898, 1899, 1908, 1914, 1917, 1918, 2041, 2043, 2045, 2047, 2089, 2091, 2094, 2097, 2117, 2121, 2144, 2660, 2662, 2664, 2692, 2693, 2695, 2752, 2760, 2764, 2768, 2979, 2983, 3487, 3488, 3492, 3493, 3494, 3502, 3510, 3511, 3514, 3521, 3522, 3526, 3529, 3531, 3565, 3567, 3568, 3570, 3573, 3584, 3585, 3588, 3589, 3591, 3949, 3962, 3969, 3977, 3980, 3982, 3984, 3985, 3986, 3987, 3990, 3995, 4001, 4007, 4014, 4027, 4029, 4032, 4033, 4035, 4039, 4045, 4073, 4078, 4089, 4091
 \l__enumext_level_h_int 127, 16, 248, 271, 284, 779, 810, 1650, 1989, 2052, 2079, 2100, 2127, 2149, 2184, 2222, 2431, 2451, 2862, 3957, 4938, 4939
 \l__enumext_level_int . 108, 16, 217, 258, 270, 285, 557, 1245, 1370, 1649, 1880, 1974, 1986, 2039, 2074, 2086, 2115, 2120, 2142, 2169, 2181, 2214, 2218, 2220, 2308, 2310, 2312, 2325, 2327, 2425, 2457, 2839, 2849, 2855, 2861, 2868, 2877, 2882, 3240, 3666, 3859, 3906, 3907, 3918, 3929, 3947, 3960, 3991, 4119, 4485, 4991, 5001, 5226, 6115, 6119, 6125, 6129, 6202
 __enumext_list_arg_two_i: 3822
 __enumext_list_arg_two_ii: 3822
 __enumext_list_arg_two_iii: 3822
 __enumext_list_arg_two_iv: 3822
 __enumext_list_arg_two_v: 104, 3822, 4224, 4401
 __enumext_list_arg_two_vii: 3865, 4918
 __enumext_list_arg_two_viii: 3865, 5190
 \l__enumext_listoffset_v_dim . 4156, 4210, 4213
 \l__enumext_listparindent_vii_dim 5138, 5142
 \l__enumext_listparindent_viii_dim 5412, 5416
 __enumext_log_answer_vars: . 38, 350, 358, 3247
 __enumext_log_global_vars: . 38, 350, 350, 3246
 __enumext_make_label: ... 100, 3545, 3545, 3853
 __enumext_make_label_box: ... 3545, 3549, 3554, 3577
 __enumext_make_label_std: ... 3545, 3557, 3561
 \l__enumext_mark_answer_sym_tl 86, 2604, 2818, 2995, 3429, 5335, 5342
 \l__enumext_mark_answer_sym_v_tl . 3429, 3461
 \l__enumext_mark_answer_sym_viii_tl ... 5335
 \l__enumext_mark_position_str 122, 2610, 2611, 2612, 2816, 3431, 5336, 5353
 \l__enumext_mark_position_v_str .. 122, 3431
 \l__enumext_mark_position_viii_str 122, 5336, 5353
 \l__enumext_mark_ref_sym_tl .. 2592, 2957, 3362
 \l__enumext_mark_sep_tmpa_dim 122, 3407, 3417, 3422
 \l__enumext_mark_sep_tmpb_dim 122, 3412, 3416, 3421, 3430
 \l__enumext_mark_sym_sep_dim . 2607, 2967, 2969, 2972, 2975, 2977
 \l__enumext_mark_sym_sep_v_dim ... 3425, 3427, 3430, 3443, 3475
 \l__enumext_mark_sym_sep_viii_dim 5329, 5331, 5338, 5355
 \l__enumext_meta_path_str 118, 5689, 5699, 5725
 __enumext_mini_addvspace_vii: 63, 1629, 1629, 4800
 __enumext_mini_addvspace_viii: 63, 1629, 1635, 4865
 __enumext_mini_env* 555
 __enumext_mini_page 1681, 1708, 4039, 4141, 4802, 4867, 4888
 __enumext_mini_right_cmd:n 64, 1666, 1668, 1668
 __enumext_mini_set_vskip_vii: 62, 1572, 1572, 1631
 __enumext_mini_set_vskip_viii: 62, 1572, 1594, 1637
 __enumext_minipage:w 39, 365, 373, 562, 4825, 5137, 5411
 \l__enumext_minipage_active_v_bool 4138, 4161, 4186
 \g__enumext_minipage_active_vii_bool .. 125, 4814, 4823, 4845
 \l__enumext_minipage_active_vii_bool . 4796, 4807
 \g__enumext_minipage_active_viii_bool 4878, 4886, 4905
 \l__enumext_minipage_active_viii_bool 4861, 4872
 \g__enumext_minipage_active_X_bool ... 175
 \l__enumext_minipage_active_X_bool 76
 __enumext_minipage_add_space: . 59, 110, 1321, 1347, 4037
 \g__enumext_minipage_after_skip 76, 1576, 1588, 4843, 4903
 \l__enumext_minipage_after_skip .. 58, 111, 76, 1334, 1374, 1376, 1381, 1384, 1388, 1393, 1397, 1400, 1404, 1416, 1421, 1424, 1428, 1433, 1437, 1440, 1444, 1455, 1460, 1463, 1467, 1472, 1476, 1479, 1483, 1495, 1509, 1542, 1544, 1549, 1551, 1553, 1557, 1561, 1563, 1565, 1596, 1609, 1623, 1677, 1704, 4196
 \g__enumext_minipage_center_vii_bool . 4829, 4846
 \g__enumext_minipage_center_viii_bool 4890, 4906
 \g__enumext_minipage_center_X_bool ... 175
 \l__enumext_minipage_hsep_v_dim 4136
 \l__enumext_minipage_hsep_vii_dim 4794
 \l__enumext_minipage_hsep_viii_dim ... 4859
 \l__enumext_minipage_left_skip 76, 1496, 1574, 1579, 1583, 1597, 1601, 1615, 1633, 1639
 \l__enumext_minipage_left_v_dim .. 4134, 4141
 \l__enumext_minipage_left_vii_dim 4790, 4802
 \l__enumext_minipage_left_viii_dim 4855, 4867
 \l__enumext_minipage_left_X_dim 76
 \g__enumext_minipage_right_skip 76, 1575, 1580,

1584, 4828, 4889
 \l__enumext_minipage_right_skip . 58, 76, 1323,
 1329, 1334, 1336, 1338, 1497, 1498, 1504, 1509, 1510,
 1511, 1516, 1598, 1605, 1619, 1683, 1710
 \l__enumext_minipage_right_v_dim . 1699, 1708,
 4132, 4136
 \g__enumext_minipage_right_vii_dim 124, 4798,
 4825, 4848
 \l__enumext_minipage_right_vii_dim 124, 4788,
 4793, 4799
 \g__enumext_minipage_right_viii_dim .. 4863,
 4888, 4908
 \l__enumext_minipage_right_viii_dim .. 4853,
 4858, 4864
 \g__enumext_minipage_right_X_dim 175
 \g__enumext_minipage_right_X_skip 175
 __enumext_minipage_set_skip: . 58, 1321, 1321,
 1349
 \g__enumext_minipage_stat_int .. 110, 76, 1688,
 1715, 4036, 4047, 4052, 4139, 4188, 4193
 \l__enumext_minipage_temp_skip 76, 1395, 1405,
 1408, 1435, 1445, 1448, 1474, 1484, 1487, 1559, 1566,
 1568
 \l__enumext_miniright_code_vii_box 4836, 4840
 \g__enumext_miniright_code_vii_tl 125, 4831,
 4838, 4847
 \l__enumext_miniright_code_viii_box .. 4897,
 4901
 \g__enumext_miniright_code_viii_tl 4892, 4899,
 4907
 \l__enumext_miniright_code_X_box 175
 \l__enumext_mode_box_bool 636, 3552, 3735
 __enumext_multi_addvspace: 57, 109, 1276, 1276,
 3998
 __enumext_multi_set_vskip: 56, 1231, 1231, 1278
 \l__enumext_multicols_above_ii_skip ... 1250
 \l__enumext_multicols_above_iii_skip .. 1259
 \l__enumext_multicols_above_iv_skip ... 1268
 \l__enumext_multicols_above_v_skip 1295, 1309,
 1319, 1510
 \l__enumext_multicols_above_X_skip 68
 \l__enumext_multicols_below_ii_skip .. 1377,
 1386, 1390, 1402, 1407
 \l__enumext_multicols_below_iii_skip . 1417,
 1426, 1430, 1442, 1447
 \l__enumext_multicols_below_iv_skip .. 1456,
 1465, 1469, 1481, 1486
 \l__enumext_multicols_below_v_skip 1299, 1313,
 1511, 1545, 1552, 1554, 1564, 1567, 4178
 \l__enumext_multicols_below_X_skip 68
 \g__enumext_multicols_right_X_skip 68
 __enumext_multicols_start: 109, 110, 3974, 3974,
 4041
 __enumext_multicols_stop: 110, 1673, 4004, 4004,
 4057
 __enumext_nested_base_line_fix: 51, 108, 979,
 985, 3925
 __enumext_newlabel:nn 34, 40, 87, 410, 410, 2893,
 3324
 \l__enumext_newlabel_arg_one_tl 34, 40, 87, 95,
 161, 2886, 2894, 2956, 3313, 3325, 3360
 \l__enumext_newlabel_arg_two_tl 34, 40, 86, 161,
 2842, 2852, 2865, 2880, 2895, 3300, 3305, 3310, 3326
 __enumext_parse_foreach_keys:n .. 5742, 5758,
 5775
 __enumext_parse_foreach_keys:nn . 5742, 5765,
 5777
 __enumext_parse_keys:n 51, 71, 3913, 3913, 4085
 __enumext_parse_keys_vii:n 71, 4913, 4951, 4951
 __enumext_parse_keys_viii:n . 5186, 5231, 5231
 __enumext_parse_save_key:n 84, 2685, 2690, 2690
 __enumext_parse_save_key_vii:n 84, 2680, 2690,
 2698
 __enumext_parse_series:n 67, 71, 108, 127, 2070,
 2070, 3923, 3932, 4963
 __enumext_parse_store_keys:n 108
 \l__enumext_parsep_i_skip 1248, 1252
 \l__enumext_parsep_ii_skip 1257, 1261
 \l__enumext_parsep_iii_skip 1266, 1270
 \l__enumext_parsep_vii_skip 5139
 \l__enumext_parsep_viii_skip 5413
 \l__enumext_partopsep_v_skip . 1311, 1315, 1506,
 1529
 \l__enumext_partopsep_viii_skip 1607
 __enumext_phantomsection: 39, 376, 403, 407, 423
 __enumext_pre_itemsep_skip: 58, 59, 1339, 1368,
 1368
 __enumext_print_footnote: .. 425, 447, 511, 516
 __enumext_print_footnote_mini: 425, 477, 538,
 543
 __enumext_print_footnote_standar: 489, 505,
 569
 __enumext_print_footnote_starred: 489, 534,
 549, 553
 __enumext_print_keyans_box:NN 86, 2810, 2810,
 2823, 2971, 2982, 3442, 3474, 5337, 5354
 \l__enumext_print_keyans_cmd_bool 122, 1842,
 1858, 3921, 3930, 4063, 4961, 4982, 5528, 5535
 \l__enumext_print_keyans_i_tl ... 5496, 5529
 \l__enumext_print_keyans_ii_tl ... 5502, 5530
 \l__enumext_print_keyans_iii_tl .. 5508, 5531
 \l__enumext_print_keyans_iv_tl ... 5514, 5532
 \l__enumext_print_keyans_star_bool . 51, 139,
 122, 991, 1003, 5553, 5558
 \l__enumext_print_keyans_starred_tl 138, 139,
 122, 5490, 5551
 \l__enumext_print_keyans_vii_tl 138, 5520, 5533
 \l__enumext_print_keyans_X_tl 122
 __enumext_printkeyans:nnn 139, 5525, 5534, 5538
 __enumext_redefine_item: 100, 3534, 3534, 3852
 \l__enumext_ref_key_arg_t 46
 \l__enumext_ref_key_arg_tl 37, 737, 738, 747, 778,
 781, 789, 795, 803, 842, 843, 851
 \l__enumext_ref_the_count_tl . 46, 37, 743, 746,
 786, 789, 800, 803, 848, 851
 __enumext_register_default_label_wd:Nn 600,
 600, 605, 606, 607, 608, 609
 __enumext_remove_extra_parsep_vii: .. 4931,
 5163, 5163
 __enumext_remove_extra_parsep_viii: . 5203,
 5441, 5441
 \l__enumext_renew_counter_v_tl . 849, 857, 859
 \l__enumext_renew_counter_vii_tl 787, 812, 814
 \l__enumext_renew_counter_viii_tl . 801, 819,
 821
 \l__enumext_renew_counter_X_tl 37
 __enumext_renew_footnote: .. 425, 429, 495, 500
 __enumext_renew_footnote_mini: 425, 459, 525,
 530

`\__enumext_renew_footnote_standar:` [489](#), [489](#), [561](#)
`\__enumext_renew_footnote_starred:` [489](#), [521](#), [5133](#), [5407](#)
`\__enumext_reset_count_resume:nn` . [2234](#), [2262](#), [2266](#), [2268](#), [2273](#), [2278](#), [2285](#)
`\__enumext_reset_count_resume_all:n` . [2234](#), [2238](#), [2274](#)
`\__enumext_reset_count_resume_levels:n` [2234](#), [2243](#), [2281](#)
`\__enumext_reset_global_bool:` . [326](#), [329](#), [338](#)
`\__enumext_reset_global_int:` . [326](#), [328](#), [332](#)
`\__enumext_reset_global_tl:` . [326](#), [330](#), [344](#)
`\__enumext_reset_global_vars:` . [38](#), [94](#), [326](#), [326](#), [3254](#)
`\__enumext_resume:n` . [72](#), [1853](#), [2113](#), [2113](#)
`\l__enumext_resume_count_bool` . [46](#), [864](#), [1849](#), [1862](#), [2064](#)
`\l__enumext_resume_count_vii_bool` [127](#), [2001](#), [4953](#)
`\l__enumext_resume_count_X_bool` . [46](#)
`\__enumext_resume_counter:` [69](#), [1962](#), [1962](#), [2067](#), [2209](#), [2212](#), [2226](#), [2229](#)
`\__enumext_resume_integer_series:` . [73](#), [2140](#), [2147](#), [2154](#), [2157](#)
`\__enumext_resume_last_counter:` [71](#), [127](#), [2037](#), [2061](#), [2077](#), [2082](#), [4955](#)
`\g__enumext_resume_last_keys_vii_tl` . [2056](#), [2057](#), [2224](#), [2230](#)
`\g__enumext_resume_last_keys_X_tl` . [46](#)
`\__enumext_resume_save_counter:` . [111](#), [128](#)
`\__enumext_resume_series:n` . [72](#), [73](#), [2119](#), [2131](#), [2140](#), [2140](#)
`\l__enumext_resume_series_vii_bool` [2054](#), [2151](#)
`\l__enumext_resume_series_X_bool` . [46](#)
`\__enumext_resume_star:` . [74](#), [1864](#), [2203](#), [2203](#)
`\l__enumext_resume_star_key_X_bool` . [46](#)
`\l__enumext_rightmargin_vii_dim` . [4644](#), [4648](#), [4653](#)
`\l__enumext_rightmargin_viii_dim` . [4675](#), [4679](#), [4684](#)
`\__enumext_safe_exec:` . [42](#), [108](#), [3902](#), [3902](#), [4084](#)
`\__enumext_safe_exec_vii:` . [42](#), [4912](#), [4934](#), [4934](#)
`\__enumext_safe_exec_viii:` [133](#), [5185](#), [5207](#), [5207](#)
`\__enumext_save_last_keys:n` [71](#), [2037](#), [2037](#), [2076](#), [2081](#)
`\__enumext_second_part:` . [111](#), [4043](#), [4043](#), [4098](#)
`\__enumext_second_part_v:` . [4128](#), [4184](#), [4232](#)
`\l__enumext_series_name_str` . [68](#), [108](#), [127](#), [1839](#), [1895](#), [1898](#), [1903](#), [1939](#), [1942](#), [1946](#), [2072](#), [2089](#), [2091](#), [2094](#), [2097](#), [2102](#), [2104](#), [2106](#), [2108](#), [3917](#), [4959](#)
`\l__enumext_series_name_tl` [68](#), [73](#), [46](#), [1844](#), [1845](#), [1901](#), [1914](#), [1917](#), [1944](#), [1955](#), [1958](#), [2065](#), [2145](#), [2146](#), [2152](#), [2153](#), [2161](#), [2165](#), [2198](#)
`\__enumext_set_error:nn` . [5644](#), [5681](#), [5683](#)
`\__enumext_set_item_width:` [111](#), [4068](#), [4068](#), [4094](#)
`\__enumext_set_parse:n` . [5644](#), [5655](#), [5671](#)
`\l__enumext_setkey_tmpa_int` . [113](#), [5648](#), [5652](#)
`\l__enumext_setkey_tmpa_seq` . [113](#), [5646](#), [5656](#), [5662](#), [5664](#), [5666](#), [5678](#)
`\l__enumext_setkey_tmpa_tl` . [113](#), [5654](#), [5658](#)
`\l__enumext_setkey_tmpb_seq` . [113](#), [5647](#), [5650](#), [5654](#), [5655](#)
`\l__enumext_setkey_tmpb_tl` [113](#), [5673](#), [5675](#), [5676](#)
`\l__enumext_show_answer_bool` . [2579](#), [2598](#), [2989](#), [3383](#), [3396](#), [3437](#), [3749](#), [5333](#), [5363](#)
`\__enumext_show_length:nnn` . [53](#), [225](#), [225](#), [5891](#), [5892](#), [5893](#), [5894](#), [5895](#), [5896](#), [5897](#), [5898](#), [5899](#), [5900](#), [5906](#), [5907](#), [5908](#), [5909](#), [5910](#), [5911](#), [5912](#), [5913](#), [5914](#), [5915](#)
`\l__enumext_show_pos_tmp_int` . [122](#), [3450](#), [3453](#), [3468](#)
`\l__enumext_show_position_bool` . [2582](#), [2601](#), [2993](#), [3384](#), [3397](#), [3457](#), [5340](#)
`\g__enumext_standar_bool` [36](#), [108](#), [22](#), [247](#), [250](#), [269](#), [341](#), [491](#), [507](#), [1877](#), [2456](#), [2470](#), [2847](#), [2860](#), [2875](#), [3944](#)
`\l__enumext_standar_bool` [108](#), [111](#), [22](#), [1657](#), [2848](#), [3909](#), [4062](#), [4948](#)
`\l__enumext_standar_first_bool` [37](#), [108](#), [22](#), [274](#), [1977](#), [2172](#), [2359](#), [2366](#)
`\__enumext_standar_item_vii:w` . [129](#), [5015](#), [5033](#), [5035](#)
`\__enumext_standar_item_viii:w` [134](#), [135](#), [5258](#), [5276](#), [5278](#)
`\__enumext_standar_ref:` . [46](#), [735](#), [751](#), [3854](#)
`\__enumext_standar_ref:nn` . [727](#), [735](#), [735](#)
`\__enumext_standar_save_counter:` . [68](#), [1875](#), [1875](#), [4065](#)
`\__enumext_standar_save_counter_aux:` . [1875](#), [1879](#), [1890](#), [1893](#)
`\__enumext_standar_unknown_keys:n` [3661](#), [3668](#), [3673](#)
`\__enumext_standar_unknown_keys:nn` [3661](#), [3675](#), [3677](#)
`\__enumext_standard_ref:nn` . [46](#)
`\__enumext_standard_reset:nn` . [2234](#), [2252](#), [2260](#)
`\__enumext_standard_reset_key:` [76](#), [2292](#), [2306](#), [2306](#)
`\__enumext_standard_reset_key_star:` [76](#), [2294](#), [2306](#), [2316](#)
`\g__enumext_starred_bool` [36](#), [127](#), [22](#), [257](#), [260](#), [283](#), [342](#), [1656](#), [1924](#), [2430](#), [2476](#), [2838](#), [3294](#), [4849](#)
`\l__enumext_starred_bool` [127](#), [128](#), [133](#), [22](#), [2876](#), [2911](#), [2917](#), [2965](#), [3910](#), [4947](#), [4981](#), [5219](#), [5223](#)
`\__enumext_starred_columns_set_vii:` . [4626](#), [4626](#), [4922](#)
`\__enumext_starred_columns_set_viii:` . [4626](#), [4657](#), [5194](#)
`\l__enumext_starred_first_bool` [37](#), [127](#), [22](#), [288](#), [989](#), [1002](#), [1992](#), [2187](#), [2359](#), [2366](#)
`\__enumext_starred_item_vii:w` . [129](#), [130](#), [5015](#), [5032](#), [5049](#)
`\__enumext_starred_item_vii_aux_i:w` . [5015](#), [5054](#), [5057](#)
`\__enumext_starred_item_vii_aux_ii:w` . [5015](#), [5055](#), [5060](#), [5062](#)
`\__enumext_starred_item_vii_aux_iii:w` [5015](#), [5065](#), [5072](#)
`\__enumext_starred_item_viii:w` [134](#), [135](#), [5275](#), [5293](#), [5293](#)
`\__enumext_starred_item_viii_aux_i:w` . [135](#), [5293](#), [5299](#), [5302](#)
`\__enumext_starred_item_viii_aux_ii:w` . [135](#), [5293](#), [5300](#), [5314](#), [5316](#)
`\__enumext_starred_joined_item_vii:n` [123](#), [129](#), [4688](#), [4688](#), [5030](#)
`\__enumext_starred_joined_item_viii:n` . [123](#), [134](#), [4688](#), [4737](#), [5273](#)

```

__enumext_starred_ref: . . . . 47, 776, 808, 3887
__enumext_starred_ref:n . . . . 47, 770, 776, 776
__enumext_starred_reset:n . . . 2234, 2247, 2264
__enumext_starred_reset_key: . 76, 2301, 2303,
    2306, 2330
__enumext_starred_save_counter: . . 68, 1875,
    1922, 4984
__enumext_starred_save_counter_aux: . 1875,
    1926, 1934, 1937
__enumext_starred_unknown_keys:n 3640, 3644,
    3647
__enumext_starred_unknown_keys:nn 3640, 3649,
    3651
__enumext_start_from:NNn 48, 862, 862, 876, 898,
    904
\l__enumext_start_i_int . . . . . 1980, 2175
__enumext_start_item_tmp_vii: 126, 4925, 5015,
    5015
__enumext_start_item_tmp_viii: . . 5197, 5258,
    5258
__enumext_start_item_vii:w 129, 131, 5041, 5046,
    5069, 5076, 5124, 5124
__enumext_start_item_viii:w . . 135, 5285, 5290,
    5319, 5398, 5398
\g__enumext_start_line_tl 37, 22, 276, 290, 347,
    2500, 2505, 2510, 2524, 2529, 2534
__enumext_start_list:nn 39, 105, 365, 367, 4088,
    4221, 4916, 5188
__enumext_start_list_tag:n . . 4234, 4258, 5134,
    5408
__enumext_start_mini_vii: 128, 4786, 4786, 4973
__enumext_start_mini_viii: . . . 133, 4851, 4851,
    5242
__enumext_start_save_ans_msg: . . 76, 77, 2343,
    2343, 2368
__enumext_start_store_level: . 108, 3938, 3938,
    4087
__enumext_start_store_level_vii: 128, 4915,
    4987, 4987
\l__enumext_start_vii_int 1995, 2003, 2190, 2196
\l__enumext_start_X_int . . . . . 89
__enumext_stop_item_tmp_vii: . . 126, 129, 131,
    4924, 4930, 5017, 5126
__enumext_stop_item_tmp_viii: 134, 5196, 5202,
    5260, 5400
__enumext_stop_item_vii: 131, 5124, 5126, 5146
__enumext_stop_item_viii: . . . 5398, 5400, 5424
__enumext_stop_list: 39, 125, 128, 365, 368, 4009,
    4017, 4174, 4181, 4809, 4817, 4874, 4881
__enumext_stop_list_tag:n . . . 4234, 4274, 5149,
    5427
__enumext_stop_mini_vii: 125, 128, 4786, 4805,
    4977
__enumext_stop_mini_viii: 134, 4851, 4870, 5246
__enumext_stop_save_ans_msg: . 76, 2343, 2348,
    3244
__enumext_stop_start_list_tag: . . 4234, 4266,
    5136, 5410
__enumext_stop_store_level: . . 109, 110, 3967,
    3967, 4010, 4018
__enumext_stop_store_level_vii: . . 125, 128,
    4810, 4818, 4987, 4997
\l__enumext_store_active_bool 33, 77, 101, 1978,
    1993, 2173, 2188, 2375, 3062, 3190, 3942, 3955, 4106,
    4114, 4481, 4989, 4999, 5209, 5225
__enumext_store_active_keys:n . 83, 108, 2658,
    2658, 3935
__enumext_store_active_keys_vii:n . 83, 127,
    2658, 2668, 4965
__enumext_store_addto_prop:n 84, 95, 2734, 2734,
    2742, 2902, 3275, 5324
__enumext_store_addto_seq:n 85, 96, 2743, 2743,
    2747, 2754, 2768, 2776, 2785, 2799, 2807, 2960, 3365
__enumext_store_anskey_arg:n . . 88, 90, 93, 94,
    2899, 2899, 3055, 3233
\l__enumext_store_anskey_arg_tl . . 33, 88, 106,
    2908, 2913, 2915, 2920, 2927, 2930, 2940, 2945, 2948,
    2954, 2960
__enumext_store_anskey_env:n . 94, 3184, 3188,
    3218
\l__enumext_store_anskey_env_tl . . 33, 94, 106,
    3220, 3222, 3224, 3227, 3235
__enumext_store_anskey_safe_outer: . . 91, 93
\l__enumext_store_columns_break_bool . 2910,
    3011, 3097
\l__enumext_store_current_label_tl 33, 95, 96,
    135, 101, 3259, 3262, 3265, 3271, 3273, 3275, 3332,
    3335, 3338, 3344, 3346, 3356, 3365, 5304, 5309, 5310,
    5323, 5324, 5326
\l__enumext_store_current_opt_arg_tl . 33, 97,
    135, 101, 3375, 3380, 3387, 3393, 3400, 5312
__enumext_store_internal_ref: . . 86, 88, 2824,
    2824, 2905
\l__enumext_store_item_join_int . . 2918, 2922,
    3014, 3100
\l__enumext_store_item_star_bool . 2925, 3016,
    3102
\l__enumext_store_item_symbol_sep_dim 2937,
    2942, 3021, 3107
\l__enumext_store_item_symbol_tl . 2928, 2932,
    3019, 3105
\l__enumext_store_keyans_item_opt_sep_v_-
    tl . . . . . 3269, 3271, 3342, 3344
\l__enumext_store_keyans_item_opt_sep_-
    viii_tl . . . . . 5307, 5309
__enumext_store_level_close: . 85, 2748, 2772,
    3971
__enumext_store_level_close_vii: . 85, 2779,
    2803, 5003
__enumext_store_level_open: 85, 109, 2748, 2748,
    3950, 3963
__enumext_store_level_open_vii: . . 85, 2779,
    2779, 4993
\g__enumext_store_name_tl 33, 77, 101, 346, 353,
    354, 355, 356, 2351, 2377, 2499, 2504, 2509, 2523,
    2528, 2533, 3242
\l__enumext_store_name_tl 33, 77, 78, 101, 1882,
    1885, 1905, 1927, 1930, 1948, 1982, 1997, 2177, 2192,
    2346, 2355, 2356, 2377, 2378, 2380, 2381, 2383, 2385,
    2386, 2388, 2390, 2391, 2415, 2736, 2738, 2745, 2888,
    2889, 3001, 3315, 3316, 3467, 5348
\l__enumext_store_ref_key_bool 88, 2595, 2903,
    2951, 3279, 3353
\l__enumext_store_save_key_vii_bool . . 2670,
    2700
\l__enumext_store_save_key_vii_tl 2672, 2673,
    2701, 2702, 2783, 2791, 2795, 2799
\l__enumext_store_save_key_X_bool . . 83, 122
\l__enumext_store_save_key_X_tl . . . . 83, 122

```


<code>\l__enumext_store_upper_level_X_bool</code> ..	122
<code>__enumext_storing_exec:</code> ..	77 , 2353 , 2369 , 2373
<code>__enumext_storing_set:n</code>	76 , 77 , 2338 , 2353 , 2353
<code>\l__enumext_the_counter_v_tl</code>	848
<code>\l__enumext_the_counter_vii_tl</code>	786
<code>\l__enumext_the_counter_viii_tl</code>	800
<code>\l__enumext_the_counter_X_tl</code>	37
<code>__enumext_tmp:n</code>	32 , 36 , 39 , 45 , 48 , 55 , 60 , 67 , 68 , 75 , 83 , 88 , 89 , 100 , 132 , 139 , 164 , 168 , 175 , 195 , 591 , 599 , 632 , 641 , 1964 , 1986 , 2159 , 2181 , 2205 , 2220 , 2288 , 2298 , 2318 , 2327 , 2334 , 2342 , 2394 , 2412 , 2588 , 2657 , 2676 , 2689 , 2826 , 2833 , 2834 , 2855 , 2868 , 2871 , 2882 , 3281 , 3288 , 3615 , 3625 , 3661 , 3672 , 3822 , 3864 , 3865 , 3901
<code>__enumext_tmp:nn</code>	642 , 663 , 664 , 698 , 699 , 714 , 893 , 918 , 919 , 934 , 1015 , 1037 , 1038 , 1058 , 1112 , 1120 , 1121 , 1135 , 1200 , 1216 , 1217 , 1230 , 1724 , 1740 , 1835 , 1874 , 2553 , 2587 , 3599 , 3614
<code>__enumext_tmp:nnn</code>	715 , 731 , 732 , 733 , 734 , 758 , 774 , 775
<code>__enumext_tmp:nnnnnn</code>	935 , 960 , 963 , 966 , 968 , 970 , 973 , 976
<code>__enumext_tmp:w</code>	5469 , 5472
<code>\l__enumext_tmpa_vii_int</code>	4636 , 4639 , 4648 , 4679
<code>\l__enumext_tmpa_viii_int</code>	4667 , 4670
<code>\l__enumext_tmpa_X_dim</code>	175
<code>\l__enumext_tmpa_X_int</code>	175
<code>\l__enumext_topsep_v_skip</code>	1297 , 1301 , 1500 , 4474
<code>\l__enumext_topsep_vii_skip</code> ..	1577 , 1586 , 1590
<code>\l__enumext_topsep_viii_skip</code> ..	1599 , 1621 , 1625
<code>__enumext_unskip_unkern:</code> ..	36 , 231 , 231 , 1350 , 1522 , 4012 , 4013 , 4053 , 4176 , 4177 , 4194 , 5140 , 5141 , 5414 , 5415
<code>\l__enumext_vspace_a_star_v_bool</code>	1773
<code>\l__enumext_vspace_a_star_vii_bool</code> ...	1795
<code>\l__enumext_vspace_a_star_viii_bool</code> ...	1806
<code>\l__enumext_vspace_a_star_X_bool</code>	89
<code>__enumext_vspace_above:</code>	65 , 110 , 1741 , 1741 , 4023
<code>__enumext_vspace_above_v:</code> ..	66 , 1769 , 1769 , 4130
<code>\l__enumext_vspace_above_v_skip</code> ..	1771 , 1775 , 1777
<code>__enumext_vspace_above_vii:</code>	66 , 128 , 1791 , 1791 , 4970
<code>\l__enumext_vspace_above_vii_skip</code>	1793 , 1797 , 1799
<code>__enumext_vspace_above_viii:</code> ..	66 , 1791 , 1802 , 5240
<code>\l__enumext_vspace_above_viii_skip</code>	1804 , 1808 , 1810
<code>\l__enumext_vspace_b_star_v_bool</code>	1784
<code>\l__enumext_vspace_b_star_vii_bool</code> ...	1817
<code>\l__enumext_vspace_b_star_viii_bool</code> ...	1828
<code>\l__enumext_vspace_b_star_X_bool</code>	89
<code>__enumext_vspace_below:</code>	66 , 111 , 1755 , 1755 , 4061
<code>__enumext_vspace_below_v:</code> ..	66 , 1780 , 1780 , 4203
<code>\l__enumext_vspace_below_v_skip</code> ..	1782 , 1786 , 1788
<code>__enumext_vspace_below_vii:</code>	67 , 128 , 1813 , 1813 , 4980
<code>\l__enumext_vspace_below_vii_skip</code>	1815 , 1819 , 1821
<code>__enumext_vspace_below_viii:</code> ..	67 , 1813 , 1824 , 5248
<code>\l__enumext_vspace_below_viii_skip</code>	1826 , 1830 , 1832
<code>__enumext_widest_from:nnn</code> ..	49 , 877 , 877 , 892 , 911
<code>\g__enumext_widest_label_tl</code>	31 , 44 , 56 , 620 , 624 , 628
<code>\l__enumext_wrap_label_opt_v_bool</code>	3695
<code>\l__enumext_wrap_label_opt_vii_bool</code>	129 , 5040
<code>\l__enumext_wrap_label_opt_viii_bool</code> ..	135 , 5284
<code>\l__enumext_wrap_label_opt_X_bool</code>	89
<code>\l__enumext_wrap_label_v_bool</code>	3691 , 3695 , 3702 , 3748 , 3756 , 4524
<code>\l__enumext_wrap_label_vii_bool</code> ..	129 , 5040 , 5044 , 5052 , 5116
<code>\l__enumext_wrap_label_viii_bool</code> ..	135 , 5284 , 5288 , 5297 , 5362 , 5371
<code>\l__enumext_wrap_label_X_bool</code>	89
<code>__enumext_wrapper_label_v:n</code> ..	3754 , 3758 , 4543
<code>__enumext_wrapper_label_vii:n</code>	5118
<code>__enumext_wrapper_label_viii:n</code> ..	5369 , 5373
<code>\l__enumext_write_anskey_env_bool</code> ..	33 , 106 , 3113 , 3138
<code>\l__enumext_write_anskey_env_file_iow</code> ..	33 , 106 , 3163 , 3164 , 3165
<code>\l__enumext_write_anskey_env_file_name_- tl</code>	33 , 106 , 3114 , 3224
<code>\l__enumext_write_aux_file_tl</code> ..	34 , 87 , 96 , 161 , 2891 , 2897 , 3322 , 3328
<code>enumext*</code>	5 , 4910
<code>enumXi</code>	574
<code>enumXii</code>	574
<code>enumXiii</code>	574
<code>enumXiv</code>	574
<code>enumXv</code>	574
<code>enumXvi</code>	574
<code>enumXvii</code>	574
<code>enumXviii</code>	574
Environments provide by enumext :	
<code>anskey*</code>	30 , 33 , 77 , 82 , 83 , 87 , 89 , 92 , 108 , 109 , 128 , 138 , 139 , 145 , 147
<code>enumext*</code>	30 , 31 , 35 , 36 , 40-44 , 47 , 49 , 50 , 52 , 53 , 55 , 62 , 63 , 66-68 , 71 , 72 , 74-79 , 82-85 , 87 , 88 , 90 , 94 , 95 , 101 , 102 , 105 , 107-109 , 114 , 122 , 123 , 125 , 128 , 130-134 , 136-140 , 142 , 146 , 148 , 150
<code>enumext</code> ..	30 , 31 , 35 , 36 , 40-44 , 46-58 , 61 , 63-68 , 71 , 72 , 74-79 , 82-85 , 87 , 88 , 90 , 94 , 95 , 99-101 , 103 , 105 , 109 , 111 , 112 , 117 , 122 , 124 , 127 , 128 , 130 , 133 , 138 , 140 , 142 , 145 , 147 , 149
<code>keyans*</code>	30 , 31 , 33-37 , 40-43 , 47-50 , 52 , 53 , 55 , 62 , 63 , 66 , 67 , 77 , 78 , 81 , 82 , 84 , 93 , 95 , 97 , 102 , 105 , 107 , 114 , 122 , 123 , 132 , 133 , 146 , 148 , 150
<code>keyanspic</code>	30 , 31 , 33 , 34 , 37 , 43 , 48 , 77 , 78 , 81 , 84 , 85 , 93 , 95-97 , 102 , 114-120 , 148
<code>keyans</code>	30 , 31 , 33 , 34 , 36 , 37 , 40 , 41 , 43 , 44 , 48-50 , 52 , 53 , 55 , 57 , 61 , 63-66 , 77 , 78 , 81 , 82 , 84 , 85 , 93 , 95-98 , 102-105 , 112 , 114 , 116 , 117 , 120 , 124 , 134 , 146 , 148
Environments:	
<code>center</code>	122
<code>description</code>	101 , 122
<code>enumerate</code>	122
<code>flushleft</code>	122
<code>flushright</code>	122
<code>itemize</code>	122
<code>list</code> ..	35 , 38 , 39 , 50 , 90 , 101 , 105 , 110 , 111 , 114 , 116-118 ,

122, 125

lrbox

131

minipage 35, 38, 39, 41, 42, 55, 58, 59, 116, 119, 121, 122,

125, 131

multicols

56–59, 64, 109–111

quotation

122

quote

122

tabbing

122

trivlist

122

verbatim

122

verse

122

exp commands:

\exp_after:wN

5472

\exp_args:Ne 1968, 2120, 2163, 2213, 3232, 3928, 5460

\exp_args:NV

3027, 3124, 3628, 3649, 3675, 5777

\exp_not:N 43, 623, 746, 789, 803, 851, 1068, 1071, 1082,

1083, 1084, 1095, 1096, 1107, 1108, 2956, 2998, 2999,

3358, 3464, 3465, 5345, 5346, 5469

\exp_not:n 278, 292, 305, 313, 321, 689, 709, 746, 747,

789, 803, 851, 1069, 2026, 2035, 2566, 2615, 2719,

2732, 2894, 2922, 2932, 2942, 2956, 2957, 3325, 3360,

3362, 4343, 5587, 5595, 5809, 5814

F

\fbox

2622

\fboxrule

2622

\fboxsep

2622

file commands:

\file_if_exist:nTF

3140

\file_input_stop:

6208

first

1121

font

642

\footnote

40

\footnote

40, 431, 461

\footnotemark

441, 471

\footnotesize

2999, 3465, 5346

\footnotetext

427

force-eol

3095

\foreachkeyans

19, 143, 5742

G

\getkeyans

19, 138, 5458

group commands:

\group_begin:

2997, 3042, 3463, 5344, 5527

\group_end:

3004, 3058, 3471, 5351, 5536

H

\hbadness

5151, 5429

hbox commands:

\hbox_overlap_left:n

2814, 3530, 5109

\hbox_set:Nn

612, 4406

\hbox_set_end:

5150, 5428

\hbox_set_to_wd:Nnw

5127, 5401

\hfill 672, 677, 683, 684, 1680, 1707, 2956, 3358, 4813, 4877

hook commands:

\hook_gput_code:nnn

5, 205, 209, 213, 376

\hook_gset_rule:nnnn

377

\hyperlink

89, 96

\hyperlink

2956, 3358

\hypertarget

39

\hypertarget

402

I

\IfDocumentMetadataT 4260, 4268, 4276, 4310, 4318, 4326,

4430, 4439, 4447, 4452, 4457, 4502, 4511, 4601, 4609,

4811, 4875, 4921, 4929, 5085, 5193, 5201

\IfDocumentMetadataTF

493, 509, 523, 536, 3547, 3730

\IfHyperBoolean

384

\IfPackageLoadedT

380

\IfPackageLoadedTF

7, 392

\ignorespaces

1071, 1084, 1096, 1108, 4419, 4926, 5013,

5046, 5069, 5076, 5122, 5142, 5198, 5256, 5290, 5319,

5396, 5416

\inputlineno

278, 292, 305, 313, 321

int commands:

\int_add:Nn

4721, 4770

\int_case:nn

1245, 1370, 2425, 2451, 2490, 2514

\int_case:nnTF

233

\int_compare:nNnTF

557, 779, 793, 810, 817, 1340,

1359, 1513, 1531, 1643, 1662, 1674, 1702, 1880, 1974,

1989, 2039, 2052, 2074, 2079, 2086, 2100, 2115, 2127,

2142, 2149, 2169, 2184, 2218, 2222, 2308, 2325, 2538,

2544, 3066, 3070, 3074, 3082, 3194, 3198, 3202, 3240,

3260, 3298, 3303, 3308, 3333, 3448, 3907, 3918, 3947,

3960, 3976, 3991, 4006, 4047, 4115, 4119, 4147, 4172,

4188, 4378, 4485, 4489, 4691, 4701, 4717, 4740, 4750,

4766, 4939, 4943, 4991, 5001, 5153, 5165, 5214, 5226,

5431, 5443, 5652, 5792

\int_compare_p:nNn

248, 258, 270, 271, 284, 285,

1649, 1650, 2250, 2251, 2431, 2457, 2839, 2849, 2861,

2862, 2877, 2918, 3957

\int_decr:N

4720, 4769

\int_eval:n

363, 906, 2254, 2738, 2889, 2999, 3316,

3465, 3839, 3886, 4709, 4758, 5346

\int_from_alph:n

871, 885

\int_from_roman:n

873, 887

\int_gadd:Nn

4722, 4771

\int_gdecr:N

2434, 2439, 2443, 2447, 2460

\int_gincr:N 2901, 3368, 3484, 3518, 3708, 4036, 4139,

4529, 5019, 5095, 5262, 5328

\int_gset:Nn

439, 469, 2483

\int_gset_eq:NN

436, 466, 1884, 1897, 1907, 1916,

1929, 1941, 1950, 1957

\int_gzero:N

334, 335, 336, 1688, 1715, 2312, 2322,

2332, 2550, 4052, 4193, 5176, 5455

\int_if_exist:NTF 1882, 1913, 1927, 1955, 2093, 2106,

2161, 2310, 2320, 2388

\int_incr:N 3081, 3450, 3906, 4110, 4377, 4938, 5018,

5213, 5261

\int_mod:nn

5167, 5445

\int_new:N 16, 17, 18, 19, 20, 21, 76, 93, 115, 130, 142,

143, 154, 155, 156, 158, 169, 170, 178, 179, 180, 181,

182, 2096, 2108, 2391

\int_set:Nn 867, 871, 873, 1968, 1980, 1995, 2003, 2163,

2175, 2190, 2196, 3666, 4595, 4596, 4636, 4667, 4690,

4696, 4712, 4739, 4745, 4761, 5151, 5429, 5648, 5794

\int_set_eq:NN

3837, 3882, 4719, 4768

\int_sign:n

2485

\int_step_function:nN

1986, 2181, 2220

\int_step_function:nnN

2327, 2855, 2868, 2882

\int_step_function:nnnN

5798

\int_step_inline:nn

5685, 5715

\int_step_inline:nnn

4597

\int_to_roman:n

217, 1966, 1968, 1970, 2161, 2163,

2165, 2207, 2215, 2262, 2310, 2312, 2320, 2322, 2835,

2872

\int_use:N 356, 361, 362, 1341, 1360, 1675, 1970, 1982,

1997, 2005, 2120, 2165, 2177, 2192, 2198, 2214, 3839,

3859, 3886, 3929, 3977, 3986, 4001, 4007, 4694, 4695,

4707, 4743, 4744, 4756, 6115, 6119, 6125, 6129, 6202

\int_zero:N

3453, 5157, 5435

©2024–2026 by Pablo González L

162 / 167

iow commands:

<code>\iow_char:N</code>	3221, 3222
<code>\iow_close:N</code>	3165
<code>\iow_new:N</code>	110
<code>\iow_now:Nn</code>	3164
<code>\iow_open:Nn</code>	3163
<code>\item</code>	99, 103, 129, 131, 134, 137, 369, 2756, 2762, 2787, 2793, 2915, 3335, 3338, 3536, 3712, 4434, 4435, 4923, 4925, 5195, 5197, 5326
<code>\item*</code>	5, 17, 81, 3710
<code>item-join</code>	3009, 3095
<code>item-pos*</code>	3009, 3095, 3599
<code>item-star</code>	3009, 3095
<code>item-sym*</code>	3009, 3095, 3599
<code>\itemindent</code>	106
<code>\itemindent</code>	105
<code>itemindent</code>	1015
<code>\itemsep</code>	4423
<code>\itemwidth</code>	573, 2622, 4070, 4076, 4207, 4213, 4730, 4734, 4779, 4783

K

<code>keyans</code>	16, 4216
<code>keyans*</code>	16, 5183
<code>keyanspic</code>	17, 4425

Keys for `\anskey` provide by `enumext`:

<code>break-col</code>	88, 90
<code>force-eol</code>	91
<code>item-join</code>	88, 90
<code>item-pos*</code>	88, 90
<code>item-star</code>	88, 90
<code>item-sym*</code>	88, 90
<code>overwrite</code>	91
<code>write-env</code>	91

Keys for `anskey*` provide by `enumext`:

<code>break-col</code>	88, 90
<code>force-eol</code>	91
<code>item-join</code>	88, 90
<code>item-pos*</code>	88, 90
<code>item-star</code>	88, 90
<code>item-sym*</code>	88, 90
<code>overwrite</code>	91
<code>write-env</code>	91

Keys for environments provide by `enumext`:

<code>above*</code>	32, 51, 65, 66, 110, 128
<code>above</code>	32, 51, 65, 66, 110, 128, 133
<code>after</code>	53, 54, 111, 128, 134
<code>align</code>	32, 45, 98–100, 104, 130, 144
<code>base-fix</code>	51, 70, 84, 108
<code>before*</code>	53, 54, 110, 128, 133
<code>before</code>	53, 54
<code>below*</code>	32, 65–67, 111, 128
<code>below</code>	32, 65–67, 111, 128, 134
<code>check-ans</code>	34, 35, 37, 76–81, 84, 94, 96, 111, 112, 128, 132, 146
<code>columns-sep</code>	55, 109, 132
<code>columns</code>	32, 55, 65, 109
<code>first</code>	53, 54, 131
<code>font</code>	44, 100, 104, 120, 130
<code>item-pos*</code>	99, 101
<code>item-sym*</code>	33, 99, 101
<code>itemindent</code>	32, 52, 99, 103–105, 131
<code>itemsep</code>	50, 107, 132
<code>label-pos</code>	116, 117, 119, 120
<code>label-sep</code>	116

<code>labelsep</code>	44, 106, 130
<code>labelwidth</code>	43, 44, 46–49, 106, 130
<code>label</code>	31, 43, 44, 46, 48, 49, 117, 121
<code>layout-sep</code>	116
<code>layout-sty</code>	116, 121
<code>layout-top</code>	116
<code>lisparindent</code>	107
<code>list-indent</code>	32, 52, 117
<code>list-offset</code>	52, 111, 113
<code>listparindent</code>	52, 131
<code>mark-ans*</code>	81, 84, 98
<code>mark-ans</code>	82, 84, 89
<code>mark-pos*</code>	81, 84, 98
<code>mark-pos</code>	33, 82, 84, 144
<code>mark-ref</code>	82, 84, 86, 89
<code>mark-sep*</code>	81, 84, 98
<code>mark-sep</code>	33, 82, 84, 135
<code>mini-env</code>	32, 40–42, 55, 64, 65, 84, 110, 122, 124, 125, 128, 133
<code>mini-right*</code>	32, 35, 55, 84, 125, 128
<code>mini-right</code>	32, 35, 55, 63, 84, 125, 128
<code>mini-sep</code>	32, 55, 84, 110
<code>mode-box</code>	44, 99–101, 104, 105
<code>no-store</code>	34, 76–78, 84, 90, 93, 99
<code>noitemsep</code>	50
<code>nosep</code>	50
<code>overwrite</code>	33, 92
<code>parindent</code>	107
<code>parsep</code>	50, 107, 117, 131
<code>partopsep</code>	50
<code>ref</code>	31, 46–48, 105, 146
<code>reset*</code>	70, 75, 76, 84, 140
<code>reset</code>	70, 75, 76, 84, 140
<code>resume*</code>	31, 43, 67, 69–71, 74, 76, 77, 84, 111, 128, 140
<code>resume</code>	31, 38, 43, 49, 67–74, 76, 77, 84, 111, 127, 128, 140, 150
<code>rightmargin</code>	52, 122
<code>save-ans</code>	33, 38, 68–70, 73, 76–78, 80, 83–85, 90, 91, 93–96, 103, 112, 119, 130, 133–135, 138, 140, 146
<code>save-key</code>	33, 70, 83, 84, 108, 127
<code>save-pos</code>	84
<code>save-ref</code>	34, 40, 82, 84, 86, 88, 89, 95, 96, 103, 135
<code>save-sep</code>	81, 84, 95, 135
<code>series</code>	31, 67, 68, 70, 71, 73, 74, 84, 108, 111, 127, 128, 140
<code>show-ans</code>	33, 81, 82, 84, 86, 88, 89, 97, 98, 120, 135
<code>show-length</code>	36, 53, 105, 145, 146
<code>show-pos</code>	33, 81, 82, 86, 88, 89, 97, 120, 135
<code>start*</code>	32, 48, 49, 70, 71
<code>start</code>	32, 36, 48, 49, 70, 71
<code>store-key</code>	83
<code>topsep</code>	50, 51, 117
<code>widest</code>	31, 36, 46, 49
<code>wrap-ans*</code>	34, 81, 84, 104, 120
<code>wrap-ans</code>	43, 82, 84, 86, 89
<code>wrap-label*</code>	32, 44, 99, 100, 103, 104, 129, 130, 135
<code>wrap-label</code>	32, 44, 99, 100, 103, 104, 117, 120, 129, 130, 135
<code>wrap-opt</code>	81, 84, 97, 103, 120
<code>wrap-sep</code>	89
<code>write-env</code>	33, 92

keys commands:

<code>\keys_define:nn</code>	634, 644, 666, 701, 717, 760, 825, 895, 921, 937, 979, 1017, 1040, 1114, 1123, 1202, 1219, 1726, 1837, 2290, 2299, 2336, 2396, 2555, 2590, 2678, 2683, 3009, 3095, 3601, 3617, 3640, 3663, 4332, 5486,
------------------------------	--

5597, 5687, 5695, 5703, 5734, 5742

\keys_if_exist_p:nn 5730, 5731

\l_keys_key_str 90, 92, 3027, 3124, 3628, 3649, 3675, 5777, 5881

\keys_precompile:nnN . . 139, 201, 201, 5488, 5494, 5500, 5506, 5512, 5518, 5760

\keys_set:nn . 658, 996, 1008, 1225, 1731, 1736, 2120, 2132, 2213, 2230, 2626, 2627, 2631, 2632, 2636, 2637, 2641, 2642, 2646, 2647, 2651, 2652, 3047, 3176, 3920, 3928, 4126, 4350, 4352, 4354, 4356, 4358, 4360, 4362, 4364, 4366, 4368, 4388, 4960, 5235, 5600, 5606, 5612, 5618, 5624, 5629, 5630, 5631, 5632, 5633, 5634, 5635, 5636, 5668, 5724, 5786

keyval commands:

\keyval_parse:NNn 2015, 2708, 5576

L

label 715, 758, 825

label-pos 4332

label-sep 4332

Labels provide by enumext:

\Alph* 43, 44

\Roman* 43, 44

\alph* 43, 44

\arabic* 43, 44

\roman* 43, 44

labelsep 642

\labelwidth 44

labelwidth 642

\lastnodetype 233

layout-sep 4332

layout-sty 4332

layout-top 4332

\leftmargin 106

\leftmargin 105, 4418

legacy commands:

\legacy_if:nTF 5080, 5083, 5379, 5382

\legacy_if_gset_false:n 563, 4826

\legacy_if_set_false:n 5082, 5381

\legacy_if_set_true:n 5045, 5068, 5075, 5089, 5289, 5318

\linewidth 110

\linewidth 4031, 4070, 4136, 4207, 4594, 4639, 4670, 4792, 4857

\list 367

list-indent 1015

list-offset 1015

\listparindent 4421

listparindent 1015

M

\makebox 121

\makebox 2816, 3583, 3780, 4519, 4532, 5113, 5391

\makelabel 99, 100, 104, 121

\makelabel 99, 103, 3563, 3579, 3764, 3776

mark-ans 2588, 4332

mark-ans* 2553, 2588

mark-pos 2588, 4332

mark-pos* 2553, 2588

mark-ref 2588

mark-sep 2588, 4332

mark-sep* 2553, 2588

midpenalty 919

mini-env 1200

mini-sep 1200

\minipage 373

\miniright 12, 63, 1641, 1692, 1719, 4050, 4191

mode commands:

\mode_if_math:TF 3090, 3213

\mode_if_vertical:TF 1279, 1307, 1327, 1351, 1502, 1523

\mode_leave_vertical: 994, 1005, 1068, 1082, 2812, 3528, 5107

mode-box 632

msg commands:

\msg_error:nn . . 1694, 1721, 3051, 3084, 3088, 3180, 3211, 4117, 4121, 4380, 4437, 4487, 4941, 5216, 5228, 5637, 5711

\msg_error:nnn 740, 783, 797, 845, 1645, 1652, 1659, 1690, 1717, 2124, 2136, 2254, 2361, 3033, 3092, 3130, 3192, 3196, 3200, 3204, 3215, 3634, 3655, 3681, 4945, 5221, 5474, 5483, 5569, 5732, 5769, 5790

\msg_error:nnnn 3036, 3064, 3068, 3072, 3076, 3133, 3637, 3658, 3684, 4108, 4483, 4491, 5211, 5548, 5684, 5772

\msg_error:nnnnn 688, 708, 2565, 2614, 4342

\msg_fatal:nn 3908

\msg_fatal:nnn 577, 594

\msg_info:nnn 9, 12, 382, 394

\msg_line_context: . . 5846, 5851, 5856, 5885, 5890, 5905, 5920, 5924, 5928, 5932, 5939, 5946, 5952, 5966, 5970, 5975, 5979, 5983, 5988, 5992, 5996, 6000, 6047, 6051, 6056, 6061, 6065, 6070, 6146, 6150, 6155, 6160, 6165, 6169, 6173, 6177, 6181, 6185, 6189, 6193, 6198, 6203

\msg_log:nnn 2380, 2385, 2390

\msg_log:nnnnn 360, 2523, 2528, 2533

\msg_log:nnnnnn 352

\msg_new:nnn 5817, 5821, 5825, 5830, 5843, 5848, 5853, 5858, 5867, 5875, 5879, 5883, 5888, 5903, 5918, 5922, 5926, 5930, 5934, 5943, 5949, 5955, 5959, 5963, 5968, 5973, 5977, 5981, 5986, 5990, 5994, 5998, 6033, 6037, 6041, 6045, 6049, 6054, 6059, 6063, 6068, 6144, 6148, 6153, 6158, 6163, 6167, 6171, 6175, 6179, 6183, 6187, 6191, 6195, 6200

\msg_new:nnnn . . 5834, 6003, 6012, 6021, 6027, 6072, 6082, 6092, 6102, 6112, 6122, 6132, 6138, 6205

\msg_term:nnnn . 2345, 2350, 3848, 3858, 3892, 3897

\msg_term:nnnnn 2504

\msg_warning:nn 4049, 4190

\msg_warning:nnn 3144, 3148, 3153

\msg_warning:nnnn 2541, 2547, 3794, 3799, 4693, 4706, 4742, 4755

\msg_warning:nnnnn 2499, 2509

\multicolsep 109

\multicolsep 1344, 1516, 3997, 4163

N

\NeedsTeXFormat 3

\NewCommandCopy 369

\newcounter 580, 596

\NewDocumentCommand 1641, 2234, 3039, 4479, 5458, 5525, 5644, 5708, 5779

\NewDocumentEnvironment . 3169, 4082, 4216, 4425, 4910, 5183

\newlabel 40

\newlabel 414

no-store 2394

\noindent 4038, 4801, 4866, 5156, 5434

\nointerlineskip 1353, 1356, 1525, 1528, 1682, 1709, 4801, 4866

noitemsep	935	\refstepcounter	5092, 5384
\nopagebreak 1290, 1318, 1353, 1356, 1525, 1528, 1632, 1638		regex commands:	
\normalfont	2998, 3464, 5345	\regex_if_match:nnTF	221, 870, 872, 884, 886
nosep	935	\renewcommand	746, 789, 803, 851
O		\RenewDocumentCommand .	431, 461, 1692, 1719, 3221, 3536, 3563, 3579, 3712, 3764, 3776, 4435
\obeyedline	3221, 3222	\RequirePackage	13
overwrite	3095	reset	2288
P		reset*	2288
Packages:		\resetenumext	11, 74, 2234
caption	125	resume	1835
enumext . 30, 43, 46, 76, 81, 101, 105, 106, 116, 144, 145		resume*	1835
enumitem	43	rightmargin	1015
expl3	121	\Roman	44, 48, 49
footnotehyper	39, 41, 42	\Roman	608
hyperref	34, 35, 39, 40, 89, 96, 130, 144	\roman	44, 48, 49
latex-lab-block	39	\roman	609, 733, 5510
ltxcmd	39, 91	S	
ltsockets	114	save-ans	2334
lua-visual-debug	58	save-key	2676
multicol	30, 144	save-ref	2588
scontents	91	save-sep	2553, 2588, 4332
shortlst	121, 126, 131	scan commands:	
tagpdf	114	\scan_stop:	4434, 4923, 5195, 5469, 5472
\par .. 1290, 1318, 1356, 1528, 1632, 1638, 1677, 1682, 1704, 1709, 2964, 4014, 4178, 4196, 4465, 4468, 4614, 4828, 4843, 4889, 4903, 5156, 5434		seq commands:	
para commands:		\seq_clear:N	5646, 5797
\para_end:	5173, 5452	\seq_const_from_clist:Nn	5639
\parbox	2622	\seq_count:N	355, 4620, 5650
\parindent	5138, 5412	\seq_gclear:N	456, 457, 486, 487
\parsep	56, 117	\seq_gput_right:Nn	442, 443, 472, 473, 2745
\parsep	995, 3883, 4402, 4411	\seq_if_empty:NnTF	449, 479, 5542, 5664
parsep	935	\seq_if_exist:NnTF	2383, 5540
\parskip	5139, 5413	\seq_if_in:NnTF	5546
\partopsep	3884, 4194, 4422	\seq_item:Nn	4607
partopsep	935	\seq_map_function:NN	5655
peek commands:		\seq_map_inline:Nn	5555, 5563, 5665, 5666
\peek_meaning:NnTF 5024, 5038, 5053, 5064, 5267, 5282, 5298		\seq_map_pairwise_function:NNN	451, 481
\peek_meaning_remove:NnTF	5031, 5274	\seq_new:N 116, 117, 119, 140, 171, 172, 173, 174, 2386	
\peek_remove_spaces:n	3717	\seq_pop_left:NN	5654
\phantomsection	39	\seq_put_right:Nn	4493, 5662, 5678, 5807
\phantomsection	403	\seq_set_from_clist:Nn	5647
prg commands:		\seq_set_map_e:NNn	5656
\prg_do_nothing:	407	\seq_use:Nn	201, 202, 5803
\prg_new_protected_conditional:Npnn 219, 3136		series	1835
\prg_replicate:nn	228	\setcounter	881, 885, 887, 3839, 3886, 4462
\prg_return_false:	223, 3149, 3157	\setenumext	6, 140, 5644
\prg_return_true:	222, 3145, 3154	\setenumextmeta	6, 142, 5685
\printkeyans	20, 138, 5525	show-ans	2553, 2588, 4332
prop commands:		show-length	1112
\prop_count:N 354, 2738, 2889, 3001, 3316, 3467, 5348, 5795		show-pos	2553, 2588, 4332
\prop_gput_if_not_in:Nnn	2736	skip commands:	
\prop_if_exist:NnTF	2378, 5478, 5788	\skip_add:Nn 1250, 1259, 1268, 1281, 1285, 1309, 1313, 1329, 1387, 1389, 1403, 1406, 1427, 1429, 1443, 1446, 1466, 1468, 1482, 1485, 1504, 1553, 1554, 1565, 1567, 4411, 4420	
\prop_item:Nn	5480, 5812	\skip_gset:Nn	1580, 1584, 1588
\prop_new:N	2381	\skip_gzero_new:N	1575, 1576
\ProvidesExplPackage	4	\skip_horizontal:N .. 1083, 1095, 1107, 5110, 5122, 5160, 5396, 5438	
R		\skip_horizontal:n .. 1069, 2813, 2821, 3529, 3531, 4537, 5009, 5108, 5142, 5252, 5416	
\raggedcolumns	4000, 4166	\skip_if_eq:nnTF 1248, 1257, 1266, 1373, 1413, 1453, 1541, 1577, 1599, 1743, 1757, 1771, 1782, 1793, 1804, 1815, 1826	
\raisebox	4556		
\ref	86, 95		
ref	715, 758, 825		

<code>\skip_new:N</code>	70, 71, 72, 77, 78, 79, 80, 81, 82, 193
<code>\skip_set:Nn</code>	1233, 1237, 1295, 1299, 1323, 1376, 1377, 1395, 1416, 1417, 1435, 1455, 1456, 1474, 1498, 1544, 1545, 1559, 1579, 1583, 1601, 1605, 1609, 1615, 1619, 1623, 4395
<code>\skip_set_eq:NN</code>	1334, 1335, 1337, 1344, 1509, 1510, 1511, 1516, 3835, 3880, 3883, 5139, 5413
<code>\skip_sub:Nn</code>	1383, 1385, 1399, 1401, 1423, 1425, 1439, 1441, 1462, 1464, 1478, 1480, 1551, 1552, 1563, 1564
<code>\skip_use:N</code>	1235, 1239, 1283, 1287, 1291, 1311, 1315, 1325, 1331, 1744, 1748, 1751, 1758, 1762, 1765, 4014
<code>\skip_vertical:N</code>	564, 567, 1007, 4827, 4841, 5175, 5454
<code>\skip_vertical:n</code>	1006, 5174, 5453
<code>\skip_zero:N</code>	1343, 1357, 1495, 1496, 1497, 1515, 1529, 3884, 3997, 4163, 4422, 4423
<code>\skip_zero_new:N</code>	1574, 1596, 1597, 1598
<code>\c_zero_skip</code>	564, 567, 1007, 1248, 1257, 1266, 1414, 1453, 1577, 1599, 1744, 1758, 1771, 1782, 1793, 1804, 1815, 1826, 4827, 4841, 5175, 5454
<code>\small</code>	5493, 5499, 5505, 5511, 5517, 5523
<code>\smash</code>	3581, 3778
socket commands:	
<code>\socket_assign_plug:nn</code>	4262, 4270, 4278, 4312, 4320, 4328
<code>\socket_new:nn</code>	4234, 4282
<code>\socket_new_plug:nnn</code>	4235, 4242, 4250, 4283, 4290, 4299
<code>\socket_use:n</code>	4313, 4321, 4329
<code>\socket_use:nn</code>	4263, 4271, 4279
<code>start</code>	893
<code>start*</code>	893
<code>start-list-tags</code>	4234, 4282
<code>\stepcounter</code>	435, 465, 4405, 4549
<code>stop-list-tags</code>	4234, 4282
<code>stop-start-tags</code>	4234, 4282
str commands:	
<code>\c_backslash_str</code>	3092, 5846, 5851, 5856, 5861, 5863, 5865, 5870, 5872, 5957, 5961, 5965, 5975, 5983, 5984, 5988, 6000, 6001, 6022, 6024, 6028, 6030, 6070, 6133, 6135, 6139, 6141, 6150, 6151, 6155, 6160, 6161, 6165, 6169
<code>\c_circumflex_str</code>	112
<code>\c_colon_str</code>	2888, 3315, 5469
<code>\c_left_brace_str</code>	5938, 5945, 5951
<code>\c_percent_str</code>	112
<code>\c_right_brace_str</code>	5938, 5945, 5951
<code>\str_case:nn</code>	241, 298, 3408
<code>\str_case:nnTF</code>	2022, 2030, 2715, 2723, 5583, 5591
<code>\str_clear:N</code>	3917, 4959
<code>\str_const:Nn</code>	111
<code>\str_count:n</code>	228
<code>\str_if_empty:NTF</code>	1895, 1903, 1939, 1946, 2072
<code>\str_if_eq:nnTF</code>	2246, 3840, 3888, 5710
<code>\str_if_in:nnTF</code>	5465
<code>\str_new:N</code>	73, 118, 125, 126, 127, 145, 188
<code>\str_set:Nn</code>	673, 679, 685, 704, 705, 706, 2561, 2562, 2563, 2610, 2611, 2612, 4337, 4340, 5689, 5699
<code>\str_set_eq:NN</code>	3431, 5336, 5353
<code>\str_use:N</code>	3585
<code>\strut</code>	3581, 3778
<code>\strutbox</code>	1362, 1365, 1376, 1377, 1388, 1390, 1405, 1408, 1416, 1417, 1428, 1430, 1445, 1448, 1455, 1456, 1467, 1469, 1484, 1487, 1533, 1536, 1544, 1545, 1553, 1554, 1566, 1568, 1579, 1580, 1583, 1590, 1603, 1611, 1617,

1625, 4414, 4420, 4465, 4473, 4562

T

tag commands:

<code>\tag_mc_begin:n</code>	4240, 4288, 4297
<code>\tag_mc_end:</code>	4244, 4292, 4301
<code>\tag_resume:n</code>	4237, 4285, 4441, 4449, 4513, 4611, 4811, 4875
<code>\tag_struct_begin:n</code>	4238, 4239, 4246, 4247, 4248, 4286, 4287, 4294, 4295, 4296
<code>\tag_struct_end:n</code>	4245, 4252, 4253, 4254, 4255, 4293, 4302, 4303, 4304, 4305, 4459, 4929, 5201
<code>\tag_suspend:n</code>	4256, 4306, 4432, 4443, 4454, 4504, 4603, 4921, 5193
<code>\tag_tool:n</code>	4442

TeX and $\TeX$ commands:

<code>\@auxout</code>	412
<code>\@currentenv</code>	241, 298
<code>\protected@write</code>	412

text commands:

<code>\text_expand:n</code>	5461
<code>\textasteriskcentered</code>	2558, 2605
<code>\textborn</code>	3605
<code>\textreferencemark</code>	2593
<code>\thepage</code>	418

tl commands:

<code>\c_space_tl</code>	3387, 3400, 5890, 5905, 5928, 5932, 6114, 6115, 6124, 6125, 6181, 6185, 6203
<code>\tl_clear:N</code>	671, 678, 2145, 2152, 2551, 2662, 2672, 2693, 2701, 2908, 3259, 3332, 5304
<code>\tl_clear_new:N</code>	618
<code>\tl_const:Nn</code>	602
<code>\tl_gclear:N</code>	346, 347, 348, 2045, 2056, 3574, 3594, 4847, 4907, 5111
<code>\tl_gclear_new:N</code>	2088, 2102
<code>\tl_gput_right:Nn</code>	603
<code>\tl_greplace_all:Nnn</code>	624
<code>\tl_gset:Nn</code>	275, 276, 289, 290, 2046, 2057, 2090, 2103, 2377, 3505, 5059
<code>\tl_gset_eq:NN</code>	620, 3501, 5104
<code>\tl_if_blank:nTF</code>	3031, 3049, 3128, 3178, 3632, 3653, 3679, 5102, 5767
<code>\tl_if_empty:NTF</code>	738, 753, 781, 795, 812, 819, 843, 857, 1845, 1901, 1905, 1944, 1948, 2207, 2224, 2356, 2415, 2752, 2783, 2928, 3242, 3269, 3342, 3380, 3393, 3526, 4618, 5307, 5676
<code>\tl_if_empty_p:N</code>	2065
<code>\tl_if_exist:NTF</code>	2117, 2129
<code>\tl_if_novalue:nTF</code>	433, 463, 2241, 3045, 3174, 3267, 3340, 3373, 3480, 3499, 3507, 3689, 3915, 4386, 4957, 5233, 5305
<code>\tl_map_inline:Nn</code>	621
<code>\tl_new:N</code>	29, 30, 31, 34, 37, 38, 41, 42, 46, 53, 57, 58, 94, 95, 96, 102, 103, 104, 105, 106, 107, 109, 113, 114, 120, 121, 122, 131, 134, 135, 152, 161, 162, 163, 166, 187
<code>\tl_put_left:Nn</code>	2760, 2791, 2913, 4831, 4892, 5323, 5326
<code>\tl_put_right:Nn</code>	619, 849, 2764, 2795, 2842, 2852, 2865, 2880, 2886, 2891, 2915, 2920, 2927, 2930, 2940, 2945, 2948, 2954, 3227, 3262, 3265, 3271, 3273, 3300, 3305, 3310, 3313, 3322, 3335, 3338, 3344, 3346, 3356, 5309, 5310
<code>\tl_remove_all:Nn</code>	5675
<code>\tl_remove_once:Nn</code>	2830, 3285
<code>\tl_replace_all:Nnn</code>	623, 3222
<code>\tl_retokenize:n</code>	3235

167 / 167